

Кіріспе. Микроконтроллер архитектурасының негіздері

Дәріс 1

Литература

- Хартов, В. Я. Микропроцессорные системы : учебное пособие для студентов высших учебных заведений, обучающихся по направлению "Информатика и вычислительная техника", (квалификация бакалавр) / Вячеслав Яковлевич Хартов. – 2-е изд., испр. и доп.. – Москва : Академия, 2014. – 367, [1] с
- Кравченко, А. В. 10 практических устройств на AVR-микроконтроллерах [Электронный ресурс] / Алексей Владимирович Кравченко. – Москва : Додэка-XXI ; Киев : МК-Пресс, 2014. – 217, [7] с
- Кравченко, А. В. 10 практических устройств на AVR-микроконтроллерах [Текст] / Алексей Владимирович Кравченко ; главный редактор: Ю.А. Шпак. – Киев : МК-Пресс ; Санкт-Петербург : КОРОНА-ВЕК, 2015. – 318, [2] с.
- Белов А.В. Микроконтролеры AVR: от азов программирования до создания практических устройств. – СПб.: Наука и Техника, 2016. – 544с.
- Соммер У. Программирование микроконтроллерных плат Arduino/Freeduino. - СПб.: БХВПетербург, 2014. - 256 с.

Дополнительная литература

- Ревич Ю. В. Занимательная электроника. – 3-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2015. – 576 с
- Платт Ч. Электроника для начинающих. СПб.: БХВ-Петербург, 2017. – 416 с.
- Петин В.А. Проекты с использованием контроллера Arduino. – СПб.: БХВ-Петербург, 2016. – 464 с.
- Карвинен, Торо, Карвинен, Киммо, Валтокари, Вилле. Делаем сенсоры: проекты сенсорных устройств на базе Aduino и Raspberry Pi.:Пер. с англ. - М.: ООО "И.Д. Вильяме": 2015. - 432 с.
- Francis Perea Arduino Essentials. – BIRMINGHAM - MUMBAI : Packt Publishing, 2015. – 206 p.

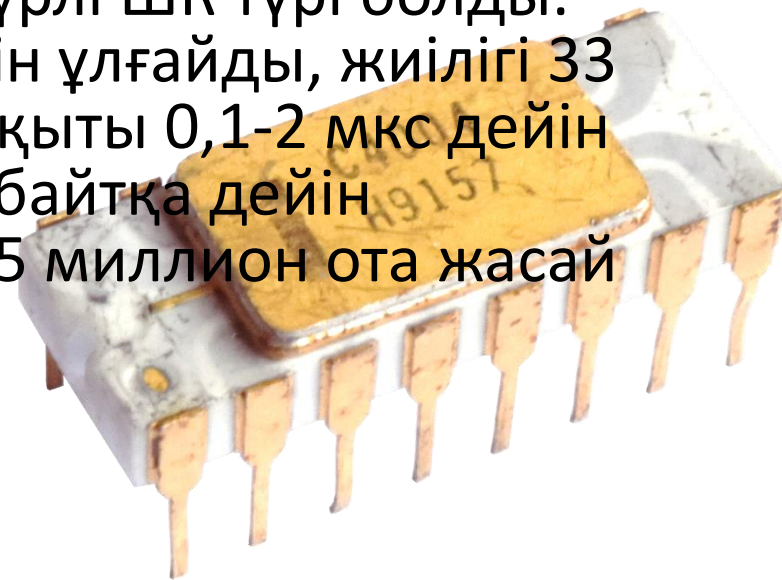
Микропроцессор

бұл сандық немесе ұқсас ақпаратты өңдеуге - бастапқы деректерді енгізуге, арифметикалық, логикалық және басқа операцияларды орындауға, ол жадтан есептейтін командаларға сәйкес нәтижелер беруге, сондай-ақ осы өңдеу процесін басқаруға арналған бағдарламалық-басқарылатын құрылғы. Микропроцессор орындайтын командалардың реттілігі бағдарлама деп аталады.



Микропроцессорлардың құрылу тарихы

1970 жылы Intel (АҚШ) фирмасынан Маршиан Эдвард Хофф өзінің функциялары бойынша үлкен ЭЕМ орталық процессорына ұқсас интегралдық схеманы құрастырды - бірінші 4 разрядты микропроцессор Intel-4004, ол 1971 жылы бағдарламаланатын калькуляторды әзірлеу барысында шығарылды. Ол 4 үлкен интегралды микросхемадан (BIS) тұрды, 4,5 кбайт жадты жібере алды, 740 кГц жиілікте жұмыс істеді және 10-20 мкс уақытымен 45 нұсқаулығы болды. Оның соңынан 1974 жылы 8-разрядты МП келді, ал 1976 жылы 50-ден астам түрлі ШК түрі болды. 1989 жылға қарай МП разрядтылығы 16-32 бит дейін ұлғайды, жиілігі 33 МГц дейін ұлғайды, ал нұсқаулықтарды орындау уақыты 0,1-2 мкс дейін төмендеді. Адрестелетін жадтың көлемі ондаған Мбайтқа дейін ұлғайды. Мұндай микропроцессорлар секундына 25 миллион ота жасай алады.



Қазіргі заманғы микропроцессорлар

Қазіргі заманғы микропроцессорлар

AMD Threadripper 2990WX 3 ГГц жиілікте жұмыс істейді, 32 ядросы бар және бір уақытта 64 нұсқау ағынын орындай алады.



Intel Core i9-9980XE 3 ГГц жиілікте жұмыс істейді, 18 ядросы бар және бір уақытта 36 пәрмен ағынын орындай алады.



Микропроцессорлық құрылғы

Микропроцессорға мыналар кіреді:

- Арифметикалық және логикалық амалдарды орындауға арналған **арифметикалық логикалық бірлік (АЛУ)**;
- Командалар кодтарына сәйкес микропроцессордың барлық блоктары үшін басқару сигналдарын шығаратын **басқару құрылғысы**;
- Командаларды, деректерді және мекенжайларды сақтауға арналған ұяшықтар - **регистер блогы**;
- Жад және енгізу/шығару құрылғыларынан командалар мен деректерді оқу/жазу үшін **байланыстырушы құрылғы**, сыртқы шинаны басқару командаларын генерациялау.

Шина - есептеу жүйесінің барлық блоктарын біріктіретін және олардың арасындағы деректермен алмасуды қамтамасыз ететін ақпараттық арна.

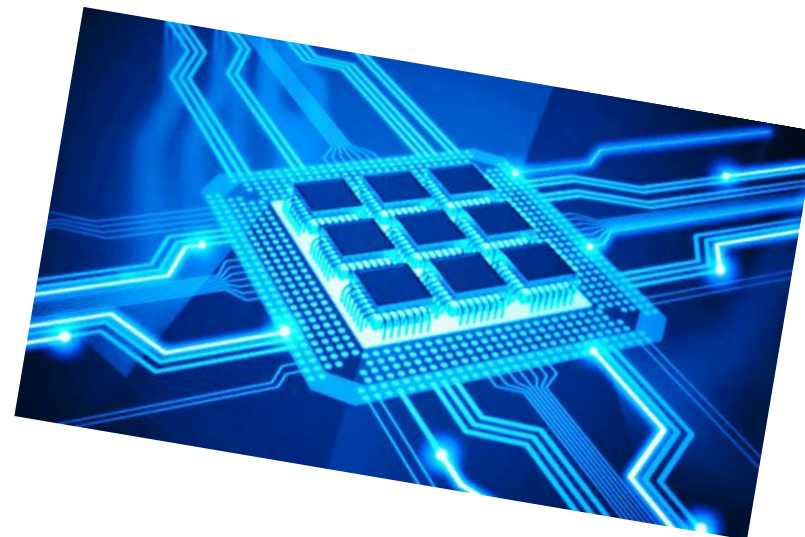
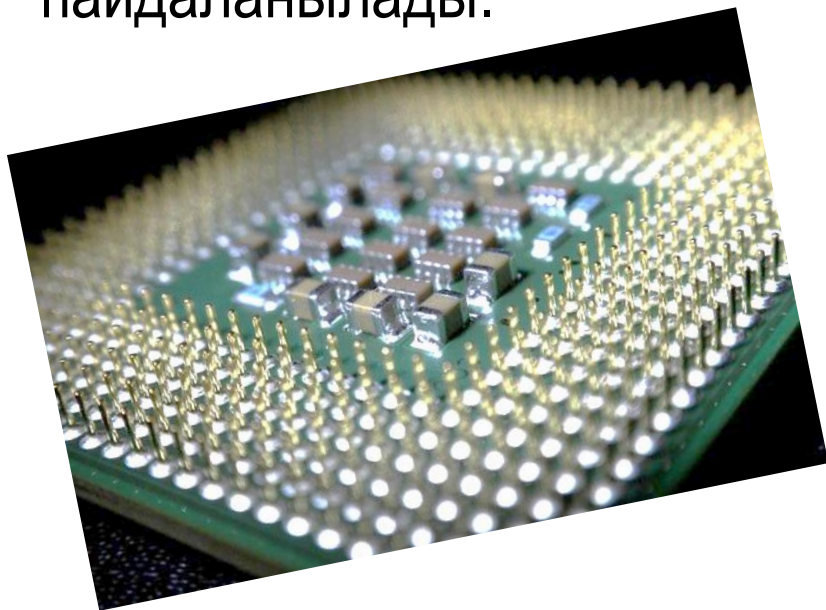
Микропроцессорлардың негізгі сипаттамалары



- Биттік тереңдік – бір уақытта өңделетін деректердің максималды мөлшері (бит биттері). Қазіргі уақытта 8/16/32/64 разрядты микропроцессорлар бар
- Сағат жиілігі – процессор уақыт бірлігінде орындай алатын циклдар саны (элементарлық операциялар). Әрбір нұсқау бір немесе бірнеше процессор циклін талап етеді. Мысалы, 10 МГц процессор жиілігімен секундына 10 миллион элементар операцияны орындайды.

Қандай микропроцессорлар болады?

Әмбебап - ақпаратты есептеу, өңдеу және басқару міндеттерінің кең сыныбын шешуге арналған. Олар, мысалы, дербес компьютерлер мен ноутбуктер құрамында пайдаланылады.



Мамандандырылған - белгілі бір сыныптың міндеттерін шешуге арналған. Олардың қатарына сигналдық, мультимедиялық, транспьютерлер, нейропроцессорлар және басқалар жатады.

Сигналдарды цифрлық өңдеу процессорлары (Digital Signal Processing - DSP), сондай-ақ цифрлық сигнал процессорлары деп аталатын, сигналдарды цифрлық өңдеуге арналған (мысалы, сигналдарды сүзу, корреляциялық функция, сигналды шектеу және түрлендіру, тікелей және кері Фурье түрлендіру үшін)

DSP процессорлары «Гарвард архитектурасына» негізделген, оның айрықша ерекшелігі бағдарламалар мен деректердің әртүрлі жад құрылғыларында — бағдарламалар мен деректерде сақталуы болып табылады.

DSP мысалдары TMS320Cxxxx (Texas Instruments, АҚШ), ADSP-21XX (Аналогтық құрылғы, АҚШ), DSP568xx (Motorola, АҚШ)



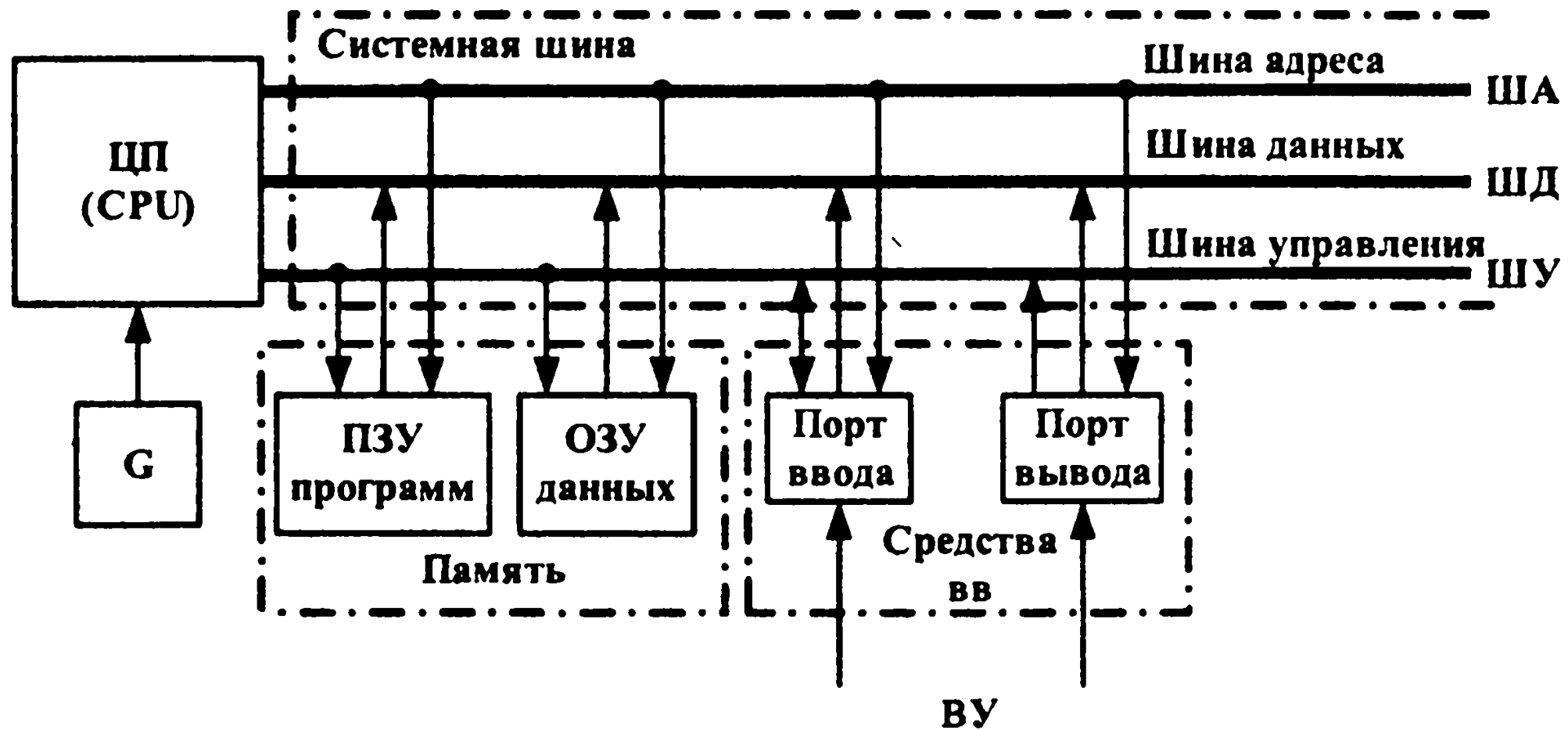
На рисунке:
ША – шина адреса;
ШД – шина данных

Микропроцессорлық жүйе

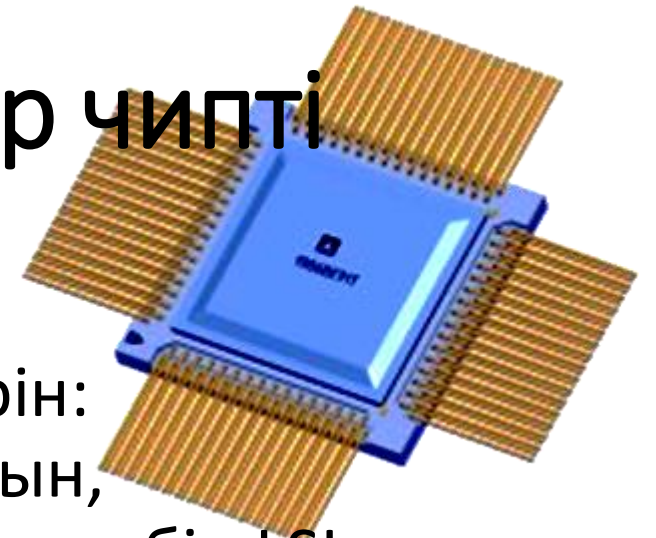
Бұл негізгі ақпаратты өңдеу құрылғысы микропроцессор болып табылатын мамандандырылған есептеу, ақпараттық, басқару немесе бақылау-өлшеу жүйесі. Микропроцессорлық жүйенің типтік құрылымына орталық процессор кіреді, ол жоғарыда қарастырылған микропроцессор түрлерінің бірі бола алады, ол MPS міндеттеріне, бағдарлама мен деректер жадысына, енгізу/шығару порттарына және жүйелік шинаға байланысты. сыртқы құрылғыларды қосу. MPS мысалы ретінде дербес компьютерді айтуға болады.



Микропроцессорлық жүйенің типтік құрылымы



Микроконтроллер (МС) немесе бір чипті микрокомпьютер



микропроцессорлық жүйенің барлық компоненттерін: процессорды, бағдарламаны және деректер жадысын, енгізу/шығару порттарын қамтитын құрылымдық түрде бір LSI модулінде жасалған құрылғы. Қазіргі микроконтроллерлер сонымен қатар көптеген қосымша бағдарламаланатын перифериялық құрылғылармен және байланыс интерфейстерімен толықтырылған. Әдетте, микроконтроллер контроллер (ағылшын тілінен controller - реттеуіш, басқару құрылғысы), микропроцессорлық контроллер немесе микропроцессорды басқару құрылғысы деп аталатын басқару және бақылау құрылғысының өзегі болып табылады.

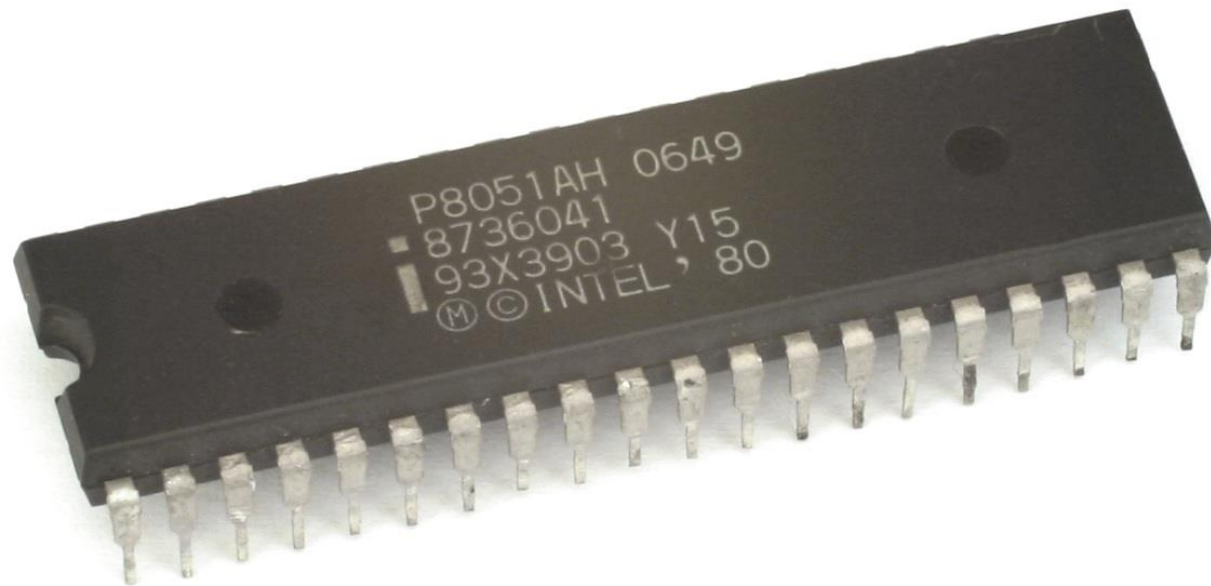
Микроконтроллерлер

Қазіргі уақытта бір чипті микроконтроллерлер микропроцессорлық технологияның ең танымал өкілдері болып табылады, олардың шығарылымы жылына шамамен 2,5 млрд. Оларды бірқатар компаниялар (Intel, Motorola, Philips, Siemens, Atmel, Dallas, Temic, Oki, AMD және т.б.) шығарады, олар тұтынушыларға салыстырмалы түрде қарапайым 8-биттік үлгілерден 32-ге дейінгі өнімдердің үлкен ассортиментін ұсынады. -биттік құрылғылар, олардың ядросы CISC және RISC процессорлары.

MCS-48



MCS-51



Микроконтроллерлердің негізгі түрлері

Микроконтроллер – бұл үлкен интегралдық схемалар (БИС) технологиясы арқылы жасалған және микропроцессорды, жадты, енгізу-шығару порттарын, сонымен қатар нақты уақыттағы қолдау құралдарын (таймерлер, үзілістерді өңдеушілер және т.б.) бір микросхемада біріктіретін құрылғы.

Бір чипті микроконтроллерлер Гарвард архитектурасын пайдаланады. Оның концепциялары бойынша программа жады мен деректер жадында әртүрлі адрестеу механизмдері арқылы қол жеткізілетін жеке адрестік кеңістіктер бар. Бағдарлама жадының мазмұны зауытта (маскалық бағдарламалау) немесе пайдаланушы бағдарламалаушысы (электрлік бағдарламалау) арқылы конфигурацияланады.

Микроконтроллерлер

Қазіргі микроконтроллерлерді үш негізгі түрге бөлуге болады: а) енгізілген 8-разрядты микроконтроллерлер; б) 16 және 32 разрядты микроконтроллерлер; в) цифрлық сигнал процессорлары.

Кірістірілген микроконтроллерлер

Кірістірілген микроконтроллерлерде барлық қажетті ресурстар (жад, таймерлер, енгізу-шығару порттары және т.б.) процессор өзегі бар бір микросхемада орналасады, бұл олардың қосымша компоненттердің ең аз санын пайдалана отырып, басқару жүйесіне қосылуын қамтамасыз етеді. Олар қазіргі концепциялар бойынша шағын көлемді бағдарлама жадысымен (ROM 1-ден 128 Кбайтқа дейін) және деректер жадысымен (RAM 64 байттан 4 Кбайтқа дейін) сипатталады.

Сонымен қатар, бағдарлама жадына жазу мүмкіндігі әдетте алынып тасталады, бұл қателердің ықтималдығын азайтады және осы жақтың мазмұнын рұқсатсыз көшіруден қорғау бар. Енгізілген қолданбаларға арналған әдеттегі микроконтроллер микроамперден ондаған миллиамперге дейін ток тартады және әдетте -55 пен +125°C температура диапазонында жұмыс істейді. Мұндай контроллерлер ассемблер тілінде немесе C тілінде бағдарламаланған.

16 және 32 разрядты микроконтроллерлер

Неғұрлым күрделі микроконтроллерлер (16-разрядты және 32-разрядты) тек сыртқы жадты пайдаланады, оған бағдарлама жады (ROM) және деректер жады (RAM) кіреді. Олар жадтың үлкен көлемін және енгізу-шығару құрылғыларының (порттардың) салыстырмалы түрде аз санын қажет ететін жүйелерде қолданылады. Мұндай МК-ны сыртқы жадымен пайдаланудың типтік мысалы ретінде үлкен көлемдегі мәліметтерді (бірнеше мегабайт тәртібі бойынша) аралық сақтауды және таратуды қамтамасыз ететін буферлік кэш жады бар қатты диск контроллері (HDD) болып табылады. Сыртқы жад мұндай микроконтроллерге енгізілген микроконтроллерге қарағанда жоғары жылдамдықпен жұмыс істеуге мүмкіндік береді.

Сандық сигналдық процессорлар (DSP)

Цифрлық сигнал процессорлары (DSP) аналогтық сигналдарды цифрлық өңдеуге арналған, яғни. аналогтық схемамен дәстүрлі түрде шешілетін мәселелерді шешу. Олар аналогтық көзден деректерді енгізуді, оларды өңдеуді және нақты уақытта сәйкес шығыс сигналын қалыптастыруды қамтамасыз етеді. Сигнал процессорларына арнайы талаптар қойылады. Олар максималды жылдамдықты, шағын өлшемдерді, аналогты-цифрлық және цифрлық-аналогтық түрлендіргіштермен оңай қондыруды, өңделген деректердің үлкен бит тереңдігін және міндетті түрде көбейту-жинақтау операциясын және аппараттық ұйымдастыруды қамтитын математикалық операциялардың шағын жиынтығын талап етеді. циклдар. Модемдердің көпшілігі осы типтегі процессорларда, сондай-ақ дыбыс пен бейне өңдеудің әртүрлі жүйелерінде жасалады.

CISC және RISC микроконтроллерлері

Сондай-ақ CISC және RISC микроконтроллерлерінің арасында айырмашылықты жасау керек. RISC процессорының нұсқаулары өте қарапайым және әрқашан дерлік бір тактілік циклде орындалады. CISC процессорымен өңделген операцияны бір команда ретінде жүзеге асыру үшін RISC микронұсқаулар тізбегін орындауы керек. CISC микроконтроллерлерінің артықшылығы - бұл бағдарлама кодының көлемін азайту және оны жазуға жұмсалған бағдарламашының күш-жігерін үнемдеу. Дегенмен, RISC процессорлары CISC-ке қарағанда жылдамырақ, чипте аз орын алады және қуатты аз тұтынады. Сондықтан соңғы уақытта RISC архитектурасы бар микроконтроллерлердің көбірек түрлері пайда болды.

Микроконтроллер жады

Микроконтроллерлерде жадтың үш негізгі түрі қолданылады: бағдарлама жады, деректер жады және регистрлер.

Бағдарлама жады – бұл бағдарлама кодын және тұрақты мәндерді сақтауға арналған тек оқуға арналған жад. Ол бағдарламаны орындау кезінде мазмұнды өзгертпейді.

Деректер жады бағдарламаны орындау кезінде айнымалыларды сақтауға арналған.

Регистрлер ішкі процессор регистрлерінің құрылғыларды басқару үшін қолданылады.

Айта кету керек, MCS-48/51 отбасыларының микроконтроллерлерінде регистрлер ретінде ішкі, (резиденттік) деректер жадысының арнайы аймағы қолданылады. Сонымен қатар, ішкі жадымен қатар микроконтроллерлердің көпшілігі сыртқы бағдарлама мен деректер жадысын пайдалана алады.

Бағдарлама жады

Бағдарламаларды сақтау үшін әдетте тек оқуға арналған жад түрлерінің бірі қолданылады: ROM (маска ROM), PROM (бір реттік бағдарламаланатын ROM), EPROM (ультра күлгін өшіретін электрлік бағдарламаланатын ROM) немесе EEPROM (электрлік жазу және өшіру мүмкіндігі бар ROM , қазіргі микросұлбалар да осы түрдегі флэш жадыға жатады). Жадтың бұл түрлерінің барлығы тұрақты емес, яғни бұл жадтың мазмұны қуат өшірілгеннен кейін сақталады.

Программирование микроконтроллеров

Микроконтроллерлерді бағдарламалау үшін (яғни, оларға бағдарламалар жазу) әдетте арнайы құрылғылар пайдаланылады - дербес компьютерге қосылған бағдарламашылар. Ерекшелік - ROM жады бар микроконтроллерлер, олар өндіріс процесінде бағдарламаланады. Дегенмен, қазіргі уақытта EEPROM жадын бағдарламалау жобаланатын немесе дайын құрылғының тақтасында тікелей жүзеге асырылуы мүмкін. Бұл әдіс ISP (Жүйелік бағдарламалауда) деп аталады, ол микроконтроллердің бағдарламалық құралын платадан шығармай-ақ жаңартуға мүмкіндік береді, бұл әзірлеу кезеңінде көп уақытты үнемдейді.

Флэш жады

Микроконтроллерлердің соңғы үлгілерінде функционалдық жағынан EEPROM-дан аз ғана айырмашылығы бар кірістірілген Flash жады бар. Негізгі айырмашылық - жазылған ақпаратты өшіру мүмкіндігі. EEPROM жадында өшіру әрбір ұяшық үшін бөлек орындалады, ал Flash жадыда өшіру тұтас блоктарда жүзеге асырылады.

Деректер жады

Деректерді сақтау үшін RAM - жедел есте сақтау құрылғысы пайдаланылады. Мұндай жадтағы оқу және жазу циклдерінің саны шектеусіз, бірақ қоректендіруді ажырату кезінде барлық ақпарат жоғалады.

Қуат және энергияны басқару

Алғашқы микроконтроллерлер 5В қоректену кернеуіне арналған. Дегенмен, жақында 3,3 В немесе одан аз номиналды кернеуі бар көптеген құрылғылар пайда болды. Бұл, ең алдымен, өздігінен жұмыс істейтін құрылғылардағы микроконтроллерлердің кеңеюіне байланысты.

Қуат және энергияны басқару

Сонымен қатар, көптеген микроконтроллерлер бағдарламалық қамтамасыз ету арқылы жұмыс режиміне немесе тоқтату режиміне ауыстыру арқылы қуат тұтынуды басқару мүмкіндігін береді. *Жұмыссыз режим процессор өзегінің жұмысы тоқтатылуымен сипатталады, тек сағаттық сигнал генераторы мен бақылау таймері жұмыс жағдайында қалады.* Регистрлер өз құндылығын сақтайды. Бұл режимнен үзу немесе қалпына келтіру сигналы арқылы шығуға болады. Сондай-ақ, белсенді әрекеттерді орындау үшін микроконтроллерді мезгіл-мезгіл «оятатын» арнайы бос таймерді қамтамасыз етуге болады. Бұл режимде микроконтроллер номиналды қуаттың шамамен 30% тұтынады. Тоқтату режимінде кристалдың ішіндегі жұмыс толығымен тоқтатылады. Режимден шығу қалпына келтіру сигналы арқылы мүмкін болады (кейде сыртқы үзу сигналы арқылы). Тоқтату режимінде ток тұтынуы 1 мкА немесе одан аз болуы мүмкін.

Микроконтроллердің перифериялық құрылғылары

- енгізу және шығару үшін теңшеуге болатын әмбебап цифрлық порттар;
- түрлі енгізу/шығару интерфейстері, мысалы, UART, I²C, SPI, CAN, USB, IEEE1394, Ethernet, SpaceWire және т.б.;
- аналогты-цифрлық және цифрлық-аналогты түрлендіргіштер;
- компараторлар;
- импульстік ені модуляторлары;
- таймерлер;
- қозғалтқыш контроллері;
- дисплей мен пернетақта контроллері;
- радиожиілік қабылдағыштары мен таратқыштары;
- кірістірілген флэш-жад массивтері;
- кірістірілген сағат генераторы және күзетші таймері және т.б.

Қолдану

Қазіргі уақытта микропроцессорлар мен микроконтроллерлер компьютерлер мен ноутбуктерде, автомобильдер мен пойыздарда, ұшақтар мен ғарыш кемелерінде, кемелер мен суасты қайықтарында, технологиялық процестер мен өндірісті басқаруда, телекоммуникациялық жүйелер мен құрылғыларда, медициналық жабдықтарда, ғылыми зерттеулерде, тұрмыстық техникада – теледидарларда, тоңазытқыштар, ұялы телефондар, кір жуғыш машиналар және тіпті кофеқайнатқыштар.

Arduino

Arduino - бұл электронды жинақ және жаңадан бастағандар мен кәсіпқойлар үшін электронды құрылғыларды жылдам дамытуға арналған ыңғайлы платформа. Платформа программалау тілінің ыңғайлылығы мен қарапайымдылығы, сонымен қатар ашық архитектурасы мен бағдарлама коды арқасында бүкіл әлемде өте танымал. Құрылғы бағдарламалаушылардың көмегінсіз USB арқылы бағдарламаланады.

Arduino компьютерге виртуалды әлемнен физикалық әлемге өтуге және онымен өзара әрекеттесуге мүмкіндік береді. Arduino негізіндегі құрылғылар әртүрлі сенсорлар арқылы қоршаған орта туралы ақпаратты ала алады, сонымен қатар әртүрлі жетектерді басқара алады.

Arduino

Тақтадағы микроконтроллер Arduino тілі (Wring тіліне негізделген) және Arduino әзірлеу ортасы (Өңдеу ортасына негізделген) арқылы бағдарламаланған. Arduino негізіндегі құрылғы жобалары өздігінен жұмыс істей алады немесе компьютердегі бағдарламалық құралмен өзара әрекеттесе алады (мысалы: Flash, Processing, MaxMSP). Тақталарды пайдаланушы өз бетімен жинай алады немесе жиынтық ретінде сатып алады. Бағдарламалық құралды тегін жүктеп алуға болады. Түпнұсқа схема сызбалары (CAD файлдары) жалпыға қолжетімді, пайдаланушылар оларды өз қалауы бойынша пайдалана алады

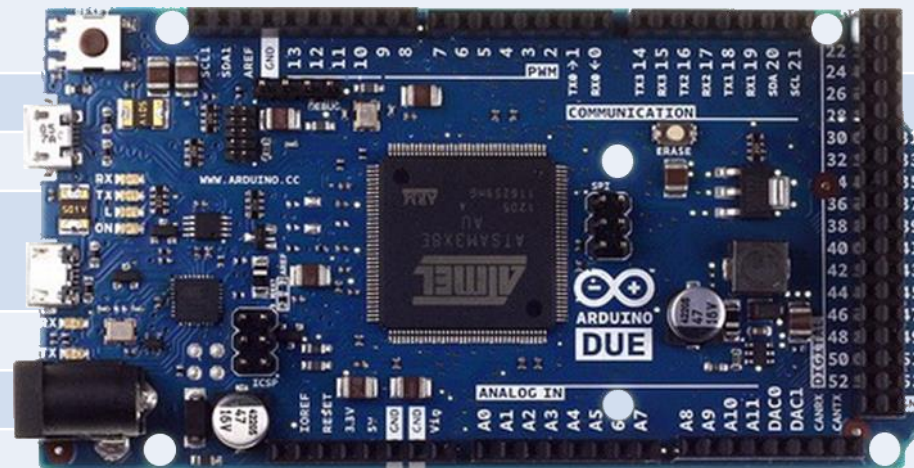
Аппаратная часть платформы Arduino

Arduino платформаларының бірнеше нұсқалары бар. Леонардоның соңғы нұсқасы ATmega32u4 микроконтроллеріне негізделген. Uno, Duemilanove бағдарламасының алдыңғы нұсқасы сияқты, Atmel ATmega328 микроконтроллерінде құрастырылған. Diecimila платформасының ескі нұсқалары және алғашқы жұмыс істейтін Duemilanoves Atmel ATmega168 негізінде әзірленді, бұрынғы нұсқаларында ATmega8 пайдаланылды. Arduino Mega2560, өз кезегінде, ATmega2560 микроконтроллерінде құрастырылған.

Arduino платформасының нұсқалары:

Due - бұл ARM микропроцессорының 32 биттік Cortex-M3 ARM SAM3U4E негізіндегі жаңа тақтасы

Микроконтроллер	AT91SAM3X8E
Жұмыс кернеуі	3,3 В
Кіріс кернеуі (ұсынылады)	7-12 В
Кіріс кернеуі (шектеу)	6-20 В
Сандық Кірістер/Шығулар	54 (на 12 из которых реализуется выход ШИМ)
Аналогтық кірістер	12
Аналогтық шығыстар	2 (ЦАП)
Барлық кіріс/шығыстардағы жалпы шығыс тұрақты ток	50 мА
3,3 В контакті арқылы тұрақты ток	800 мА
5 В контакті арқылы тұрақты ток	800 мА
Флэш жады	512 КБ доступно всего для пользовательских приложений
Жедел Жадтау Құрылғысы	96 КБ (два банка: 64 КБ и 32 КБ)
Сағат жиілігі	84 МГц



Arduino платформасының нұсқалары:

Леонардо - ATmega32u4 микроконтроллеріндегі Arduino платформасының соңғы нұсқасы. Оның өлшемі UNO сияқты бірдей microUSB қосқышы бар.

Микроконтроллер	ATmega32u4
Жұмыс кернеуі	5 В
Кіріс кернеуі (ұсынылады)	7-12 В
Кіріс кернеуі (шектеу)	6-20 В
Сандық Кірістер/Шығулар	20 (7 из которых могут использоваться как выходы ШИМ)
Аналогтық арналар	12
Кіріс/шығыс арқылы тұрақты ток	40 мА
Тұрақты ток шығысы 3,3 В	50 мА
Флеш-память	32 Кб (ATmega32u4) из которых 4 Кб используются для загрузчика
ОЗУ	2 Кб (ATmega32u4)
EEPROM	1 Кб (ATmega32u4)
Сағат жиілігі	16 МГц



Arduino платформасының нұсқалары:

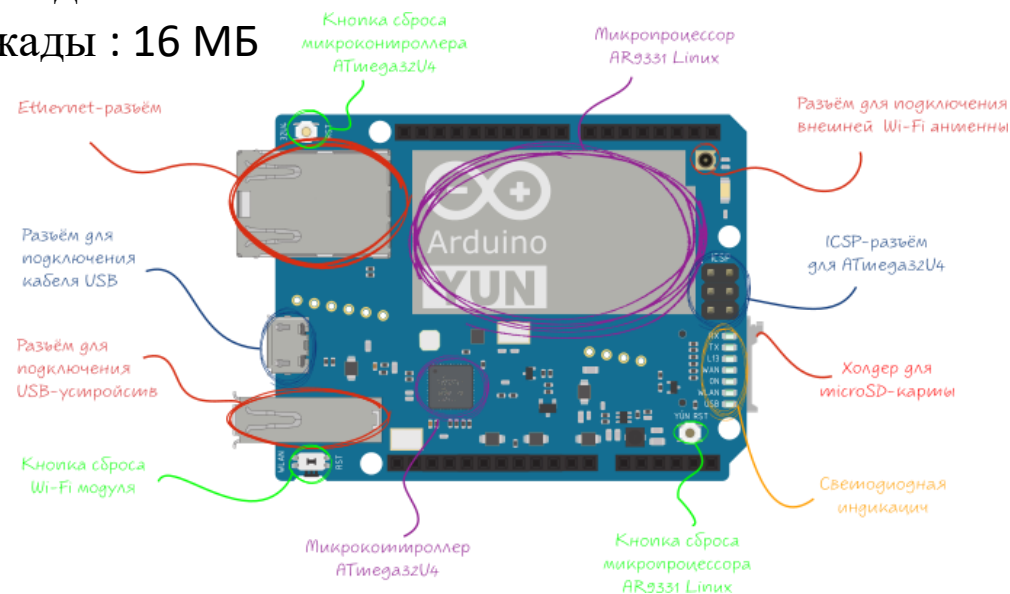
Yun - ATmega32u4 және Atheros AR9331 негізіндегі кірістірілген WiFi қолдауы бар жаңа тақта

Arduino микроконтроллер ATmega32U4

- Архитектура: AVR
- Сағат жиілігі: 16 МГц
- Логикалық деңгей кернеуі: 5V
- Жалпы мақсаттағы енгізу/шығару порттары: 20
- Енгізу/шығару істікшесінен максималды ток: 20 мА
- PWM порттары : 6
- ADC-ге қосылған порттар : 6
- ADC бит тереңдігі : 10 бит
- Флэш жады : 32 КБ
- EEPROM жады: 1 КБ
- Жедел жады: 2,5 КБ

Linux микропроцессор

- Процессор: Atheros AR9331
- Архитектура MIPS: 400 МГц
- Жұмыс кернеуі : 3,3 В
- Ethernet: IEEE 802.3 10/100 Мбит/сек
- Wi-Fi: IEEE 802.11b/g/n
- USB: Type-A 2.0 хост/устройство
- SD-модуль: Micro-SD
- Жедел жады : 64 МБ DDR2
- Флэш жады : 16 МБ



Arduino платформасының нұсқалары:

Arduino Micro

Микроконтроллер	ATmega32u4
Жұмыс кернеуі	5 В
Кіріс кернеуі (ұсынылады)	7-12 В
Кіріс кернеуі (шектеу)	6-20 В
Сандық Кірістер/Шығулар	20
PWM арналары	7
Аналогтық кіріс арналары	12
Кіріс/шығыс арқылы тұрақты ток	40 мА
Тұрақты ток шығысы 3,3 В	50 мА
Флэш жады	32 Кб (ATmega32u4) из которых 4 Кб используются для загрузчика
ОЗУ	2,5 Кб (ATmega32u4)
EEPROM	1 Кб (ATmega32u4)
Сағат жиілігі	16 МГц



Arduino платформасының нұсқалары:

Arduino Uno

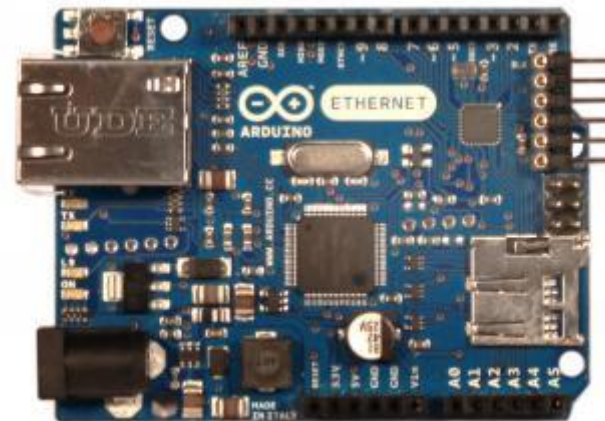
Микроконтроллер	ATmega328
Жұмыс кернеуі	5 В
Кіріс кернеуі (ұсынылады)	7-12 В
Кіріс кернеуі (шектеу)	6-20 В
Сандық Кірістер/Шығулар	14 (6 из которых могут использоваться как выходы ШИМ)
Аналогтық кірістер	6
Кіріс/шығыс арқылы тұрақты ток	40 мА
Тұрақты ток шығысы 3,3 В	50 мА
Флэш жады	32 Кб (ATmega328) из которых 0.5 Кб используются для загрузчика
ОЗУ	2 Кб (ATmega328)
EEPROM	1 Кб (ATmega328)
Сағат жиілігі	16 МГц



Arduino платформасының нұсқалары:

Arduino Ethernet

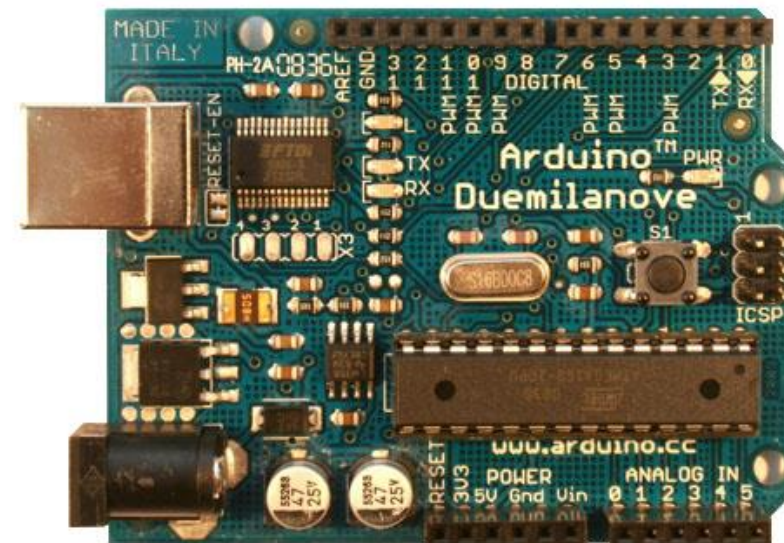
Микроконтроллер	ATmega328
Жұмыс кернеуі	5 В
Кіріс кернеуі (шектеу)	6-18 В
POE арқылы кіріс кернеуі (шектеу).	36-57 В
Сандық Кірістер/Шығулар	14 (4 из которых могут использоваться как выходы ШИМ)
Сақталған Arduino түйреуіштері	с 10 по 13 используются для SPI 4 используется для SD-карты 2 – прерывание W5100 (когда соединен)
Кіріс/шығыс арқылы тұрақты ток	6
Тұрақты ток шығысы 3,3 В	40 мА
Кіріс/шығыс арқылы тұрақты ток	50 мА
Флэш жады	32 Кб (ATmega328) из которых 0.5 Кб используются для загрузчика
ОЗУ	2 Кб (ATmega328)
EEPROM	1 Кб (ATmega328)
Сағат жиілігі	16 МГц



Arduino платформасының нұсқалары:

Arduino Duemilanove

Микроконтроллер	ATmega168
Жұмыс кернеуі	5 В
Кіріс кернеуі (ұсынылады)	7-12 В
Кіріс кернеуі (шектеу)	6-20 В
Сандық Кірістер/Шығулар	14 (6 из которых могут использоваться как выходы ШИМ)
Аналогтық кірістер	6
Кіріс/шығыс арқылы тұрақты ток	40 мА
Тұрақты ток шығысы 3,3 В	50 мА
Флэш жады	16 Кб (ATmega168) или 32 Кб (ATmega328) при этом 2 Кб используются для загрузчика
ОЗУ	1 Кб (ATmega168) или 2 Кб (ATmega328)
EEPROM	512 байт (ATmega168) или 1 Кб (ATmega328)
Сағат жиілігі	16 МГц



Arduino платформасының нұсқалары:

Arduino Nano

Микроконтроллер	Atmel ATmega168 или ATmega328
Жұмыс кернеуі	5 В
Кіріс кернеуі (ұсынылады)	7-12 В
Кіріс кернеуі (шектеу)	6-20 В
Сандық Кірістер/Шығулар	14 (6 из которых могут использоваться как выходы ШИМ)
Аналогтық кірістер	8
Кіріс/шығыс арқылы тұрақты ток	40 мА
Флэш жады	16 Кб (ATmega168) или 32 Кб (ATmega328) при этом 2 Кб используются для загрузчика
ОЗУ	1 Кб (ATmega168) или 2 Кб (ATmega328)
EEPROM	512 байт (ATmega168) или 1 Кб (ATmega328)
Сағат жиілігі	16 МГц
Өлшемдері	1.85 см x 4.2 см



Arduino платформасының нұсқалары:

Arduino Mega ADK для Android

Микроконтроллер	ATmega2560
Жұмыс кернеуі	5В
Кіріс кернеуі (ұсынылады)	7-12В
Кіріс кернеуі (шектеу)	5.5-16В
Сандық Кірістер/Шығулар	54 (14 из которых могут работат также выходы ШИМ)
Аналогтық кірістер	16
Кіріс/шығыс арқылы тұрақты ток	40 mA
Тұрақты ток шығысы 3,3 В	50 mA
Флэш жады	256 KB (из которых 8 KB используются для загрузчика)
ОЗУ	8 KB
Энергияға тәуелді жад	4 KB
Сағат жиілігі	16 MHz



Arduino платформасының нұсқалары:

Arduino Mega 2560

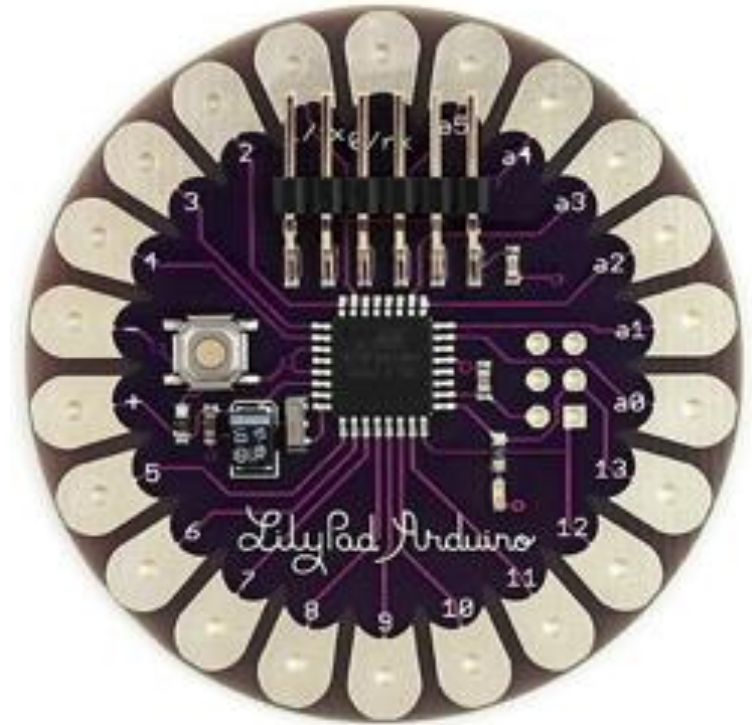
Микроконтроллер	ATmega2560
Жұмыс кернеуі	5В
Кіріс кернеуі (ұсынылады)	7-12В
Кіріс кернеуі (шектеу)	6-20В
Сандық Кірістер/Шығулар	54 (14 из которых могут работат также к выходы ШИМ)
Аналогтық кірістер	16
Кіріс/шығыс арқылы тұрақты ток	40 mA
Тұрақты ток шығысы 3,3 В	50 mA
Флэш жады	256 KB (из которых 8 KB используются для загрузчика)
ОЗУ	8 KB
Энергияға тәуелді жад	4 KB
Сағат жиілігі	16 MHz



Arduino платформасының нұсқалары:

Arduino Lilypad

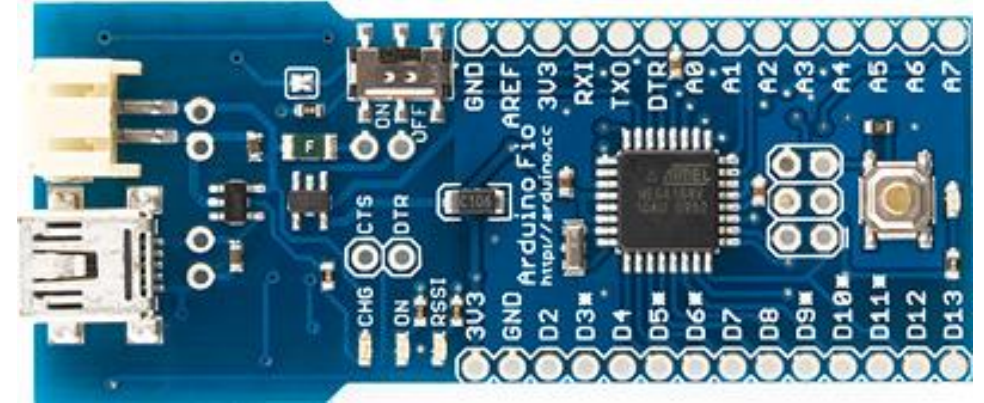
Микроконтроллер	ATmega168V или ATmega328V
Рабочее напряжение	2,7-5,5 В
Входное напряжение	2,7-5,5 В
Цифровые Входы/Выходы	14 (6 из которых могут использоваться как выходы ШИМ)
Аналоговые входы	6
Постоянный ток через вход/выход	40 мА
Флеш-память	16 Кб (2 используются для загрузчика)
ОЗУ	1 Кб
Энергонезависимая память	512 байт
Тактовая частота	8 МГц



Arduino платформасының нұсқалары:

Arduino Fio

Микроконтроллер	ATmega328P
Рабочее напряжение	3.3 В
Входное напряжение	3,35-12 В
Входное напряжение заряда	3.7 - 7 В
Цифровые Входы/Выходы	14 (6 из которых могут использоваться как выходы ШИМ)
Аналоговые входы	8
Постоянный ток через вход/выход	40 мА
Флеш-память	32 Кб (2 используются для загрузчика)
ОЗУ	2 Кб
EEPROM	1 Кб
Тактовая частота	8 МГц



Arduino платформасының нұсқалары:

Arduino Mini

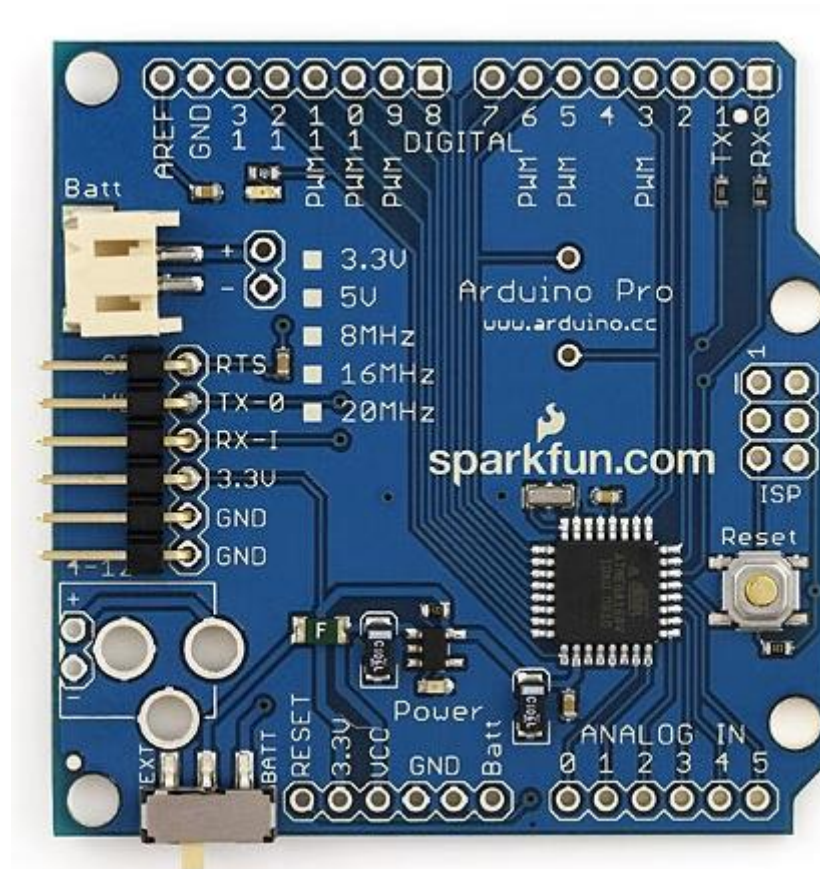
Микроконтроллер	ATmega168
Рабочее напряжение	5 В
Входное напряжение	7-9 В
Цифровые Входы/Выходы	14 (6 из которых могут использоваться как выходы ШИМ)
Аналоговые входы	8 (4 из которых имеют выводы)
Постоянный ток через вход/выход	40 мА
Флеш-память	16 Кб (2 используются для загрузчика)
ОЗУ	1 Кб
Энергонезависимая память	512 байт
Тактовая частота	16 МГц



Arduino платформасының нұсқалары:

Arduino Pro

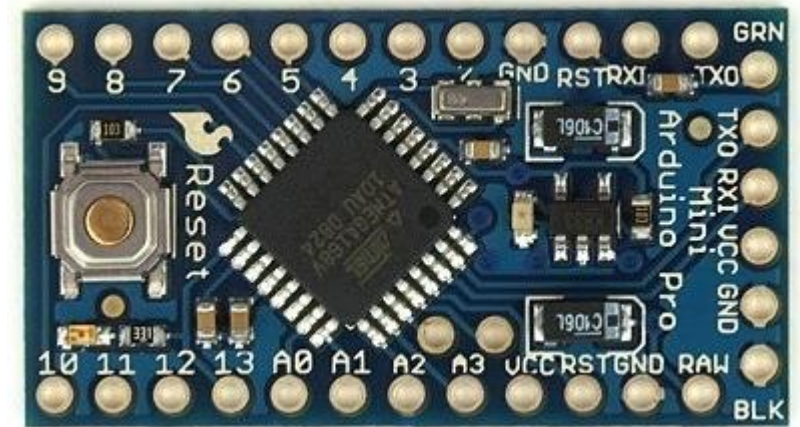
Микроконтроллер	ATmega168 или ATmega328
Рабочее напряжение	3.3 В или 5 В
Входное напряжение	3.35-12 В (версии 3.3 В) или 5-12 В (версии 5 В)
Цифровые Входы/Выходы	14 (6 из которых могут использоваться как выходы ШИМ)
Аналоговые входы	6
Постоянный ток через вход/выход	40 мА
Флеш-память	16 Кб (ATmega168) или 32 Кб (ATmega328) при этом 2 Кб используются для загрузчика
ОЗУ	1 Кб (ATmega168) или 2 Кб (ATmega328)
EEPROM	512 байт (ATmega168) или 1 Кб (ATmega328)
Тактовая частота	8 МГц (версии 3.3 В) или 16 МГц (версии 5 В)



Arduino платформасының нұсқалары:

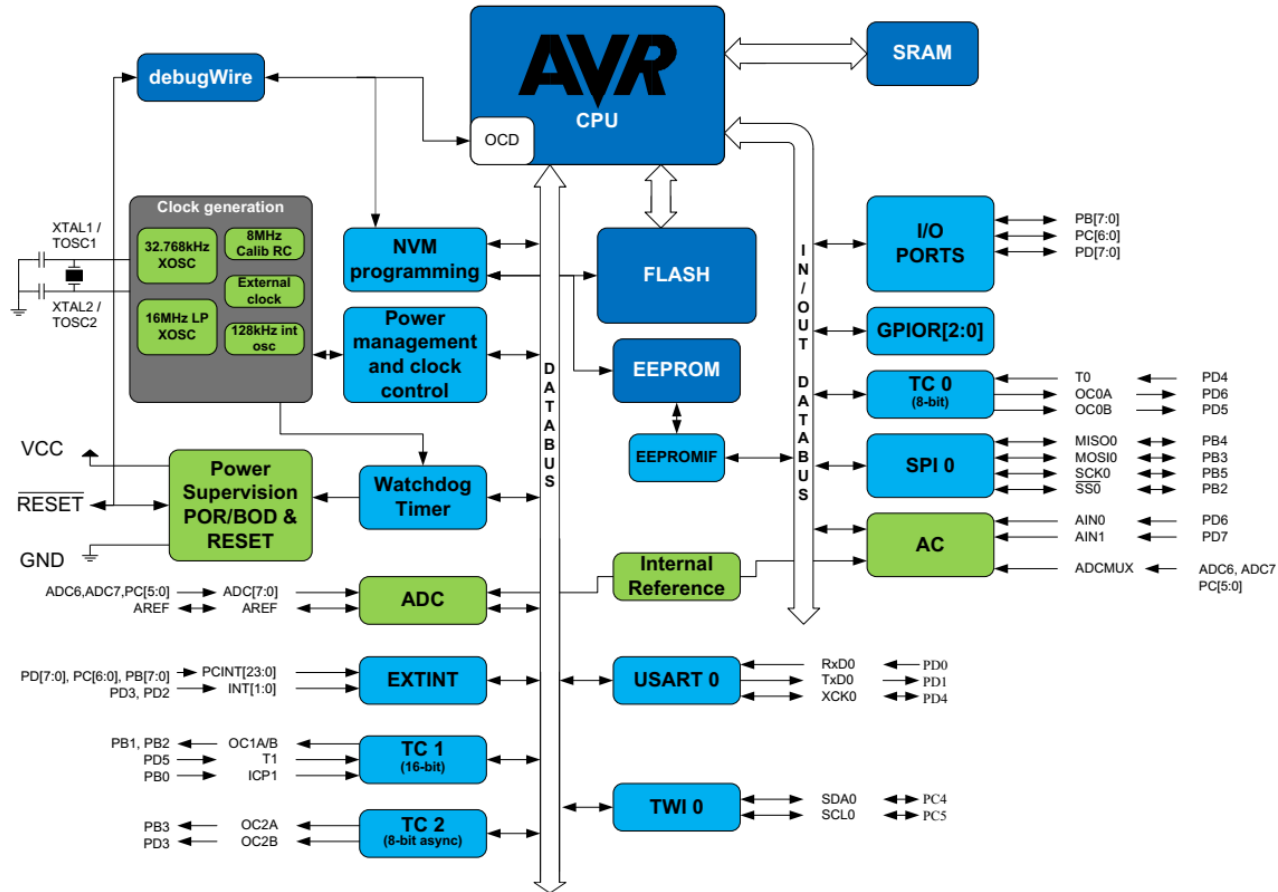
Arduino Pro Mini

Микроконтроллер	ATmega168
Рабочее напряжение	3.3 В или 5 В (в зависимости от модели)
Входное напряжение	3.35-12 В (модель 3.3 В) или 5-12 В (модель 5 В)
Цифровые Входы/Выходы	14 (6 из которых могут использоваться как выходы ШИМ)
Аналоговые входы	6
Постоянный ток через вход/выход	40 мА
Флеш-память	16 Кб (2 используются для загрузчика)
ОЗУ	1 Кб
EEPROM	512 байт
Тактовая частота	8 МГц (модель 3.3 В) или 16 МГц (модель 5 В)

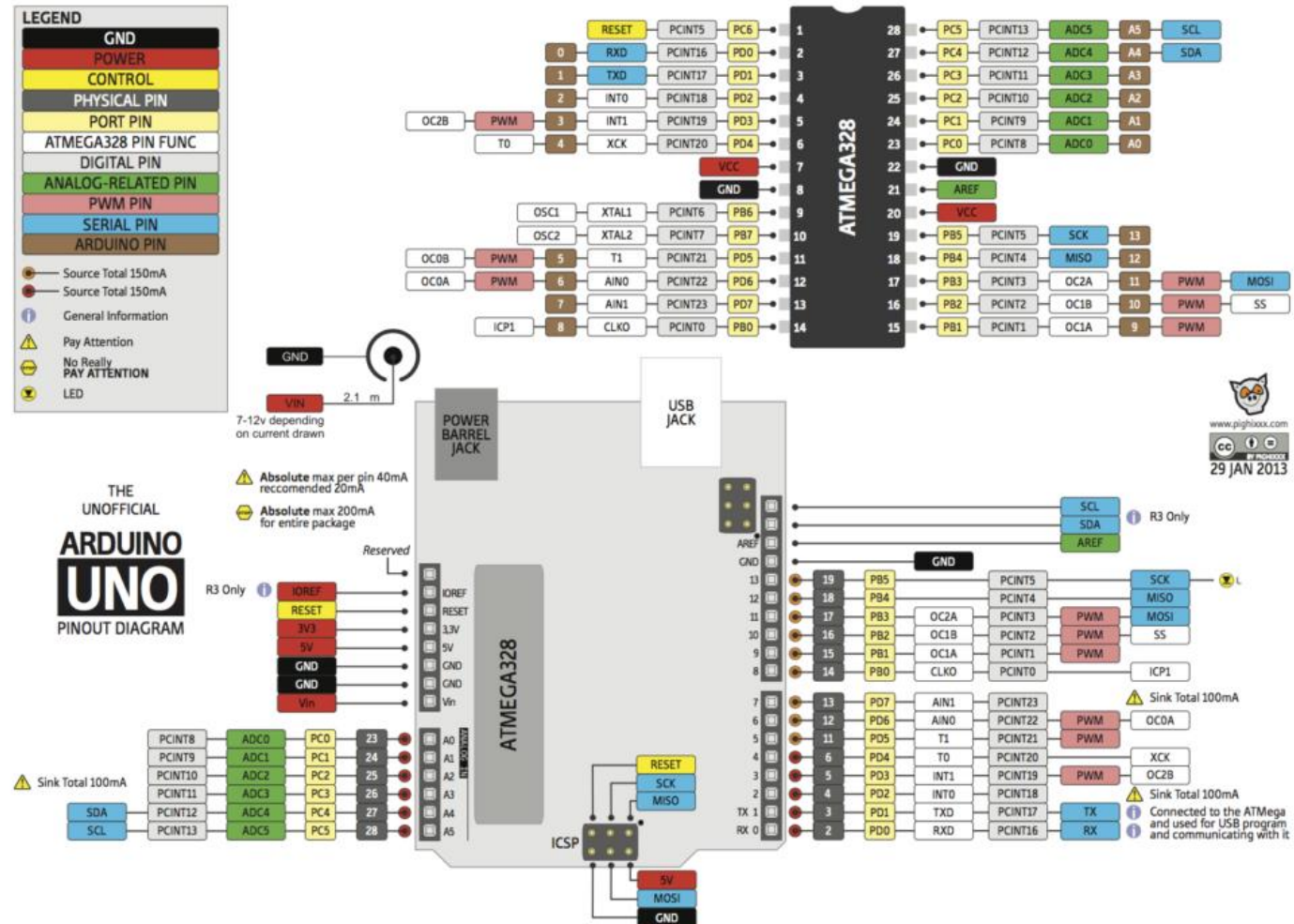


Arduino Uno

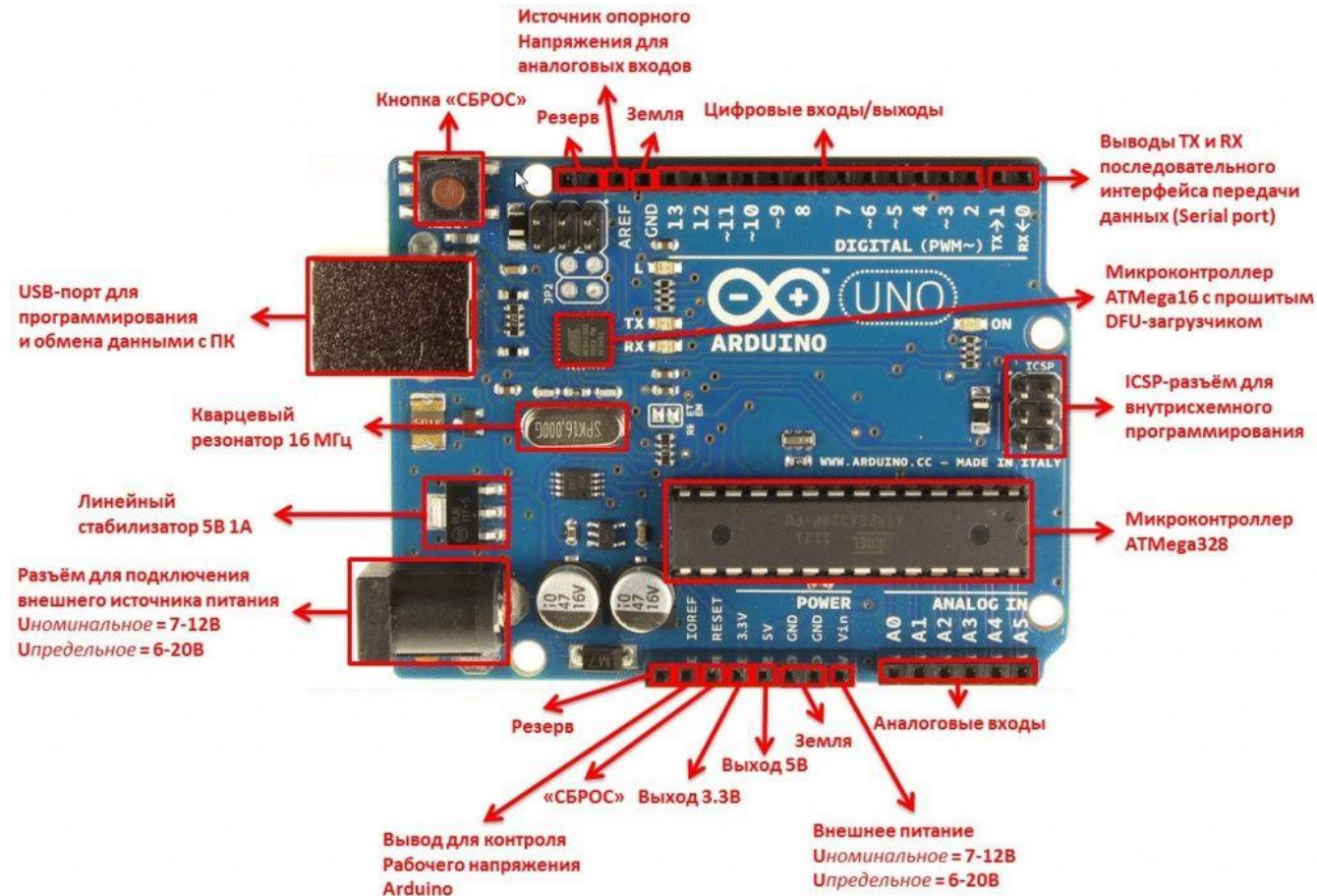
АТmega328 микроконтроллерінің құрылымдық схемасы



ATmega328-Arduino распиновка



Arduino Uno R3 тақтасының элементтерінің сипаттамасы



АТmega328Р микроконтроллерінің сипаттамалары

АТmega328Р - бұл AVR отбасының микроконтроллері, басқалары сияқты, оның 8-биттік процессоры бар және көптеген командаларды бір тактілік циклде орындауға мүмкіндік береді.

Жады:

- 32 kB Flash (өзін-өзі бағдарламалау мүмкіндігі бар бағдарлама жады)
- 2 kB ОЗУ
- 1 kB EEPROM (тұрақты деректер жады)

АТmega328Р микроконтроллерінің сипаттамалары

Перифериялық құрылғылар:

- Салыстырғыштары мен жиілік бөлгіштері бар екі 8-биттік таймер/санауыш
- Салыстырғыш және жиілікті бөлгіш және жазу режимі бар 16-биттік таймер/санауыш
- Жеке генераторы бар нақты уақыт есептегіші
- Алты PWM арнасы (DAC-ға ұқсас)
- Кірістірілген температура сенсоры бар 6 арналы DAC
- Бағдарламаланатын USART сериялық порты
- Сериялық интерфейс SPI
- I2C интерфейсі
- Бөлек ішкі осцилляторы бар бағдарламаланатын бақылау таймері
- Ішкі кернеуді салыстыру тізбегі
- Микроконтроллердің түйреуіштеріндегі кернеу өзгерген кезде ұзу және ояту өңдеу блогы

ATmega328P микроконтроллерінің сипаттамалары

Микроконтроллердің арнайы функциялары:

- Қосылған кезде ысыру және төмен кернеуді анықтау бағдарламалық құралы
- Ішкі калибрленетін сағат
- Ішкі және сыртқы үзулерді өңдеу
- 6 ұйқы режимі (дәлірек ADC түрлендіру үшін қуатты аз тұтыну және шуды азайту)

Жеткізу кернеулері және процессор жылдамдығы:

- 4 МГц жиілікте 1,8 - 5,5 В
- 10 МГц жиілікте 2,7 - 5,5 В
- 20 МГц жиілікте 4,5 - 5,5 В

Техникалық сипаттама

- Микроконтроллердің қоректендіру кернеуі 5 В
- Ұсынылатын борттық қоректендіру кернеуі 7 - 12 В
- Тақтаның максималды рұқсат етілген кернеуі 6 - 20 В
- Сандық енгізу/шығару 14 (оның ішінде 6 PWM қолдайды)
- PWM модуляция шығыстары 6
- Аналогтық кірістер 6
- Сандық шығыстардың рұқсат етілген тогы 20 мА
- Рұқсат етілген шығыс тогы 3,3 В 50 мА
- Флэш жады (FLASH) 32 кБ (оның 0,5 кБ жүктеуші пайдаланады)
- ЖЖҚ көлемі (SRAM) 2 кБ
- Тұрақты жад көлемі (EEPROM) 1 кБ
- Сағат жиілігі 16 МГц

Бағдарламалау

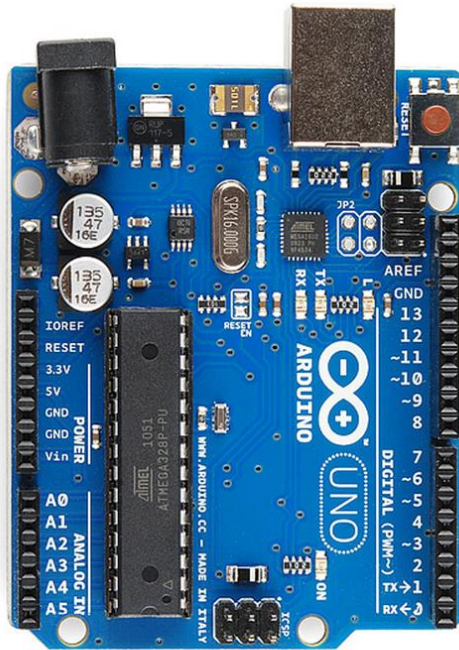
Контроллер Arduino Integrated Software Environment (IDE) арқылы бағдарламаланған. Бағдарламалау STK500 хаттамасы арқылы резидент жүктеушінің бақылауымен жүзеге асады. Аппараттық бағдарламашы қажет емес.

Микроконтроллерді ICSP ұясы арқылы жүктеушіні пайдаланбай бағдарламалауға болады. Жүктеп алушының бастапқы коды еркін қол жетімді.

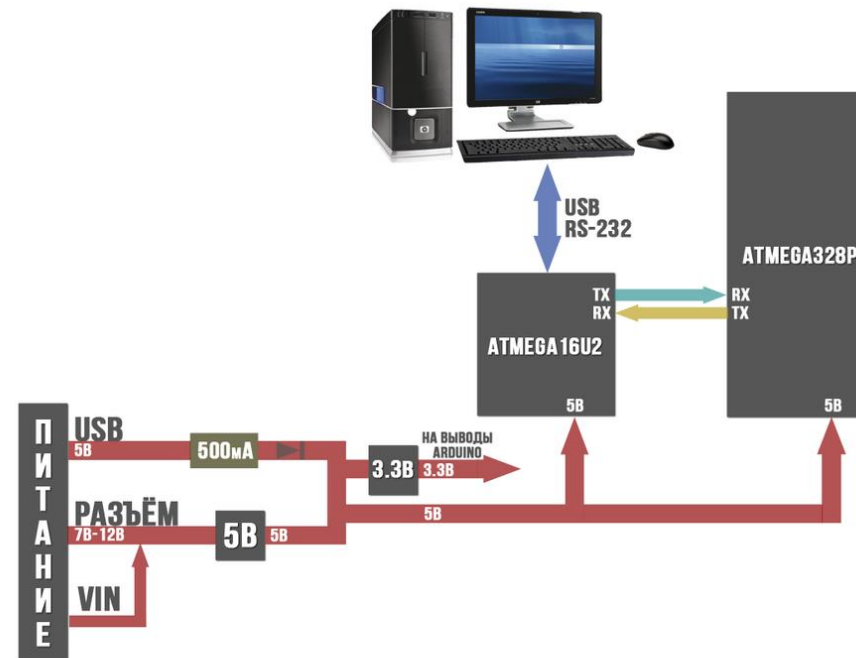
Басқа Arduino контроллерлерінен айырмашылығы

Arduino UNO R3, алдыңғы нұсқалардан айырмашылығы, компьютерге қосылу үшін USB-UART FTDI көпірін пайдаланбайды. Бұл функцияны ATmega16U2 микроконтроллері орындайды.

ARDUINO UNO R3



УПРОЩЕННАЯ ФУНКЦИОНАЛЬНАЯ СХЕМА



Қуат жүйесі

UNO тақтасын USB портынан немесе сыртқы көзден қуаттандыруға болады. Қуат көзі автоматты түрде таңдалады. Айнымалы ток адаптерін немесе батареяны сыртқы қуат көзі ретінде пайдалануға болады. Адаптер 2,1 мм қосқыш арқылы қосылған (орталық түйреуіш оң). Батарея POWER қосқышының GND және Vin түйреуіштеріне қосылған.

Сыртқы қуат көзінің кернеуі 6 - 20 В диапазонында болуы мүмкін. Бірақ құрылғының тұрақсыз жұмысына байланысты кернеудің 7 В-тан төмен түсуіне жол бермеу ұсынылады. Сондай-ақ қоректену кернеуін 12 В-тан жоғары арттыру қажет емес, өйткені тұрақтандырғыш қызып кетуі және істен шығуы мүмкін. Яғни, ұсынылатын қоректендіру кернеуінің диапазоны 7 - 12 В.

Қуат жүйесі

Қуатты қосу үшін келесі түйреуіштерді пайдалануға болады.

Vin	Тақта сыртқы қуат көзінен қуат алады. 5V USB қуатына немесе басқа тұрақтандырғыш шығыстарына қатысты емес. Тақта адаптерден қуат алатын болса, осы контакт арқылы құрылғыңызға қуат алуға болады.
5 V	Тақта кернеу реттегішінің шығысы. Оның кез келген қуат көзімен 5 В кернеуі бар. Тақтаны осы шығыс арқылы қуаттандыру ұсынылмайды, өйткені тұрақтандырғыш пайдаланылмайды, бұл микроконтроллердің істен шығуына әкелуі мүмкін.
3 V 3	Тақтадағы кернеу реттегішінен кернеу 3,3 В. Осы шығыстан максималды рұқсат етілген ток тұтынуы 50 мА құрайды.
GND	Жалпы сым.
IOREF	Тақтаның жұмыс кернеуі туралы шығыс ақпаратында. Кеңейту тақтасы сигнал мәнін оқи алады және 5В немесе 3,3В қуат беру режиміне ауыса алады.

Кіріс және шығыс

14 сандық түйреуіштердің әрқайсысын шығыс немесе кіріс ретінде пайдалануға болады. Шығулардағы кернеу деңгейі 5 В. Әрбір шығыстың шығыс және кіріс тогын 20 мА деңгейінде шектеу ұсынылады. Бұл параметр үшін максималды рұқсат етілген мән - 40 мА. Әрбір түйреуіште ішкі 20-50 кОм тартылатын резистор бар. Резисторды бағдарламалық құрал арқылы өшіруге болады.

Кіріс және шығыс

Кейбір түйреуіштер қосымша функцияларды орындауы мүмкін.

Сериялық интерфейс: түйреуіштер 0 (Rx) және 1 (Tx). TTL логикалық деңгейлерінің дәйекті деректерін қабылдау (Rx) және беру (Tx) үшін пайдаланылады. Бұл қорытындылар USB-UART көпірі ретінде пайдаланылатын ATmega16U2 микросхемасының деректерін беру қорытындыларына қосылған.

- **Сыртқы үзілістер 2 және 3-қорытындылар.** Бұл түйреуіштерді сыртқы ұзу кірістері ретінде пайдалануға болады. Бағдарламалық құрал арқылы төмен деңгейде, көтерілу немесе төмендеу жиегінде ұзу немесе сигнал деңгейін өзгерту үшін орнатуға болады.
- **PWM: түйреуіштер 3, 5, 6, 9, 10, 11.** 8 разрядты рұқсатпен PWM модуляция режимінде жұмыс істей алады..

Кіріс және шығыс

- **Сериялық интерфейс SPI:** түйреуіштер 10(SS), 11(MOSI), 12(MISO), 13(SCK).
- **Жарық диодты:** 13 түйреуіш. Шығаруға қосылған жарықдиод 13. Шығуда сигнал деңгейі жоғары болғанда жарқырайды.
- **I2C интерфейсі:** шығыс A4 немесе SDA және A5 немесе SCL. I2C коммуникациялық интерфейс.
- **Arduino UNO** тақтасында **A0-A5** деп белгіленген **6 аналогтық кіріс** бар. Аналогты цифрлық түрлендірудің рұқсаты 10 бит. Әдепкі бойынша кіріс кернеуі 0-5В диапазонында жерге сілтеме бойынша өлшенеді, бірақ AREF істікшесі мен бағдарламалық құрал параметрлерін пайдаланып өзгертуге болады.

Кіріс және шығыс

Тақтаның тағы 2 түйреуіштері келесі функцияларға ие:

- **AREF.** Микроконтроллердің ADC эталондық кернеуі.
- **RESET.** Бұл түйреуіштегі төмен деңгей микроконтроллердің бастапқы қалпына келуіне әкеледі.

Коммуникациялық интерфейстер

Arduino UNO модулінде компьютермен, басқа UNO тақтасымен немесе басқа микроконтроллерлермен байланысу құралдары бар. Ол үшін платада 0 (RX) және 1 (TX) түйреуіштерге қосылған TTL (5 В) логикалық деңгейлері бар UART интерфейсі бар. Тақтадағы ATmega16U2 чипі UART интерфейсін компьютердің USB портына қосады. Компьютер портына қосылған кезде компьютерлік бағдарламалар Arduino-мен жұмыс істейтін виртуалды COM порты пайда болады. ATmega16U2 микробағдарламасы стандартты USB-COM драйверлерін пайдаланады және қосымша драйверді орнату қажет емес.

Коммуникациялық интерфейстер

Windows амалдық жүйесі сәйкес .inf файлын қажет етеді. Arduino Integrated Software Environment (IDE) тақтадан қарапайым мәтіндік деректерді жіберуге және алуға мүмкіндік беретін сериялық байланыс мониторын қамтиды. Тақтада USB байланысы үшін сәйкес сигналдардың күйін көрсететін RX және TX жарық диодтары бар (бірақ 0 және 1 түйреуіштеріндегі сериялық интерфейс үшін емес).

ATmega328 микроконтроллері I2C (TWI) және SPI байланыс интерфейстерін де қолдайды.

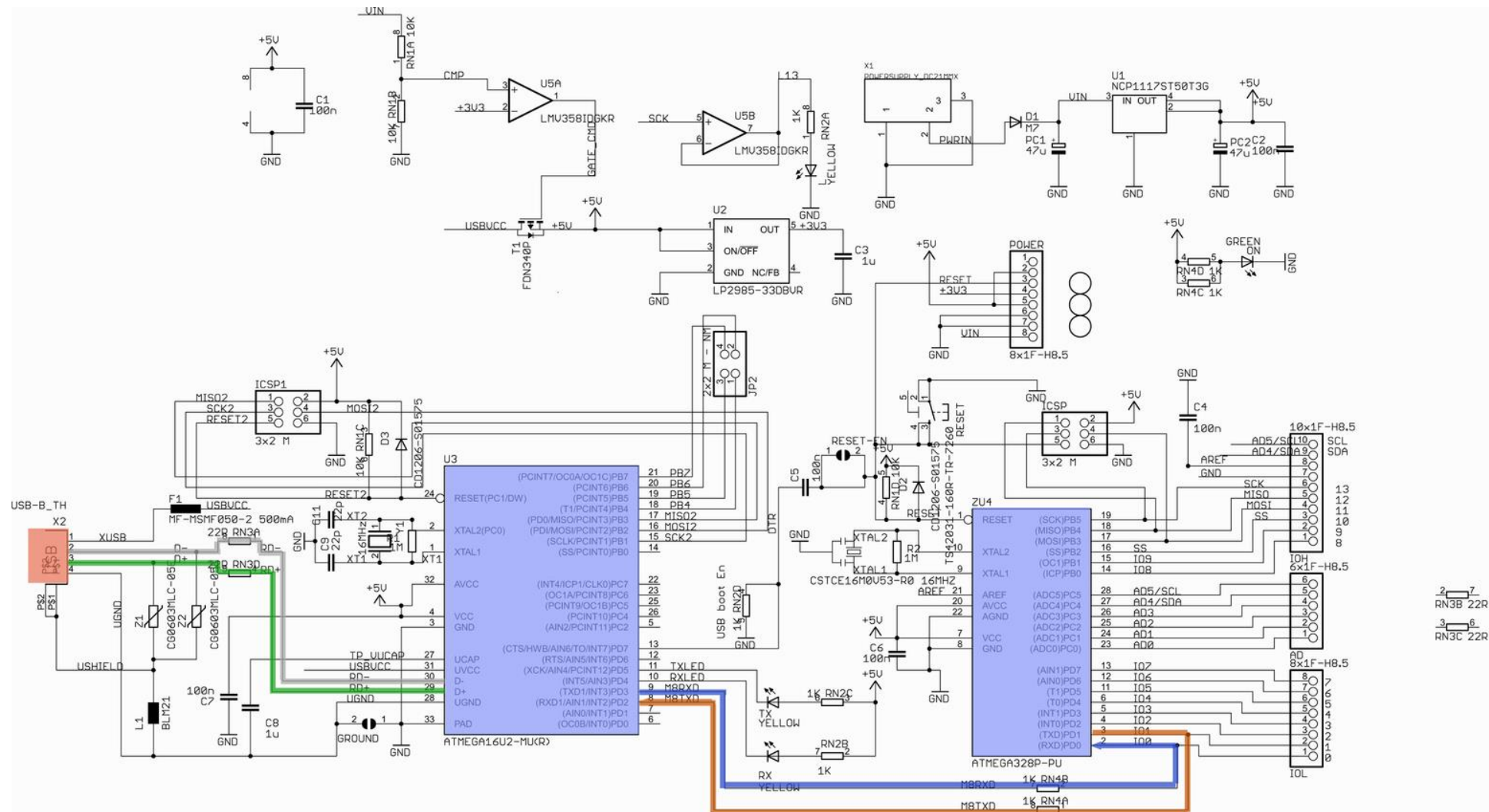
Автоматты (бағдарламалық құрал) қалпына келтіру

Бағдарламаны жүктемес бұрын бастапқы қалпына келтіру түймесін әр уақытта басу қажет болмас үшін UNO тақтасында қосылған компьютерден іске қосылған аппараттық құралдарды қалпына келтіру функциясы бар. ATmega16U2 деректер ағынын басқару (DTR) сигналдарының бірі 0,1 мкФ конденсатор арқылы ATmega328 қалпына келтіру істікшесіне қосылған. DTR сигналы төмендеген кезде микроконтроллерді қалпына келтіру импульсі жасалады. Бұл шешім Arduino Integrated Programming Environment (IDE) жүйесінен бір түймені басу арқылы бағдарламаны жүктеп алуға мүмкіндік береді.

Автоматты (бағдарламалық құрал) қалпына келтіру

Бірақ мұндай функция теріс салдарға әкелуі мүмкін. UNO тақтасын Mac Os X немесе Linux жұмыс істейтін компьютерге қосқанда, бағдарлама тақтаға қосылған сайын микроконтроллер бастапқы күйін қалпына келтіреді. Жарты секунд ішінде жүктеуші UNO тақтасында іске қосылады. Жүктеп алу бағдарламасы бөгде деректерді елемесе де, қосылым орнатылғаннан кейін бірден пакеттен бірнеше байтты алуы мүмкін. Егер Arduino тақтасындағы бағдарлама бірінші іске қосу кезінде кез келген деректерді алуды қамтамасыз етсе, қосылымнан кейін шамамен 1 секунд кідіріспен деректерді жіберу керек.

Arduino UNO контроллерінің схемасы



Цифрлық шығулар

Arduino платформасының түйреуіштері кіріс немесе шығыс ретінде жұмыс істей алады. Сондай-ақ, Arduino аналогтық кірістерінің көпшілігі (Atmega) сандық енгізу/шығару порттары сияқты конфигурациялануы және жұмыс істеуі мүмкін екенін ескеріңіз.

Кіріс порты ретінде конфигурацияланған енгізу/шығару портының (пин) қасиеттері

Arduino (Atmega) түйреуіштері әдепкі бойынша кіріс порттары ретінде конфигурацияланады, сондықтан `pinMode()` функциясында ешқандай айқын мәлімдеме қажет емес. Бұл кіріс порты өзі қосылған тізбекке тым аз жүктемені білдіреді. Эквивалентті ішкі кедергі микросұлбаның шығысына қосылған 100 MΩ резистор болып табылады. Осылайша, кіріс портын бір күйден екінші күйге ауыстыру үшін аз мөлшерде ток қажет. Бұл сыйымдылық сенсорлық сенсорды, фотодиодты, RC тізбегіне ұқсас схемасы бар аналогтық сенсорды қосу үшін микросұлбаның шығыстарын пайдалануға мүмкіндік береді.

Екінші жағынан, егер бұл түйреуішке ештеңе қосылмаған болса, онда ондағы мәндер электр шуы немесе көрші істікшемен сыйымдылық өзара байланысы арқылы туындаған кездейсоқ шамаларды қабылдайды.

Тартылатын (жүктемелі) резисторлар

Егер кіріс порты сигнал қабылдамаса, онда бұл жағдайда портты белгілі күйге қою ұсынылады. Бұл кірісті +5 В (тартқыш резисторлар) немесе жерге (жерге тартылатын резисторлар) қосу арқылы 10 кОм тартылатын резисторларды қосу арқылы жүзеге асырылады.

Atmega микроконтроллерінде бағдарламаланатын кірістірілген 20 кОм тартылатын резисторлар бар. Бұл резисторларды бағдарламалау келесідей жүзеге асырылады.

```
pinMode(pin, INPUT);      // Шығаруға енгізу портын тағайындау  
digitalWrite(pin, HIGH);   // тартқыш резисторды қосу
```

Тартылатын (жүктемелі) резисторлар

Тартқыш резистор енгізу порты ретінде жұмыс істейтін шығуға қосылған жарық диодының сәл жарқырауы үшін жеткілікті тоқты өткізеді. Сондай-ақ, жарық диодтарының жеңіл жарықтандырылуы бағдарламалау кезінде шығарылым `pinMode()` функциясына шығару порты ретінде теңшелмегенін білдіреді.

Тартқыш резисторлар сол регистрлермен (микроконтроллер жадының ішкі мекенжайлары) басқарылады, ол HIGH немесе LOW шығарудың жай-күйін басқарады. Сондықтан, егер түйреуіш HIGH мәні бар кіріс порты ретінде әрекет етсе, бұл тартылатын резистордың қосулы екенін білдіреді, онда чиптің сол істікшесінде шығыс портының `pinMode()` функциясын конфигурациялау мәнді өткізеді HIGH. Бұл процедура да қарама-қарсы бағытта жұмыс істейді, яғни. егер пин HIGH болса, чиптің істікшесін `pinMode()` көмегімен кіріс порты ретінде конфигурациялау тартылатын резисторды қосады.

Тартылатын (жүктемелі) резисторлар

Ескертпе: 13-ші чипті кіріс порты ретінде пайдалану жарық диоды мен оған қосылған резисторға байланысты қиын. Қуат көзіне 20 кОм тартылатын резисторды қосқанда, кіріс 5 В орнына 1,7 В болады, себебі жарық диоды мен тізбектей жалғанған резисторда кернеудің төмендеуі байқалады. 13-ші чипті сандық кіріс порты ретінде пайдаланғыңыз келсе, түйреуіш пен жерге сыртқы тартылатын резисторды қосу керек.

Шығыс порты ретінде конфигурацияланған енгізу/шығару портының сипаттары

Шығыс порттары ретінде конфигурацияланған түйреуіштер төмен кедергі күйінде. Бұл түйреуіштер өздері арқылы айтарлықтай үлкен ток өткізе алады. tmega микросхемасының қорытындылары басқа құрылғылар үшін 40 мА дейінгі токтың көзі (оң) немесе қабылдағышы (теріс) болуы мүмкін. Бұл ток мәні жарық диодты (сериялық резистор қажет), сенсорларды қосу үшін жеткілікті, бірақ көптеген реле, электромагниттік және қозғалтқыштар үшін жеткіліксіз.

Шығыс порты ретінде конфигурацияланған енгізу/шығару портының сипаттары

Arduino шығарылымдарының қысқа тұйықталуы немесе энергия қажет ететін құрылғыларды қосу әрекеті шығу транзисторларына немесе бүкіл Atmega микроконтролдеріне зиян келтіруі мүмкін. Көп жағдайда бұл әрекеттер микроконтроллерде шығаруды өшіруге әкеледі, бірақ схеманың қалған бөлігі бағдарламаға сәйкес жұмыс істейтін болады. Егер құрылғыға жұмыс істеу үшін үлкен ток қажет болмаса, платформаның шығу жолдарына құрылғыларды 470 Ом немесе 1 кОм резисторлар арқылы қосу ұсынылады.

Аналогты сандық түрлендіргіш (ADC)

Arduino-да қолданылатын Atmega микроконтроллерлерінде алты арналы аналогты-сандық түрлендіргіш (ADC) бар. Түрлендіргіштің рұқсаты 10 бит, бұл шығыс 0-ден 1023-ке дейінгі мәндерді қабылдауға мүмкіндік береді. Көптеген Arduino платформаларының аналогтық кірістерінің негізгі қолданылуы аналогтық сенсорларды оқу болып табылады, бірақ сонымен бірге оларда жалпы мақсаттағы кіріс/шығыс (GPIO) функционалдығы (0 - 13 сандық енгізу/шығару порттары сияқты).

Осылайша, қосымша енгізу/шығару порттары қажет болса, пайдаланылмаған аналогтық кірістерді конфигурациялауға болады.

Цоколевка

Аналогтық кірістерге сәйкес Arduino түйреуіштері 14-тен 19-ға дейін нөмірленген. Бұл Atmega микроконтроллерінің физикалық істік нөмірлеріне емес, тек Arduino істіктеріне қатысты. Аналогтық кірістерді енгізу/шығару порттарының сандық шығыстары ретінде пайдалануға болады. Мысалы, HIGH мәні бар шығыс портына аналогтық кірістің 0 PIN орнату бағдарламасының коды:

```
pinMode(14, OUTPUT);  
digitalWrite(14, HIGH);
```

Тартылатын резисторлар

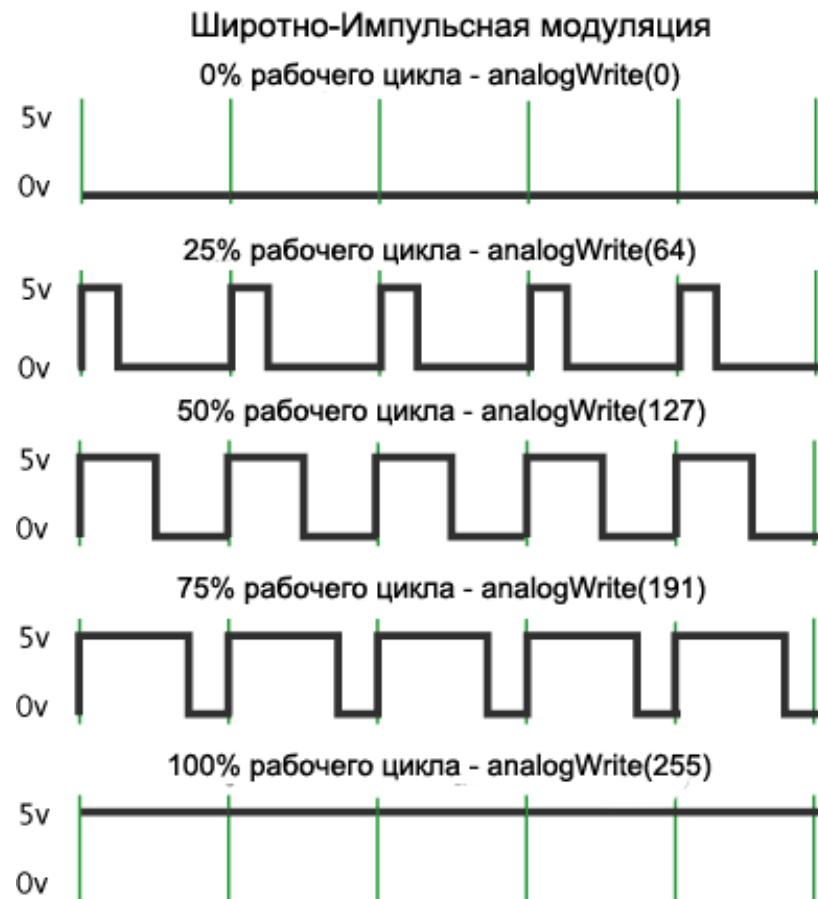
Аналогтық кіріс шығыстарында сандық шығыстар сияқты жұмыс істейтін тартылатын резисторлар бар. Резисторлар командамен қосылады `digitalWrite(14, HIGH);` // 0 аналогтық кіріс істікшесіне резисторды қосыңыз, ал істік кіріс порты ретінде әрекет етеді.

Резисторды қосу кейбір сенсорларды пайдаланған кезде `analogRead()` функциясы хабарлаған мәнге әсер етеді. Көптеген пайдаланушылар сандық режимде аналогтық кіріс пинін қолданғанда тартылатын резисторды пайдаланады.

Импульстік ені модуляциясы (ағыл. PWM)

Импульстік ені модуляциясы немесе PWM – сандық құрылғылар арқылы өзгермелі аналогтық мәнді алу операциясы. Құрылғылар тікбұрышты импульстарды қабылдау үшін қолданылады - максималды және ең төменгі мәндер арасында үнемі ауысатын сигнал. Бұл сигнал 0 В қосуға қатысты 5 В қосу уақытының ұзақтығын өзгерте отырып, максималды мән (5 В) мен ең төменгі мән (0 В) арасындағы кернеуді модельдейді. Бұрылыс ұзақтығы - ең үлкен мән импульс ені деп аталады. Әртүрлі аналогтық мәндерді алу үшін импульстің ені өзгертіледі. Қосу-өшіру кезеңдерін жеткілікті жылдам өзгерту арқылы жарық диодыға 0 және 5 В арасындағы тұрақты сигналды қолдануға болады, осылайша оның жарқырауының жарықтығын басқара аласыз.

Импульстік ені модуляциясы (ағыл. PWM)



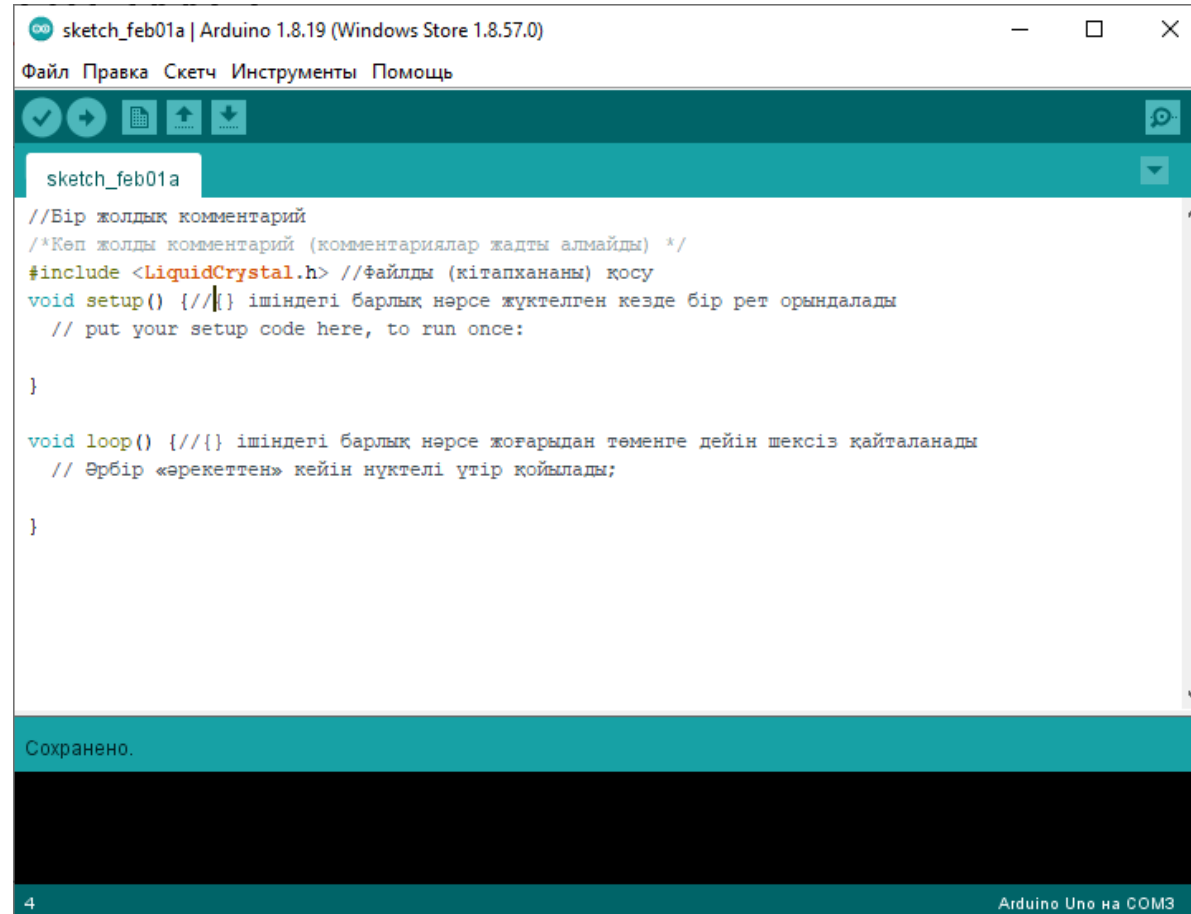
Дәріс 3

Деректер және айнымалылар

Бақылау мәлімдемелері



Бағдарлама құрылымы



```
sketch_feb01a | Arduino 1.8.19 (Windows Store 1.8.57.0)
Файл Правка Скетч Инструменты Помощь

sketch_feb01a

//Бір жолдық комментарий
/*Көп жолдық комментарий (комментариялар жақты алмайды) */
#include <LiquidCrystal.h> //файлды (кітапхананы) қосу
void setup() { //[] ішіндегі барлық нәрсе жүктелген кезде бір рет орындалады
    // put your setup code here, to run once:

}

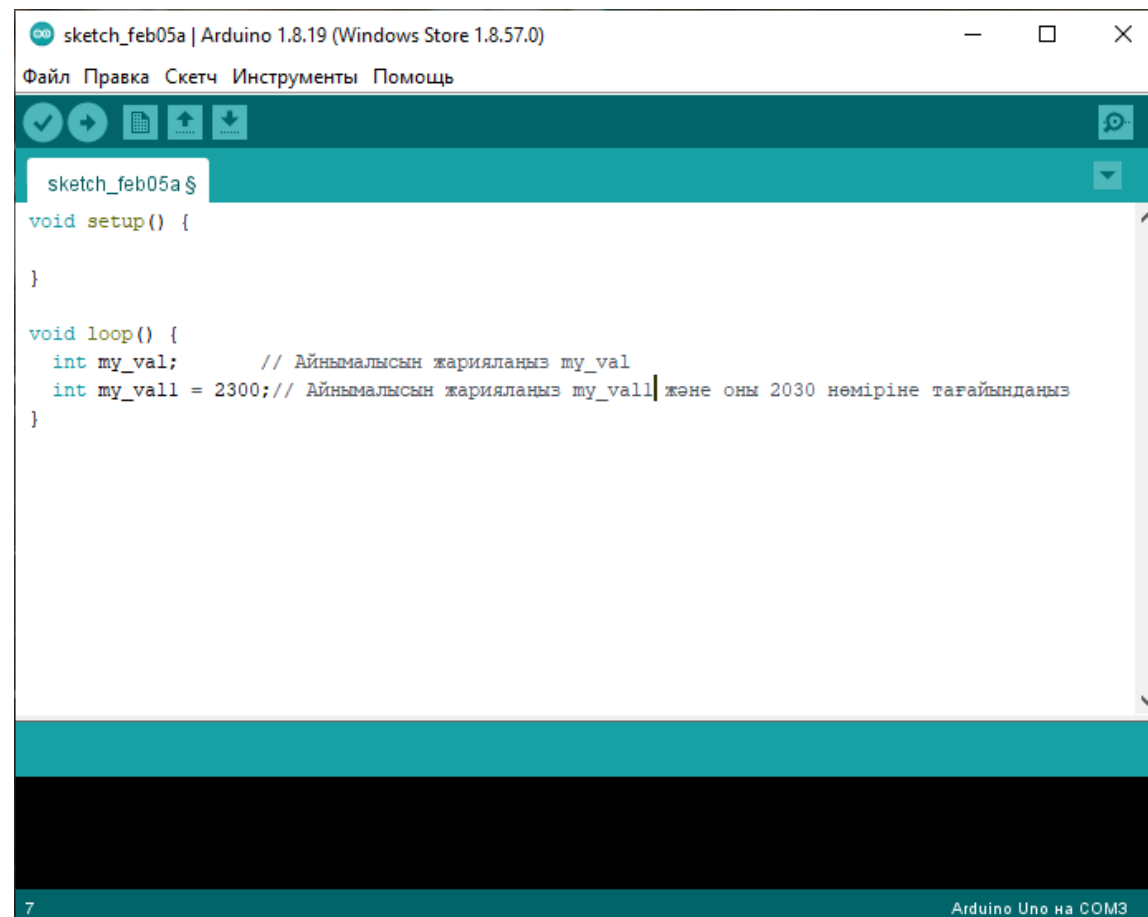
void loop() { //[] ішіндегі барлық нәрсе жоғарыдан төменге дейін шексіз қайталанады
    // Әрбір «әрекеттен» кейін нүктелі үтір қойылады;

}

Сохранено.

4 Arduino Uno на COM3
```


Деректер түрлері



```
sketch_feb05a $
void setup() {
}

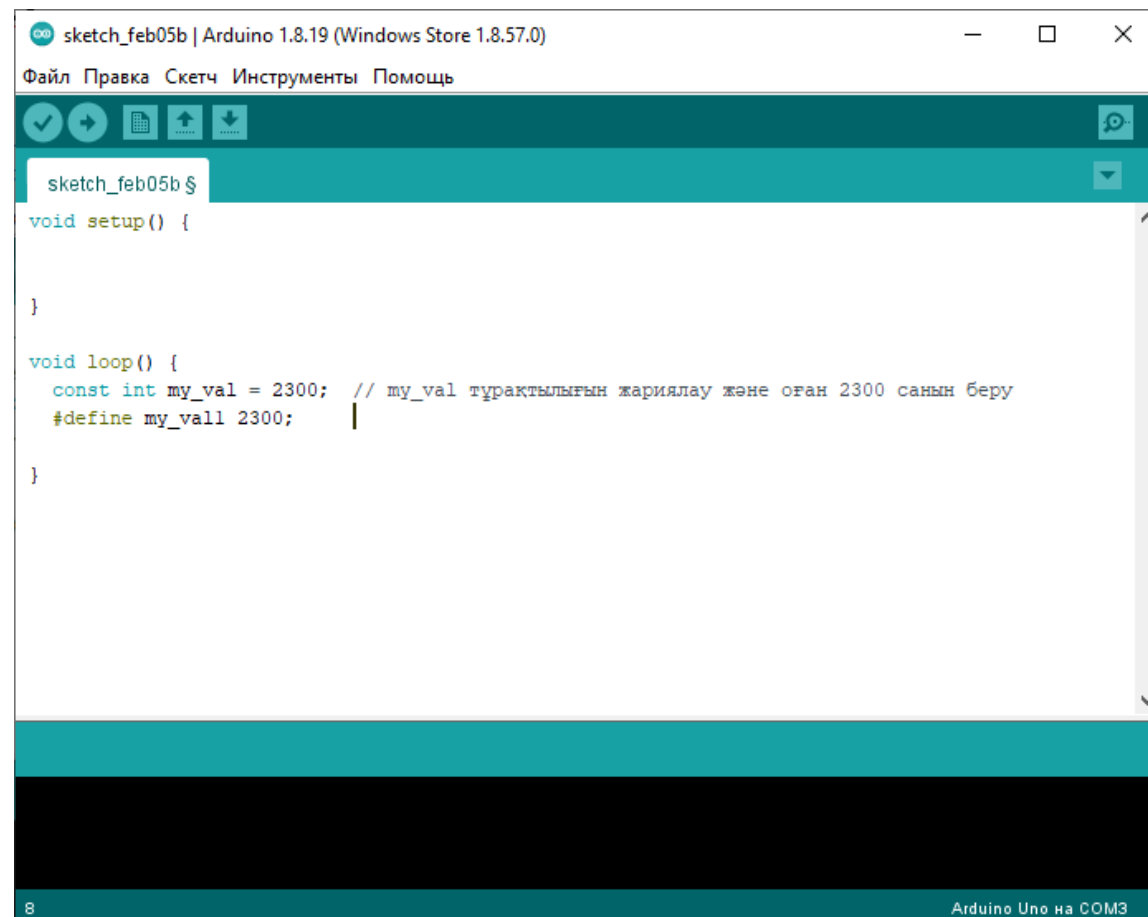
void loop() {
    int my_val;           // Айнымалысын жариялаңыз my_val
    int my_val1 = 2300; // Айнымалысын жариялаңыз my_val1 және оны 2300 нөміріне тағайындаңыз
}
```

7 Arduino Uno на COM3

Деректер түрлері

Атау	Салмақ	Ауқым	Ерекшілік
boolean	1 байт	0 или 1	Логикалық айнымалы, true (1) және false (0) мәндерін қабылдай алады.
char	1 байт	-128... 127	Таңба нөмірін ASCII кестесінен сақтайды
byte	1 байт	0... 255	
int	2 байта	-32 768... 32 767	
unsigned int	2 байта	0... 65 535	
word	2 байта	0... 65 535	unsigned int сияқты
long	4 байта	-2 147 483 648... 2 147 483 647	- 2 миллиард ... 2 миллиард
unsigned long	4 байта	0... 4 294 967 295	0... 4 миллиард ...
float	4 байта	-3.4028235E+38... 3.4028235E+38	Қалқымалы нүктесі бар сандар (ондық бөлшектер) сақталады. Дәлдігі: 6-7 таңба.
double	4 байта		float сияқты

Тұрақтылар



The screenshot shows the Arduino IDE window titled "sketch_feb05b | Arduino 1.8.19 (Windows Store 1.8.57.0)". The menu bar includes "Файл", "Правка", "Скетч", "Инструменты", and "Помощь". The toolbar contains icons for checking, running, serial monitor, upload, and download. The sketch editor shows the following code:

```
sketch_feb05b $  
void setup() {  
  
}  
  
void loop() {  
  const int my_val = 2300; // my_val тұрақтылығын жариялау және оған 2300 санын беру  
  #define my_val1 2300;    |  
  
}
```

The status bar at the bottom indicates "8" on the left and "Arduino Uno на COM3" on the right.

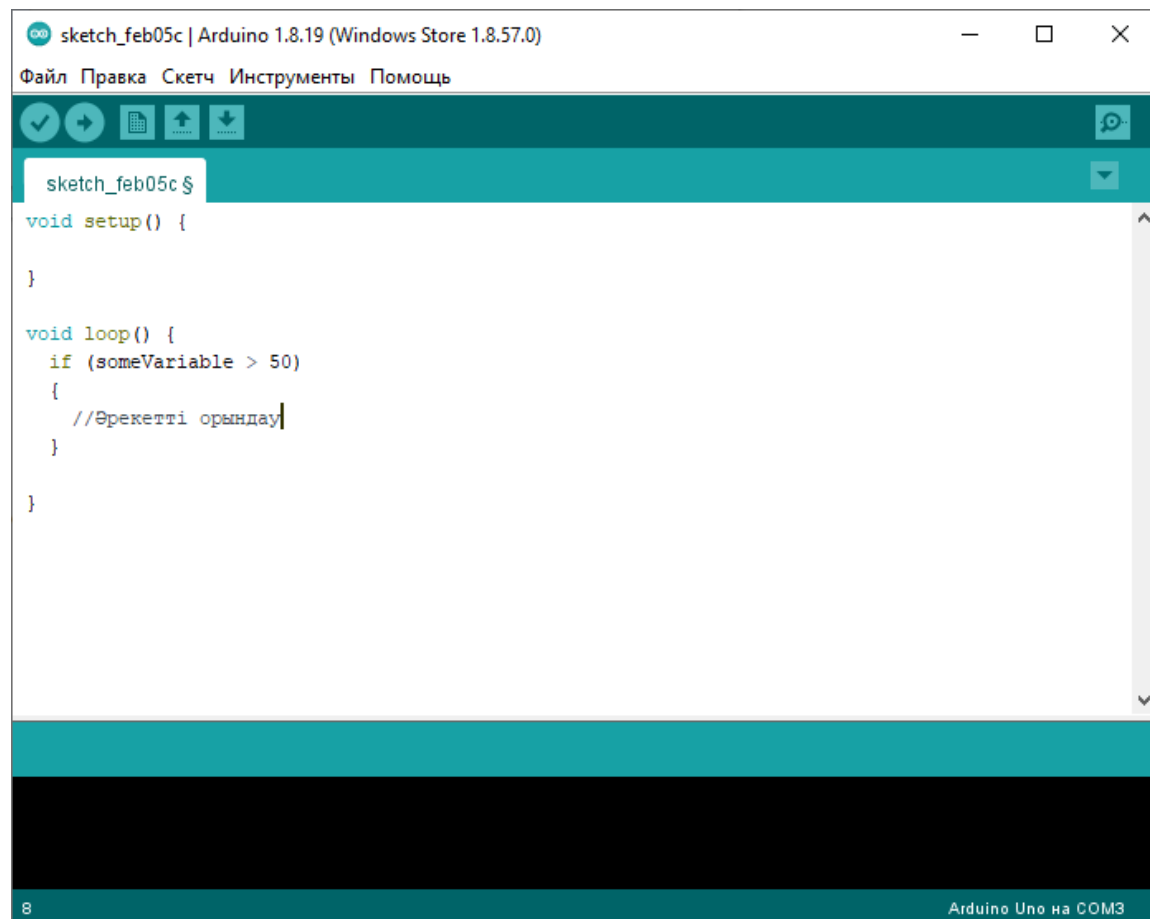
Математикалық операторлар

- **+, -, *, /** - бүктеу, шегеру, көбейту, бөлу
- **pow(x, a);** - «x» «a» (x^a) дәрежесіне шығару, pow бөлшек дәрежеге қоя алады! **sq (x);** - «x» санын квадратқа (x^2) шығару.
- **sqrt(x);** - x санының квадратың түбірін алу
- **abs(x);** - Санның модулін табыңыз, $|x|$
- **sin(x), cos(x), tan(x);** - синус, косинус, тангенс
- **round(x);** - Математикалық жуықтау (егер үтірден кейін 5-тен артық немесе тең болса, онда үлкен жаққа жуықтаймыз)
- **ceil(x);** - үлкен жаққа жуықтау
- **floor(x);** - кіші жаққа жуықтау

Математикалық операторлар

- **$x += a;$** // $x = x + a$
- **$x -= a;$** // $x = x - a$
- **$x *= a;$** // $x = x * a$
- **$x /= a;$** // $x = x / a$
- **$x++;$** - 1-ге « x » ұлғайту
- **$x--;$** - « x » 1-ге азайту

If операторы



The screenshot shows the Arduino IDE interface. The title bar reads "sketch_feb05c | Arduino 1.8.19 (Windows Store 1.8.57.0)". The menu bar includes "Файл", "Правка", "Скетч", "Инструменты", and "Помощь". The toolbar contains icons for checking, running, serial monitor, upload, and download. The sketch editor shows the following code:

```
sketch_feb05c $  
void setup() {  
  
}  
  
void loop() {  
  if (someVariable > 50)  
  {  
    //Өрекетті орындау  
  }  
}
```

The status bar at the bottom indicates "8" on the left and "Arduino Uno на COM3" on the right.

If операторы

```
if (x > 120) digitalWrite(LEDpin, HIGH);
```

```
if (x > 120)
```

```
digitalWrite(LEDpin, HIGH);
```

```
if (x > 120){ digitalWrite(LEDpin, HIGH); }
```

```
if (x > 120){
```

```
digitalWrite(LEDpin1, HIGH);
```

```
digitalWrite(LEDpin2, HIGH);
```

```
}
```

Салыстыру операциялары

$x == y$ ("x" "y"-ке тең)

$x != y$ ("x" "y"-ке тең емес)

$x < y$ ("x" "y"-ден аз)

$x > y$ ("x" "y"-ден үлкен)

$x <= y$ ("x" аз немесе тең "y")

$x >= y$ ("x" артық немесе тең "y")

If.. else операторы

```
if (pinFiveInput < 500)
```

```
{
```

```
    // А әрекеті
```

```
}
```

```
Else
```

```
{
```

```
    // В әрекеті
```

```
}
```

If.. else операторы

```
if (pinFiveInput < 50)
{
    // А әрекетін орындау
}
else if (pinFiveInput >= 1000)
{
    // В әрекетін орындау
}
else
{
    // С әрекетін орындау
}
```

Switch операторы (Мысал)

```
switch (var) {  
    case 1:  
        //VAR 1 болған кезде орындалады  
        break;  
    case 2:  
        //VAR 2 болған кезде орындалады  
        break;  
    default:  
        // егер бірде-бір балама таңдалмаса, орындалады  
        // default міндетті емес  
}
```

Синтаксис

```
switch (var) {  
  case label:  
    // Орындау үшін код  
    break;  
  case label:  
    // Орындау үшін код  
    break;  
  default:  
    // Орындау үшін код  
}
```

For операторы

for (initialization; condition; increment) {циклде орындалатын операторлар}

Мысал

```
// PWM шығысын пайдаланып, жарық диодтарын күңгірт
int PWMpin = 10; // 470 м резисторы бар 10 шығаруға арналған жарықдиод
void setup()
{
  // Орнату қажет емес
}
void loop()
{
  for (int i=0; i <= 255; i++){
    analogWrite(PWMpin, i);
    delay(10);
  }
}
```

Мысал

```
for(int x = 2; x < 100; x = x * 1.5){  
    println(x);  
}
```

Мысал

```
void loop()
{
    int x = 1;
    for (int i = 0; i > -1; i = i + x){
        analogWrite(PWMPin, i);
        if (i == 255) x = -1;          // Максимумды басқаруды ауыстыру
        delay(10);
    }
}
```


while циклдері

```
while(өрнөк){  
    // оператор (лар)  
}
```

Мысал

```
var = 0;  
while(var < 200){  
    // бір нәрсені 200 рет қайталап орындау  
    var++;  
}
```

do ... while

do

{

 // операторлардың реттілігі

} while (шартты тексеру);

Мысал

```
do
{
    delay(50);      // подождать, пока датчики стабилизируются
    x = readSensors(); // проверить датчики
} while (x < 100);
```

break

```
for (x = 0; x < 255; x ++)  
{  
    digitalWrite(PWMpin, x);  
    sens = analogRead(sensorPin);  
    if (sens > threshold){    // егер датчиктен сигнал бар болса, циклдан шығамыз  
        x = 0;  
        break;  
    }  
    delay(50);  
}
```

continue

```
for (x = 0; x < 255; x ++)  
{  
    if (x > 40 && x < 120){ // егер ақиқат болса, онда циклдің келесі  
                           //иттерациясына бірден секіреміз  
        continue;  
    }  
  
    digitalWrite(PWMpin, x);  
    delay(50);  
}
```

return

- *return;*
- *return мән; //* екі формасы да жарамды

Мысалдар :

```
int checkSensor(){  
    if (analogRead(0) > 400) {  
        return 1;  
    }  
    else{  
        return 0;  
    }  
}
```


Мысалдар :

```
void loop(){  
    // Міне, керемет кодты сынау идеясы  
    return;  
    // дұрыс жұмыс істемейтін нұсқаның қалған бөлігі осында  
    // бұл код ешқашан орындалмайды  
  
}
```

goto операторы

label:

//

// Кез келген код

//

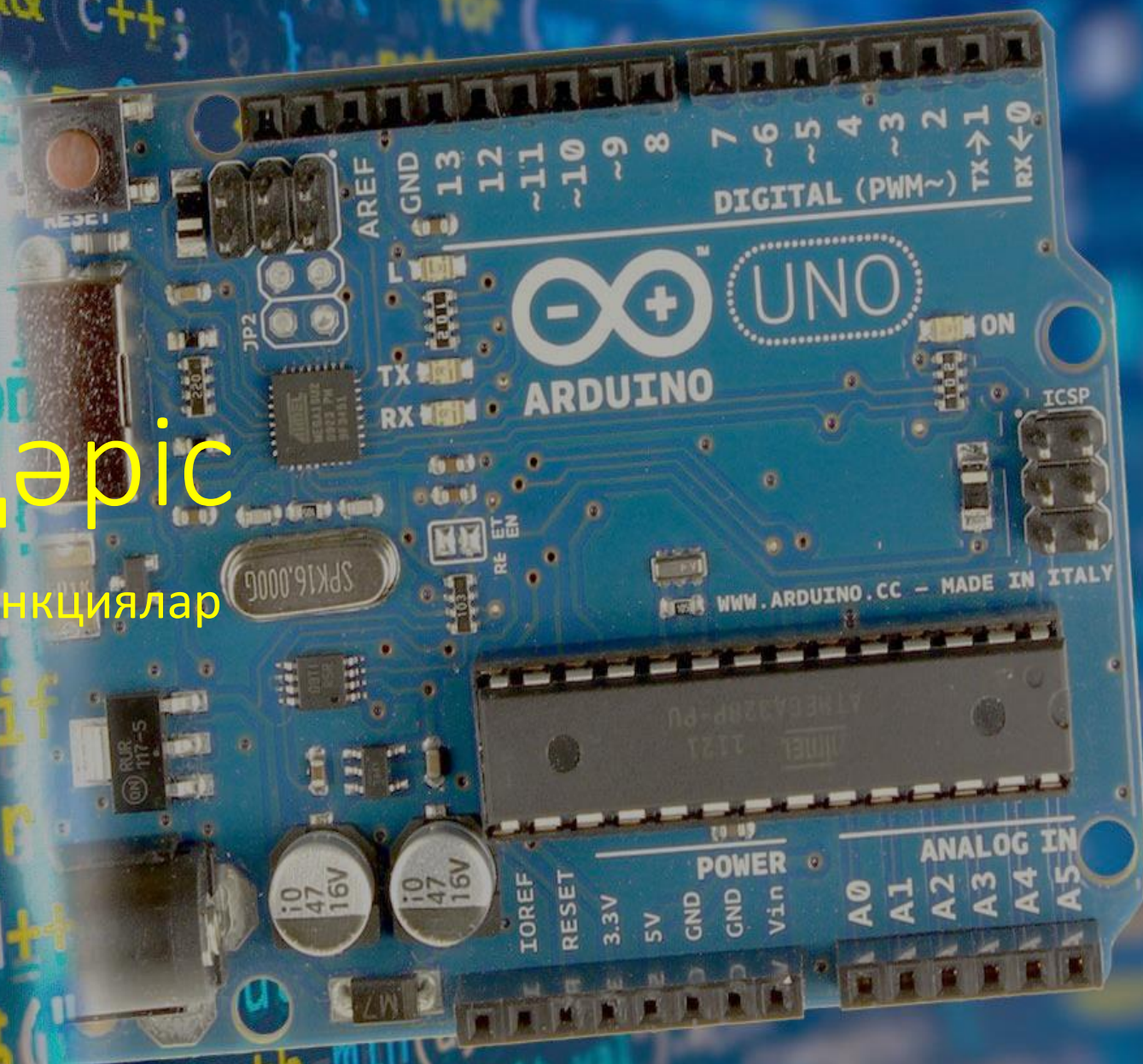
goto label; // label белгісіне өту

Мысал

```
for(byte r = 0; r < 255; r++){  
    for(byte g = 255; g > -1; g--){  
        for(byte b = 0; b < 255; b++){  
            if (analogRead(0) > 250){ goto bailout;}  
            // еще код  
        }  
    }  
}  
bailout:
```

Дәріс

Функциялар



Мысал

Функцияның синтаксисі

Егер функция ештеңе
қайтармаса, қайтарылатын
«void» мәнінің түрі

Функцияға
берілетін
параметрлер

Функция атауы

```
int myMultiplyFunction(int x, int y){  
    int result;  
    result = x * y;  
    return result;  
}
```

Тиісті түрдің мәнін
қайтаратын
оператор

Фигуралық жақшалар
міндетті

The diagram illustrates the syntax of a function in C. It shows a code snippet for a function named 'myMultiplyFunction' that takes two integer parameters 'x' and 'y' and returns an integer 'result'. Blue arrows point from descriptive labels to specific parts of the code: 'Егер функция ештеңе қайтармаса, қайтарылатын «void» мәнінің түрі' points to the 'int' return type; 'Функция атауы' points to 'myMultiplyFunction'; 'Функцияға берілетін параметрлер' points to the parameters '(int x, int y)'; 'Тиісті түрдің мәнін қайтаратын оператор' points to the 'return result;' statement; and 'Фигуралық жақшалар міндетті' points to the curly braces '{ }'.

Мысал

```
void loop(){
```

```
    int i = 2;
```

```
    int j = 3;
```

```
    int k;
```

```
    k = myMultiplyFunction(i, j); // k құрамында 6 бар
```

```
}
```

Мысал

```
void setup(){
  Serial.begin(9600);
}

void loop(){
  int i = 2;
  int j = 3;
  int k;
  k = myMultiplyFunction(i, j); // k құрамында 6 бар
  Serial.println(k);
  delay(500);
}

int myMultiplyFunction(int x, int y){
  int result;
  result = x * y;
  return result;
}
```

pinMode функциясы

pinMode(pin, mode)

- **pin**: Орнатқыңыз келетін кіру/шығу нөмірі (pin)
- **mode**: режим екі мәннің бірі - INPUT немесе OUTPUT, кіруге немесе шығуға тиісінше орнатады.

Мысал

```
int ledPin = 13;           // Жарықдиод қатынауға/шығуға қосылған 13
void setup()
{
  pinMode(ledPin, OUTPUT); // жұмыс режимін - шығуды белгілейді
}

void loop()
{
  digitalWrite(ledPin, HIGH); // жарық диодын қосады
  delay(1000);                // Бір секунд күту
  digitalWrite(ledPin, LOW);  // Жарық диодын өшіреді
  delay(1000);                // Бір секунд күту
}
```

digitalWrite() функциясы

digitalWrite(pin, value)

- **pin:** кіру/шығу нөмірі (pin)
- **value:** значение HIGH или LOW

Мысал

```
int ledPin = 13;           // Жарықдиод қатынауға/шығуға қосылған 13
void setup()
{
  pinMode(ledPin, OUTPUT); // жұмыс режимін - шығуды белгілейді
}

void loop()
{
  digitalWrite(ledPin, HIGH); // жарық диодын қосады
  delay(1000);                // Бір секунд күту
  digitalWrite(ledPin, LOW);  // Жарық диодын өшіреді
  delay(1000);                // Бір секунд күту
}
```

digitalRead() функциясы

digitalRead(pin)

- **pin:** Сіз есептегіңіз келетін кіру/шығу нөмірі (pin)

қайтарылатын мән

HIGH немесе LOW

Мысал

```
int ledPin = 13;           // Кіру/шығуға қосылған жарықдиод 13
int inPin = 7;             // «7» кіре берісіндегі батырма
int val = 0;               // мәнді сақтау үшін айнымалы
void setup()
{
    pinMode(ledPin, OUTPUT); // жұмыс режимін орнатады - 13-ші кіріс / шығыс (pin) үшін шығыс
    pinMode(inPin, INPUT);   // жұмыс режимін орнатады - 7-ші кіріс / шығыс үшін кіріс (pin)
}
void loop()
{
    val = digitalRead(inPin); // кірістен мәнді оқу
    digitalWrite(ledPin, val); // жарық диодындағы мәнді түйменің кіріс мәніне тең етіп орнатыңыз
}
```

analogRead() функциясы

analogRead(pin)

- **pin:** оқу орындалатын аналогтық кіріс портының нөмірі (көптеген тақталар үшін 0..5, Mini және Nano үшін 0..7 және Mega үшін 0..15)

Қайтарылатын мән

- int (0 to 1023)

Мысал

```
int analogPin = 3;    // потенциометр қосылған порт нөмірі
int val = 0;          // оқылатын мәнді сақтау үшін айнымалы
void setup()
{
  Serial.begin(9600);    // сериялық байланыс орнату
}
void loop()
{
  val = analogRead(analogPin); // мәнін оқыңыз
  Serial.println(val);        // алынған мәнді шығару
}
```

analogReference() функциясы

Мүмкін параметрлер:

- DEFAULT: стандартты эталондық кернеу 5 В (қоректену кернеуі 5 В платформаларда) немесе 3,3 В (қоректену кернеуі 3,3 В платформаларда)
- INTERNAL: ATmega168 және ATmega328 микроконтроллерлерінде кірістірілген эталондық кернеу 1,1 В және ATmega8 микроконтроллерлерінде 2,56 В.
- INTERNAL1V1: кірістірілген эталондық кернеу 1,1 В (Arduino Mega)
- INTERNAL2V56: кірістірілген эталондық кернеу 2.56 (Arduino Mega)
- EXTERNAL: шығаруға қосылған тірек кернеуінің сыртқы көзі AREF

analogReference() функциясы

analogReference(type)

type: қолданылатын анықтамалық кернеуді анықтайды (**DEFAULT**, **INTERNAL** немесе **EXTERNAL**).

analogWrite() функциясы

analogWrite(pin, value)

- **pin:** PWM сигнал беретін кіру/шығу порты.
- **value:** Жұмыс циклінің кезеңі - 0 (толық өшірулі) және 255 (әрдайым қосулы) арасындағы мән.

Мысал

```
int ledPin = 9;  // 9 шығысына қосылған жарық диоды
int analogPin = 3; // 3 шығысына қосылған потенциометр
int val = 0;     // мәнді сақтау үшін айнымалы
void setup()
{
    pinMode(ledPin, OUTPUT);  // шығыс портын орнату
}
void loop()
{
    val = analogRead(analogPin); // потенциометрге қосылған порттан мәнді оқыңыз
    analogWrite(ledPin, val / 4); // analogRead 0-ден 1023-ке дейінгі мәндерді қайтарады, analogWrite 0-ден
    255-ке дейінгі аралықта болуы керек
}
```

tone()

tone(pin, frequency)

tone(pin, frequency, duration)

- pin: сигнал шығарылатын кіру/шығу портының нөмірі
- frequency: Герцтегі сигнал жиілігі
- duration: миллисекундтағы сигнал ұзақтығы

shiftOut()

shiftOut(dataPin, clockPin, bitOrder, value)

- dataPin: Биттер шығарылатын кіріс/шығыс портының нөмірі (int).
- clockPin: Синхрондау жүргізілетін порттың нөмірі (int).
- bitOrder: пайдаланылған бит шығыс тізбегі.

MSBFIRST (Most Significant Bit First) – солға жақта немесе **LSBFIRST** (Least Significant Bit First) – оң жақта .

- value: Шығаруға арналған мән (байт) (byte)

Мысал

```
// Шығу бірінші (сол) биттен MSBFIRST болады.  
int data = 500;  
// жоғары байт басып шығару  
shiftOut(dataPin, clock, MSBFIRST, (data >> 8));  
// шығару төмен бит  
shiftOut(dataPin, clock, MSBFIRST, data);  
// соңғы биттен LSBFIRST шығару  
data = 500;  
// шығару төмен бит  
shiftOut(dataPin, clock, LSBFIRST, data);  
// жоғары битті шығарамыз  
shiftOut(dataPin, clock, LSBFIRST, (data >> 8));
```

Мысал

```
//ST_CP 74HC595 қосылған порт
int latchPin = 8;

//SH_CP 74HC595 портына қосылған
int clockPin = 12;

//DS 74HC595 портына қосылған
int dataPin = 11;

void setup() {
    //шығыс портының режимін орнатыңыз
    pinMode(latchPin, OUTPUT);
    pinMode(clockPin, OUTPUT);
    pinMode(dataPin, OUTPUT);
}

void loop() {
    for (int j = 0; j < 256; j++) {
        //Байтты тасымалдау аяқталғанша latchPin-де LOW параметрін орнатыңыз
        digitalWrite(latchPin, LOW);
        shiftOut(dataPin, clockPin, LSBFIRST, j);
        //тізілімге тасымалдаудың аяқталғанын хабарлау үшін latchPin-де HIGH мәнін орнатыңыз.
        digitalWrite(latchPin, HIGH);
        delay(1000);
    }
}
```

pulseIn()

pulseIn(pin, value)

pulseIn(pin, value, timeout)

- pin: сигнал күтілетін кіріс/шығыс портының нөмірі. (int)
- value: күтілетін сигнал түрі— **HIGH** немесе **LOW**.
- timeout (опционалды түрде): микросекундтарда сигнал күту уақыты (тайм-аут); әдепкі – бір секунд. (unsigned long)

Мысал

```
int pin = 7;  
unsigned long duration;  
void setup()  
{  
    pinMode(pin, INPUT);  
}  
void loop()  
{  
    duration = pulseIn(pin, HIGH);  
}
```

millis()

Arduino тақтасындағы ағымдағы бағдарлама орындала бастағандағы миллисекундтар санын қайтарады. Шамамен 50 күннен кейін мәннің асып кетуіне байланысты бұл сан нөлге қайтарылады.

Мысал

```
unsigned long time;
```

```
void setup(){
```

```
  Serial.begin(9600);
```

```
}
```

```
void loop(){
```

```
  Serial.print("Time: ");
```

```
  time = millis();
```

```
  //Бағдарламаны орындау басталған сәттен бастап миллисекунд санын көрсетеді
```

```
  Serial.println(time);
```

```
  // циклді келесі қайталаудың алдында секунд күтеді.
```

```
  delay(1000);
```

```
}
```

micros()

Arduino тақтасындағы ағымдағы бағдарлама орындала бастағаннан бергі микросекундтар санын қайтарады. Мән толып кетеді және шамамен 70 минуттан кейін нөлге қайта оралады. 16 МГц Arduino тақталарында (Duemilanove және Nano) micros() функциясы 4 микросекунд ажыратымдылығына ие (қайтару мәні әрқашан 4-ке еселік). 8 МГц тақталарында (Arduino Lilypad) функция рұқсаты 8 микросекундты құрайды.

Мысал

```
unsigned long time;
```

```
void setup(){
```

```
  Serial.begin(9600);
```

```
}
```

```
void loop(){
```

```
  Serial.print("Time: ");
```

```
  time = micros();
```

```
  //Бағдарламаны орындау сәтінен бастап микросекундтар санын көрсетеді
```

```
  Serial.println(time);
```

```
  // циклді келесі қайталаудың алдында секунд күтеді.
```

```
  delay(1000);
```

```
}
```

delay()

Параметрде берілген миллисекунд санына (1 секундта 1000 миллисекунд) бағдарламаны орындауды тоқтатады.

delay(ms)

ms: бағдарламаның орындалуы кідірітілген миллисекундтар саны.
(unsigned long)

Мысал

```
int ledPin = 13;          // Жарық диодты 13 портқа қосылған

void setup()
{
  pinMode(ledPin, OUTPUT); // порт режимі шығуға орнатылған
}

void loop()
{
  digitalWrite(ledPin, HIGH); // жарықдиодты қосамыз
  delay(1000);                // бір секунд күтеміз
  digitalWrite(ledPin, LOW);  // жарықдиодты өшіреміз
  delay(1000);                // бір секунд күтеміз
}
```

delayMicroseconds()

Параметрде көрсетілген микросекундтар саны үшін бағдарламаның орындалуын тоқтатады (1 секундта 1 000 000 микросекунд).

Arduino-ның бұл нұсқасында дұрыс ойнатуға болатын максималды үзіліс - 16383. Мүмкін бұл Arduino-ның болашақ нұсқаларында өзгертілуі мүмкін. Бағдарламаның орындалуын бірнеше мың микросекундтан астам уақытқа тоқтату үшін delay() функциясын пайдалану ұсынылады.

delayMicroseconds(us)

us: программаның орындалуы кідірітілген микросекундтар саны.
(unsigned int)

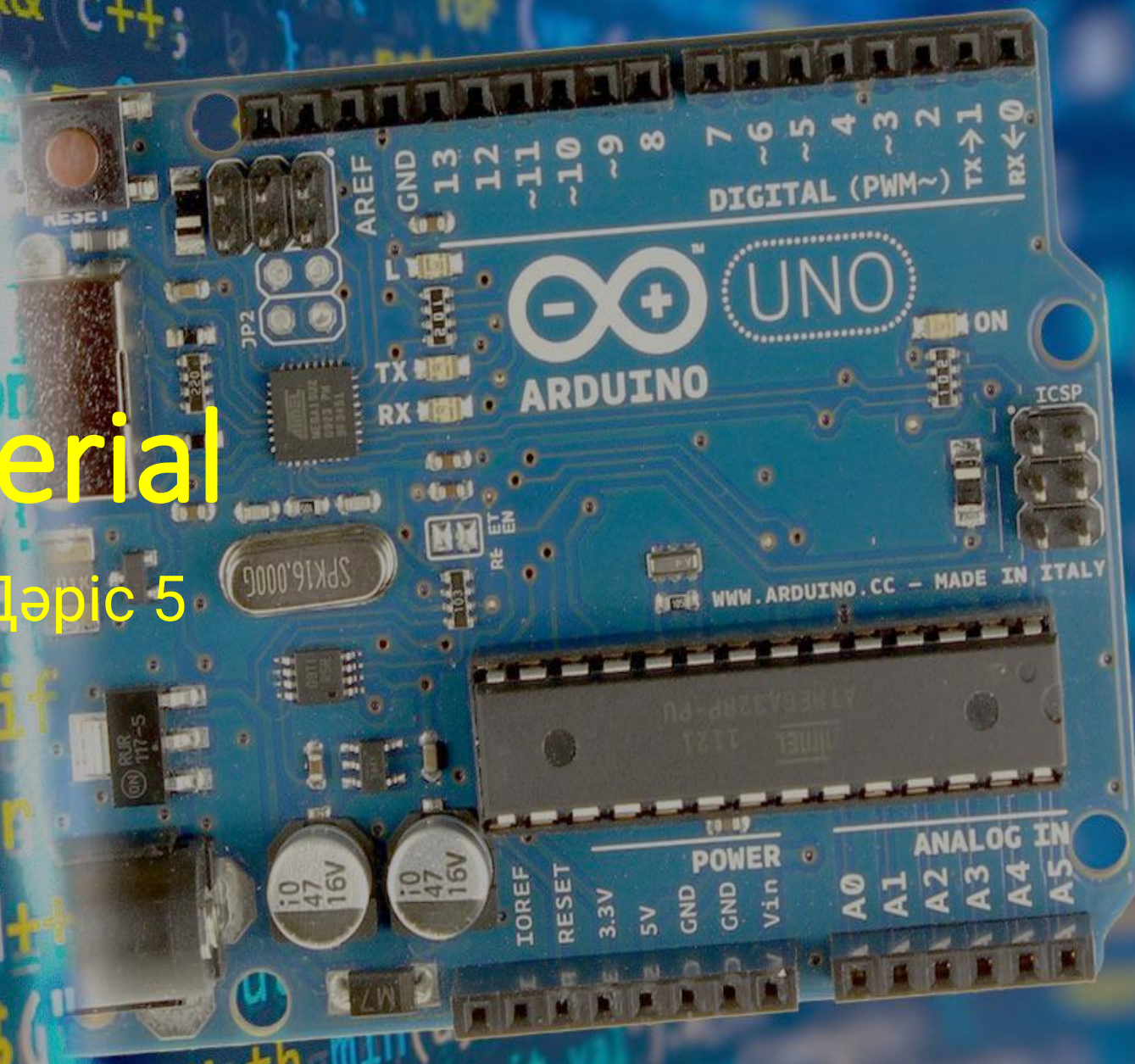
Мысал

```
int outPin = 8;           // сандық кіріс/шығыс порты 8
void setup()
{
  pinMode(outPin, OUTPUT); // порт режимі шығуға орнатылған
}

void loop()
{
  digitalWrite(outPin, HIGH); // шығуға HIGH береміз
  delayMicroseconds(50);      // 50 микросекунд күтеміз
  digitalWrite(outPin, LOW);  // Шығысқа «LOW» орнатамыз
  delayMicroseconds(50);      // 50 микросекунд күтеміз
}
```

Serial

Дәріс 5



Сериялық класс

Сериялық класс Arduino тақтасын компьютермен немесе басқа құрылғылармен байланыстыру үшін қолданылады. Барлық Arduino тақталарында кем дегенде бір сериялық порт бар (сонымен қатар UART немесе USART ретінде белгілі): Сериялық. Ол 0 (RX) және 1 (TX) сандық түйреуіштерге қосылған, сонымен қатар USB арқылы компьютермен байланысу үшін қолданылады. Осылайша, сериялық портты пайдалану кезінде 0 және 1 түйреуіштерді сандық кіріс немесе шығыс ретінде пайдалану мүмкін емес.

Сериялық класс

Arduino-мен байланысу үшін Arduino бағдарламалық құралына енгізілген арнайы сериялық порт бақылау бағдарламасын пайдалануға болады. Бағдарламаға қоңырау шалу үшін құралдар тақтасындағы сәйкес түймені басыңыз және `begin()` әдісін шақырған кезде бағдарламаңызда көрсетілгендей жіберу жылдамдығын орнатыңыз.

Arduino Mega қосымша үш сериялық портқа ие:

- 19 (RX) және 18 (TX) түйреуіштері бар **"Serial1"**,
- 17 (RX) және 16 (TX) түйреуіштері бар **"Serial2"**,
- 15 (RX) және 14 (TX) түйреуіштері бар **"Serial3"**.

Arduino Mega қосымша үш сериялық портқа ие:

Бұл түйреуіштер Mega тақтасындағы USB-UART түрлендіргішіне қосылмаған, сондықтан осы түйреуіштер арқылы компьютермен байланысты ұйымдастыру үшін қосымша сыртқы USB-UART түрлендіргіші қажет болады.

Сериялық TTL порты бар басқа сыртқы құрылғымен байланысу үшін тек үш түйреуішті жалғау жеткілікті: TX істікшелі құрылғының RX істікшесіне, RX істікшелі құрылғының TX істікшесіне және Mega жерлендіруге тиісінше , сыртқы құрылғының жерге.

Бұл түйреуіштерді RS232 сериялық портына тікелей қоспаңыз, себебі соңғысы +/- 12В кернеумен жұмыс істейді және Arduino тақтасын зақымдауы мүмкін.

Arduino Due TTL деңгейі бар үш қосымша тізбекті порты бар 3.3V

- 19 (RX) және 18 (TX) түйреуіштері бар **"Serial1"**,
- 17 (RX) және 16 (TX) түйреуіштері бар **"Serial2"**,
- 15 (RX) және 14 (TX) түйреуіштері бар **"Serial3"**.

Arduino Due TTL деңгейі бар үш қосымша тізбекті порты бар 3.3V

Выводы 0 и 1 по-прежнему соединены с соответствующими выводами преобразователя интерфейсов USB-Serial TTL, реализованным на микросхеме ATmega16U2, связанной с отладочным портом USB. Помимо этого, благодаря возможностям микросхем SAM3X, на плате реализована и аппаратная поддержка порта USB, SerialUSB.

Arduino Leonardo Serial1

Arduino Leonardo Serial1 TTL (5V) сериялық порты арқылы 0 (RX) және 1 (TX) түйреуіштері арқылы байланысу үшін пайдаланылады. "Serial" USB CDC байланысы үшін сақталған. Қосымша ақпарат алу үшін Леонардо тақтасының нұсқаулығын қараңыз.

Serial.begin()

Дәйекті қосылымды бастайды және деректерді бит/с (бод) жылдамдығын орнатады. Компьютермен деректер алмасу үшін мынадай мәндерді пайдаланыңыз: 300, 1200, 2400, 4800, 9600, 14400, 19200, 28800, 38400, 57600 немесе 115200. 0 және 1 кіру/шығу порттары арқылы жалғау кезінде құрылғы талап ететін деректермен алмасу жүзеге асырылатын жылдамдықтың басқа да мәндері пайдаланылуы мүмкін.

Serial.begin()

Синтаксис

Serial.begin(speed)

Сонымен қатар, тек **Arduino Mega** үшін:

- Serial1.begin(speed)
- Serial2.begin(speed)
- Serial3.begin(speed)

Параметрлер

speed: бит/с жылдамдығы (бод). (long)

Мәндері :

300, 1200, 2400, 4800, 9600, 14400, 19200, 28800, 38400, 57600 или 115200

Қайтарылатын мән

Жоқ

Мысал

```
void setup() {
```

```
    Serial.begin(9600); // Сериялық портты ашады, жылдамдықты  
    9600 бит/с орнатады}
```

```
void loop() {}
```

Барлық 4 сериялық порттарды пайдаланатын Arduino Mega мысалы

```
void setup(){  
  Serial.begin(9600);  
  Serial1.begin(38400);  
  Serial2.begin(19200);  
  Serial3.begin(4800);  
  Serial.println("Hello Computer");  
  Serial1.println("Hello Serial 1");  
  Serial2.println("Hello Serial 2");  
  Serial3.println("Hello Serial 3");  
}  
void loop() {}
```

Serial.end()

Сериялық қосылымды жабады, RX және TX порттары босатылады және енгізу/шығару үшін пайдаланылуы мүмкін. Сериялық қосылымды қалпына келтіру үшін Serial.begin() пайдаланыңыз.

Serial.end()

Синтаксис

Serial.end()

Сонымен қатар, тек Arduino Mega үшін:

Serial1.end()

Serial2.end()

Serial3.end()

Параметрлер

Жоқ

Қайтарылатын мән

Жоқ

Serial.available()

Функция сериялық байланыс интерфейсінен оқуға қолжетімді байттардың (таңбалар) санын алады. Бұл әлдеқашан келген және сериялық порт буферіне жазылған байттар. Буфер 64 байтқа дейін сақтай алады.

Serial.available()

Синтаксис

Serial.available()

Arduino Mega үшін :

Serial1.available()

Serial2.available()

Serial3.available()

Параметрлер

Жоқ

Қайтарылатын мән

Оқуға қолжетімді байттардың саны.(int)

Мысал

```
int incomingByte = 0; // алынған байтты сақтау үшін
void setup() {
  Serial.begin(9600); // сериялық қосылымды орнату
}
void loop() {
  if (Serial.available() > 0) { //қолжетімді деректер бар болса
    // байт оқимыз
    incomingByte = Serial.read();
    // алғанымызды жібереміз
    Serial.print("I received: ");
    Serial.println(incomingByte, DEC);
  }
}
```

Arduino Mega үшін мысал:

```
void setup() {  
    Serial.begin(9600);  
    Serial1.begin(9600);  
}  
  
void loop() {  
    // 0 портынан оқу және 1 порттан жіберу  
    if (Serial.available()) {  
        int inByte = Serial.read();  
        Serial1.print(inByte, BYTE);  
    }  
    // 1 порттан оқу, сериялық порт 0 арқылы жіберу  
    if (Serial1.available()) {  
        int inByte = Serial1.read();  
        Serial.print(inByte, BYTE);  
    }  
}
```

Serial.read()

Сериялық қосылым буферінен келесі қолжетімді байтты оқиды.

Синтаксис

Serial.read()

Arduino Mega үшін :

Serial1.read()

Serial2.read()

Serial3.read()

Параметрлер

Жоқ

Қайтарылатын мән

Келесі қолжетімді байт немесе жоқ болса -1 (int)

Мысал

```
int incomingByte = 0; // қабылданған байтты сақтауға арналған айнымалы
void setup() {
    Serial.begin(9600); // сериялық қосылымды орнату
}

void loop() {
    if (Serial.available() > 0) { // қолжетімді деректер бар болса
        // байт оқимыз
        incomingByte = Serial.read();
        // алғанымызды жібереміз
        Serial.print("I received: ");
        Serial.println(incomingByte, DEC);
    }
}
```

Serial.flush()

Шығыс деректерді тасымалдаудың аяқталуын күтеді (Arduino 1.0 нұсқасына дейін функция сериялық қосылым буферін тазартты).

Синтаксис

Serial.flush()

Для Arduino Mega:

Serial1.flush()

Serial2.flush()

Serial3.flush()

Параметрлер

Жоқ

Қайтарылатын мән

Жоқ

Serial.print()

Деректерді сериялық порт арқылы ASCII мәтіні ретінде жібереді. Бұл функция әртүрлі деректер түрлерін қабылдай алады. Осылайша бүтін сандар сәйкес ASCII таңбаларымен басып шығарылады. Нақты сандар бүтін және бөлшек бөліктер үшін екі ASCII таңбасын пайдаланып шығарылады. Байттар сәйкес нөмірі бар таңба ретінде беріледі. Таңбалар мен жолдар сол күйінде жіберіледі.

Serial.print()

Синтаксис

Serial.print(val)

Serial.print(val, format)

Параметрлер

val: дәйекті қосылыс арқылы жіберуге арналған деректер

format: Бүтін сандар үшін негіз немесе нақтылар үшін ондық таңбалар саны.

Қайтарылатын мән

Жоқ

Мысал

`Serial.print(78);` //«78» ретінде жіберілді

`Serial.print(1.23456);` //«1,23» ретінде жіберіледі

`Serial.print(byte(78));` //«N» ретінде беріледі (Осылайша ASCII «N» кестесінде 78 нөмірмен)

`Serial.print('N');` //«N» ретінде беріледі

`Serial.print("Hello world.");` // «Hello world.» ретінде өтті.

Мысал

`Serial.print(78, BYTE)` «N» шығарады

`Serial.print(78, BIN)` "1001110" шығарады

`Serial.print(78, OCT)` "116" шығарады

`Serial.print(78, DEC)` "78" шығарады

`Serial.print(78, HEX)` "4E" шығарады

`Serial.println(1.23456, 0)` "1" шығарады

`Serial.println(1.23456, 2)` "1.23" шығарады

`Serial.println(1.23456, 4)` "1.2346" шығарады

Мысал

```
int x = 0;  // айнымалы
void setup() {
  Serial.begin(9600);  // дәйекті қосылысты ашамыз
}
void loop() {
  // print labels
  Serial.print("NO FORMAT");
  Serial.print("\t");      // выводим tab
  Serial.print("DEC");
  Serial.print("\t");
  Serial.print("HEX");
  Serial.print("\t");
```

```
Serial.print("OCT");  
Serial.print("\t");  
Serial.print("BIN");  
Serial.print("\t");  
Serial.println("BYTE");  
for(x=0; x< 64; x++){ // ASCII баспа таңбаларын шығарамыз  
    // түрлі базистерде шығарамыз  
    Serial.print(x);    // «DEC» сияқты ондық түрінде шығарамыз  
    (әдепкі)  
    Serial.print("\t");
```

```
Serial.print(x, DEC);  
  Serial.print("\t");  
  Serial.print(x, HEX);  
  Serial.print("\t");  
  Serial.print(x, OCT);  
  Serial.print("\t");  
  Serial.print(x, BIN);  
  Serial.print("\t");  
  Serial.println(x, BYTE); // println деректерден кейін жолды  
  тасымалдау таңбасын қосады  
  delay(200);  
}  
Serial.println(""); // Келесі жолға тағы бір тасымалдау.  
}
```

Serial.println()

Деректерді келесі жолды ауыстыру символымен (ASCII символы 13 немесе '\r ') және жаңа жолдың символымен (ASCII 10 немесе '\n') ASCII мәтіні ретіндегі дәйекті қосылыс арқылы береді. Бұл функция үшін параметрлер мен деректер түрлері Serial.print () сияқты.

Синтаксис

Serial.println(val)

Serial.println(val, format)

Параметрлер

val: дәйекті қосылыс арқылы жіберуге арналған деректер

format: Бүтін сандар үшін негіз немесе нақтылар үшін ондық таңбалар саны

Қайтарылатын мән

Жоқ

Мысал

```
/* 0 аналогтық кірісінен деректерді оқиды және мәнді жібереді */
int analogValue = 0; // оқылатын мәнді сақтау үшін айнымалы
void setup() {
    Serial.begin(9600); //сериялық қосылымды орнату
}
void loop() {
    // 0 кірісінен деректерді оқимыз
    analogValue = analogRead(0);
    // түрлі базистерде шығарамыз
    Serial.println(analogValue); // print as an ASCII-encoded decimal
    Serial.println(analogValue, DEC); // print as an ASCII-encoded decimal
    Serial.println(analogValue, HEX); // print as an ASCII-encoded hexadecimal
    Serial.println(analogValue, OCT); // print as an ASCII-encoded octal
    Serial.println(analogValue, BIN); // print as an ASCII-encoded binary
    Serial.println(analogValue, BYTE); // print as a raw byte value
    delay(10);
}
```

Serial.write()

Функция деректерді екілік код ретінде сериялық қосылым арқылы жібереді. Деректер бір немесе бірнеше байт ретінде жіберіледі. Деректерді таңбалар ретінде тасымалдау үшін басқа print() функциясын пайдалану керек.

Синтаксис

Serial.write(val)

Serial.write(str)

Serial.write(buf, len)

Arduino Mega үшін : Serial1, Serial2, Serial3

Параметрлер

val: бір байт

str: жолды байттар тізбегі ретінде

buf: байт массиві

len: массив ұзындығы

Serial.peek()

Кіріс сериялық қосылым буферінен келесі қолжетімді байтты (таңбаны) сол буферден жоймай қайтарады. Яғни, осы функцияға сәтті шақыру read() функциясына келесі шақыру сияқты мәнді қайтарады.

Синтаксис

Serial.peek()

Для Arduino Mega:

Serial1.peek()

Serial2.peek()

Serial3.peek()

Параметрлер

Жоқ

Қайтарылатын мән

Келесі қолжетімді байт немесе жоқ болса -1(int)

random()

random() функциясы псевдокездейсоқ сандарды жасайды.

Синтаксис

random(max)

random(min, max)

Параметрлер

min - кездейсоқ шаманың төменгі шекарасы, қоса алғанда (міндетті емес параметр).

max - кездейсоқ шаманың жоғарғы шегі, қосатын емес..

Қайтарылатын мән

min мен max-1 ауқымындағы кездейсоқ сан (*long*)

random()

Ескертпе

Егер `random ()` функциясын циклдік пайдалану кезінде түрлі кездейсоқ сандар тізбегін алу маңызды болса, онда кездейсоқ сандар генераторын баптандыру үшін белгілі бір дәрежеде кездейсоқ шаманы білдіретін параметрі бар `randomSeed ()` функциясын пайдаланыңыз. Мұндай параметр, мысалы, қосылмаған шығарудан `analogRead ()` функциясы саналған кернеу шамасы болуы мүмкін.

Кейде керісінше - жалған-кездейсоқ сандардың дәл қайталанатын тізбектерін пайдалану қажет.

Мысал

```
long randNumber;
void setup(){
  Serial.begin(9600);
  randomSeed(analogRead(0));
}
void loop() {
  // 0 мен 299 аралығындағы кездейсоқ санды шығарамыз
  randNumber = random(300);
  Serial.println(randNumber);
  // 10-дан 19-ға дейінгі ауқымда кездейсоқ сан шығарамыз
  randNumber = random(10, 20);
  Serial.println(randNumber);
  delay(50);
}
```

lowByte()

Айнымалының кіші (шеткі оң) байтын шығарады (мысалы, word).

Синтаксис

lowByte(x)

Параметрлер

x: кез келген түрдегі мән

Қайтарылатын мән

Byte

highByte()

word түріндегі айнымалының аға (шеткі сол жақ) байты алынады (немесе айнымалының екінші кіші байты, егер оның түрі екіден көп болса).

Синтаксис

highByte(x)

Параметрлер

x: кез келген түрдегі мән

Қайтарылатын мән

Byte

bitRead()

Санның көрсетілген битінің күйін оқиды

Синтаксис

bitRead(x, n)

Параметрлер

x: бит оқылуы тиіс сан

n: күйі оқылатын биттің нөмірі. Нөмірлеу ең аз маңызды биттен (ең оң жақта) 0 санымен басталады

Қайтарылатын мән

Бит күйі (0 немесе 1)

bitWrite()

Айнымалыдағы көрсетілген биттің күйін өзгертеді.

Синтаксис

bitWrite(x, n, b)

bitWrite(x, n, b)

Параметрлер

x - битті өзгерту қажет сандық айнымалысы

n: күйін өзгерту қажет биттің нөмірі. Нөмірлеу 0 нөмірі бар кіші мәнді биттен (шеткі оң) басталады

b: жаңа бит мәні (0 немесе 1)

Қайтарылатын мән

Жоқ

bitSet()

Көрсетілген битті (1 жазады) сандық айнымалыға орнатады

Синтаксис

bitSet(x, n)

Параметрлер

x: бит орнатуға арналған сандық айнымалы

n: орнатылатын биттік нөмір. Нөмірлеу ең аз маңызды биттен (ең оң жақта) 0 санымен басталады

Қайтарылатын мән

Жоқ

bitClear()

Көрсетілген битті (0 жазады) сандық айнымалыға тазартады.

Синтаксис

bitClear(x, n)

Параметрлер

x: битті қалпына келтіру үшін сандық айнымалы

n: қалпына келтірілетін бит саны. Нөмірлеу ең аз маңызды биттен (ең оң жақта) 0 санымен басталады

Қайтарылатын мән

Жоқ

bit()

bit дәрежесіне 2 қайтарады

Синтаксис

bit(n)

Параметрлер

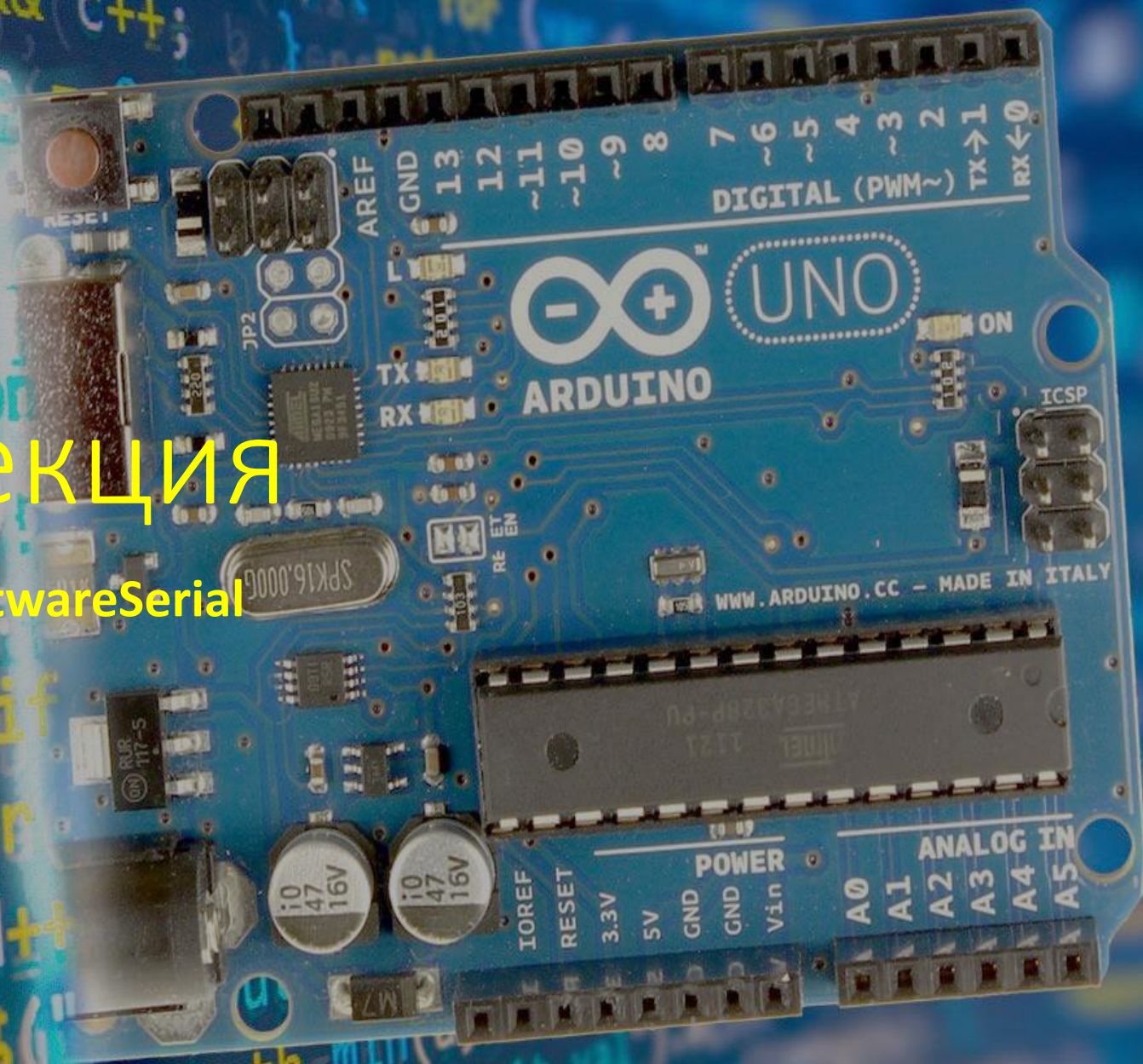
n: мәнін есептеу қажет бит.

Қайтарылатын мән

Бит мәні

Лекция

SoftwareSerial



SoftwareSerial кітапханасы

(Жүктеп алу <https://github.com/PaulStoffregen/SoftwareSerial>).

Arduino 0 және 1 түйреуіштер арқылы сериялық деректер интерфейсіне аппараттық қолдауды жүзеге асырады (олар компьютермен USB арқылы байланысу үшін де қолданылады). Сериялық интерфейсі бар аппараттық жұмыс UART трансивері деп аталатын микроконтроллерге орнатылған арнайы құрылғының көмегімен жүзеге асырылады. Ол Atmega микроконтроллеріне басқа тапсырмалармен жұмыс істегенде де кіріс деректерді өңдеуге мүмкіндік береді.

SoftwareSerial кітапханасы

SoftwareSerial кітапханасы UART функционалдығын қайталайтын бағдарламалық құралдарды пайдалана отырып, кез келген Arduino сандық түйреуіштерінде сериялық интерфейсті жүзеге асыруға мүмкіндік береді (осыдан «SoftwareSerial» атауы). Кітапхана 115200 бит/с жылдамдықпен жұмыс істейтін бірнеше сериялық порттарды бағдарламалы түрде жасауға мүмкіндік береді. Инверттелген сигналмен жұмыс істейтін құрылғылар үшін кітапхана инверсияны қамтитын сәйкес параметрді береді.

Шектеулер

- Бірнеше сериялық порттарды пайдаланған кезде олардың тек біреуі бір уақытта деректерді қабылдай алады.
- Arduino Mega және Mega2560 тақталарында кейбір түйреуіштер сигнал деңгейі өзгерген кезде пайда болатын үзілістерді қолдамайды. Осыған байланысты, бұл тақталарда RX пин ретінде тек келесі түйреуіштерді пайдалануға болады: 10, 11, 12, 13, 14, 15, 50, 51, 52, 53, A8 (62), A9 (63), A10 (64), A11 (65), A12 (66), A13 (67), A14 (68), A15 (69).
- Arduino Leonardo-да кейбір түйреуіштер сигнал деңгейі өзгерген кезде пайда болатын үзілістерді қолдамайды. Сондықтан, осы тақтадағы RX істікшесі ретінде тек келесі түйреуіштерді пайдалануға болады: 8, 9, 10, 11, 14 (MISO), 15 (SCK), 16 (MOSI).

SoftwareSerial()

SoftwareSerial(rxPin, txPin) функциясы жаңа SoftwareSerial нысанын жасайды және оны көрсетілген айнымалыға тағайындайды.

SoftwareSerial.begin() функциясы байланысты бастау үшін пайдаланылады.

SoftwareSerial(rxPin, txPin)

- **rxPin:** Дәйекті интерфейс бойынша деректерді қабылдау үшін шығару
- **txPin:** Дәйекті интерфейс бойынша деректерді жіберу үшін шығару

Мысал

```
#define rxPin 2
```

```
#define txPin 3
```

```
// Жаңа дәйекті портты баптандырамыз.
```

```
SoftwareSerial mySerial = SoftwareSerial(rxPin, txPin);
```

SoftwareSerial: available()

Дәйекті програмалық порты арқылы алынған оқылмаған байттардың (таңбалар) санын қайтарады. Оқылмаған деректер дәйекті кіріс буферінде жинақталады.

```
mySerial.available()
```

Мысал

// SoftwareSerial кітапханасын оның функцияларын пайдалану үшін қосамыз:

```
#include <SoftwareSerial.h>
```

```
#define rxPin 10
```

```
#define txPin 11
```

// жаңа дәйекті портты баптандырамыз

```
SoftwareSerial mySerial = SoftwareSerial(rxPin, txPin);
```

```
void setup() {
```

```
    // tx, rx шығыстарының жұмыс режимін орнату:
```

```
    pinMode(rxPin, INPUT);
```

```
    pinMode(txPin, OUTPUT);
```

```
    // сериялық порттың жіберу жылдамдығын орнатыңыз
```

```
    mySerial.begin(9600);
```

```
}
```

```
void loop() {
```

```
    if (mySerial.available()>0){
```

```
        mySerial.read();
```

```
    }
```

```
• }
```

SoftwareSerial: begin(speed)

Сериялық порттың жіберу жылдамдығын орнатады. Қолдау көрсетілетін мәндер: 300, 600, 1200, 2400, 4800, 9600, 14400, 19200, 28800, 31250, 38400, 57600 и 115200.

Мысал

// SoftwareSerial кітапханасын оның функцияларын пайдалану үшін қосамыз:

```
#include <SoftwareSerial.h>
```

```
#define rxPin 10
```

```
#define txPin 11
```

```
// жаңа дәйекті портты баптандырамыз
```

```
SoftwareSerial mySerial = SoftwareSerial(rxPin, txPin);
```

```
void setup() {
```

```
    // tx, rx шығыстарының жұмыс режимін орнатух:
```

```
    pinMode(rxPin, INPUT);
```

```
    pinMode(txPin, OUTPUT);
```

```
    // сериялық порттың жіберу жылдамдығын орнатыңыз
```

```
    mySerial.begin(9600);
```

```
}
```

```
void loop() {
```

```
    // ...
```

```
}
```

```
}
```

SoftwareSerial: isListening()

Бағдарламалық құралдың сериялық портының белсенді екенін тексеруге мүмкіндік береді. Егер порт деректерді күтіп тұрса, функция "true" мәнін қайтарады.

```
mySerial.isListening()
```

Мысал

```
#include <SoftwareSerial.h>

// TX = сандық шығарылым 10, RX =  
сандық шығарылым 11
```

```
SoftwareSerial portOne(10,11);
```

```
void setup()
```

```
{
```

```
  // аппараттық сериалық портты  
баптандырамыз
```

```
  Serial.begin(9600);
```

```
  // бағдарламалық сериалық портты  
баптандырамыз
```

```
  portOne.begin(9600);
```

```
}
```

```
void loop()
```

```
{
```

```
  if (portOne.isListening()) {
```

```
    Serial.println("Port One is listening!");
```

```
  }
```

```
}
```

SoftwareSerial: overflow()

Бағдарламалық құралдың сериялық портының кіріс буферінің толып кетуін тексереді. Бұл функция шақырылғанда, буфердің толып кету жалауы қалпына келтіріледі. Сондықтан, барлық келесі шақыруларда функция деректердің келесі байты алынғанша (және еленбейді) жалған мәнін қайтарады. Сериялық порттың кіріс буфері 64 байт.

```
mySerial.overflow()
```


Мысал

```
#include <SoftwareSerial.h>
```

```
// бағдарламалық құралдың сериялық порты: TX = сандық шығарылым 10, RX = сандық шығарылым 11
```

```
SoftwareSerial portOne(10,11);
```

```
void setup()
```

```
{
```

```
  // аппараттық сериалық портты баптандырамыз
```

```
  Serial.begin(9600);
```

```
  // бағдарламалық сериалық портты баптандырамыз
```

```
  portOne.begin(9600);
```

```
}
```

```
void loop()
```

```
{
```

```
  if (portOne.overflow()) {
```

```
    Serial.println("SoftwareSerial overflow!");
```

```
}
```

SoftwareSerial: peek

Бағдарламалық құрал сериялық порты RX істікшелі арқылы алынған таңбаны қайтарады. Дегенмен, `read()` функциясынан айырмашылығы, бұл функцияны бірнеше рет шақыру бірдей мәнді қайтарады.

Бір уақытта тек бір бағдарлама порты кіріс деректерді қабылдай алатынын есте сақтаңыз. Қабылдаушы портты таңдау үшін (`SoftwareSerial` класының данасы) `listen()` функциясын пайдаланыңыз.

Параметрлер

Жоқ

Мәндерді қайтару

алынған таңба немесе жоқ болса -1

Мысал

```
SoftwareSerial mySerial(10,11);
```

```
void setup()  
{  
  mySerial.begin(9600);  
}
```

```
void loop()  
{  
  char c = mySerial.peek();  
}
```

SoftwareSerial: read

Бағдарламалық құрал сериялық порты RX істікшелі арқылы алынған таңбаны қайтарады. Бір уақытта тек бір бағдарлама порты кіріс деректерді қабылдай алатынын есте сақтаңыз. Қабылдаушы портты таңдау үшін (SoftwareSerial класының данасы) listen() функциясын пайдаланыңыз.

Параметрлер

Жоқ

Мәндерді қайтару

алынған таңба немесе жоқ болса -1

Мысал

```
SoftwareSerial mySerial(10,11);
```

```
void setup()  
{  
  mySerial.begin(9600);  
}
```

```
void loop()  
{  
  char c = mySerial.read();  
}
```

SoftwareSerial: print(data)

Деректерді бағдарламалық құралдың сериялық портының TX пин арқылы шығарады. Бұл функцияның жұмысы Serial.print() функциясына ұқсас.

Параметрлер

Әртүрлі болуы мүмкін, Serial.print() функциясының сипаттамасын қараңыз.

Мәндерді қайтару

byte

print() функциясы жіберілген байттардың санын қайтарады. Бұл мәнді оқу міндетті емес.

Мысал

```
SoftwareSerial serial(10,11);
int analogValue;
void setup()
{
  serial.begin(9600);
}
void loop()
{
  // 0 аналогтық кірісінен оқу мәні:
  analogValue = analogRead(A0);
  // оны түрлі форматта шығарамыз:
  serial.print(analogValue);    // ASCII таңбалары
  ретінде ондық түрінде шығарамыз
  serial.print("\t");          // қадамдау таңбасын
  шығарамыз
  serial.print(analogValue, DEC); // ондық түрдегі ASCII
  таңбалары ретінде шығару
  serial.print("\t");          // қадамдау таңбасын
  шығарамыз
  serial.print(analogValue, HEX); // он алтылық пішінде
```

```
ASCII таңбалары ретінде шығару
  serial.print("\t");          // қадамдау таңбасын
  шығарамыз
  serial.print(analogValue, OCT); // сегіздік санында
  ASCII таңбалары ретінде шығару
  serial.print("\t"); қадамдау таңбасын шығарамыз
  serial.print(analogValue, BIN); // екілік пішінде ASCII
  таңбалары ретінде шығару
  serial.print("\t");          // қадамдау таңбасын
  шығарамыз
  serial.print(analogValue/4, BYTE); // шикі түрде шығару
  //(бұрын 4-ке бөлінген, өйткені analogRead() 0-ден
  //1023-ке дейінгі диапазондағы мәндерді қайтарады
  //және деректер байтында 255-тен аспайтын санды
  сақтауға болады)
  serial.print("\t");          // қадамдау таңбасын
  шығарамыз
  serial.println();            // Жолды тасымалдау
  таңбасын шығарамыз
  // кезекті оқу алдында 10 миллисекундты кешіктіру:
  delay(10);
}
```

SoftwareSerial: println(data)

Деректерді бағдарламалық құралдың сериялық портының TX істікшелі арқылы шығарады, содан кейін каретканы қайтару, содан кейін жол беру. Бұл функцияның жұмысы Serial.println() функциясына ұқсас.

Параметрлер

Әртүрлі болуы мүмкін, Serial.println() функциясының сипаттамасын қараңыз.

Мәндерді қайтару

byte

println() функциясы жіберілген байттардың санын қайтарады. Бұл мәнді оқу міндетті емес.

SoftwareSerial: println(data)

```
SoftwareSerial serial(10,11);
int analogValue;
void setup()
{
  serial.begin(9600);
}
void loop()
{
  // 0 аналогтық кірісінен оқу мәні:
  analogValue = analogRead(A0);
  // оны түрлі форматта шығарамыз:
  serial.print(analogValue);    // ASCII таңбалары ретінде ондық
  түрінде шығарамыз
  serial.print("\t");          // қадамдау таңбасын шығарамыз
  serial.print(analogValue, DEC); // ондық түрдегі ASCII таңбалары
  ретінде шығару
  serial.print("\t");          // қадамдау таңбасын шығарамыз
  serial.print(analogValue, HEX); // он алтылық пішінде ASCII таңбалары
  ретінде шығару
```

```
  serial.print("\t");          // қадамдау таңбасын шығарамыз
  serial.print(analogValue, OCT); // сегіздік санында ASCII таңбалары
  ретінде шығару
  serial.print("\t"); қадамдау таңбасын шығарамыз
  serial.print(analogValue, BIN); // екілік пішінде ASCII таңбалары
  ретінде шығару
  serial.print("\t");          // қадамдау таңбасын шығарамыз
  serial.print(analogValue/4, BYTE); // шикі түрде шығару //(бұрын 4-ке
  бөлінген, өйткені analogRead() 0-ден //1023-ке дейінгі диапазондағы
  мәндерді қайтарады //және деректер байтында 255-тен аспайтын
  санды сақтауға болады)
  serial.print("\t");          // қадамдау таңбасын шығарамыз
  serial.println();            // Жолды тасымалдау таңбасын шығарамыз
  // кезекті оқу алдында 10 миллисекундты кешіктіру:
  delay(10);
}
```

SoftwareSerial: listen()

Көрсетілген сериялық портты деректер үшін күту режиміне орнатады. Бір уақытта тек бір бағдарламалық құрал порты деректерді қабылдай алады; дегенмен, басқа порттарға келетін деректер еленбейді. Егер тыңдау() функциясы шақырылған кезде ағымдағы белсенді порт басқа портқа өзгертілсе, онда бұрын алынған барлық деректер өшіріледі.

SoftwareSerial: listen()

`mySerial.listen()`

Параметры

`mySerial`: имя экземпляра класса `SoftwareSerial`, который должен принимать данные

Возвращаемые значения

нет

Мысал

```
#include <SoftwareSerial.h>
// бағдарламалық құралдың сериялық порты: TX =
сандық шығарылым 10, RX = сандық шығарылым 11
SoftwareSerial portOne(10, 11);
// бағдарламалық құралдың сериялық порты: TX =
сандық шығарылым 8, RX = сандық шығарылым 9
SoftwareSerial portTwo(8, 9);
void setup()
{
    // аппараттық сериалық портты баптандырамыз
    Serial.begin(9600);
    // бағдарламалық құрал порттарының екеуін де
    инициализациялаңыз
    portOne.begin(9600);
    portTwo.begin(9600);
}
```

```
void loop()
{
    portOne.listen();
    if (portOne.isListening()) {
        Serial.println("Port One is listening!");
    }else{
        Serial.println("Port One is not listening!");
    }
    if (portTwo.isListening()) {
        Serial.println("Port Two is listening!");
    }else{
        Serial.println("Port Two is not listening!");
    }
}
```

SoftwareSerial: write(data)

Деректерді бағдарламалық құралдың сериялық портының TX пин арқылы байт тізбегі ретінде шығарады. Бұл функцияның жұмысы Serial.write() функциясына ұқсас.

Параметрлер

Serial.write() функциясының сипаттамасын қараңыз.

Мәндерді қайтару

byte

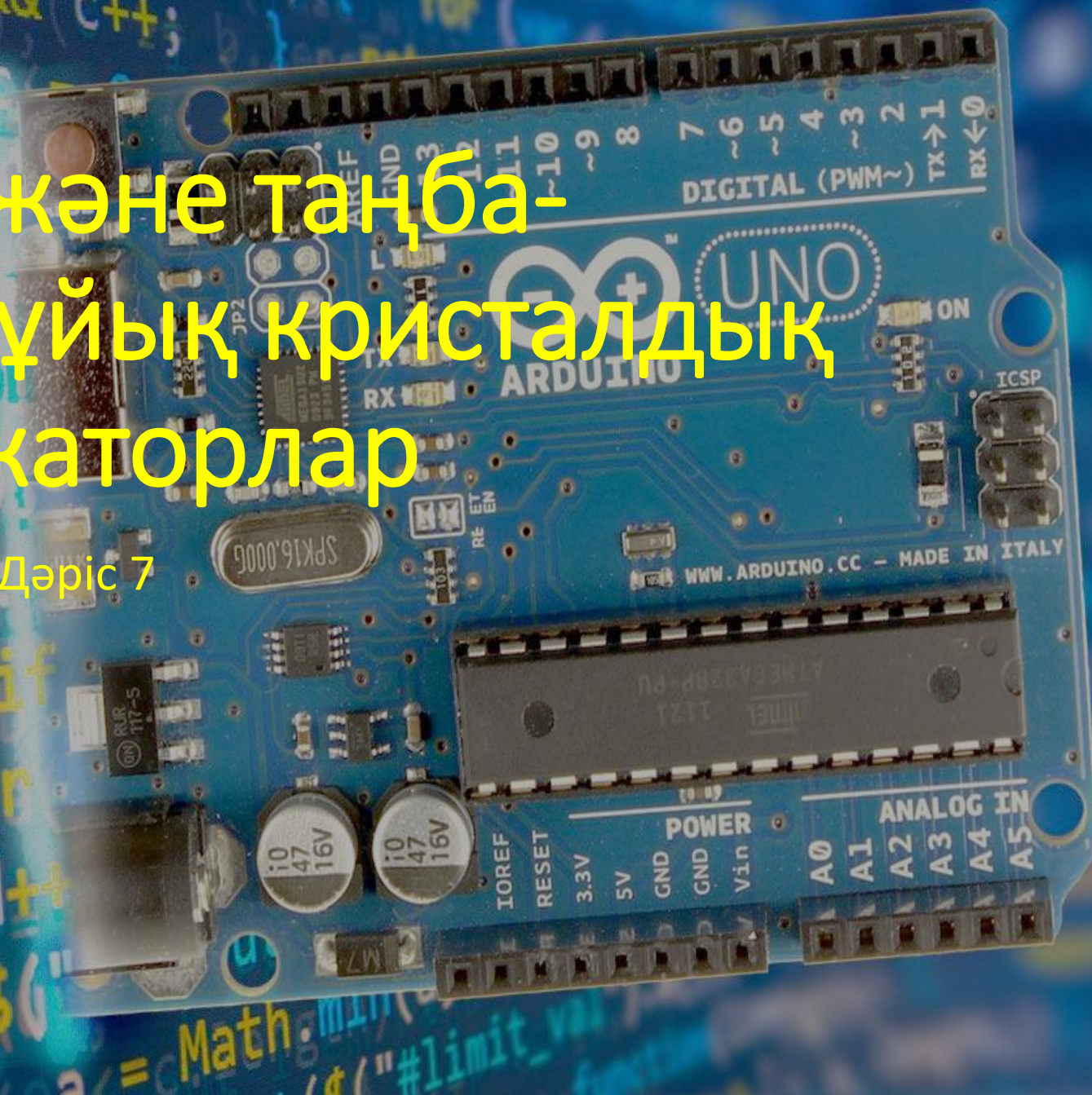
write() функциясы жазылған байттардың санын қайтарады. Бұл мәнді оқу міндетті емес.

SoftwareSerial: write(data)

```
SoftwareSerial mySerial(10, 11);  
void setup()  
{  
  mySerial.begin(9600);  
}  
void loop()  
{  
  mySerial.write(45); // 45 мәні бар деректер байты жіберу  
  int bytesSent = mySerial.write("hello"); //сол жолдың ұзындығын оқу арқылы «сәлем»  
  жолын жіберіңіз.  
}
```


Arduino және таңба- синтездеуші сұйық кристалдық индикаторлар

Дәріс 7



Сұйық кристалды дисплейлер (LCD)

Деректерді шығару мониторы ретінде сериялық портты пайдалану өте ыңғайлы емес екені түсінікті және жеке жобалар үшін бұл мүмкін емес. Бұл мақсат үшін, сондай-ақ жасалып жатқан жобаларды жөндеу кезінде ақпаратты көрсету үшін сұйық кристалды индикаторды пайдалану орынды.

Әртүрлі компаниялардың (POWERTIP, MICROTIPS, WINSTAR, Fordata, Anshan Yes, Data Vision, Newtec Sunlike) сұйық кристалды индикаторлары (LCD) бүгінгі әлемдік электроника нарығын сенімді түрде жаулап алуда және бірте-бірте ескі жақсы көрсеткіш құрылғыларын алмастыруда.

Сұйық кристалды дисплейлер (LCD)

Физикалық эксперименттер мен өнеркәсіп үшін жабдықтардың заманауи конструкторы осы ыңғайлы және пайдалы құрылғылардың жұмысының негізгі принциптерін білуі керек. Әртүрлі елдерде, соның ішінде бізде шығарылатын сұйық кристалды индикаторлардың барлығы бағдарламашы көзқарасы бойынша бір типті. Бірақ әзірлеушілер берген нұсқаулықтар әдетте ағылшын тілінде жазылған, үнемді және түсіну қиын. Дегенмен, әртүрлі елдерде шығарылатын құрылғыларда стандарттаудың айқын тенденциясы бар екенін және өндірілген СКД көрсеткіштерінің барлығы дерлік HD44780 контроллерін пайдалануға бағытталғанын атап өткен жөн.

Сұйық кристалды дисплейлер (LCD)

Arduino жобалары жиі WINSTAR Display Co., Ltd компаниясының қымбат емес көрсеткіштерін пайдаланады. – жоғары сапалы сипаттағы және графикалық (оның ішінде түсті) СКД индикаторларының әлемдегі ең ірі әзірлеушілері мен өндірушілерінің бірі. Компанияның штаб-пәтері Тайваньда, ал өндіріс орындары 3000 шаршы метрден астам аумақта орналасқан. метр, жылына миллионнан астам түрлі көрсеткіштерді шығаруды қамтамасыз етеді. Өндіріс ISO 9001 жүйесі бойынша сертификатталған.

WINSTAR 1604A-NGG-CT



WINSTAR WH1604 СКД пин тағайындау

Шығару нөмірі	Аты	Функция
1	Vss	Жалпы (GND)
2	Vdd	Қоректендіру кернеуі (3 немесе 5 В)
3	Vo	Контрастілік
4	RS	Командалар/Деректер
5	R/W	Оқу/жазу
6	E	Оқу/жазу рұқсаты
7	DB0	Деректер жолы 0
8	DB1	Деректер жол 1
9	DB2	Деректер жол 2
10	DB3	Деректер жол 3
11	DB4	Деректер жол 4
12	DB5	Деректер жол 5
13	DB6	Деректер жол 6
14	DB7	Деректер жол 7
15	A	Артқы жарық кернеуі (+)
16	K	Артқы жарық кернеуі (–)

WINSTAR WH1604 LCD пин тағайындау

Мұнда 8 жол (DB0...DB7) стандартты деректер шинасы болып табылады. DB0...DB7 түйреуіштеріндегі деңгейлер операциялық кодтар немесе деректер болып табылады. Үш басқару сызығы бар (RS, R/W, E). RS, R/W, E кірістеріндегі деңгейлер режимдерді анықтайды.

Оқу/жазу операцияларын орындау шарттары

RS	R/W	E	Операция
0	0	1->0	Пәрмендер тіркеліміне жазу
1	0	1->0	Деректер тіркеліміне жазу
1	1	0->1->0	Деректер тіркелімінен оқу
0	1	0->1->0	Бос емес жалаушасын оқу (DB7) және адрес есептегіші (DB0-DB6)

Сұйық кристалды дисплейлер (LCD)

LCD контроллері сыртқы контроллермен бірге әртүрлі әрекеттерді орындай алады: деректерді екі бағытта параллель интерфейс арқылы жіберу, LCD дисплейінің таңбалар кеңістігін толтыру бағытын өзгерту, жолдарды өңдеу және т.б.

Қуат қосылғаннан кейін LCD модулінің жұмысы әрқашан оның баптандыру процедурасынан басталады, оның параметрлері құрылғы өшірілгенге дейін сақталады. Баптандыру процедурасының соңында LCD модулі деректермен жұмыс істеуге дайын күйге жеткізіледі. Егер LCD модулін инициализациялау сәтті болса, модуль жұмыс істеуге дайын - ол үшін жолдар саны мен таңбалар кеңістігінің өлшемі орнатылады (мұнда таңбалар кеңістігі 5x7 немесе 5x10 нүктелік матрица болып табылады).

Сұйық кристалды дисплейлер (LCD)

Матрицаны таңдау LCD модулін баптандыру процедурасы кезінде бір рет жасалады, бірақ жұмыс кезінде матрица түрін өзгертуге болады. Бірлік оған қолданылғанда нүкте «ерекшеленеді», егер нүктеге нөл қолданылса, онда ол «өшіріледі».

Матрицалардың екі түрінде де қосымша (8-ші немесе 11-ші) нүктелер жолы бар - курсор жолы деп аталады. Курсор қосылған кезде (бағдарламалық түрде) осы жолдың барлық 5 нүктесі келесі дисплей циклінде көрсетілетін таңбаның астында «ерекшеленеді» (курсор ыңғайлы элемент және барлық құрылғыларда қажет емес). Матрицаның көршілес элементтері бір-бірінен 0,7 мм саңылаумен бөлінген. 5×7 нүктелік матрицаны жиі пайдаланады, бұл жағдайда таңбаларды таңдау 5×10 нүктелік матрицаны пайдаланғанға қарағанда көбірек.

Сұйық кристалды дисплейлер (LCD)

Дисплейге арналған таңбалар таңба генераторының тұрақты жады аймағында орналастырылған. Бұл ROM болғандықтан, таңбалар жиынын пайдаланушы өзгерте алмайды. Дегенмен, тұрақты жад аймағынан басқа, таңбалар генераторы CG RAM аймағын қамтиды, онда бағдарламашы «жобалаған» таңбаларды жазуға болады. Әзірлеушілер бағдарламашыға өз таңбаларын жасау мүмкіндігін қалдырады: бір CG RAM ұяшығындағы 5×7 нүктелік матрица үшін 8 таңбаға дейін немесе 5×10 нүктелік матрица үшін 4 таңбаға дейін (CG RAM екі ұяшықтағы бір символ) . Дегенмен, «тұрақты» белгілердің саны соншалықты үлкен және әртүрлі, олар жеткілікті.

Сұйық кристалды дисплейлер (LCD)

Таңбалар генераторы әдеттегі ROM чіпінің жұмысын басқаратын бағдарламаланатын есептегіш болып табылады. Таңбалар генераторының екілік көрінісіндегі сандар сұралған мекенжайлардың ROM ұяшықтарынан шығарылады. Таңбалар генераторының кестесі белгіленген адрестерде символдар жазылатын ұяшықтарға бөлінген.

Деректерді жазу режимін «бастау» үшін алдымен LCD модулінің RS кірісінде «1» мәнін орнату керек (R/W – корпуста), содан кейін «1»-ден «0» аралығында строб жиігін қалыптастыру керек. пәрменді инициализациялайтын LCD модулінің E кірісі. E шығысын «1» «0» күйіне өзгерткеннен кейін таңба генераторының ұяшығының алдын ала таңдалған мекенжайынан алынған мазмұн LCD бейне жадының ағымдағы танысуына шығарылады. LCD бейне жады мекенжайының кестесі келесі кестеде көрсетілген.

LCD бейне жадының мекенжай кестесі

Таңбалы орын		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Әр жолға адресер	1	00	01	02	03	04	05	06	07	08	09	1A	1B	1C	1D	1E	1F
		20	21	22	23	24	25	26	27								
	2	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F
										28	29	2A	2B	2C	2D	2E	2F
	3	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E	1F
		30	31	32	33	34	35	36	37								
	4	50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F
										38	39	3A	3B	3C	3D	3E	3F

Сұйық кристалды дисплейлер (LCD)

Курсор бейне жадының жұмысына байланысты. Мұндай құрылғылармен жұмыс істеу кезінде мұндай байлау өте пайдалы. Курсор келесі таңба көрсетілетін таныстықты белгілейді. LCD бейне жадының ағымдағы танысуы «АС» есептегішті анықтайды - автоматты арттыру және автоматты азайту бағдарламалық құралының параметрі және санаулардың алдын ала орнатылған саны бар қайтымды есептегіш. Егер жолды символдармен толтыру барысында «АС» адресі есептегіші тудыратын адрестің мәнін ұлғайту немесе азайту қажет болса, онда сәйкес команданы программалық түрде генерациялау қажет.

Сұйық кристалды дисплейлер (LCD)

Кестеде бейне жады ұяшықтарының адрестері ақ фонмен белгіленген, оның мазмұны дисплей экранында бірден көрінеді (кестедегі адрестер СКД сызықтарына сілтеме жасап көрсетілген), сұр ұяшықтар – бұл ұяшықтар мазмұны тек циклдік ауысымнан кейін СКД экранында көрінеді. Бұл жағдайда бейне жады аймағындағы адрестер ауыспайды. Жылжыту командасы орындалғаннан кейін барлық ұяшықтардың мазмұны ұяшықтан ұяшыққа ауысады.

Ағымдағы орынды өзгерту пәрменін құру нұсқаулары келесі кестеде берілген.

Таңбаларды көрсету және LCD экранында символдық орынды орнату нұсқаулары.

Нұсқау	Режим		Екілік нұсқаулық коды							
	RS	RW	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
LCD дисплейінің символдық орнын орнату	0	0	1	Бейне жады ұяшығының адресі						
ROM-дан бейне жадыға таңбалар генераторының шығысының адресін таңдау	1	0	Таңбалы генератор ұяшығының адресі.							

Сұйық кристалды дисплейлер (LCD)

LCD басқару пәрмендерінің тізімі келесі кестеде берілген. Кестеде LCD контроллерінің жадынан оқуға қатысты пәрмендер жоқ, бірақ оларды пайдалану күрделі меншікті дисплей жүйелерін жобалау кезінде ұтымды және Arduino пайдаланатын салыстырмалы түрде қарапайым жобаларда қажет болуы екіталай.

Басқару командалары

Нұсқау	Режим		Екілік нұсқаулық коды								16-р. код
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
Дисплейді тазалау, деректерді қалпына келтіру, курсорды басына қою.	0	0	0	0	0	0	0	0	0	1	01
Деректерді жабусыз бастау үшін курсорды қайтару	0	0	0	0	0	0	0	0	1	x	02
I/D дисплей курсорының SH бағытын өзгерту	0	0	0	0	0	0	0	1	I/D	SH	04 05 06 07
Дисплей қосу/өшіру D = (1/0) Курсор суреті B астын сызу түрінде C Немесе жарқыраған көрнекті орын	0	0	0	0	0	0	1	D	C	B	08 09 0A 0B 0C 0D 0E 0F

Курсорды немесе дисплейді оңға немесе солға жылжыту Дисплей мен курсорды жылжыту $S/C = 1$ Тек курсорды жылжыту $S/C = 0$ $R/L = 1$ оңға $R/L = 0$ солға	0	0	0	0	0	1	S/C	R/L	x	x	10 18 14 1C
Интерфейстің бит тереңдігін орнату 8-DL = 1, 4-DL = 0 Матрица түрі F = 0 — 5x7 F = 0 — 5x10 Жолдар саны N = 1 4 жолдар	0	0	0	0	1	DL	N	F	x	x	40 60

LiquidCrystal кітапханасы

Көптеген таңбалы СКД-де табылған Hitachi HD44780 контроллері (немесе үйлесімді чипсет) арқылы жұмыс істейтін СКД дисплейлерді Arduino стандартты ретінде қосылған LiquidCrystal кітапханасы басқаруға болады. LiquidCrystal кітапханасы 4-биттік немесе 8-биттік режимде жұмыс істейді (яғни, ол RS, ENABLE және қосымша RW басқару жолдарына қосымша 4 немесе 8 деректер жолын пайдаланады).

LiquidCrystal кітапханасы

Кітапханада келесі функциялар
орындалады:

LiquidCrystal();
begin();
clear();
home();
setCursor();
write();
print();
cursor();
noCursor();

blink();
noBlink();
display();
noDisplay();
scrollDisplayLeft();
scrollDisplayRight();
autoscroll();
noAutoscroll();
leftToRight();
rightToLeft();
createChar().

LiquidCrystal() функциясы

LiquidCrystal() функциясы LiquidCrystal түріндегі айнымалыны жасайды. Көрсеткішті 4 немесе 8 деректер желісі арқылы басқаруға болады. 4 жолды қосылым үшін D0-D3 түйреуіштерін өткізіп жіберіп, бұл жолдарды жалғанбай қалдырыңыз. RW істікшесін Arduino тақтасының істікшесіне қосудың орнына жерге қосуға болады. Бұл сіздің схемаңызда болса, осы функция параметрін өткізіп жіберіңіз.

LiquidCrystal() функциясы

Синтаксис:

LiquidCrystal(rs, enable, d4, d5, d6, d7)

LiquidCrystal(rs, rw, enable, d4, d5, d6, d7)

LiquidCrystal(rs, enable, d0, d1, d2, d3, d4, d5, d6, d7)

LiquidCrystal(rs, rw, enable, d0, d1, d2, d3, d4, d5, d6, d7)

rs - LCD RS істікшесіне қосылған Arduino тақтасының пин нөмірі;

rw - LCD RW істікшесіне қосылған Arduino тақтасының пин нөмірі (міндетті емес);

қосу – LCD ENABLE істікшесіне қосылған Arduino тақтасының пин нөмірі;

d0, d1, d2, d3, d4, d5, d6, d7 - тиісінше LCD деректер түйреуіштеріне қосылған Arduino тақтасының істік нөмірлері. D0, D1, D2 және D3 қосымша болып табылады - олар қосылмаған болса, LCD тек 4 деректер желісі (D4, D5, D6, D7) арқылы қозғалады.

Мысал

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(12, 11, 10, 5, 4, 3, 2);
void setup()
{lcd.print("hello, world!");}
void loop() {;}
```

begin() функциясы

begin() функциясы индикатордың өлшемін (ені мен биіктігіндегі таңбалар саны) анықтайды.

Синтаксис:

```
lcd.begin(cols, rows)
```

Параметрлер:

lcd — LiquidCrystal түріндегі айнымалы;

cols — жолдағы таңбалар саны;

rows — жолдар саны.

clear() функциясы

clear() функциясы LCD экранын тазартады және курсорды жоғарғы сол жақ бұрышқа орналастырады.

Синтаксис:

```
lcd.clear();
```

Параметрлер:

lcd — LiquidCrystal түріндегі айнымалы.

home() функциясы

home() функциясы курсорды LCD жоғарғы сол жақ бұрышына орналастырады. Индикатор экранында дәйекті мәтінді көрсету кезінде бастапқы орынды анықтау үшін қолданылады. Сондай-ақ индикатор экранын тазалау үшін оның орнына clear() функциясын пайдаланыңыз.

Синтаксис:

`lcd.home()`

Параметрлер:

lcd — LiquidCrystal түріндегі айнымалы.

setCursor() функциясы

setCursor() функциясы сұйық кристалды индикатор курсорын орналастырады, яғни оның экранында келесі мәтін көрсетілетін орынды белгілейді.

Синтаксис:

```
lcd.setCursor(col, row)
```

Параметрлер:

lcd — LiquidCrystal түріндегі айнымалы;

col — жолдағы таңбалар орнының саны (бірінші таңба орнына 0-ден басталады);

row — жол нөмірі (бірінші жол үшін 0-ден басталады).

write() функциясы

write() функциясы LCD-ге таңба жазады

Синтаксис:

`lcd.write(data)`

Параметрлер:

`lcd` — LiquidCrystal түріндегі айнымалы;

`data` — индикаторға жазылатын таңба.

Мысал

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(12, 11, 10, 5, 4, 3, 2);
void setup()
{Serial.begin(9600);}
void loop()
{
if (Serial.available())
lcd.write(Serial.read());
}
```

print() функциясы

print() функциясы LCD-де мәтінді басып шығарады.

Синтаксис:

```
lcd.print(data)
```

```
lcd.print(data, BASE)
```

Параметрлер:

lcd — LiquidCrystal түріндегі айнымалы;

data — басып шығаруға арналған деректер (char, byte, int, long немесе string түрі);

BASE (міндетті емес) — сандар басып шығарылатын база: екілік үшін BIN (2-негіз), ондық үшін DEC (10-негіз), сегіздік үшін OCT (8-негіз), он алтылық үшін HEX (16-база).

cursor() функциясы

cursor() функциясы LCD экранында курсорды көрсетеді – келесі таңба жазылатын орындағы белгі орнының астын сызу.

Синтаксис:

lcd.cursor()

Параметр:

lcd — LiquidCrystal түріндегі айнымалы.

noCursor() функциясы

noCursor() функциясы курсорды LCD экранынан жасырады.

Синтаксис:

`lcd.noCursor()`

Параметр:

lcd — LiquidCrystal түріндегі айнымалы.

blink() функциясы

blink() функциясы LCD экранында жыпылықтайтын курсорды көрсетеді. Егер ол cursor() функциясымен бірге қолданылса, нәтиже нақты көрсеткішке байланысты болады.

Синтаксис:

`lcd.blink()`

Параметр:

lcd — LiquidCrystal түріндегі айнымалы.

noBlink() функциясы

noBlink() функциясы LCD экранындағы жыпылықтайтын курсорды өшіреді.

Синтаксис:

`lcd.noBlink()`

Параметр:

`lcd` — LiquidCrystal түріндегі айнымалы.

display() функциясы

Display() функциясы LCD дисплейді noDisplay() функциясымен өшіргеннен кейін қосады. Бұл функция дисплейде болған мәтінді (және курсорды) қалпына келтіреді.

Синтаксис:

`lcd.display()`

Параметр:

`lcd` — LiquidCrystal түріндегі айнымалы.

noDisplay() функциясы

noDisplay() функциясы LCD дисплейде көрсетілген ақпаратты жоғалтпай өшіреді.

Синтаксис:

`lcd.noDisplay()`

Параметр:

`lcd` — LiquidCrystal түріндегі айнымалы.

scrollDisplayLeft() функциясы

scrollDisplayLeft() функциясы индикатор экранындағы ақпаратты (мәтін және курсор) бір таңба кеңістігінде солға жылжытады.

Синтаксис:

`lcd.scrollDisplayLeft()`

Параметр:

`lcd` — LiquidCrystal түріндегі айнымалы.

scrollDisplayRight() функциясы

scrollDisplayRight() функциясы индикатор экранындағы ақпаратты (мәтін және курсор) бір таңба кеңістігінде оңға жылжытады.

Синтаксис:

`lcd.scrollDisplayRight()`

Параметр:

`lcd` — LiquidCrystal түріндегі айнымалы

autoscroll() функциясы

Autoscroll() функциясы LCD экранын автоматты түрде айналдыруға мүмкіндік береді және индикатор экранындағы әрбір таңба шығысы алдыңғы таңбаларды бір таңба кеңістігіне жылжытады. Ағымдағы таңбаның шығыс бағыты солдан оңға қарай болса (әдепкі мән), индикатор экраны солға жылжиды; егер ағымдағы таңбаның шығыс бағыты оңнан солға қарай болса, индикатор экраны оңға жылжиды. Бұл әрбір жаңа таңбаны LCD экранында бірдей белгіше орнына шығаруға әсер етеді.

Синтаксис:

`lcd.autoscroll()`

Параметр:

`lcd` — LiquidCrystal түріндегі айнымалы.

noAutoscroll() функциясы

noAutoscroll() функциясы LCD экранының автоматты айналдыруын өшіреді.

Синтаксис:

`lcd.noAutoscroll()`

Параметр:

lcd - LiquidCrystal түріндегі айнымалы.

leftToRight() функциясы

leftToRight() функциясы сұйық кристалды индикатор экранына символдарды шығару бағытын солдан оңға қарай орнатады. Бұл индикатор экранына бұдан кейін шығатын символдардың солдан оңға қарай жылжуын көрсетеді, алайда ол бұған дейін шыққан мәтінге әсер етпейді.

Синтаксис:

```
lcd.leftToRight();
```

Параметр:

lcd — LiquidCrystal түріндегі айнымалы.

rightToLeft() функциясы

rightToLeft() функциясы сұйық кристалды индикатор экранына символдарды шығару бағытын солдан оңа қарай орнатады. Бұл индикатор экранына бұдан кейін шығатын символдардың оңнан солға қарай жылжуын көрсетеді, алайда ол бұған дейін шыққан мәтінге әсер етпейді.

Синтаксис:

`lcd.rightToLeft()`

Параметр:

`lcd` — LiquidCrystal түріндегі айнымалы.

createChar() функциясы

createChar() функциясы сұйық кристалды дисплейде қолдану үшін пайдаланушы символын (глиф) жасайды. 5×8 пиксельді сегіз символға дейін шығарылады (нөмірленуі 0-ден 7-ге дейін). Әрбір қолданушы символын құру 8 байтты массивпен - әрбір жол үшін бір байт – айқындалады. Әрбір байттың мәнді бес кіші биты осы жолдағы пиксельдерді айқындайды. Экранға пайдаланушы символын шығару үшін параметр ретінде символ номерімен write() функциясын қолдану керек.

Синтаксис:

```
Lcd.createChar(num, data)
```

Параметрлер:

Lcd — LiquidCrystal түріндегі айнымалы;

num — жасалған таңбаның нөмірі (0-ден 7-ге дейін);

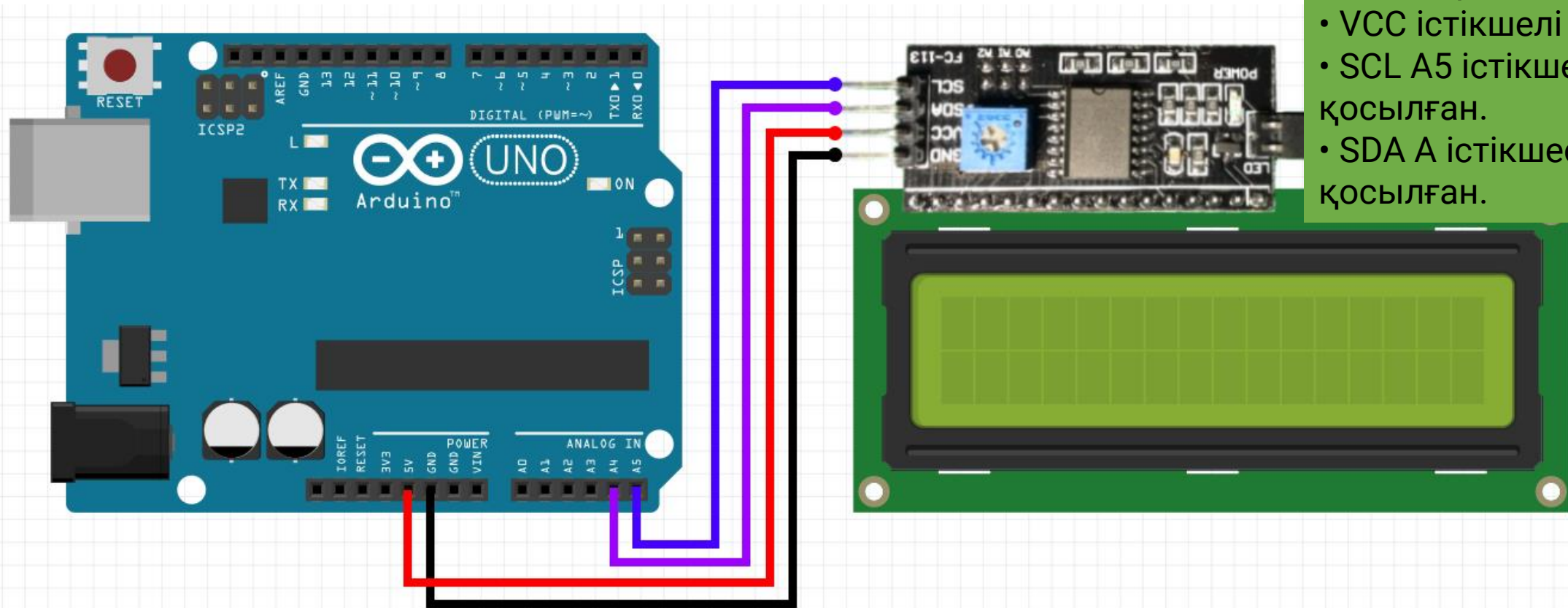
data — таңба пикселінің деректері.

Мысал

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
byte smiley[8]={
  B00000,
  B10001,
  B00000,
  B00000,
  B10001,
  B01110,
  B00000,
};
void setup()
{
  lcd.createChar(0, smiley);
  lcd.begin(16, 2);
  lcd.write(0);
}
void loop() {;}
```

LCD экранды I²C арқылы Arduino-ға қосу

I²C қосылған LCD монитор тақтаға төрт сым арқылы қосылған - деректер үшін екі сым, қуат үшін екі сым.



- GND түйреуіш тақтадағы GND-ге қосылған.
- VCC істікшелі - 5 В.
- SCL A5 істікшесіне қосылған.
- SDA A істікшесіне қосылған.

I²C LCD дисплейімен жұмыс істеуге арналған кітапханалар

Arduino-ны LCD 1602-мен I²C шинасы арқылы интерфейстеу үшін сізге кемінде екі кітапхана қажет:

- I²C-пен жұмыс істеуге арналған Wire.h кітапханасы стандартты Arduino IDE бағдарламасында бұрыннан бар.
- LiquidCrystal_I2C.h кітапханасы, ол I²C шинасы арқылы мониторды басқаруға арналған пәрмендердің кең ауқымын қамтиды және эскизді жеңіл әрі қысқартуға мүмкіндік береді. Дисплейді қосқаннан кейін LiquidCrystal_I2C.h кітапханасын қосымша орнату керек

http://image.dfrobot.com/image/data/DFR0154/LiquidCrystal_I2Cv1-1.rar

Мысал

```
#include
```

```
LiquidCrystal_I2C lcd(0x20,16,2); // set the LCD address to 0x20 for a 16 chars and 2 line display
```

```
void setup() {
```

```
  lcd.init();           // initialize the lcd
```

```
  // Print a message to the LCD.
```

```
  lcd.backlight();
```

```
  lcd.print("Hello, world I2C!");
```

```
}
```

```
void loop() {
```

```
  lcd.setCursor(0, 1);
```

```
  lcd.print(millis() / 1000);
```

```
  lcd.setCursor(4,1);
```

```
  lcd.print("display 16x2");
```

```
}
```

I²C LCD дисплейіне қосылу ақаулары

Егер sketch қотарылғаннан кейін дисплейде ешқандай жазу болмаса, келесі әрекеттерді орындап көріңіз.

Біріншіден, монитордың қарама-қарсылығын ұлғайтуға немесе азайтуға болады. Таңбалар қарама-қайшылық пен жарықтандыруға байланысты жиі көрінбейді.

Егер бұл көмектеспесе, контактілердің қосылуының дұрыстығын, жарықтандырудың қосылғанын тексеріңіз. Егер жеке I²C пернесін пайдалансаңыз, контактілерді дәнекерлеу сапасын тағы да тексеріңіз.

I²C LCD дисплейіне қосылу ақаулары

Экранда мәтіннің болмауының тағы бір жалпы себебі қате I²C мекенжайы болуы мүмкін. Алдымен эскиздегі құрылғы мекенжайын 0x27 0x20 немесе 0x3F етіп өзгертіп көріңіз. Әртүрлі өндірушілердің әдепкі мекенжайлары әртүрлі болуы мүмкін. Бұл көмектеспесе, барлық қосылған құрылғыларды сканерлейтін және олардың мекенжайын қатал күшпен анықтайтын I²C сканерінің эскизін іске қосуға болады.


```
#include <Wire.h>

void setup()
{
  Wire.begin();
  Serial.begin(9600);
  while (!Serial);           // Leonardo: wait
  for serial monitor         {
    Serial.println("\nI2C Scanner");
  }
}

void loop()
{
  byte error, address;

  int nDevices;
  Serial.println("Scanning...");
  nDevices = 0;
  for(address = 1; address < 127;
  address++ )
  {
    Wire.beginTransmission(address);
    error = Wire.endTransmission();
```

```
if (error == 0)
{
    Serial.print("I2C device found at
address 0x");
    if (address<16)
        Serial.print("0");
    Serial.print(address,HEX);
    Serial.println(" !");
    nDevices++; }
else if (error==4)
{    Serial.print("Unknown error at
address 0x");
```

```
if (address<16)
    Serial.print("0");
    Serial.println(address,HEX); }
}
if (nDevices == 0)
    Serial.println("No I2C devices
found\n");
else
    Serial.println("done\n");
    delay(5000);        // wait 5 seconds
for next scan
}
```

Arduino и двигатели

Дәріс 8



Arduino және қозғалтқыштар

Қозғалтқыштың құрылғысын ашпас бұрын, біз басқа роботтық тұжырымдаманы - актуатор қысқаша сипаттаймыз. Негізінде актуатор - бұл роботтың қозғалуына мүмкіндік беретін құрылғы. Егер робот доңғалақ болса, онда әр дөңгелек үшін бір-бірден екі актуатор бар. Күрделі жаяу жүретін роботта мұндай құрылғылар көп, аяқтарының әр буынында бірнешеу бар.

Arduino және қозғалтқыштар

Актуатордің көптеген түрлері бар, оның ішінде үш негізгі топ бар: электрлік, пневматикалық және гидравликалық (соңғысын кез келген экскаваторда көруге болады). Ең қолжетімді және қарапайым - электрлік. Біз курстарымызда осы электр актуаторінің бірін қолданамыз. Оның атауы тұрақты ток қозғалтқышы.

Қозғалтқыштардың жұмыс істеу принципі

Қозғалтқыштардың нақты қалай жұмыс істейтінін білмес бұрын, олардың не үшін қолданылатынына тоқталайық. Қозғалтқыштар қозғалысты қамтамасыз ету үшін электромагниттік күштерді пайдаланады, электр энергиясын механикалық энергияға айналдырады.

Қозғалтқыштардың жұмыс істеу принципі

Магниттік өрістер объектілерді жылжыта алатын күш тудырады. Әрбір магниттің солтүстік және оңтүстік полюсі бар магнит өрісі бар. Екі магниттің екі солтүстік полюсін жақындатуға тырыссаңыз, олар бір-бірін итереді. Екі оңтүстік полюсті жақындатуға тырыссаңыз да солай болады. Егер полюстер бірдей болса, олар бір-бірін итереді. Бір магниттің солтүстік полюсін екінші магниттің оңтүстік полюсіне жақындатсаңыз, олар белгілі бір күшпен тартылады. Яғни қарама-қарсы полюстер бір-бірін тартады.

Қозғалтқыштардың жұмыс істеу принципі

Электр қозғалтқышы қозғалыс жасау үшін магниттердің итеруші және тартымды қасиеттерін пайдаланады. Кәдімгі электр қозғалтқышында екі магнит бар: тұрақты және айнымалы. Айнымалы магнит электромагнит деп аталады. Электромагнит электр тогын өткізгіш арқылы өткізу арқылы жасалады. Тұрақты магниттің тұрақты магнит өрісі (солтүстік және оңтүстік полюстері) бар, ал электромагнит ол арқылы электр тогы өткенде ғана магнит өрісін тудырады. Электромагниттің магнит өрісінің күшін өткізгіш арқылы өтетін токты ұлғайту немесе бірнеше өткізгіш орамдарын құру арқылы арттыруға болады.

Қозғалтқыштардың жұмыс істеу принципі

Электр қозғалтқышының негізгі бөліктері:

- Тұрақты магнит(тер), ол бекітілген кезде, статор деп аталады;
- статордың ішінде ротор деп аталатын оське орнатылған және айналатын катушка бар.

Тұрақты ток қозғалтқыштары

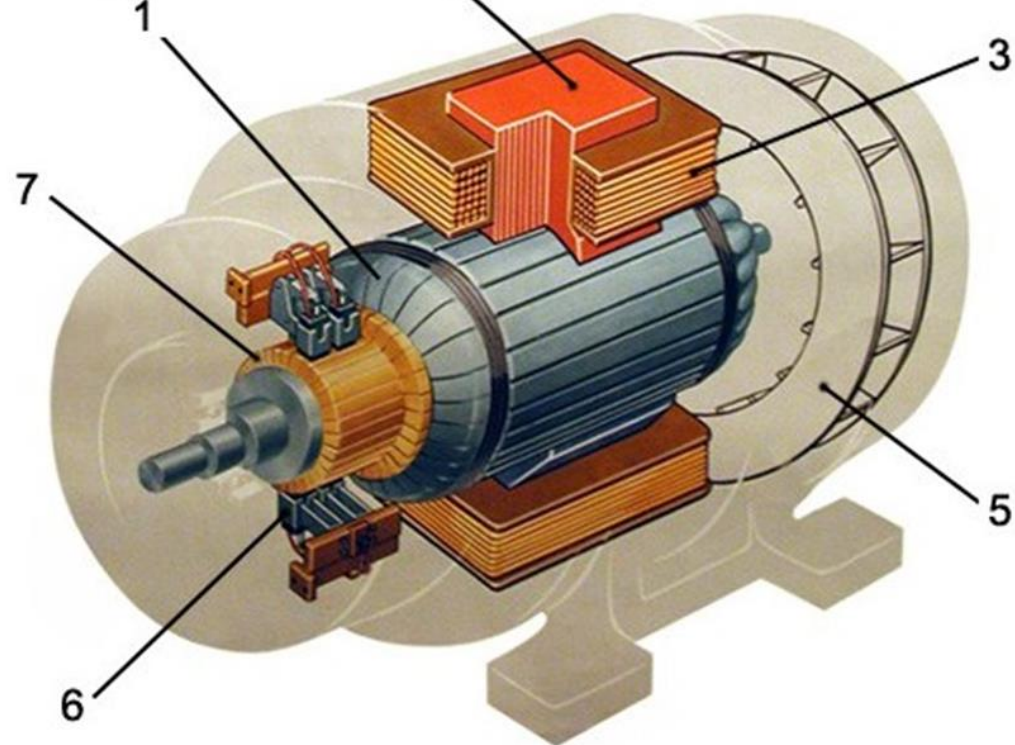
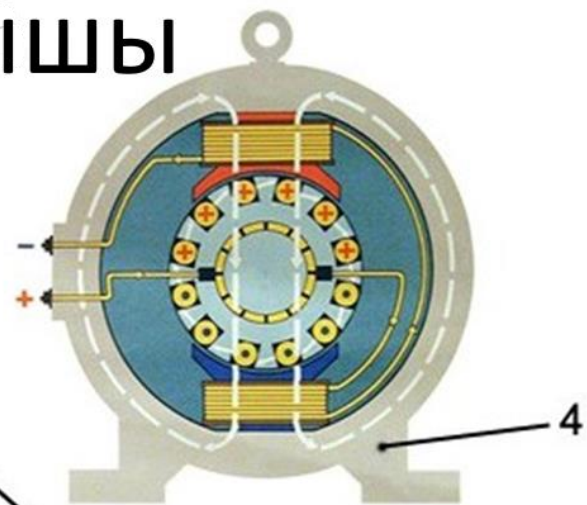
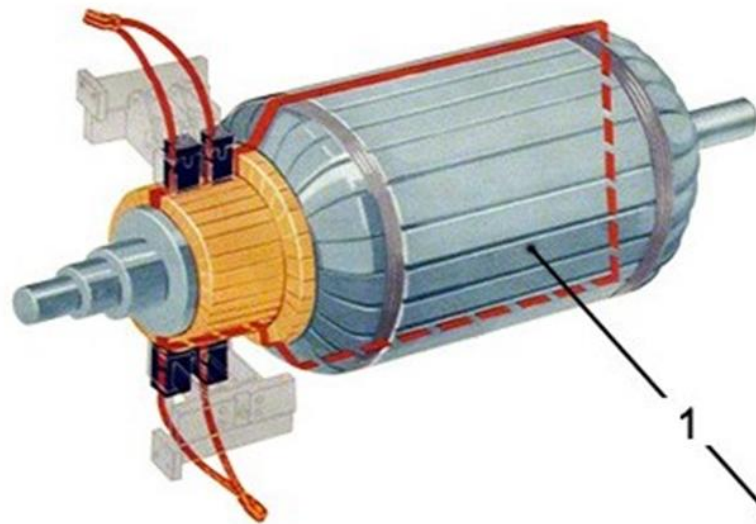
Тұрақты ток қозғалтқыштары көп жағынан қарапайым электр қозғалтқыштары болып табылады. Көптеген «щетка» қозғалтқыштары бірдей жұмыс істейді. Ротор мен статор бар. Статорда магниттер және роторда оған ток қолдану арқылы пайда болатын магнит өрісі бар катушкалар бар. Қозғалтқыштың ішінде роторды жылжытатын щеткалар бар.

Тұрақты ток қозғалтқыштары

Тұрақты ток көзін пайдаланған кезде мұндай қозғалтқышты басқару үшін дерлік ештеңе қажет емес. Оның айналу жылдамдығы қуат көзінен коммутаторға дейінгі катушкаларға түсетін ток күшіне байланысты.

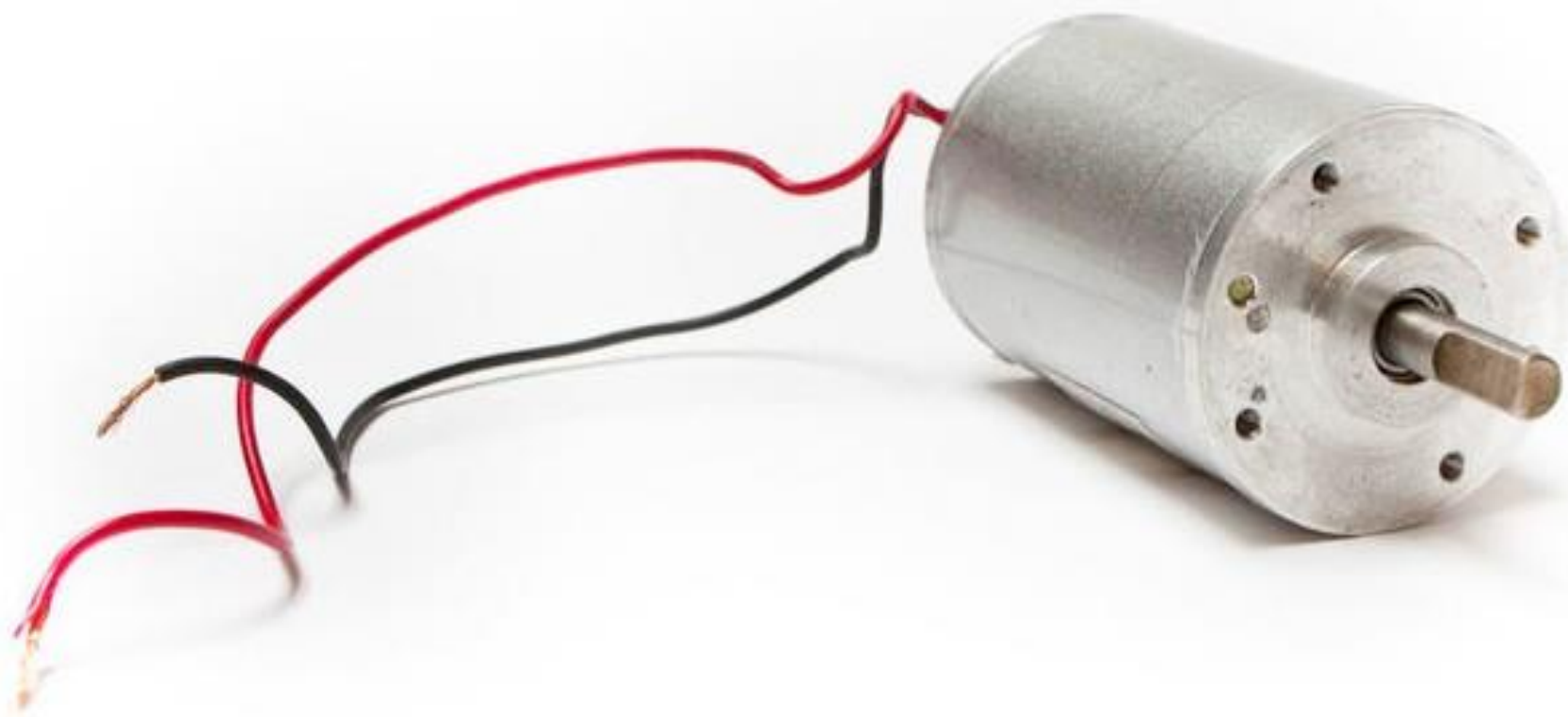
Қозғалтқыш осін кері бағытта бұру үшін қуат көзінен қозғалтқышқа контактілерді кері бағытта қосу жеткілікті.

Тұрақты ток қозғалтқышы

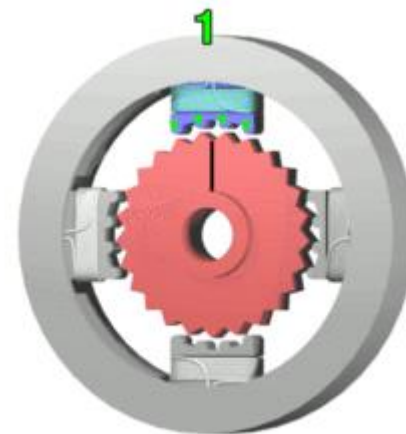
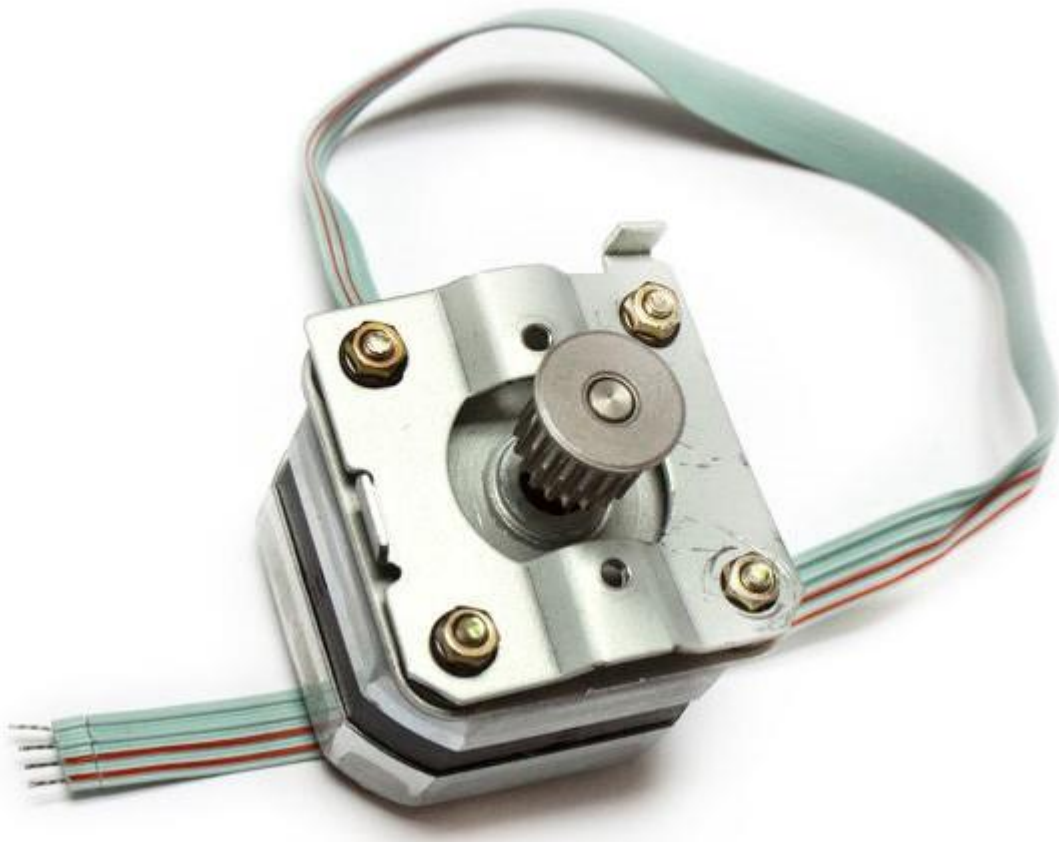


1. Якорь
2. Сердечник полюса
3. Обмотка полюса
4. Статор
5. Вентилятор
6. Щетки
7. Коллектор

Мысал

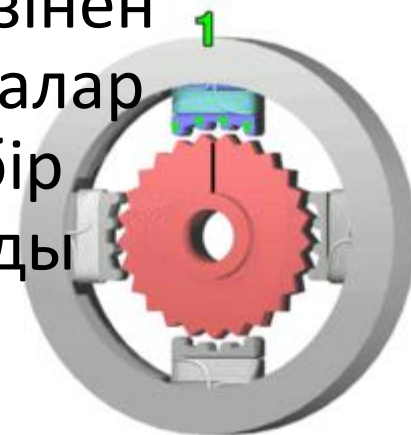


Қадамдық қозғалтқыштар



Қадамдық қозғалтқыштар

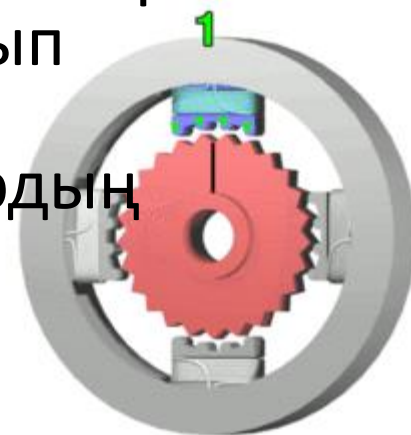
Тұрақты ток қозғалтқыштары сияқты, қадамдық қозғалтқыштар ротор мен статордан тұрады. Бірақ, басқа қозғалтқыштардан айырмашылығы, қадамдық қозғалтқыштың роторы электромагниттермен жасалған өрістердің ішінде айналатын тұрақты магнит болып табылады. Статор қозғалтқыш корпусында орналасқан бірнеше катушкалардан тұрады. Ток катушкалар арқылы өткен кезде қозғалтқыштың қозғалатын білігі (негізінен тұрақты магнит болып табылады) электромагниттік катушкалар тудыратын өріске сәйкес орналасады. Катушкалар белгілі бір ретпен зарядталғанда, қозғалтқыш білігі жаңа позицияларды таңдайды және сәйкесінше айнала бастайды.



Қадамдық қозғалтқыштар

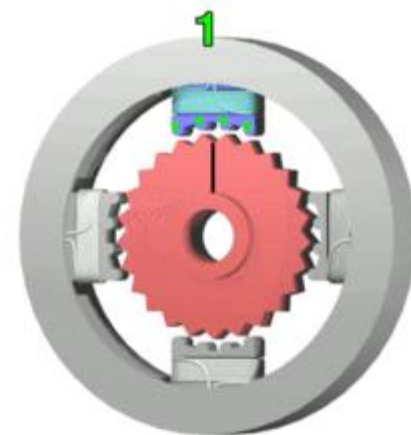
Ротор катушкаларға кернеуді дәйекті түрде беру арқылы қозғалады. Қадамдық қозғалтқыш импульстің рұқсат ету қабілетіне байланысты роторды белгілі бір қадамға айналдыру мүмкіндігіне ие.

Қадамдық қозғалтқыштар бірнеше себептерге байланысты Arduino жобалары (және т.б.) үшін тамаша таңдау болып табылады. Олар салыстырмалы түрде арзан, оларда аз қате бар, сондықтан олар ашық басқару жүйесімен басқару үшін тамаша таңдау болып табылады (қосымша позиция сенсорлары жоқ). Қадамдық қозғалтқыштар тек қолданылатын токқа байланысты ротордың берілген орнын қамтамасыз етеді.



Қадамдық қозғалтқыштар

Бұл қозғалтқыштар бір және қарама-қарсы бағытта айналуға арналған. Егер сіз қадамдық қозғалтқыштың түйреуіштеріне қуат көзін қоссаңыз, білік айнала бастайды. Өткізгіштерді керісінше қоссаңыз, ол кері бағытта айналады. Рас, кейбір қадамдық қозғалтқыштарда қарама-қарсы бағытта айналу мүмкін емес екенін ескерген жөн. Оны іске қоспас бұрын осы нүктені тексеріңіз.



Сервомоторлар



Модельдеушілер үшін кәдімгі сервомоторлар 0-ден 180 градусқа дейін белгілі бір позицияны айналдыру және ұстау үшін қолданылады. Осыған байланысты олар робототехникада, позициялау жетектерінде кеңінен қолданылады. Өндірісте сервомоторлар автомобильдердің, қайықтардың рульдік модульдерінде және заманауи бейнекамераның фокустау механизмдерінде қолданылады.

Көптеген жағдайларда сервоқозғалтқыштарда үш сым бар: қуат, жерге және сигнал. Әдетте электр сымы қызыл, жер қара немесе қоңыр болады. Сигнал сары, қызғылт сары немесе ақ.

Сервомоторлар



Сервомоторларда, мысалы, радиомен басқарылатын машиналарды басқару жүйелерінде қолданылатын электр қозғалтқышы потенциометрге қосылады. Стандартты қабылдағыш/таратқыш PWM сигналдарын сервоға жібереді. Сервомотордың ішіндегі электроника (шағын контроллер тақтасы) импульс енін позицияға аударады. Сервоға айналу сигналы берілгенде, потенциометр қабылдағыш/таратқыш арқылы алдын ала орнатылған мәнге сәйкес келетін орынға жеткенше қуат қозғалтқышқа беріледі.

Сервомоторлар



Басқару сигналы 50 Гц жиілігі бар цифрлық PWM сигналы болып табылады. Әрбір 20 миллисекунд сайын сандық басқару импульсі қолданылады. Импульстің ұзақтығы (ені) 1,0 миллисекундтан 2,0 миллисекундқа дейінгі аралықта. 1,5 - диапазонның ортасы. Үлкен импульс ендерін қалыпты жұмыс ауқымынан тыс қосымша қозғалыс үшін пайдалануға болады. PWM (импульстік ең модуляциясы) сигналы кейде PPM (позиция енін модуляциялау) деп аталады, бірақ бұл дұрыс емес.

Сервомоторлар



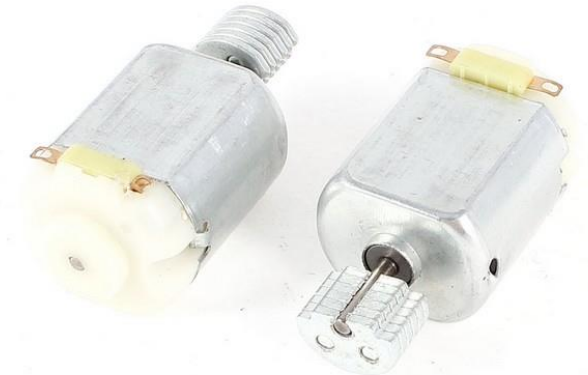
1,5 миллисекундтық импульс әдетте сервомоторды «бейтарап» күйге немесе 90 градусқа орнатады. 1,25 миллисекундтық импульс оны 0 градусқа, ал 1,75 миллисекундтық импульс оны 180 градусқа орнатуы мүмкін. Әртүрлі сервомоторлар үшін физикалық шектеулер мен уақыт әртүрлі болуы мүмкін, бірақ оның бейтарап жағдайы әрқашан 1,5 миллисекундқа сәйкес келеді.

Вибрациялық қозғалтқыштар

Қозғалыстың болуын анықтау үшін мобильді құрылғыларда жиі діріл қозғалтқыштары қолданылады.

Діріл қозғалтқыштары конструкциясы бойынша көптеген қадамдық қозғалтқыштарға және тұрақты ток қозғалтқыштарына ұқсас.

Айырмашылық - ротордың соңында эксцентрик орнатылған. Ротор қозғалған кезде эксцентрик қозғалтқыш корпусының ішіндегі механизмнің дірілдеуіне әкеледі. Дірілдің қарқындылығы қозғалтқыштың өлшеміне де, эксцентрик өлшеміне де байланысты.



Қозғалтқыштардың жүрісін реттегіш

Біз микроконтроллердің өте «сезімтал» және «нәзік» құрылғы екенін білеміз. Ол шағын токтар мен кернеулермен жұмыс істейді. Сонымен, қазіргі заманғы микроконтроллерлердің көпшілігі және жалпы компьютерлер 5 вольт, 3,3 вольт немесе одан да аз кернеумен жұмыс істейді. Бұл ретте типтік микросұлбаларда айналатын ток 1020 миллиамперден аспайды.

Қозғалтқыштардың жүрісін реттегіш

Микроконтроллердің көмегімен қуатты робот қозғалтқыштарын басқару қажет болса, не істеуіміз керек? Өйткені, тіпті біздің курсымыздағы шағын қозғалтқыштар 56 вольт кернеуді және жүздеген миллиамперге дейінгі ток күшін қажет етеді. Сонымен бірге қозғалтқышты іске қосу сәтінде ол арқылы күші бір амперден асатын ток өтеді. Мұндай жүктеме кез келген микроконтроллерді бір секундта бұзады.

Қозғалтқыштардың жүрісін реттегіш

Бұл проблеманы шешу үшін қозғалтқыштардың жүріс реттегіштері деп аталатын арнайы құрылғылар қолданылады. Мұндай құрылғылар әлсіз микроконтроллердің қатаң бақылауымен қуатты қозғалтқышқа үлкен токтарды беруге мүмкіндік береді. Біздің жұмысымызда біз екі арналы DRV8833 реттегішін қолданамыз, оның қорытындылары төменде келтірілген.

Қозғалтқыштардың жүрісін реттегіш



GND жер;

VIN қозғалтқышты қоректендіру кернеуі 2,710,8 В;

AIN1, AIN2 №1 арнаны басқару (контроллерге);

BIN1, BIN2 №2 арнаны басқару (контроллерге);

AOUT1, AOUT2 №1 арна шығыстары (қозғалтқышқа) ;

BOUT1, BOUT2 №2 арна шығыстары (қозғалтқышқа) ;

AISEN, BISEN ток шектегішін теңшеу (қосылмаймыз);

nSLEEP ұйықтау режиміне ауыстыру (белсенді төмен, қосылмаймыз);

nFAULT қате сигналы (қосылмаймыз).

Қозғалтқыштардың жүрісін реттегіш

Қозғалтқыштың айналу бағытын орнату үшін әрбір арнаны басқаруға арналған екі шығыс қажет. Мысалы, AOUT1 істікшесіне оң сигнал, ал AOUT2 істікшесіне нөлдік сигнал жіберілсе (яғни жерге қосылған), онда қозғалтқыш бір бағытта айналады. Керісінше, оң сигналды AOUT2-ге, ал нөлді AOUT1-ге беру арқылы біз қозғалтқыштың айналу бағытын өзгертеміз.

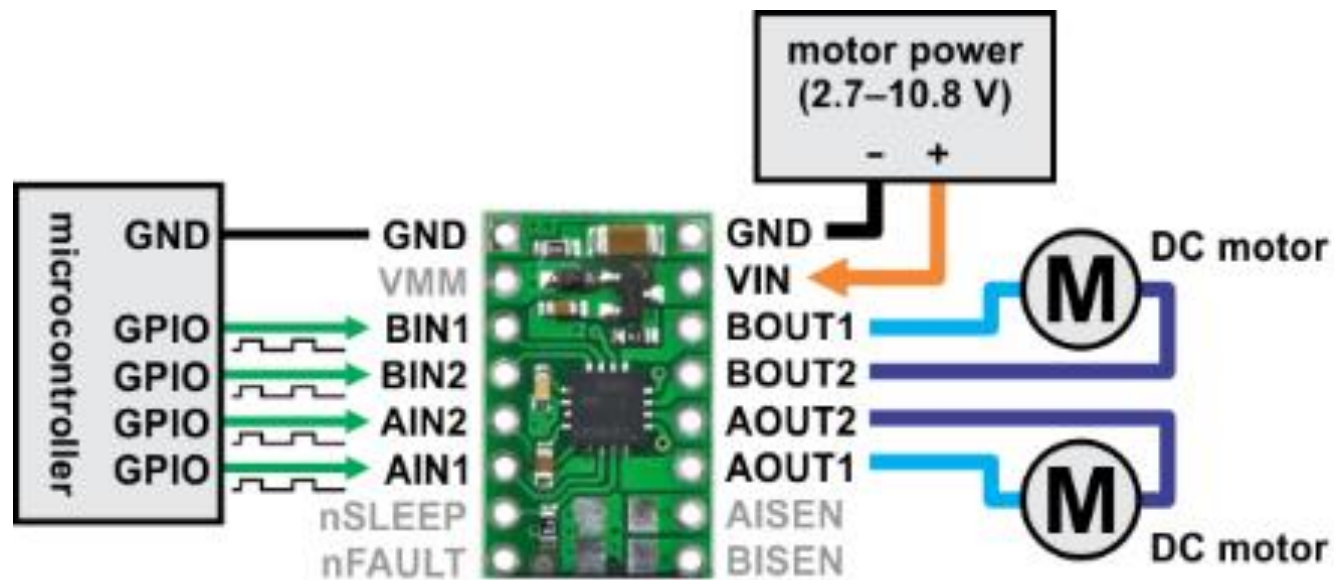
Айналу жылдамдығын басқару үшін PWM пайдалану

Алдыңғы сабақтарда айтылғандай, құрылғылардың кернеуін біркелкі басқару үшін күрделі PWM сигналдарын пайдалана аламыз. Қозғалтқыштардың айналу жылдамдығын басқару үшін біз жарықдиодты біркелкі өшіріп, қосу үшін қолданған тәсілді қолданамыз.

Мысал ретінде эксперимент жүргізейік: бағдарламаны іске қосқаннан кейін қозғалтқыш кезекпен бір бағытта, содан кейін екінші бағытта айналуы керек.

Ауыстыру кезеңі бес секунд.

Айналу жылдамдығын басқару үшін PWM пайдалану



Мысал

```
int motA_1 = 3;
int motA_2 = 5;
void setup(){
  pinMode(motA_1, OUTPUT);
  pinMode(motA_2, OUTPUT);
}
void loop(){
  analogWrite( motA_1, 0 );
  analogWrite( motA_2, 128 );
  delay(2500);
  analogWrite( motA_2, 0 );
  analogWrite( motA_1, 128 );
  delay(2500);
}
```

Сервомоторды басқару кітапханасы

Arduino IDE-де сервосерво қозғалтқыштармен жұмыс істеуге арналған арнайы кітапхана бар.

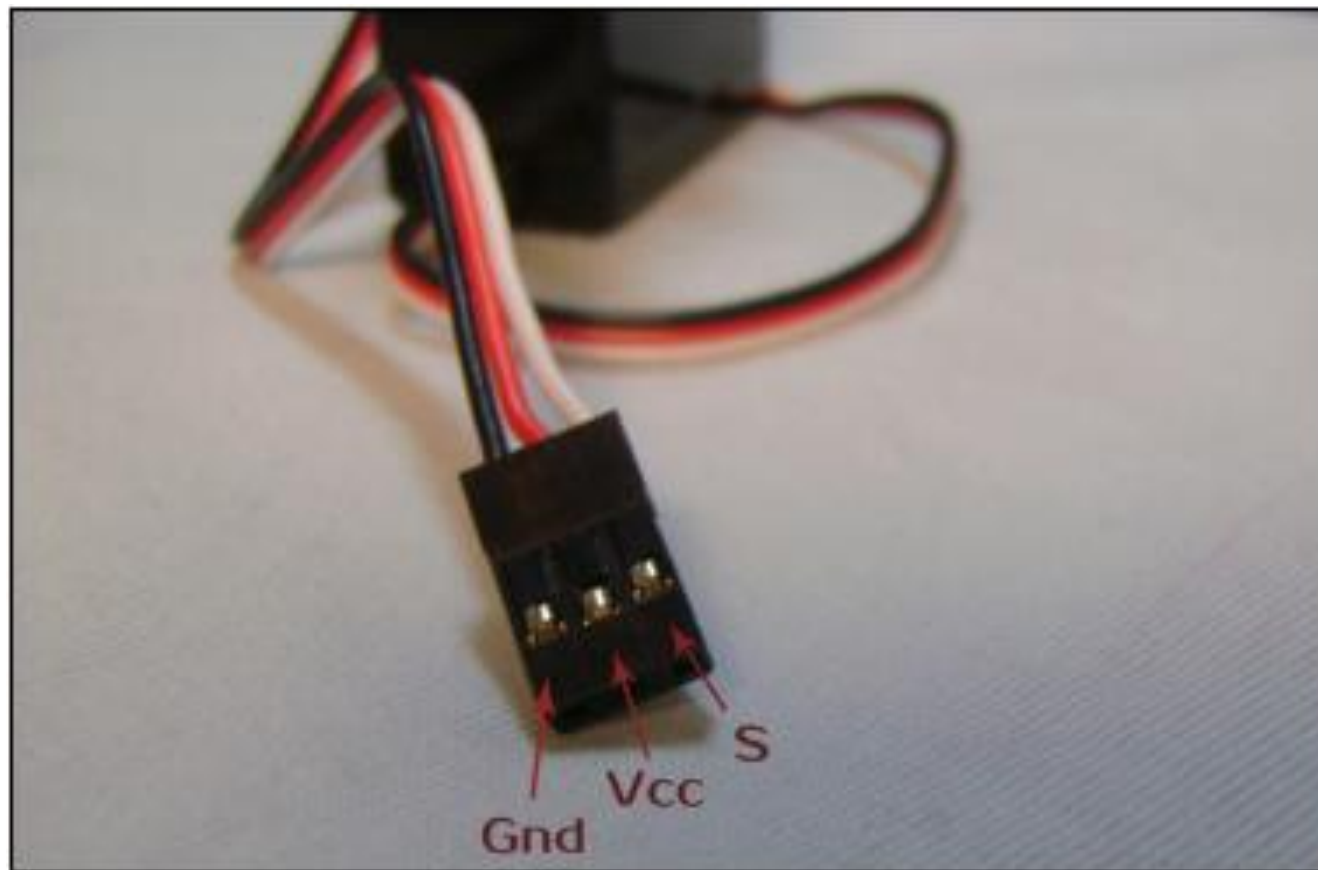
Mega емес тақталарда кітапхананы пайдалану 9 және 10 түйреуіштерде (серволар осы түйреуіштерге қосылған-қоспағанына қарамастан) `analogWrite()` (PWM) пайдалану мүмкіндігін ажыратады. Mega тақталарында PWM функциясына әсер етпестен 12 сервоға дейін пайдалануға болады, бірақ 12-ден 23-ке дейінгі серволарды пайдалану 11 және 12 түйреуіштердегі PWM-ді өшіреді. Серво үш сыммен қосылған: қуат (Vcc), жерге (Gnd) және сигнал (C) . Қуат - қызыл сым, оны Arduino тақтасындағы +5 В істікшесіне қосуға болады. Қара немесе қоңыр сым – «жер» Arduino GND істікшесіне қосылған, сигналдық (қызылт сары/сары/ақ) сым Arduino контроллерінің цифрлық істікшесіне қосылған. Айта кету керек, қуатты серволар үлкен жүктемені жасай алады - бұл жағдайда оларды бөлек қуаттандыру керек (Arduino +5V шығысы арқылы емес). Бір уақытта бірнеше серволарды қосқан жағдайда да солай.

Сервомоторды басқару кітапханасы

Servo кітапханасы функцияларының жиыны:

- `attach(int);`
- `detach();`
- `write(int);`
- `writeMicroseconds(int);`
- `read();`
- `attached();`

Сервомоторды қосу



attach() функциясы

attach() функциясы серво айнымалы мәнін қозғалтқышты басқаратын көрсетілген шығысқа қосады.

Синтаксис:

- servo.attach(pin)
servo.attach(pin, min, max)

Параметрлер:

- pin – Сервожетекті қосатын және жетек басқарылатын шығыстың нөмірі;
- min – сервожетекінің минималды жағдайына сәйкес келетін (0 градус бұрыш) микросекундтағы импульс ені;
- max – сервожетекінің максималды жағдайына сәйкес келетін (180 градус бұрыш) микросекундтағы импульс ені .

detach() функциясы

detach() функциясы серво айнымалыны attach() функциясы арқылы қосылған шығыстан ажыратады. Егер барлық серво айнымалылар өшірілсе, 9 және 10 түйреуіштерді analogWrite() көмегімен PWM режимінде пайдалануға болады.

Синтаксис:

```
servo.detach()
```

Параметрлер жоқ.

Қайтару мәні жоқ.

write(int) функциясы

write(int) функциясы жетекіні басқару үшін мәндерді береді. Стандартты серво үшін бұл айналу бұрышы. Тұрақты айналу жетегі үшін функция айналу жылдамдығын орнатады: бір бағытта максималды айналу жылдамдығы үшін 0, екінші бағытта максималды айналу жылдамдығы үшін 180 және стационарлық күй үшін шамамен 90.

Синтаксис:

- servo.write(angle)

Параметр:

- **angle** – сервоға жазылған мән (0-ден 180-ге дейін).
- Қайтару мәні жоқ.

writeMicrosconds(int) функциясы

writeMicrosconds(int) функциясы сервоны микросекундтарда (uS) басқару үшін мәнді жібереді, айналу бұрышын сол мәнге орнатады. Стандартты диск үшін: 1000 мәні - сағат тіліне қарсы максималды айналу, 2000 - сағат тіліне қарсы максималды айналу, 1500 - ортасында.

Синтаксис:

- servo.writeMicrosconds (uS)

Параметр:

- uS – микросекундтағы мән .
- Қайтару мәні жоқ.

read() функциясы

read() функциясы сервомотордың ағымдағы орнының мәнін оқиды (write() функциясына соңғы шақыру арқылы жазылған мән).

Синтаксис:

```
servo.read()
```

Параметрлер жоқ.

Қайтарылатын мән:

angle — сервомотордың позициясы (бұрышы) 0-ден 180-ге дейін.

attached() функциясы

attached () функциясы Servo айнымалысының шығуға қосылғанын тексереді.

Синтаксис:

- servo.attached()
- Параметрлер жоқ.

Қайтарылатын мәндер:

- true - егер қосылған болса, false - олай болмаған жағдайда.

Мысал

```
#include <Servo.h>
```

```
Servo myservo;
```

```
int servo_pin = 9;
```

```
void setup() {
```

```
myservo.attach( servo_pin ); // серво қозғалтқышты 9 контактімен  
байланыстырамызArduino
```

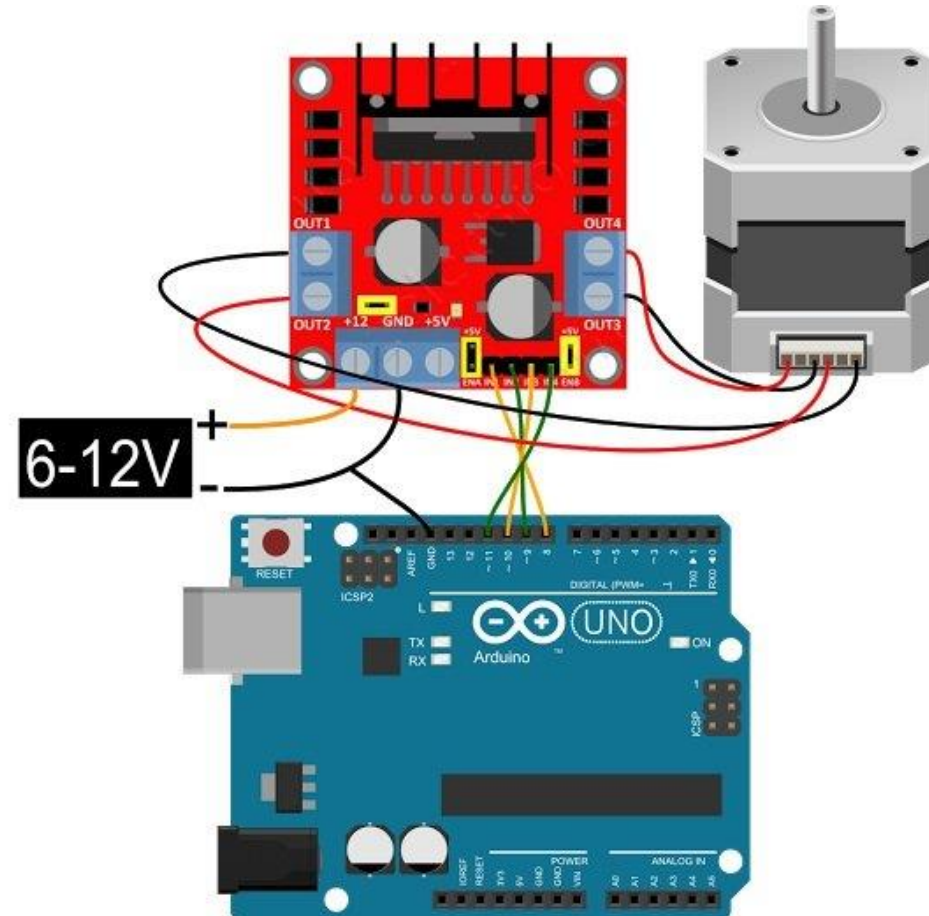
```
myservo.write( 90 ); // қозғалтқыш білігін + 90 градусқа бұрамыз
```

```
}
```

```
void loop(){
```

```
}
```


Қадамдық қозғалтқышты қосу



Stepper Arduino кітапханасы

Arduino-да қадамдық қозғалтқышты басқару үшін стандартты Stepper кітапханасы бар. Оның келесі мүмкіндіктер жинағы бар:

```
Stepper();  
setSpeed();  
step().
```

Stepper() функциясы

Stepper() функциясы Arduino-ға қосылған жалғыз қадамдық қозғалтқышқа байланыстырылған жаңа Stepper нысанын жасайды. Конструкторды Stepper класының айнымалы мәнін жариялау кезінде пайдалану керек, әдетте ең басында – setup() және loop() сыртында. Параметрлердің саны қосылу әдісіне байланысты - қозғалтқышты басқару үшін 2 немесе 4 шығыс пайдаланылады.

Stepper() функциясы

Синтаксис:

- Stepper(steps, pin1, pin2)
- Stepper(steps, pin1, pin2, pin3, pin4)

Параметрлер:

- steps – пайдаланылған қозғалтқыштың толық айналымындағы қадамдар саны. Қозғалтқыш құжаттамасында бір қадамның бұрышы көрсетілсе, біз осы бұрышқа 360° бөлуіміз керек, бұл бізге қадамдардың қажетті санын береді;
- pin1, pin2, pin3, pin4 – Қадамдық қозғалтқышты қосуға арналған Arduino шығыстары.

Қайтарылатын мән: Stepper класының жаңа данасы.

setSpeed() функциясы

setSpeed(rpm) функциясы айналу жылдамдығын минутына айналыммен орнатады. Бұл функция қозғалтқышты айналдырмайды, тек step() функциясын шақырған кезде пайдаланылатын айналу жылдамдығын орнатады.

Синтаксис:

- Stepper.setSpeed(rpm)

Параметр:

rpm – қадамдық қозғалтқыштың айналымы жүргізілетін жылдамдық минутына айналыммен көрсетіледі.

Қайтару мәні жоқ.

step (steps) функциясы

Қадам(қадамдар) функциясы қадамдық қозғалтқышты `setSpeed()` функциясымен көрсетілген жылдамдықпен қадамдардың белгілі бір санын айналдырады. Бұл функция блоктауда, яғни кодтың келесі жолына басқаруды бермес бұрын қозғалтқыштың айналуын күтеді. Ұзақ уақыт бойы эскиздік кодтың орындалуына тосқауыл қоймау үшін басқару жылдамдығы жоғары болатындай етіп ұйымдастырылуы керек және бір `step()` шақыруында бірнеше қадамдар ғана орындалады.

step (steps) функциясы

Синтаксис:

- Stepper.step(steps)

Параметр: steps – қадамдар саны:

- оң сан - бір жаққа айнаруды тудырады;
- теріс - қарама-қарсы.

Қайтару мәні жоқ .

Мысал

```
#include <Stepper.h>
const int stepsPerRevolution = 200;
Stepper myStepper(stepsPerRevolution, 8,9,10,11); //Arduino-дағы 8 ... 11 түйреуіштерге қосылу
void setup() {
  myStepper.setSpeed(60); //ротор жылдамдығын орнату
  Serial.begin(9600);}
void loop() {
  Serial.println("Move right"); //сағат тілімен
  myStepper.step(stepsPerRevolution);
  delay(1000);
  Serial.println("Move left"); //сағат тіліне қарсы
  myStepper.step(-stepsPerRevolution);
  delay(1000);
}
```

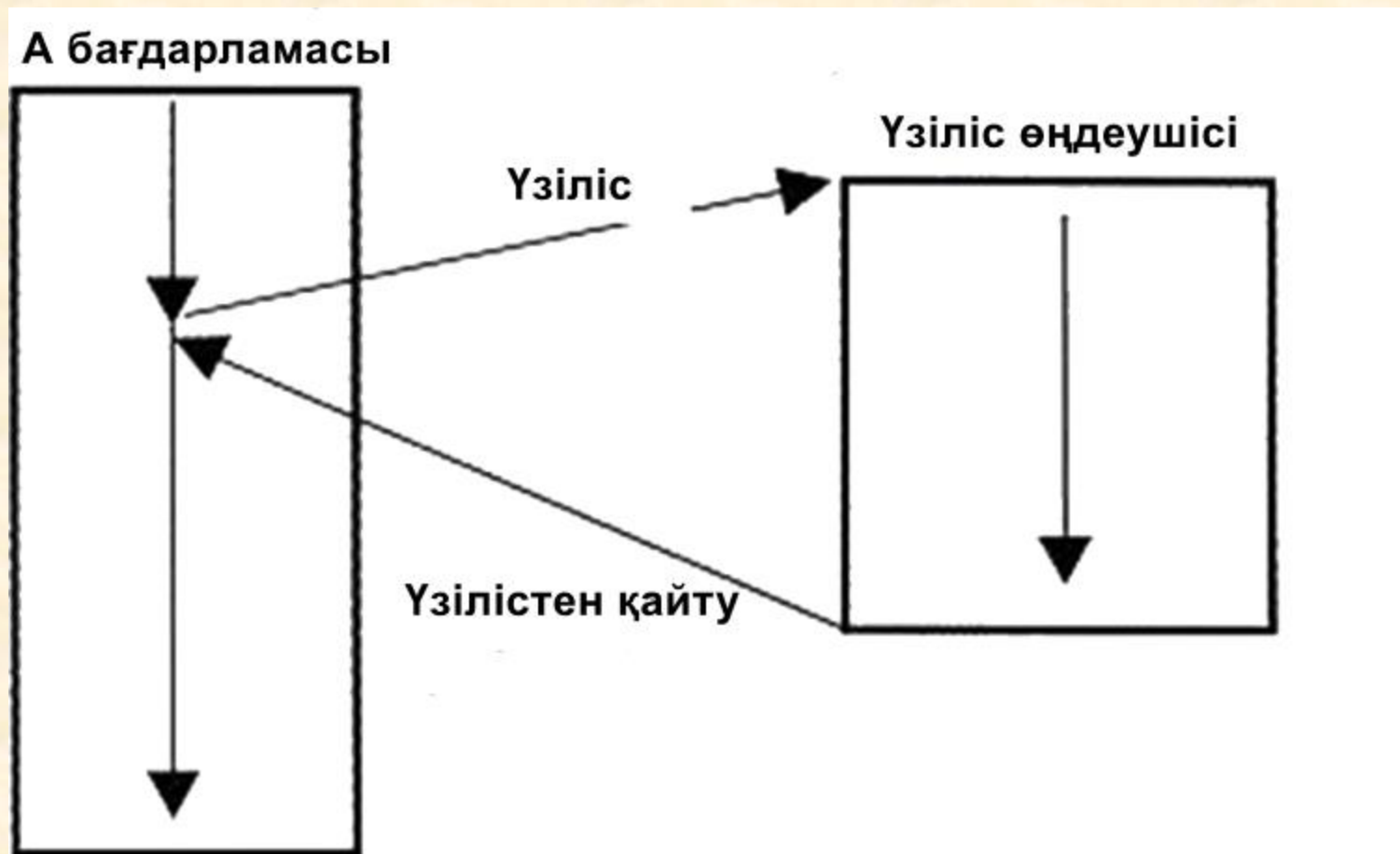

Arduino үзілістері

Үзіліс

Үзіліс - бұл дереу назар аударуды қажет ететін қандай да бір басым оқиғаның орын алуын көрсететін аппараттық құралдан сигнал. Үзіліс сұрауына жауап ретінде процессор ағымдағы кодтың орындалуын тоқтатады және арнайы функцияны шақырады - үзу қызметінің тәртібі (ISR). Бұл функция белгілі бір әрекеттерді орындау арқылы оқиғаға әрекет етеді, содан кейін ол басқаруды үзілген кодқа қайтарады.

Үзіліс жүйесі процессорлардың архитектурасына оларды қандай да бір оқиғаны күту кезінде перифериялық құрылғыларды үнемі сұрау қажеттілігінен босату үшін енгізілді. Бұл процессор баяу құрылғылармен әрекеттескенде, әсіресе дұрыс. Үзілістерді пайдаланған жағдайда құрылғының өзі процессорға оқиғаның орын алғаны туралы хабарлайды және сол уақытқа дейін процессор басқа жұмыстарды орындай алады. Бұл тәсіл процессорлық жүйелердің өнімділігін айтарлықтай арттыруға мүмкіндік берді.

Үзіліс тізбегі



Arduino-да үзілістер

- **Аппараттық үзілістер.**
- **Бағдарламалық қамтамасыз ету үзілістері (шын мәнінде бағдарламалық құралдағы аппараттық үзулерді шақыру).**
- **Ішкі (синхронды) және сыртқы (асинхронды) үзілістер.**

Сыртқы үзілістер

Бірден айта кету керек, AVR микроконтроллерлерінде сыртқы үзілістердің екі түрі бар. Бірінші түрдегі үзілістер сыртқы ұзу кірісінде көрсетілген сигнал пайда болған кезде жасалады. Әдетте, сыртқы үзілістер туралы сөйлескенде, бұл түрді білдіреді. Екінші типті үзілістер микроконтроллердің белгілі бір түйреуіштерінің күйінің кез келген өзгерісімен жасалады, сондықтан олардың атауы - Pin Change Interrupt.

Көптеген Arduino тақталарында (әсіресе ATmega328/P негізіндегі) екі сыртқы ұзу кірісі бар: INT0 және INT1. Микроконтроллер олардан келетін ұзу сұрауларына жауап бере бастауы үшін келесі қадамдарды орындау керек:

1. Ұзуді өңдеуші функциясын жазыңыз. Ол параметрлерсіз болуы керек және ешқандай мәндерді қайтармауы керек.
2. Сыртқы ұзуді қосыңыз және оған өңдегішті орнатыңыз. Осы мақсаттар үшін `attachInterrupt` функциясы қолданылады.

Әр түрлі Arduino тақталарында үзілістерді жүзеге асыру арасындағы айырмашылық

Плата	INT0	INT1	INT2	INT3	INT4	INT5
UNO және ATmega328/P негізіндегі басқалар	2	3				
Leonardo	3	2	0	1	7	
Mega2560	2	3	21	20	19	16

Arduino DUE барлық енгізу/шығару желілеріндегі үзулерді қолдайды. Ол үшін `digitalPinToInterrupt` функциясын пайдалана алмайсыз және пин нөмірін тікелей `attachInterrupt` параметрлерінде көрсете алмайсыз.

attachInterrupt() функциясы

AttachInterrupt функциясы микроконтроллерге қандай оқиғаларға жауап беру керектігін және оларға қандай өңдеуші сәйкес келетінін айтады.

Шақыру синтаксисі:

- attachInterrupt(interrupt, function, mode)

Функция аргументтері:

- interrupt - шақырылған үзіліс нөмірі (стандартты 0 - 2-ші түйреуіш үшін, Arduino Uno тақтасы үшін 1 - 3-ші істікше үшін),
- function – үзілген кезде шақырылатын функцияның аты (маңызды – функция ешқандай мәндерді қабылдамауы немесе қайтармауы керек),
- mode – үзілістің іске қосылу шарты.

Келесі іске қосу шарттарын орнатуға болады

- LOW – контакт нөлге тең болғанда төмен сигнал деңгейінде орындалады. Үзіліс циклді түрде қайталануы мүмкін - мысалы, түйме басылғанда.
- CHANGE – шетінде сигнал жоғарыдан төменге немесе керісінше өзгерген кезде үзіліс орын алады. Кез келген сигналды өзгерту кезінде бір рет орындалады.
- RISING – Сигнал «LOW» күйден «HIGH» күйге ауысқанда үзуді бір рет орындаңыз .
- FALLING – Сигнал «HIGH» күйден «LOW» күйіне өзгерген кезде үзуді бір рет орындаңыз.

attachInterrupt

attachInterrupt функциясына әрбір жаңа қоңырау жаңа үзу өңдеушісін байланыстырады, яғни басқа өңдеуші функциясы бұрын тіркелген болса, ол енді шақырылмайды. Жағдай микроконтроллер жауап беруі керек оқиғалардың түрін анықтайтын режим параметрімен ұқсас. Басқаша айтқанда, сіз, мысалы, бірдей сыртқы үзу кірісі үшін FALLING және RISING үшін бөлек өңдеушілерді анықтай алмайсыз.

Мысал

```
#define ledPin 13
#define interruptPin 2 // Сандық істікшелі 2 және GND арасындағы түйме
volatile byte state = LOW;
void setup() {
    pinMode(ledPin, OUTPUT);
    pinMode(interruptPin, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(interruptPin), blink, FALLING);
}
void loop() {
    digitalWrite(ledPin, state);
}
void blink() {
    state = !state;
}
```

Жалпы алғанда, эскиз жоспарланғандай жұмыс істейді, бірақ сіз кейде түймені басқан кезде жарық диоды жауап бермейтінін байқайсыз. Бұл әрекет түйме контактілерінің серпілуінен туындайды: 2-сандық кірістегі сигналдың бірнеше рет өзгеруі өңдеушіге қайталанатын қоңырауларға әкеледі.

«setup» функциясында біз кірістірілген жарық диодты басқару үшін шығыс ретінде он үшінші пинді орнаттық. Біз екінші түйреуішті қуатқа тартамыз, бұл оған жоғары деңгейлі сигнал береді. Әрі қарай, attachInterrupt функциясын пайдаланып, біз жыпылықтауды өңдеу функциясын орнатамыз, ол екінші түйреуіштегі сигнал жоғарыдан төменге (FALLING) өзгерген кезде шақырылуы керек. Жыпылықтау функциясының ішінде біз күй айналымының мәнін өзгертеміз, ол кейіннен цикл функциясында жарық диоды қосады және өшіреді. Осылайша, негізгі бағдарламадағы түймені сұраусыз жарық диодты басқаруға болады.

detachInterrupt() функциясы

detachInterrupt() функциясы үзіліс өңдеушісі ретінде белгілі бір функцияны өшіруге мүмкіндік береді. Осыдан кейін сіз жаңа үзіліс өңдеуші функциясын тағайындай аласыз.

- detachInterrupt(0); //үзіліс 0 өшіріледі

volatile

volatile — компиляторға айнымалы мәннің кез келген уақытта өзгеруі мүмкін екенін айтатын айнымалы түрінің квалификациясы.

Компилятор орындалатын кодты құру және оңтайландыру кезінде осы фактіні ескереді. Ұшқыштың мақсатын дерексіз түрде түсіндірмеу үшін келесі код үзіндісі бойынша компилятор жұмысын қарастырайық:

```
byte A = 0;
```

```
byte B;
```

```
void loop() {
```

```
    A++;
```

```
    B = A + 1;
```

```
}
```

А және В айнымалылары микроконтроллердің жадындағы орындар болып табылады.

Микроконтроллер олармен бірдеңе істеуі үшін

(өзгерту, салыстыру және т.б.) олардың мәндері жадтан ішкі регистрлерге жүктелуі керек. Сондықтан, осы фрагментті құрастыру кезінде келесі код жасалады:

1. А мәнін жадтан P1 регистріне жүктеңіз
2. P2 регистріне 1 тұрақтысын жүктеңіз
3. P2 мәнін P1 арқылы қосыңыз (нәтиже P2)
4. P2 регистрінің мәнін жадта А мекенжайында сақтаңыз
5. 2 тұрақтысы бар P1 регистрінің мазмұнын қосыңыз
6. P1 регистрінің мәнін В мекенжайы бойынша жадта сақтаңыз

volatile

А айнымалысының мәні регистрде кодтың ең басында оқылады. Егер жоғарыда аталған қадамдардың бірінде өңдеу кезінде А айнымалысының мәні (жад ұяшығы) өзгертілетін үзу сұрауы алынса, онда цикл функциясына оралғаннан кейін микроконтроллер өзінің маңызды емес мәнімен жұмыс істейді. тіркелім. Құбылмалы квалификацияны пайдалану мұндай жағдайларды болдырмауға мүмкіндік береді. Ұшқыш деп жарияланған айнымалыға қол жеткізу кезінде микроконтроллер әрқашан оның ағымдағы мәнін жадтан оқиды және бұрын оқылғанды пайдаланбайды (яғни компилятор мұндай кодты жасайды).

Әрине, ұшпаның болмауы әрқашан қатеге әкелмейді, бәрі бағдарламаның логикасына байланысты. Сонымен, жарықдиодты басқару үшін үзілістерді пайдаланудың жоғарыдағы мысалы ұшпасыз жұмыс істейді. Бірақ басқа жағдайларда бұл квалификацияны пайдалану табу қиын қателерді болдырмауға көмектеседі. Сондықтан үзіліс өңдеушісі басқа функциялармен бөлісетін айнымалы мәндерді жариялау кезінде әрқашан өзгермелі мәнді пайдалануды ережеге айналдырған дұрыс.

Мысал

```
#define interruptPin 2
volatile byte f = 0;
void setup() {
    pinMode(interruptPin, INPUT_PULLUP);

    attachInterrupt(digitalPinToInterrupt(interruptPin), buttonPressed, FALLING);
}
void loop() {
    while (f == 0) {
        // Түйменің басылуын күтіп тұрғанда бірдеңе жасаңыз
    }
    // Түйме басылды
}
void buttonPressed() {
```

```
    f = 1;
}
```

Цикл функциясының ішіндегі цикл f айнымалысының мәні нөлге тең болғанша орындалуы керек. Және ол түймені басқан кезде үзу өңдегішінде өзгеруі керек. Егер f айнымалысын ауыспалы квалификациясыз жарияласақ, онда цикл ішіндегі айнымалының мәні өзгермейтінін және циклден шығу шарты жалған болып қала беретінін «көрген» компилятор циклді шексізге ауыстырар еді. Компиляция кезінде кодты оңтайландыру осылай жұмыс істейді. Мұндай циклге кіру микроконтроллер жай ғана қатып қалады. Айнымалыны тұрақсыз жіктеуішпен жариялау айнымалының оңтайландырылған қатынас түрін қабылдамауын қамтамасыз етеді.

Ескертпе

Өңдеуші мен негізгі бағдарлама арасында үзілістермен жұмыс істегенде және айнымалы мәндерді ортақ пайдалану кезінде есте сақтау керек өте маңызды мәселе AVR микроконтроллерлері 8 биттік және жадтан 16 биттік немесе 32 биттік мәнді жүктеу үшін бірнеше бөлек операциялар қажет. Сәйкесінше, микроконтроллер жадтан айнымалының төменгі байттарын оқығанда, үзу сұрауы қабылданатын және өңдеуші осы айнымалының мәнін өзгертетін жағдай мүмкін. Негізгі бағдарламаға оралғаннан кейін микроконтроллер жадтан жоғары байтты оқиды, ескі мәнді ғана емес, мүлде басқа мәнді қабылдайды, бұл бағдарламада қатеге әкелуі мүмкін. Мұндай жағдайларды болдырмау үшін үзілістер және noInterrupts функцияларын пайдаланып ортақ айнымалы мәндерге қатынасу кезінде үзілістерді өңдеуді өшіруге болады немесе мұндай айнымалыға рұқсатты атомдық түрде орындалатын код блогында орналастыруға болады.

interrupts() және noInterrupts() функциялары

```
void loop()  
{  
  // нұсқаулар, бағдарламалар  
  noInterrupt();  
  // үзулерді өшіруді талап ететін маңызды бөлім  
  interrupt();  
  // нұсқаулар, бағдарламалар
```


Атомдық түрде орындалатын код блоктары

Атомдық және атомдық емес орындалатын код блоктарымен жұмыс істеуге арналған конструкциялар AVR Libc бөлігі болып табылады. Олар атомдық (яғни үзіліссіз) немесе атомдық емес түрде орындалуына кепілдік берілген блоқты анықтауға мүмкіндік береді.

Атомдық түрде орындалатын код блоктары

```
#include <util/atomic.h>
#define interruptPin 2
volatile unsigned int counter = 0;
void setup() {
    pinMode(interruptPin, INPUT_PULLUP);

    attachInterrupt(digitalPinToInterrupt(interruptPin), myISR, FALLING);
}
void myISR() {
    counter++;
}

void loop() {
    unsigned int counter_copy;
    ATOMIC_BLOCK(ATOMIC_FORCEON){
        // 2x байттық counter айнымалысына
        // атомарлық қол жеткізу:
        counter_copy = counter;
    }
    // Одан әрі жұмыс істеу үшін мәнді
    // пайдаланамыз counter_copy
    // if (counter_copy == ...)
}
```

Атомдық түрде орындалатын код блоктары

Бұл мысал атомдықты қолдайтын макростарды қамтитын `atomic.h` тақырып файлын қосу арқылы басталады. Содан кейін цикл функциясында екі байт есептегіш айнымалыға қол жеткізіледі. Қоңырау оның атомдылығына кепілдік беретін `ATOMIC_BLOCK` блогында орналастырылған. Үзілістер және `noInterrupts` функциялары сияқты атомдылыққа мұндай блокқа кіру алдында үзілістерді өңдеуді жаһандық түрде өшіру арқылы қол жеткізіледі. Блоктан шыққаннан кейін

`ATOMIC_FORCEON` параметрі жауап беретін үзілістерді өңдеу қосылады. Егер қандай да бір себептермен блокқа кірер алдында үзілістердің қосылғанын немесе қосылмағанын білмесек, блоктан шыққаннан кейін оларды қосуға мәжбүрлеу дұрыс емес шешім болар еді. Мұндай жағдайларда `ATOMIC_BLOCK` макросы `ATOMIC_RESTORESTATE` параметрін пайдалануы керек - ол жаһандық үзуді қосу жалауының алдыңғы мәнін қалпына келтіреді.

Атомдық түрде орындалатын код блоктары

Атомдық емес орындалатын код блоктарын анықтау үшін `NONATOMIC_BLOCK` макросы пайдаланылады және оның параметрі келесі мәндердің бірі болуы мүмкін:

`NONATOMIC_FORCEOFF`,
`NONATOMIC_RESTORESTATE`.

Мұндай блоктың мақсаты бұрын талқыланған нәрсеге қарама-қарсы: ол атомсыз орындалуы

керек блокты анықтайды. Бұл атомдылықты қажет етпейтін үлкен атомдық орындалатын блоктың ішінде фрагменттер болған кезде пайдалы болуы мүмкін.

Контактілер дребезгі

Контактілердің серпілісімен бұрмаланған сыртқы ұзу сигналы кіріске берілгенде, ұзу өңдеушісі бірнеше рет орындалатыны анық. Кейбір жағдайларда сөйлесу қиындық тудырмайды, мысалы, бағдарламада қандай да бір әрекетті орындау үшін түйменің басылуын күтетін болсақ: өңдеушіге кіру кезінде және кодты орындағаннан кейін бұл үзуді бақылауды өшіру жеткілікті. , оны қайтадан қосыңыз (біздің әрекеттеріміздің орындалу уақыты контактілердің серпілу ұзақтығынан асады деп есептеледі). Осылайша біз серпілуді өткізіп жібереміз және өңдеуші тек бір рет орындалады. Тағы бір нәрсе - оқиғаларды тіркеу үшін үзілістер пайдаланылған кезде. Жақсы мысал - механикалық айналу кодтары. Бұл жағдайда біз үзілістерді бақылауды өшіре алмаймыз, өйткені оқиғаны өткізіп алу қаупі бар (кодердегі импульс). Өңдеуші бұл жалған импульс пе, жоқ па деген шешім қабылдай алмайды, және бұл жай ғана оның орындауы керек тапсырма емес. Сондықтан, жалғыз ұтымды шешім - аппараттық дебонжерлерді пайдалану.

Антидребезгі бар түймешікті басу бойынша үзілістер

```
if(digitalRead(2)==HIGH) { //түймені басқанда  
//Алдыңғы басудан кейін 100 миллисекундтан астам уақыт өтсе  
if (millis() - previousMillis >= 100) {  
//Алғашқы операцияның уақыты есте қалады  
previousMillis = millis();
```

```
if (led==oldled) { //түймешік күйінің өзгермегендігі тексерілуде  
led=!led;  
}
```

Ұйқыдан ояту үшін үзілістерді пайдалану

Түймені басу бақыланатын бұрын талқыланған мысалдарды шығыстарды жүйелі түрде сұрау арқылы үзіліссіз жүзеге асыруға болады. Бірақ үзілістерден бас тартуға болмайтын тапсырма бар. Біз микроконтроллердің ұйқы режимінен шығуы туралы айтып отырмыз.

Ұйқы режиміне (қуатты үнемдеу режимі) кірген кезде бағдарламаның орындалуы тоқтатылады. Бірақ микроконтроллердің бірқатар түйіндері, атап айтқанда үзілістерді өңдеудің ішкі жүйесі үзіліс сұрауларының келуін бақылай отырып, жұмысын жалғастыруда. Егер алынған ұзу сұранысын өңдеу қосылса, микроконтроллер оянады және сәйкес өңдеушінің орындалуына көшеді, содан кейін ол негізгі бағдарламаның орындалуын жалғастырады.

Қуатты үнемдеу режимдері

14.2. Sleep Modes

The following Table shows the different sleep modes, BOD disable ability and their wake-up sources.

Table 14-1. Active Clock Domains and Wake-up Sources in the Different Sleep Modes.

Sleep Mode	Active Clock Domains					Oscillators		Wake-up Sources							Software BOD Disable
	clkCPU	clkFLASH	clkI/O	clkADC	clkASY	Main Clock Source Enabled	Timer Oscillator Enabled	INT and PCINT	TWI Address Match	Timer2	SPM/EEPROM Ready	ADC	WDT	Other I/O	
Idle			Yes	Yes	Yes	Yes	Yes ⁽²⁾	Yes	Yes	Yes	Yes	Yes	Yes	Yes	
ADC Noise Reduction				Yes	Yes	Yes	Yes ⁽²⁾	Yes ⁽³⁾	Yes	Yes ⁽²⁾	Yes	Yes	Yes		
Power-down								Yes ⁽³⁾	Yes				Yes		Yes
Power-save					Yes		Yes ⁽²⁾	Yes ⁽³⁾	Yes	Yes			Yes		Yes
Standby ⁽¹⁾						Yes		Yes ⁽³⁾	Yes				Yes		Yes
Extended Standby					Yes ⁽²⁾	Yes	Yes ⁽²⁾	Yes ⁽³⁾	Yes	Yes			Yes		Yes

Note:

1. Only recommended with external crystal or resonator selected as clock source.
2. If Timer/Counter2 is running in asynchronous mode.
3. For INT1 and INT0, only level interrupt.

Қуатты үнемдеу режимдері

- Idle Mode (күту күйі). Онда процессор тоқтатыла тұрады, бірақ қалған периферия - SPI, USART, аналогтық компаратор, TWI шинасы (I2C), таймерлер/есептегіштер, күзет таймері және ұзу жүйесі жұмыс істей береді. Сондықтан Idle режимінен шығу сыртқы және ішкі үзіліс бойынша мүмкін. Бұл режимнің негізгі артықшылығы МК оянуына әкеп соғатын оқиғаларға тез реакция болып табылады. Басқаша айтқанда, бағдарламаны орындау Idle режимінен жұмыс режиміне өткеннен кейін бірден басталады.
- ADC Noise Reduction Mode (АЦҚ шуылдарын төмендету режимі). Бұл режим өзінің құрамында АЦҚ бар МК модельдерінде ғана болады және оның жұмысы кезінде барлық ықтимал бағыттамадарды азайтуға арналған. Бұл режимде процессор жұмысын тоқтатады және енгізу-шығару синхрондауы ажыратылады, қалған құрылғылар жұмысын жалғастырады. МК-ны осы режимнен шығаруға болады, АЦҚ түрлендіру аяқталғаннан кейін үзілістен басқа: прерывание от сторожевого таймера;
 - TWI интерфейсінен мекенжай сәйкестігін үзу;
 - асинхронды таймерден/санауыштан үзу 2;
 - EEPROM дайын үзу;
 - PIN кодын өзгерту үзілуі
 - INTO және INT1 сыртқы үзілістер;
 - сондай-ақ қалпына келтіру (аппараттық құрал, бақылау таймерінен, BOD тізбегінен).

Қуатты үнемдеу режимдері

- Power down режимі (микро тұтыну режимі) - барлық AVR құрылғыларында бар ең үнемді режим. Бұл режимде барлық ішкі тактілік сигналдар өшіріледі, сәйкесінше синхронды режимде жұмыс істейтін барлық МК түйіндерінің жұмысы тоқтатылады. Бұл режимде жұмысын жалғастыратын жалғыз түйіндер асинхронды МК модульдері болып табылады: бақылау таймері (егер ол қосылған болса), үзілістерді өңдеудің сыртқы ішкі жүйесі және TWI модулінің мекенжайын салыстыру блогы. Қуатты өшіру режимінен ояту қалпына келтіру орын алған кезде (аппараттық құрал, бақылаушы, BOD) немесе келесі үзулердің пайда болуы нәтижесінде мүмкін болады:
- күзетшінің үзілуі;
- TWI интерфейсінен мекенжай сәйкестігін үзу;
- PIN кодын өзгерту үзілуі
- INT0 және INT1 сыртқы үзілістер.

Қуатты үнемдеу режимдері

- Power Save Mode (экономикалық режим). Power Down режимімен бірдей, Power Save режимінде 2-таймер/есептеуіш синхронды және асинхронды режимде жұмысын жалғастыра алады. МК осы режимнен Power Down режимін пайдаланған жағдайдағыдай оқиғаларды, сондай-ақ 2-таймерден/есептегіштен үзілістерді шығара алады.
- Standby Mode (күту күйі). Бұл режимді сыртқы резонаторды пайдаланғанда ғана іске қосу ұсынылады. Ол Power Down режимімен бірдей, сонымен қатар тактілік генератор жұмыс істеуін жалғастырады. Осының арқасында ЖК ояту және жұмыс режиміне өту 6 тактіні ғана қажет етеді.
- Extended Standby Mode (кеңейтілген күту режимі). Standby режимі сияқты бұл режимді сыртқы резонатормен бірге пайдалану ұсынылады. Ол Power Save режимімен бірдей, тактілік генератор жұмыс істеуін жалғастыруда. Осының арқасында ЖК ояту және жұмыс режиміне өту 6 тактіні ғана қажет етеді.

Қуатты үнемдеу режимдері

Arduino-ны аталған режимдердің біріне ауыстыру үшін 2 функцияны орындау жеткілікті: белгілі бір режимді орнату үшін `set_sleep_mode(<режим>)` және `sleep_mode()`. Олар `sleep.h` файлында жарияланған, сондықтан оны эскизіңіздің қосу бөліміне қосу керек.

```
#include <avr/sleep.h>
```

...

```
set_sleep_mode(SLEEP_MODE_PWR_DOWN);
```

```
sleep_mode();
```

Қуатты үнемдеу режимдері

Тұрақтылар режимді көрсету үшін қолданылады :

SLEEP_MODE_IDLE

SLEEP_MODE_ADC

SLEEP_MODE_PWR_DOWN

SLEEP_MODE_PWR_SAVE

SLEEP_MODE_STANDBY

SLEEP_MODE_EXT_STANDBY

Сыртқы үзілістер микроконтроллерді кез келген қуат үнемдеу режимінен оятуы мүмкін.

Мысал

```
#include <avr/sleep.h>
#define ledPin 13
#define interruptPin 2 // Сандық істікшелі 2
және GND арасындағы түйме
void setup() {
    pinMode(ledPin, OUTPUT);
    pinMode(interruptPin, INPUT_PULLUP);
}
void loop() {
    // Біз жарық диодты өшіреміз, FALLING үзуін
    қосамыз және ұйқыға өтеміз:
    digitalWrite(ledPin, 0);

    attachInterrupt(digitalPinToInterrupt(interruptPin), myISR, FALLING);
    set_sleep_mode(SLEEP_MODE_PWR_DOWN);

    sleep_mode();
    // Ояңаннан кейін өңдеуші бірінші
    орындалады. Сосын осында қайтамыз.
    // Үзілісті өшіріңіз, оятуды көрсету үшін жарық
    диодты қосыңыз

    detachInterrupt(digitalPinToInterrupt(interruptPin));
    digitalWrite(ledPin, 1);
    // Міне, түймені басқан кезде орындалуы
    керек код болуы мүмкін
    delay(2000);
}
void myISR() {
    //Ол бос
}
```

Өңдеушіні орындау кезінде сұрауды үзу

Үзуді өңдеушіге өту кезінде барлық басқа үзулерді өңдеу өшіріледі. Біз noInterrupts функциясын шақырған сияқтымыз. Өңдеуші орындалып, шыққаннан кейін үзілістерді өңдеу қайта қосылады. Осылайша, бір өңдеушіні екіншісінің үзуі алынып тасталады (егер бұған арнайы рұқсат болмаса). Осы себепті өңдеушілерде үзіліске негізделген функцияларды шақыру мүмкін емес.

Үзіліс өңдеушілерін жазуға арналған жалпы нұсқаулар

- Біріншіден, ұзу өңдеушілерін мүмкіндігінше қысқа ұстаңыз. Өйткені, олар негізгі бағдарламаның орындалуын үзеді, сонымен қатар басқа үзулерді өңдеуді блоктайды. Мүмкін болса, өңдеуші айнымалы мәнін өзгерту арқылы оқиға болған фактіні ғана түсіруі керек. Ал оқиғаға реакцияның өзі осы айнымалыны талдау кезінде негізгі бағдарламада орындалуы керек.
- Жоғарыда айтылғандай, ұзу өңдеушісіне кіру кезінде басқа үзулерді өңдеуге ғаламдық тыйым салынады. Ал бұл, өз кезегінде, үзілістерді пайдаланатын функциялардың жұмысына әсер етеді. Олармен абай болыңыз. Егер сіз олардың қоңырауларының қауіпсіздігіне сенімді болмасаңыз, оларды өңдеушіде пайдаланудан бас тартқан дұрыс.
- Негізгі бағдарлама мен өңдеуші арасында ортақ айнымалы мәндерді өзгермелі деп жариялау ережесін жасаңыз. Көп байтты айнымалылар жағдайында бұл квалификацияның жеткіліксіз екенін ұмытпаңыз - олармен жұмыс істеу кезінде атомдық түрде орындалатын блоктарды немесе `interrupts/noInterrupts` пайдаланыңыз.

Arduino үзілістері

2-бөлім⁺

Ұзу векторлық кестесі

Үзіліс сұрауы орын алған кезде микроконтроллер негізгі бағдарламаның орындалуын тоқтатады және ұзу өңдеушісіне барады. Бірақ ол дәл осы өңдеушіні қалай табады? Ол үшін үзілістердің векторлық кестесі қолданылады. Әрбір үзілістің өз векторы болады, ол негізінен өңдеуші функциясына өту командасы болып табылады. Бұл кесте әдетте программа жадысының төменгі адресерінде орналасады: қалпына келтіру векторы нөлдік адресіте, одан кейін ұзу векторлары орналасады. Үзіліс көздері микроконтроллердің сыртқы және перифериялық құрылғылары болғандықтан, олардың жиынтығы микроконтроллердің нақты моделіне байланысты.

АТmega328/P үшін үзіліс векторлық кестесі

№	Адрес	Дереккөз	Сипаттама	Вектор
1	0x0000	RESET	Векторды қалпына келтіру	
2	0x0002	INT0	Сыртқы үзіліс 0	INT0_vect
3	0x0004	INT1	Сыртқы үзіліс 1	INT1_vect
4	0x0006	PCINT0	Үзіліс 0, шығыстардың күйінің өзгеруі бойынша	PCINT0_vect
5	0x0008	PCINT1	Үзіліс 1, шығыстардың күйінің өзгеруі бойынша	PCINT1_vect
6	0x000A	PCINT2	Үзіліс 2, шығыстардың күйінің өзгеруі бойынша	PCINT2_vect
7	0x000C	WDT	Watchdog тайм-ауты	WDT_vect
8	0x000E	TIMER2_COMPA	"А" таймер/санауыш T2 сәйкестендіріңіз	TIMER2_COMPA_vect
9	0x0010	TIMER2_COMPB	"В" таймер/санауыш T2 сәйкестендіріңіз	TIMER2_COMPB_vect
10	0x0012	TIMER2_OVF	T2 таймері/есептегіштің толып кетуі	TIMER2_OVF_vect
11	0x0014	TIMER1_CAPT	T1 таймер/санауыш түсіру	TIMER1_CAPT_vect
12	0x0016	TIMER1_COMPA	"А" таймер/санауыш T1 сәйкестендіріңіз	TIMER1_COMPA_vect
13	0x0018	TIMER1_COMPB	"В" таймер/санауыш T1 сәйкестендіріңіз	TIMER1_COMPB_vect

14	0x001A	TIMER1_OVF	T1 таймері/есептегіштің толып кетуі	TIMER1_OVF_vect
15	0x001C	TIMER0_COMPA	"А" таймер/санауыш T0 сәйкестендіріңіз	TIMER0_COMPA_vect
16	0x001E	TIMER0_COMPB	"В" таймер/санауыш T0 сәйкестендіріңіз	TIMER0_COMPB_vect
17	0x0020	TIMER0_OVF	T0 таймері/есептегіштің толып кетуі	TIMER0_OVF_vect
18	0x0022	SPI_STC	SPI тасымалдау аяқталды	SPI_STC_vect
19	0x0024	USART_RX	USART қабылдау аяқталды	USART_RX_vect
20	0x0026	USART_UDRE	USART деректер тізілімі бос	USART_UDRE_vect
21	0x0028	USART_TX	USART тасымалдауы аяқталды	USART_TX_vect
22	0x002A	ADC	ADC түрлендіру аяқталды	ADC_vect
23	0x002C	EE_READY	EEPROM дайындығы	EE_READY_vect
24	0x002E	ANALOG_COMP	Аналогтық компаратордың үзілуі	ANALOG_COMP_vect
25	0x0030	TWI	TWI модулінің үзілуі (I2C)	TWI_vect
26	0x0032	SPM_READY	SPM дайындығы	SPM_READY_vect

Үзіліс векторының кестесі

Қалпына келтіру векторының орны BOOTRST сақтандырғышының мәнімен анықталады. Бұл сақтандырғыш бағдарламаланбаған кезде (бұл оның әдепкі күйі), қалпына келтіру векторы 0x0000 адресіте орналасады және бағдарламаның орындалуы одан басталады. Егер BOOTRST сақтандырғышы бағдарламаланған болса, қалпына келтіргеннен кейін микроконтроллер жүктеуші бөлімінде орналасқан кодты орындауға кіріседі. Arduino және ұқсас тақталар осы принцип бойынша жұмыс істейді: микроконтроллерді қалпына келтіргеннен кейін жүктеуші басқаруды алады, ол біраз уақыт компьютерден командаларды күтеді. Жаңа программа жазу командасын алған кезде жүктеуші оны қабылдап, бағдарлама жадына орналастырады. Осыдан кейін, сондай-ақ берілген уақыт ішінде командалар болмаған жағдайда, жүктеуші басқаруды негізгі бағдарламаға береді.

Үзіліс векторының кестесі

Қалпына келтіру векторымен ұқсастық бойынша үзіліс векторы кестесінің орнын көрсетуге болады: бағдарлама жадының төменгі адрестерінде немесе жүктеуші бөлімінде. Бұл үшін MCUCR регистрінің IVSEL биті жауап береді. Әдепкі бойынша (микроконтроллерді қалпына келтіргеннен кейін) бұл бит 0-ге қалпына келтіріледі және ұзу векторлары 0x0002 мекенжайынан бастап орналасады. IVSEL битін 1-ге орнату арқылы ұзу векторлары жүктеуші бөліміне «беріледі». IVSEL биті кейінірек толығырақ талқыланады. Келесі кестеде BOOTRST және IVSEL әртүрлі комбинациялары үшін қалпына келтіру және ұзу векторларының орны көрсетілген.

Үзіліс векторының кестесі

BOOTRST	IVSEL	Векторлық адресі қалпына келтіру	Үзіліс векторлық кестесінің басталу адресі
1	0	0x0000	0x0002
1	1	0x0000	Жүктеуші бөлімінің бастапқы адресі + 0x0002
0	0	Жүктеуші бөлімінің бастапқы адресі	0x0002
0	1	Жүктеуші бөлімінің бастапқы адресі	Жүктеуші бөлімінің бастапқы адресі + 0x0002

Осыны ескере отырып, ATmega328/P үшін ассемблер тілі бағдарламасының басталуы келесідей болуы мүмкін:

Адресар	Белгілер	Командалар	Комментариялар
---------	----------	------------	----------------

0x0000	jmp	RESET	; Reset
0x0002	jmp	INT0	; IRQ0
0x0004	jmp	INT1	; IRQ1
0x0006	jmp	PCINT0	; PCINT0
0x0008	jmp	PCINT1	; PCINT1
0x000A	jmp	PCINT2	; PCINT2
0x000C	jmp	WDT	; Watchdog Timeout
0x000E	jmp	TIM2_COMPA	; Timer2 CompareA
0x0010	jmp	TIM2_COMPB	; Timer2 CompareB
0x0012	jmp	TIM2_OVF	; Timer2 Overflow
0x0014	jmp	TIM1_CAPT	; Timer1 Capture
0x0016	jmp	TIM1_COMPA	; Timer1 CompareA
0x0018	jmp	TIM1_COMPB	; Timer1 CompareB
0x001A	jmp	TIM1_OVF	; Timer1 Overflow
0x001C	jmp	TIM0_COMPA	; Timer0 CompareA
0x001E	jmp	TIM0_COMPB	; Timer0 CompareB
0x0020	jmp	TIM0_OVF	; Timer0 Overflow
0x0022	jmp	SPI_STC	; SPI Transfer Complete
0x0024	jmp	USART_RXC	; USART RX Complete
0x0026	jmp	USART_UDRE	; USART UDR Empty
0x0028	jmp	USART_TXC	; USART TX Complete

0x002A	jmp	ADC	; ADC Conversion Complete
0x002C	jmp	EE_RDY	; EEPROM Ready
0x002E	jmp	ANA_COMP	; Analog Comparator
0x0030	jmp	TWI	; 2-wire Serial
0x0032	jmp	SPM_RDY	; SPM Ready
			;
0x0034	RESET:	ldi r16,high(RAMEND)	; Main program start
0x0035		out SPH,r16	; Set Stack Pointer to top of RAM
0x0036		ldi r16,low(RAMEND)	
0x0037		out SPL,r16	
0x0038		sei	; Enable interrupts

Қуат қосылғанда және қалпына келтіру сигналы Power-on Reset тізбегі арқылы жасалған кезде, микроконтроллер 0x0000 мекенжайында орналасқан пәрменді орындайды (немесе ол Arduino-дағы жағдай сияқты жүктеушіден ауыстырылады). Бұл пәрмен RESET белгісіне сөзсіз өту болып табылады, ол пайдаланушы бағдарламасын инициализациялауды және орындауды бастайды. Егер, мысалы, бағдарлама жұмысы кезінде INT1 сыртқы үзу сұрауы қабылданса және оны өңдеу қосылса, микроконтроллер INT1 векторына

(0x0004 мекенжайына) барады, ол өз кезегінде микроконтроллерді осы үзу үшін өңдеушіге қайта бағыттайды.

Егер бағдарлама үзілістерді қолданбаса, оны бірден 0x0000 мекенжайынан табуға болады.

Ал, Arduino тақталарына оралсақ, пайдаланушыға векторлық кесте жасап, оны ағымдағы мекенжайлармен толтырудың қажеті жоқ екенін қосамыз. Бұл жұмысты AVR-GCC компиляторы орындайды. .

MCUCR регистрі, IVSEL биті

Жоғарыда айтылғандай, MCUCR регистрінің IVSEL биті (MicroController Unit Control Register – микроконтроллердің басқару регистрі) ұзу векторлық кестесінің позициясына жауап береді. Ол келесі биттерді қамтиды:

Bit	7	6	5	4	3	2	1	0
		BODS	BODSE	PUD			IVSEL	IVCE
Access		R/W	R/W	R/W			R/W	R/W
Reset		0	0	0			0	0

Үзіліс өңдеу

SREG регистрінің I биті үзілістерді ғаламдық түрде қосу/өшіру үшін арналған. Үзілістерді қосу үшін оны 1-ге орнату керек, өшіру үшін - 0-ге қалпына келтіру керек. Дәл осы бит алдыңғы жарияланымда талқыланған sei, cli және interrupts, noInterrupts функцияларымен басқарылады. Сонымен қатар, әрбір үзіліс үшін оны өңдеуге мүмкіндік беретін жеке бит беріледі.

Үзіліс өңдеу

Барлық үзілістерді екі түрге бөлуге болады. Бірінші типті үзілістер қандай да бір оқиға орын алған кезде құрылады, нәтижесінде үзіліс жалаушасы орнатылады. Содан кейін үзіліс қосылса және SREG I биті орнатылса, бағдарлама санауышы үзіліс векторының адресімен жүктеледі. Бұл жағдайда осы үзілістің жалауы аппараттық құрал арқылы қалпына келтіріледі (ерекшелік тек бағдарламалық құрал арқылы қалпына келтірілетін TWI интерфейсінің жалауы болып табылады). Оны 1 мәнін жазу арқылы бағдарламалық құрал арқылы да тазалауға болады. Ұзу жалауын бағдарламалық түрде орнату мүмкін емес (мысалы, үзілістің пайда болуын эмуляциялау үшін).

Үзіліс өңдеу

Егер ұзу сұрауы оны өңдеу өшірілген уақытта келсе (жаһандық I бит немесе жеке бит бойынша), сәйкес жалауша әлі де орнатылады. Осылайша, біз бақыланатын оқиғаны жіберіп алмаймыз және үзіліс шешілген кезде оның өңдеушісі орындалады. Үзілістердің бұл қарастырылатын түрі екінші түрінен ерекшеленеді.

Үзіліс өңдеу

Екінші типті үзілістердің жалаушалары болмайды, олар генерациялау шарттары болғанша генерацияланады. Сәйкесінше, егер ұзу сұрауы оны өңдеу өшірілген уақытта келсе және ол қосылғанға дейін жоғалып кетсе, өңдеуші орындалмайды және біз бұл сұрауды «жоғалтып аламыз». Егер өңдеуші орындалғаннан кейін үзіліс тудыратын шарт әлі болса, онда ол қайта орындалады. Үзілістің бұл түрінің мысалы сыртқы төмен деңгейлі ұзулер болып табылады.

Үзіліс өңдеу

Үзіліс өңделген кезде SREG регистрінің I биті автоматты түрде тазаланады, осылайша басқа үзулерді өңдеуді өшіреді.

Өңдеуіштен негізгі бағдарламаға оралғанда I бит қайтадан орнатылады. Бұл мінез-құлық әдетте бағдарламашыға ұнайды, өйткені ол жадты жоғалту қаупін тудыратын өңдеушілердің рекурсивті шақыруын болдырмайды. Бірақ егер бағдарлама логикасы кірістірілген үзілістерді өңдеуді қажет етсе, оны өңдеушіге кіру кезінде I биті орнату арқылы қосуға болады.

Үзілімдерді өңдеу кезектілігі

Жоғарыда аталған ұзу жалаушалары ұзу сұрауларын өңдеу өшірілген кезде де тіркейді. Кейіннен қосу кезінде үзілістер басымдылығына сәйкес кезекпен қызмет көрсетіледі. Бірнеше сұраулар бір уақытта келгенде де солай болады. Ұзу басымдығы оның векторлық кестедегі орнымен анықталады. Жоғарыдағы векторлық кестеде сыртқы үзіліс INT0 ең жоғары басымдыққа ие, содан кейін INT1 және SPM READY болғанша солай жалғаса береді.

Үзілімдерді өңдеу кезектілігі

Егер ұзу сұрауларының кезегі түзілсе, онда келесі өңдеуші орындалғаннан кейін басқару әрқашан негізгі программаға қайтарылады. Келесі үзуді өңдеу негізгі программаның бір командасы орындалғаннан кейін жүреді.

Қалпына келтіру сигналы үзіліс емес, ол тәртіпсіз өңделеді.

Үзіліс сұрауына жауап беру уақыты

Кез келген үзіліс үшін ең қысқа жауап беру уақыты - 4 тактілік цикл. Осы уақыт ішінде бағдарлама есептегіші стекке сақталады және I биті тазартылады. Содан кейін вектордың адресі бағдарлама санағышына жүктеледі. Әдетте, бұл адресіте өңдеушіге өту командасы бар, оның орындалуы тағы 3 циклды алады. Егер үзіліс анықталған кезде бірнеше циклге созылатын нұсқау орындалса, онда үзілістерді өңдеу осы нұсқау аяқталғаннан кейін басталады. Егер микроконтроллер ұйқы режимінде болғанда ұзу сұрауы алынса, режим мен сақтандырғыш параметрлеріне байланысты ояту уақытына қосымша жауап беру уақыты тағы 4 циклге артады.

Өңдеушіден негізгі бағдарламаға оралу 4 циклді алады, оның барысында бағдарлама санауышы стектен қалпына келтіріледі және SREG регистрінің I биті орнатылады.

ISR кілт сөзі

Үзілістердің векторлық кестесін ұйымдастыруға компилятор жауап беретіні бұрын айтылған болатын. AVR-GCC-де микроконтроллердің әрбір түрі үшін қажетті векторлар жинағы бар кесте алдын ала анықталған. Біз тек компиляторға қандай ұзу қай өңдегішке сәйкес келетінін айтуымыз керек, сонда ол өңдеушінің ағымдағы адресін кестеге енгізеді. Бұл ISR макросы арқылы орындалады. Оның синтаксисі келесідей:

ISR (vector, attributes)

Векторлық параметр өңдеуші жасағымыз келетін үзуді көрсетеді. ATmega328/P үшін мүмкін болатын параметр мәндері Вектор бағанындағы ұзу кестесінде көрсетілген. Осы параметрдің барлық жарамды мәндері туралы ақпаратты AVR Libc құжаттамасынан таба аласыз. : https://www.nongnu.org/avr-libc/user-manual/group_avr_interrupts.html

Қосымша атрибуттар параметрі келесі мәндерді қабылдай алады: ISR_BLOCK, ISR_NOBLOCK, ISR_NAKED және ISR_ALIASOF(vect). Мәндерді бос орынмен бөліп көрсету арқылы біріктіруге рұқсат етіледі.

ISR кілт сөзі

Негізгі сөзді қолданудың мысалы ретінде күзет таймері үшін өңдегіш функциясын анықтауды қарастырамыз:

```
ISR (WDT_vect) {  
    //Наш код здесь  
}
```

ISR кілт сөзі

```
push r1
```

```
push r0
```

```
in r0, SREG
```

```
push r0
```

```
clr r1
```

```
; Біздің код осында
```

```
pop r0
```

```
out SREG, r0
```

```
pop r0
```

```
pop r1
```

```
reti
```

ISR_NOBLOCK параметрі

Егер ISR_NOBLOCK ISR макросының екінші параметрі ретінде көрсетілсе, компилятор өңдеушіге I битін орнату нұсқаулығын қосады, осылайша кірістірілген үзулерді өңдеуге мүмкіндік береді. Әрине, сіз өңдеушінің басына үзілістерді немесе sei функциясын өзіңіз енгізе аласыз, бірақ біз көргеніміздей, компилятор регистрлердің мазмұнын стекке сақтау үшін өңдегішті кодпен толықтырады және үзілістерге тек осыдан кейін ғана рұқсат етіледі. бұл код орындалды. ISR_NOBLOCK опциясы компиляторға регистрлер сақталмас бұрын өңдегіштің ең басына sei пәрменін енгізуге нұсқау береді. Бұл қажет болған жағдайда үзу сұрауына жауап беру уақытын қысқартады.

ISR_NOBLOCK параметрі

```
ISR (WDT_vect, ISR_NOBLOCK) {
```

```
    //Біздің код осында
```

```
}
```

ISR_NOBLOCK параметрі

```
sei  
push  r1  
push  r0  
in     r0, SREG  
push  r0  
clr    r1  
; Біздің код осында  
pop    r0  
out    SREG, r0  
pop    r0  
pop    r1  
reti
```

sei және cli

Мұнда біз компилятор жасаған кодта да, Arduino бағдарламаларда де көретін sei командасын айта аламыз. sei және cli - AVR микроконтроллерлеріне арналған ассемблер тілінің командалары. Бұл командалар сәйкесінше SREG регистрінің I битін орнатады және тазартады. Біз бағдарламаларда қолданатын sei және cli функциялары interrupt.h файлында (AVR Libc бөлігі) жарияланған макростар болып табылады, олар өз кезегінде жоғарыда аталған командалардың біріне жинақталған. Үзілістер және үзіліссіз функциялары Arduino IDE бөлігі болып табылады және Arduino.h файлында төмендегідей жарияланады:

```
#define interrupts() sei()
```

```
#define noInterrupts() cli()
```


ISR_BLOCK параметрі

ISR_BLOCK мәні компиляторға кірістірілген үзулерді қоспауға нұсқау береді, бұл оның әдепкі әрекеті. Сондықтан ISR_BLOCK параметрі бар өңдегіштің сипаттамасы онсыз оның сипаттамасына баламалы.

ISR_NAKED параметрі

Кейбір жағдайларда өңдеуші ішіндегі регистр мәндерін сақтау және қалпына келтіру үшін компилятор жасаған код оңтайлы болмауы мүмкін. Мысалы, жоғарыдағы WDT өңдеушісі ешнәрсе жасамайды, бірақ үш регистрдің мәндері әлі де стекте сақталады. Егер компилятор жасаған код бізді қанағаттандырмаса, онда оны өңдеушіге қосуды ISR макросының екінші параметрінде ISR_NAKED мәнін көрсету арқылы басуға болады. Бұл жағдайда өңдеушіге регистрлерді сақтау коды да, тіпті негізгі reti бағдарламасына оралу командасы да қосылмайды, өңдеушінің дұрыс жұмыс істеуі үшін жауапкершілік бізге жүктеледі.

ISR_NAKED мысалы

```
ISR(TIMER1_OVF_vect, ISR_NAKED)
{
    PORTB |= _BV(0);
    reti();
}
```

reti өңдеушіден негізгі бағдарламаға оралу үшін ассемблер нұсқауы болып табылады. Бірақ жоғарыда келтірілген фрагментте reti () шақыру макросқа шақыру болып табылады, ол бірдей interrupt.h файлында жарияланған және аттас микроконтроллер командасына жинақталған.

ISR_ALIASOF параметрі

ISR_ALIASOF пайдалану компиляторға берілген үзіліс өңдегішті басқа үзумен бөлісетінін айтуға мүмкіндік береді. Бұл екі немесе одан да көп үзілістердің өңдеушілері толығымен бірдей (немесе жалпы пішінге азайтылуы мүмкін) жағдайларда пайдалы.

Мысал: Бірнеше PCINT үзулеріне арналған ортақ өңдегіш

```
ISR(PCINT0_vect){
```

```
    // Наш код здесь
```

```
}
```

```
ISR(PCINT1_vect, ISR_ALIASOF(PCINT0_vect));
```

```
ISR(PCINT2_vect, ISR_ALIASOF(PCINT0_vect));
```

Жоғарыда келтірілген код үшін компилятор барлық 3 PCINT векторын бір жалпы өңдеушімен байланыстырады.

Үзіліс өңдеушісі функциясының прологы және эпилогы

Компилятор өңдеуші кодының алдында қосатын нұсқаулар жиыны пролог деп аталады; өңдеушіден кейін – эпилог. Функция прологы регистрлерді пайдалануға дайындайды: олардың мазмұнын стекте сақтайды; Эпилог шақырушы/үзілген бағдарлама олармен жұмысын жалғастыра алуы үшін шығу алдында регистрлерді қалпына келтіреді. Жоғарыдағы мысалдардан сақталған регистрлерді қарастырыңыз.

SREG - күй регистрі

Бұл регистрде әртүрлі командалар орындалғанда мәндері өзгертін жалаушалар бар. Мысалы, логикалық немесе арифметикалық операция нәтижесінде 0 алынса, нөлдік жалауша (Нөлдік жалауша) бір мәнге орнатылады. Басқа жалаушалар тасымалдау фактісін немесе теріс нәтижені білдіреді және т.б. Өңдеушіге кіру кезінде осы регистрдің мазмұнын сақтаңыз - алтын ереже, ол AVR-GCC-де де байқалады. Дегенмен, SREG пәрменімен стекке тікелей итеру мүмкін емес. Сондықтан оның мазмұны алдымен r0 регистрінде оқылады.

r0 – жалпы мақсаттағы регистр

r0 – жалпы мақсаттағы регистр. AVR-GCC оны жоғарыдағыдай жағдайларда аралық ұяшық ретінде пайдаланады. Сондықтан, SREG оқымас бұрын, r0 мазмұны да стекке итеріледі.

r1 – жалпы мақсаттағы регистр

r1 - жалпы мақсаттағы регистр, AVR-GCC контекстінде ол нөлдік регистр ретінде пайдаланылады («нөлдік регистр») - ол әрқашан 0-ді қамтуы керек. Осыны ескере отырып, бұл регистр пайдаланылады, мысалы, егер қажет болса. 0-мен салыстыру немесе жад ұяшығына 0 жазу кезінде. Сондықтан прологта r1 регистрін тазалау туралы нұсқау бар:

```
clr    r1
```

оның құрамында 0 болатынына көз жеткізу үшін. Неліктен оның құрамында әрқашан 0 болса, оны стекке сақтау керек? Мәселе мынада: әрқашан емес: көбейту нұсқаулары r1 регистріне нәтиженің жоғары байтын орналастырады. Егер үзіліс көбейтудің нәтижесі негізгі программамен әлі өңделмеген уақытта орын алса, онда r1 регистрінде нөлдік емес мән болуы мүмкін. Сондықтан оның мазмұны да стекке итеріледі.

BADISR_vect векторы

Қосылған үзу үшін өңдеуші көрсетілмеген жағдай қате болып табылады. AVR-GCC, өз өңдеушісі жоқ барлық үзулер үшін векторлық кестені құру кезінде «әдепкі» өңдегішті орнатады. Бұл өңдеуші бір пәрменді қамтиды - қалпына келтіру векторына өту. Қажет болса, осы өңдегішті қайта анықтауға болады, ол үшін BADISR_vect векторы пайдаланылады:

```
ISR(BADISR_vect) {  
    //Наш код здесь  
}
```

Осылайша, әдеттегі қалпына келтірудің орнына күтпеген үзулер үшін басқа әрекетті көрсетуге болады.

EMPTY_INTERRUPT кілт сөзі

Сирек жағдайларда өңдеушіден ешқандай әрекетті орындау талап етілмейді. Мысалы, егер үзуді микроконтроллерді ұйқы режимінен ояту үшін ғана қолдансақ. Бұл жағдайда оған бос өңдегішті анықтауға болады (ISR көмегімен), бірақ неғұрлым ақылды шешім EMPTY_INTERRUPT макросын пайдалану болады. ISR макросынан айырмашылығы, ол өңдеушіге пролог пен эпилогты қоспайды және оның жалғыз командасы негізгі бағдарламаға - reti-ге оралу болады. Төменде осы макросты WDT үзуі үшін пайдалану мысалы берілген:

```
EMPTY_INTERRUPT(WDT_vect);
```

INTx сыртқы үзілістер

Үзілістерді өңдеу логикасын және ISR макросының синтаксисін қарастыра отырып, сіз жаңа білімді тәжірибеде қолдана аласыз. Өткен лекцияларда сыртқы үзулермен жұмыс істеуге арналған `attachInterrupt` және `detachInterrupt` функцияларымен таныстық. Енді IDE функцияларынсыз істеуге тырысайық және сыртқы үзулерді өзіміз өңдеу үшін барлық қажетті қадамдарды орындаймыз.

INTx сыртқы үзілістер

Жоғарыда айтылғандай, жаһандық деңгейде үзілістерді өңдеуге мүмкіндік беретін 1 битке қосымша, үзілістерді жеке өңдеуге мүмкіндік беретін биттер бар. Сыртқы үзілістер үшін бұл EIMSK (External Interrupt Mask Register) регистрінің екі ең аз маңызды биттері:

Bit	7	6	5	4	3	2	1	0
							INT1	INT0
Access							R/W	R/W
Reset							0	0

INT0 кірісінде үзілістерді өңдеуді қосу үшін EIMSK регистрінде аттас бит орнату керек. INT1 үзулерін қосу үшін тізілімнің INT1 битін сәйкесінше орнатыңыз. Әдепкі бойынша (микроконтроллерді қалпына келтіргеннен кейін) екі бит де қалпына келтіріледі және сыртқы үзулерді өңдеу өшіріледі.

INTx сыртқы үзілістер

EICRA (External Interrupt Control Register A) регистрі INTx кірістерінде бақыланатын оқиғалардың түрін орнату үшін пайдаланылады. Оның биттерінің мақсаты келесідей:

Bit	7	6	5	4	3	2	1	0
					ISC11	ISC10	ISC01	ISC00
Access					R/W	R/W	R/W	R/W
Reset					0	0	0	0

ISC00 және ISC01 биттерінің мәндері оқиғалардың қандай түрі INT0 үзуінің пайда болуына әкелетінін анықтайды:

00 - төмен деңгейлі сигнал болған кезде;

01 - сигнал жоғарыдан төменге және керісінше өзгергенде;

10 - сигнал жоғарыдан төменге ауысқанда;

11 - сигнал төменнен жоғарыға өзгергенде.

ISC10 және ISC11 биттерінің тағайындалуы ISC0x-қа ұқсас, айырмашылығы олар INT1 үзілуі үшін оқиға түрін анықтайды.

INTx сыртқы үзілістер

Ал соңғы, үшінші, сыртқы үзулерді өңдеуге қатысты регистр – EIFR жалауша регистрі (External Interrupt Flag Register):

Bit	7	6	5	4	3	2	1	0
							INTF1	INTF0
Access							R/W	R/W
Reset							0	0

EICRA регистрінің ISC0x биттерінің конфигурациясына сәйкес INT0 кірісінде үзілістерді генерациялау шарты орындалғанда INTF0 биті 1-ге орнатылады. Содан кейін, егер INT0 үзуін өңдеу қосылса және I биті 1-ге орнатылса, сәйкес өңдеуші орындалады және бит мәні тазаланады. Сондай-ақ, INTF0 битінің мәнін бағдарламалық құрал арқылы оған «1» мәнін жазу арқылы қалпына келтіруге болады. EICRA регистрінің ISC0x биттерінің конфигурациясы INT0 кірісінде төмен деңгей болған кезде үзілістерді генерациялау үшін анықталған болса, онда бұл жалау өңдеуде пайдаланылмайды және оның мәні «0» болады.

INTF1 биті INTF0-ге ұқсас, бірақ INT1 кірісінде үзу сұрауын анықтау туралы сигнал береді.

INTx сыртқы үзілістер

Сонымен, сыртқы үзілістерді қосу және оларды өңдеу режимін орнату үшін келесі қадамдарды орындау керек:

1. ISR кілт сөзін пайдаланып өңдегішті көрсетіңіз.
2. Ұзу сұрауын тудыратын енгізу оқиғаларының түрін анықтаңыз (EICRA тіркелімі).
3. Сыртқы үзілістерді өңдеуді қосыңыз (EIMSK тіркеу).
4. Жаһандық үзіліс өңдеуді қосу үшін I битін орнатыңыз (SREG тіркеуі).

Барлық осы регистрлер мен биттер AVR Libc құрамындағы тақырып файлдарында анықталған және Arduino әзірлеу ортасында аты бойынша қол жеткізуге болады.

Мысал

```
#define ledPin 13
#define interruptPin 2 // 2-сандық пин (INT0 кірісі) мен GND арасындағы түйме
volatile byte state = LOW;
void setup() {
    pinMode(ledPin, OUTPUT);
    pinMode(interruptPin, INPUT_PULLUP); // Біз екінші түйреуішті қуатқа тартамыз
    EICRA &= ~(1 << ISC00); //ISC00 қалпына келтіріңіз
    EICRA |= (1 << ISC01); // ISC01 орнату - INT0 бойынша FALLING мониторы
    EIMSK |= (1 << INT0); // Үзілуге рұқсат етеміз INT0
}
void loop() {
    digitalWrite(ledPin, state);
}
ISR(INT0_vect) {
    state = !state;
}
```

Шығару күйін өзгерту кезіндегі үзілістер (Pin Change Interrupts, PCINT)

Аты айтып тұрғандай, осы түрдегі үзілістер шығыс күйінің кез келген өзгерісінде жасалады. Төмен деңгейлі үзілістер бізге қолжетімді болмаса да, сыртқы INTx үзілістеріндегідей сигналдың тек көтерілуінде немесе төмендеу шетінде ғана қол жетімді емес, бірақ біз енді екі кіріспен шектелмейміз: сигналды өзгертуге арналған үзілістер. шығыс күйі барлық дерлік Arduino шығыстарында қол жетімді. Arduino UNO (және ATmega328/P негізіндегі басқа тақталар) үшін бұл түйреуіштер:

- + D8 .. D13 - үзу сұрауын жинақтайды PCINT0
- + A0 .. A5 - үзу сұрауын жинақтайды PCINT1
- + D0 .. D7 - үзу сұрауын жинақтайды PCINT2

Шығару күйін өзгерту кезіндегі үзілістер (Pin Change Interrupts, PCINT)

PCINT-пен жұмыс істеу үшін PCICR, PCIFR және үш PCMSKx регистрлері қамтамасыз етілген. Олардың әрқайсысын қарастырайық.

PIN өзгерту үзілісін басқару регистрінің (PCCR) бит тағайындауы:

Bit	7	6	5	4	3	2	1	0
						PCIE2	PCIE1	PCIE0
Access						R/W	R/W	R/W
Reset						0	0	0

- + PCIE0 – бұл биттегі «1» мәні PCINT0 тобының үзілістерін өңдеуге мүмкіндік береді.
- + PCIE1 – осы биттегі «1» мәні PCINT1 тобының үзілістерін өңдеуге мүмкіндік береді.
- + PCIE2 – бұл биттегі «1» мәні PCINT2 тобының үзілістерін өңдеуге мүмкіндік береді.

Шығару күйін өзгерту кезіндегі үзілістер (Pin Change Interrupts, PCINT)

PCIFR (Pin Change Interrupt Flag Register) биттерінің мақсаты:

Bit	7	6	5	4	3	2	1	0
						PCIF2	PCIF1	PCIF0
Access						R/W	R/W	R/W
Reset						0	0	0

- + PCIF0 – осы биттегі «1» мәні PCINT0 үзу сұрауын анықтауды білдіреді.
- + PCIF1 – осы биттегі «1» мәні PCINT1 үзу сұрауын анықтауды білдіреді.
- + PCIF2 – осы биттегі «1» мәні PCINT2 үзу сұрауын анықтауды білдіреді.

Регистр PCMSK0

Бит	Шығаруды белгілеу (IDE Arduino)	Шығару нөмірі (Микроконтрол лер)	PCINTx
Регистр PCMSK0			
0	D8	14	PCINT0
1	D9	15	PCINT1
2	D10	16	PCINT2
3	D11	17	PCINT3
4	D12	18	PCINT4
5	D13	19	PCINT5
6	-	9	PCINT6
7	-	10	PCINT7

Регистр PCMSK1

Бит	Шығаруды белгілеу (IDE Arduino)	Шығару нөмірі (Микроконтролл ер)	PCINTx
Регистр PCMSK1			
0	A0	23	PCINT8
1	A1	24	PCINT9
2	A2	25	PCINT10
3	A3	26	PCINT11
4	A4	27	PCINT12
5	A5	28	PCINT13
6	-	1	PCINT14

Регистр PCMSK2

Бит	Шығаруды белгілеу (IDE Arduino)	Шығару нөмірі (Микроконтролл ер)	PCINTx
Регистр PCMSK2			
0	D0	2	PCINT16
1	D1	3	PCINT17
2	D2	4	PCINT18
3	D3	5	PCINT19
4	D4	6	PCINT20
5	D5	11	PCINT21
6	D6	12	PCINT22
7	D7	13	PCINT23

Шығару күйін өзгерту кезіндегі үзілістер (Pin Change Interrupts, PCINT)

ATmega328/P 9 және 10 түйреуіштерін Arduino резонаторды қосу үшін пайдаланады; pin 1 - Қалпына келтіру кірісі. Сондықтан оларды үзу кірістері ретінде пайдаланудан бас тартуға тура келеді. Негізінде ATmega328/P-де PCINT15 бит жоқ. PIN күйі өзгерген кезде үзілістерді қосу үшін келесі әрекеттерді орындаңыз:

1. ISR макросын пайдаланып сәйкес PCINT үзуіне өңдегішті орнатыңыз.
2. Қызықты микроконтроллер пин арқылы үзулерді генерациялауды қосыңыз (PCMSKx топ тізілімі).
3. Қызығушылықты тудыратын PCINT үзуін өңдеуді қосыңыз (PCICR тіркелімі).
4. Жаһандық үзіліс өңдеуді қосу үшін I битін орнатыңыз (SREG тіркеуі).

Мысал

```
#define ledPin 13    // Пин для светодиода
#define setLedOnPin 8 // Пин кнопки включения светодиода
#define setLedOffPin 9 // Пин кнопки выключения светодиода
volatile uint8_t state = 0;
uint8_t oldPINB = 0xFF;
void pciSetup(byte pin) {
    *digitalPinToPCMSK(pin) |= bit (digitalPinToPCMSKbit(pin)); //
    Разрешаем PCINT для указанного пина
    PCIFR |= bit (digitalPinToPCICRbit(pin)); // Очищаем признак
    запроса прерывания для соответствующей группы пинов
    PCICR |= bit (digitalPinToPCICRbit(pin)); // Разрешаем PCINT для
    соответствующей группы пинов
}
ISR (PCINT0_vect) { // Обработчик запросов прерывания от
пинов D8..D13
    uint8_t changedbits = PINB ^ oldPINB;
    oldPINB = PINB;
    if (changedbits & (1 << PB0)) { // Изменился D8
        state = 1; // Зажигаем светодиод
    }
    if (changedbits & (1 << PB1)) { // Изменился D9
        state = 0; // Гасим светодиод
    }
    //if (changedbits & (1 << PB2)) { ... } - аналогичные условия для
```

```
остальных пинов
}
ISR (PCINT1_vect) { // Обработчик запросов прерывания от
пинов A0..A5
    // Обработка аналогична PCINT0_vect, только изменить на
    PINC, oldPINC, PCx
}
ISR (PCINT2_vect) { // Обработчик запросов прерывания от
пинов D0..D7
    // Обработка аналогична PCINT0_vect, только изменить на
    PIND, oldPIND, PDx
}
void setup() {
    pinMode(ledPin, OUTPUT);
    pinMode(setLedOnPin, INPUT_PULLUP); // Подтянем пины-
    источники PCINT к питанию
    pinMode(setLedOffPin, INPUT_PULLUP);
    pciSetup(setLedOnPin); // И разрешим на них прерывания
    pciSetup(setLedOffPin);
}
void loop() {
    digitalWrite(ledPin, state);
}
```

Күзет таймерінен үзілістер

Үзілістермен жұмыс істеудің тағы бір мысалын қарастырайық, бұл жолы Watchdog Timer (WDT). Біз WDT үзілістерін ұйқы режимінен ояту үшін қолдандық, енді оларды толығырақ зерттей аламыз.

Жалпы, микроконтроллерді ояту үшін үзіліс жасау күзет таймерінің жалғыз функциясы емес. Өлшенетін уақыт аралықтарының дәлдігі талап етілмесе, оны қарапайым таймер ретінде пайдалануға болады. Қалпына келтіру сигналын генерациялау режимінде күзет таймері жүйенің тоқтап қалуы мүмкін болғанда пайдаланылады: қалыпты жұмыс кезінде бағдарлама микроконтроллердің бастапқы қалпына келуіне жол бермей, күзет таймерін жүйелі түрде қалпына келтіреді; егер бағдарлама көрсетілген уақыт өткеннен кейін тоқтап қалса, күзет таймері бастапқы қалпына келтіру сигналын жасайды. Бұл микроконтроллер жүйесінің сенімділігін арттырады. Сондай-ақ үзілістерді жасау және қалпына келтіру режимдерін біріктіруге болады, бұл жағдайда бірінші кезекте үзу өңдеушісі шақырылады, ол маңызды деректерді сақтайды, содан кейін келесі WDT күту уақытында микроконтроллерді қалпына келтіру сигналы жасалады.

Күзет таймерінен үзілістер

WDTCSR (Watchdog Timer Control Register) Битті тағайындау :

Bit	7	6	5	4	3	2	1	0
	WDIF	WDIE	WDP[3]	WDCE	WDE	WDP[2:0]		
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Күзет таймерінен үзілістер

WDIF (Watchdog Interrupt Flag) – ұзу сұрауының жалаушасы. Күзет таймері үзілістерді жасау үшін конфигурацияланған кезде көрсетілген уақыт аралығы өткеннен кейін 1 мәніне орнатылады. Жалау өңдеуші орындалған кезде аппараттық құрал арқылы немесе оған «1» мәнін бағдарламалық түрде жазу арқылы қалпына келтіріледі.

WDIE (Watchdog Interrupt Enable) - WDT үзілістерін өңдеуге мүмкіндік беретін бит. Бұл бит орнатылғанда және WDE өшірілгенде, бақылау таймері үзілістерді жасау үшін конфигурацияланады. WDIE және WDE орнатылған кезде бақылаушы таймері алдымен WDIE қалпына келтіре отырып, ұзу сұрауын жасайды, содан кейін келесі таймер күту уақыты микроконтроллерді қалпына келтіру сигналын бастайды.

WDP[3] (Watchdog Timer Prescaler 3) - WDT жиілік бөлгішінің үшінші биті.

WDCE (Watchdog Change Enable) – WDE биті мен бөлгіш мәнін өзгертуге мүмкіндік беретін бит (WDP[3:0]).

Оларды өзгерту үшін WDCE параметрін «1» күйіне орнату керек. Төрт циклден кейін бұл бит автоматты түрде тазаланады, сондықтан оны WDE және WDP[3:0] өзгертпес бұрын бірден орнату керек.

WDE (Watchdog жүйесін қалпына келтіруді қосу) - Бұл бит бақылау таймері арқылы қалпына келтіру сигналын жасауға мүмкіндік береді. Оның өзгеруі WDCE биті арқылы басқарылады және MCUSR регистріндегі WDRF биті орнатылғанша оны тазалау мүмкін емес.

WDP[2:0] (Watchdog Timer Prescaler 2, 1 және 0) - WDT сағат бөлгішінің төменгі үш биті. WDP[3:0] биттерінің рұқсат етілген комбинациялары және олардың сәйкес уақыт интервалдары кестеде көрсетілген

WDP3	WDP2	WDP1	WDP0	Число тактов генератора WDT	Интервал времени
0	0	0	0	2K (2048)	16мс
0	0	0	1	4K (4096)	32мс
0	0	1	0	8K (8192)	64мс
0	0	1	1	16K (16384)	0.125с
0	1	0	0	32K (32768)	0.25с
0	1	0	1	64K (65536)	0.5с
0	1	1	0	128K (131072)	1с
0	1	1	1	256K (262144)	2с
1	0	0	0	512K (524288)	4с
1	0	0	1	1024K (1048576)	8с
1	0	1	0	Зарезервировано	
1	0	1	1		
1	1	0	0		
1	1	0	1		
1	1	1	0		
1	1	1	1		

Күзет таймерінен үзілістер

WDE және WDP[3:0] мәндерін өзгерту үшін келесі әрекеттерді орындаңыз:

WDCE және WDE биттерін орнатыңыз (алдыңғы WDE мәніне қарамастан).

Келесі төрт цикл ішінде WDE, WDP[3:0] биттерінің қажетті мәнін орнатыңыз және WDCE битін қалпына келтіріңіз. Бұл бір пәрмен ішінде орындалуы керек.

Сипатталған реттілікке WDE және WDP [3:0] өзгерту кезінде WDIE битін орнату және үзулерді өшіру қосу қалады.

Мысал

```
volatile bool f = 0;
void setup() {
  Serial.begin(9600);
  cli(); // Запрещаем прерывания на время
изменения WDE и WDP
  asm("wdr"); // Сбрасываем WDT
  // Разрешаем изменение значения
предделителя WDT:
  WDTCR |= (1 << WDCE) | (1 << WDE);
  // Устанавливаем бит WDP3 для выбора
интервала 4с и разрешаем прерывания от
WDT:
  WDTCR = (1 << WDP3) | (1 << WDIE );
```

```
sei(); // Разрешаем прерывания
}
void loop() {
  if (f) {
    Serial.print(millis());
    Serial.println(" WDT!");
    f = 0;
  }
}
ISR(WDT_vect){
  f = 1;
}
```

Мысал

```
#include <avr/wdt.h>
#include <avr/sleep.h>
#define ledPin 13 // Пин для светодиода
bool f = 0;
void setup() {
    pinMode(ledPin, OUTPUT);
}
void loop() {
    // Здесь может быть условие для перехода
    // в спящий режим
    wdt_enable(WDTO_4S); // Разрешаем
    // работу таймера и задаем интервал
    // Определенные в wdt.h значения:
    WDTO_15MS, WDTO_30MS, WDTO_60MS,
    WDTO_120MS,
    // WDTO_250MS, WDTO_500MS, WDTO_1S,
```

```
WDTO_2S, WDTO_4S, WDTO_8S
    WDTCSR |= (1 << WDIE); // Разрешаем
    прерывания от сторожевого таймера
    set_sleep_mode(SLEEP_MODE_PWR_DOWN);
    //Устанавливаем интересующий нас режим
    sleep_mode(); // Переводим МК в спящий
    режим. Пробуждение по WDT
    // После выполнения обработчика WDT
    // работа программы продолжится отсюда
    wdt_disable(); // Останавливаем WDT, он
    нам больше не нужен
    // Далее выполняем какие-либо
    действия и опять уходим в сон
    digitalWrite(ledPin, (f=!f));
}
    EMPTY_INTERRUPT(WDT_vect);
```


Мысал

Соңында, микроконтроллерді бақылау күту уақытында қалпына келтіру үшін пайдалануға болатын код бөлігі:

```
#include <avr/wdt.h>
```

```
...
```

```
wdt_enable(WDTO_15MS); // Сброс по WDT через ~16мс
```

```
while(1); // Ожидаем сброс
```

Ардуино бағдарламаларындағы класстар. Батырма нысан ретінде

Дәріс 10

Класс анықтамасы

class класс_атауы {класс мүшелері};

Батырмаларға арналған айнымалылар

```
boolean flagPress= false;    // белгісі батырмасы енді басылды  
boolean flagClick= false;   // белгісі батырмасы басылды (нұқыңыз)  
byte  buttonCount= 0;       // тұрақты күйді растау есептеуіші  
#define TIME_BUTTON 15      // тұрақты күй уақыты (* 2 мс)
```

Батырмалар сигналдарын өңдеу класының сипаттамасы

```
class Button {  
    boolean flagPress;    // белгісі батырмасы енді басылды  
    boolean flagClick;    // белгісі батырмасы басылды (нұқыңыз)  
    byte buttonCount;    // тұрақты күйді растау есептеуіші  
    byte timeButton;    // батырма күйін растау уақыты  
    byte _pin;           // шығыс нөмірі  
};
```

Қолжетімділік модификаторы

- public түрлендіргіштен кейінгі функциялар мен айнымалылар бағдарламаның кез келген орнынан қол жетімді.
- private модификаторынан кейін жабық функциялар мен айнымалылар орналастырылады. Олармен тек өз класының әдістері ғана жұмыс істей алады. Егер public түрлендіргіші болмаса, онда сыныптың барлық мүшелері жабық болып есептеледі.

Батырмалар сигналдарын өңдеу кдассының сипаттамасы

```
class Button {  
    public:  
        boolean flagPress;    // признак кнопка сейчас нажата  
        boolean flagClick;    // признак кнопка была нажата (клик)  
    private:  
        byte _buttonCount;    // счетчик подтверждений стабильного  
        состояния  
        byte _timeButton;     // время подтверждения состояния кнопки  
        byte _pin;            // номер вывода  
};
```

Батырмалар сигналдарын өңдеу классының сипаттамасы

```
class Button {  
    public:  
        boolean flagPress;    // признак кнопка сейчас нажата  
        boolean flagClick;    // признак кнопка была нажата (клик)  
        void scanState();    // метод проверки состояние сигнала  
        void setPinTime(byte pin, byte timeButton); // метод установки номера  
        вывода и времени подтверждения  
    private:  
        byte _buttonCount;    // счетчик подтверждений стабильного  
        состояния  
        byte _timeButton;    // время подтверждения состояния кнопки  
        byte _pin;           // номер вывода  
};
```


Батырма күйін тексеру тәсілдері

```
// метод проверки состояния кнопки
// flagPress= true - нажата
// flagPress= false - отжата
// flagClick= true - была нажата (клик)
void Button::scanState() {
    if ( flagPress == (! digitalRead(_pin)) ) {
        // состояние сигнала осталось прежним
        _buttonCount= 0; // сброс счетчика состояния сигнала
    }
    else {
        // состояние сигнала изменилось
        _buttonCount++; // +1 к счетчику состояния сигнала
        if ( _buttonCount >= _timeButton ) {
            // состояние сигнала не менялось заданное время
            // состояние сигнала стало устойчивым
            flagPress= ! flagPress; // инверсия признака состояния
            _buttonCount= 0; // сброс счетчика состояния сигнала
        }
        if ( flagPress == true ) flagClick= true; // признак клика на нажатие
    }
}
```

*шығару номері мен растау уақытын
орнату тәсілі*

```
void Button::setPinTime(byte pin, byte timeButton) {  
    _pin= pin;  
    _timeButton= timeButton;  
    pinMode(_pin, INPUT_PULLUP); // кіріс ретінде анықтаймыз  
}
```

```
int x;    // мы создали переменную типа int с именем x
Button button1; // мы создали объект типа Button с именем button1
Button buttonPlus; // мы создали еще один объект типа Button с именем buttonPlus

button1.flagClick= false; // переменная flagClick j, объекта button1 = false
button1.scanState();    // вызов метода scanState(), объекта button1
button1.setPinTime(12, 20); // вызов метода setPinTime (), объекта button1 с параметрами 12, 20
if ( button1.flagPress == true ) {
    // кнопка нажата }
```

Мысал * Әрбір батырманы басу жарық диодының күйін өзгертеді

```
#define LED_PIN 13    // светодиод подключен к выводу 13
#define BUTTON_PIN 12 // кнопка подключена к выводу 12

// Описание класса обработки сигналов кнопок
class Button {
public:
    boolean flagPress;    // признак кнопка сейчас нажата
    boolean flagClick;    // признак кнопка была нажата (клик)
    void scanState();     // метод проверки состояние сигнала
    void setPinTime(byte pin, byte timeButton); // метод установки номера вывода
и времени (числа) подтверждения

private:
    byte _buttonCount;    // счетчик подтверждений стабильного состояния
    byte _timeButton;     // время подтверждения состояния кнопки
    byte _pin;            // номер вывода
};
```

```
boolean ledState;    // переменная состояния светодиода
Button button1; // создание объекта типа Button с именем button1
void setup() {
  pinMode(LED_PIN, OUTPUT);    // определяем вывод 13 (светодиод) как выход
  button1.setPinTime(BUTTON_PIN, 15); // вызов метода установки объекта button1
  // с параметрами: номер вывода 12, число подтверждений 15
}

// бесконечный цикл с периодом 2 мс
void loop() {
  button1.scanState(); // вызов метода сканирования сигнала кнопки

  // блок управления светодиодом
  if ( button1.flagClick == true ) {
    // было нажатие кнопки
    button1.flagClick = false; // сброс признака клика
    ledState = !ledState;      // инверсия состояния светодиода
    digitalWrite(LED_PIN, ledState); // вывод состояния светодиода
  }

  delay(2); // задержка на 2 мс
}
```

```
// метод проверки состояния кнопки
// flagPress= true - нажата
// flagPress= false - отжата
// flagClick= true - была нажата (клик)
void Button::scanState() {
    if ( flagPress == (! digitalRead(_pin)) ) {
        // состояние сигнала осталось прежним
        _buttonCount= 0; // сброс счетчика состояния сигнала
    }
    else {
        // состояние сигнала изменилось
        _buttonCount++; // +1 к счетчику состояния сигнала
        if ( _buttonCount >= _timeButton ) {
            // состояние сигнала не менялось заданное время
            // состояние сигнала стало устойчивым
            flagPress= !flagPress; // инверсия признака состояния
            _buttonCount= 0; // сброс счетчика состояния сигнала
            if ( flagPress == true ) flagClick= true; // признак клика на нажатие
        }
    }
}
```

```
// метод установки номера вывода и времени подтверждения  
void Button::setPinTime(byte pin, byte timeButton) {  
    _pin= pin;  
    _timeButton= timeButton;  
    pinMode(_pin, INPUT_PULLUP); // определяем вывод как вход  
}
```

Ардуино бағдарламаларындағы класстар конструкторлары.

- Конструктор класс мүшесі болып табылады;
- қайтарылатын мәннің түрі жоқ, тіпті void;
- класс атымен бірдей.

Конструкторлар сипаттамасы

Button(byte pin, byte timeButton); // описание конструктора

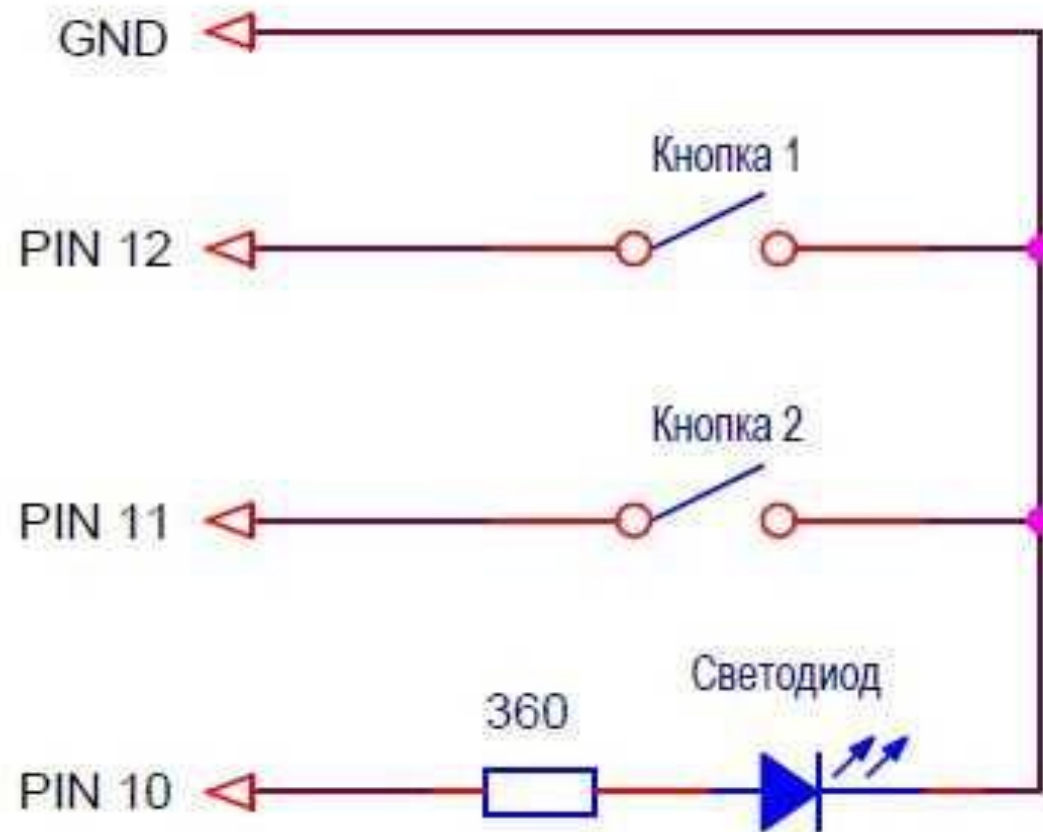
Button класы конструкторының сипаттамасы

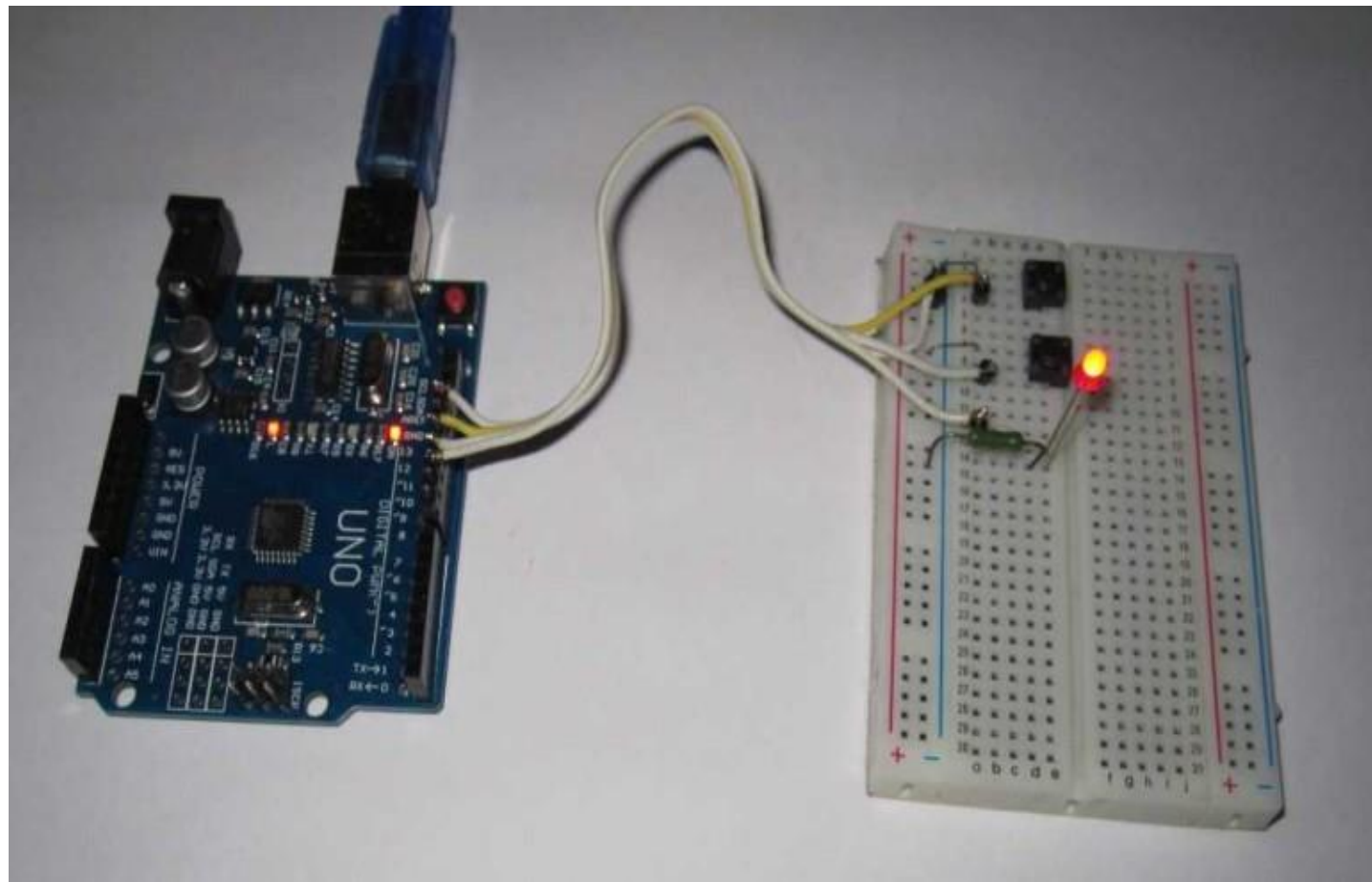
```
Button::Button(byte pin, byte timeButton) {  
    _pin= pin;  
    _timeButton= timeButton;  
    pinMode(_pin, INPUT_PULLUP); // определяем вывод как вход  
}
```

МЫСАЛЫ

Button button1(BUTTON_1_PIN, 15); // создание объекта для кнопки 1 с параметрами BUTTON_1_PIN и 15

Екі нысанды бағдарлама (батырмалармен).





Мысалы

/ Қосылған екі батырма мен жарық диоды
* 1-түймешікті әрбір басу Ардуино платасындағы жарық
диодының күйін теріске шығарады
* 2-түймешікті әрбір басу макеттік платта жарық диодының
күйін теріске шығарады*/*

*#define LED_1_PIN 13 // номер вывода светодиода 1 равен 13
#define BUTTON_1_PIN 12 // номер вывода кнопки 1 равен 12
#define BUTTON_2_PIN 11 // номер вывода кнопки 2 равен 11
#define LED_2_PIN 10 // номер вывода светодиода 2 равен 10*

// Описание класса обработки сигналов кнопок

class Button {

public:

Button(byte pin, byte timeButton); // описание конструктора

boolean flagPress; // признак кнопка сейчас нажата

boolean flagClick; // признак кнопка была нажата (клик)

void scanState(); // метод проверки состояние сигнала

*void setPinTime(byte pin, byte timeButton); // метод установки номера
вывода и времени (числа) подтверждения*

private:

*byte _buttonCount; // счетчик подтверждений стабильного
состояния*

byte _timeButton; // время подтверждения состояния кнопки

byte _pin; // номер вывода

};

```
boolean ledState1;    // переменная состояния светодиода 1  
boolean ledState2;    // переменная состояния светодиода 2  
  
Button button1(BUTTON_1_PIN, 15); // создание объекта для кнопки 1  
Button button2(BUTTON_2_PIN, 15); // создание объекта для кнопки 2  
  
void setup() {  
    pinMode(LED_1_PIN, OUTPUT);    // определяем вывод светодиода 1  
    как выход  
    pinMode(LED_2_PIN, OUTPUT);    // определяем вывод светодиода 2  
    как выход  
}
```


// бесконечный цикл с периодом 2 мс

void loop() {

button1.scanState(); // вызов метода сканирования сигнала кнопки 1

button2.scanState(); // вызов метода сканирования сигнала кнопки 2

// блок управления светодиодом 1

if (button1.flagClick == true) {

// было нажатие кнопки

button1.flagClick= false; // сброс признака клика

ledState1= ! ledState1; // инверсия состояния светодиода 1

digitalWrite(LED_1_PIN, ledState1); // вывод состояния светодиода 1

}

// блок управления светодиодом 2

if (button2.flagClick == true) {

// было нажатие кнопки

button2.flagClick= false; // сброс признака клика

ledState2= ! ledState2; // инверсия состояние светодиода 2

digitalWrite(LED_2_PIN, ledState2); // вывод состояния светодиода 2

}

delay(2); // задержка на 2 мс

}

```
// метод проверки состояния кнопки
// flagPress= true - нажата
// flagPress= false - отжата
// flagClick= true - была нажата (клик)
void Button::scanState() {
    if ( flagPress == (! digitalRead(_pin)) ) {
        // состояние сигнала осталось прежним
        _buttonCount= 0; // сброс счетчика состояния сигнала
    }
    else {
        // состояние сигнала изменилось
        _buttonCount++; // +1 к счетчику состояния сигнала
        if ( _buttonCount >= _timeButton ) {
            // состояние сигнала не менялось заданное время
            // состояние сигнала стало устойчивым
            flagPress= !flagPress; // инверсия признака состояния
            _buttonCount= 0; // сброс счетчика состояния сигнала

            if ( flagPress == true ) flagClick= true; // признак клика на нажатие
        }
    }
}
```

// метод установки номера вывода и времени подтверждения
void Button::setPinTime(byte pin, byte timeButton) {

_pin= pin;

_timeButton= timeButton;

pinMode(_pin, INPUT_PULLUP); // определяем вывод как вход
}

// описание конструктора класса Button

Button::Button(byte pin, byte timeButton) {

_pin= pin;

_timeButton= timeButton;

pinMode(_pin, INPUT_PULLUP); // определяем вывод как вход
}

Arduino үшін кітапхана құру

Дәріс 11

Кітапханада кемінде екі файл болуы керек:

- тақырып файлы (кеңейтімі .h);
- бастапқы код файлы (.cpp кеңейтімі).

кітапхана ақпараты

*/**

кітапхана ақпараты/*

Кітапхана әлі қосылмағанын тексеру

```
#ifndef Button_h // егер Button кітапханасы қосылмаған болса  
#define Button_h // сонда оны қосамыз
```

```
// .....
```

```
#endif
```

```
#include "Arduino.h"
```


*/**

Button.h - түймелер контактілерін және басқа компоненттердің сигналдарын параллельді процесс арқылы цифрлық сигналдарды өңдеуге арналған кітапхана

Әдістердің бірі параллельді процесте жүйелі түрде шақырылуы керек:

void scanState(); // сынақ әдісі тұрақты сигнал күйін күту
void filterAvarage(); // сигналды орташа мән бойынша сүзу әдісі

Нәтижесінде белгілер қалыптасады:

scanState() әдісі үшін:

- түйме басылғанда flagPress= true*
- түйме басылғанда flagPress= false*
- түймесін басыңыз flagClick= true*

filterAvarage() әдісі үшін :

- сигнал төмен болғанда flagPress= true*
- жоғары сигналда flagPress= false*
- күйді жоғарыдан төменге өзгерту кезінде flagClick= true*

Button түріндегі нысан жасалған кезде параметрлері болады:

- түйме немесе сигнал қосылған PIN нөмірі*
- сигналды өңдеу уақыты (scanState() немесе filterAvarage() әдісін шақыру кезеңіне көбейтілген*

Button button1(12, 15); // сүзгі уақыты 30 мс (2 мс циклімен) 12 түйреуішке қосылған түйме үшін нысан жасау/*

```
// Kітапхана әлі қосылмағанын тексеру
#ifndef Button_h // если библиотека Button не подключена
#define Button_h // тогда подключаем ее

#include "Arduino.h"

// класс обработки сигналов
class Button {
public:
    Button(byte pin, byte timeButton); // конструктор
    boolean flagPress; // признак кнопка нажата (сигнал в низком уровне)
    boolean flagClick; // признак клика кнопки (фронт)
    void scanState(); // метод проверки ожидание стабильного состояния сигнала
    void filterAvarage(); // метод фильтрации по среднему значению
    void setPinTime(byte pin, byte timeButton); // установка номера вывода и времени
    фильтрации

private:
    byte _buttonCount; // счетчик времени фильтрации
    byte _timeButton; // время фильтрации
    byte _pin; // номер вывода
};

#endif
```

Button.cpp кітапханасының бастапқы файлы

```
#include "Arduino.h"  
#include "Button.h"
```

*/**

*Button.h - библиотека для цифровой обработки сигналов контактов кнопок
и сигналов других компонентов параллельным процессом*

В параллельном процессе должен регулярно вызываться один из методов:

*void scanState(); // метод проверки ожидание стабильного состояния сигнала
void filterAvarage(); // метод фильтрации сигнала по среднему значению*

В результате формируются признаки:

для метода scanState():

- при нажатой кнопке flagPress= true*
- при отжатой кнопке flagPress= false*
- при нажатии на кнопку flagClick= true*

для метода filterAvarage() :

- при сигнале низкого уровня flagPress= true*
- при сигнале высокого уровня flagPress= false*
- при изменении состояния с высокого на низкий flagClick= true*

Объект типа Button при создании имеет параметры:

- номер вывода, к которому подключена кнопка или сигнал*
- время обработки сигнала (умножается на период вызова метода scanState() или filterAvarage())*

*Button button1(12, 15); // создание объекта для кнопки, подключенной к 12 выводу
с временем фильтрации 30 мс (при цикле 2 мс)*

<http://mypractic.ru/urok-8-cifrovaya-filtraciya-signalov-v-programmax-dlya-arduino.html>
**/*

```
#include "Arduino.h"
#include "Button.h"
// метод фильтрации сигнала по среднему значению
// при сигнале низкого уровня flagPress= true
// при сигнале высокого уровня flagPress= false
// при изменении состояния с высокого на низкий flagClick= true
void Button::filterAvarage() {
    if ( flagPress != digitalRead(_pin) ) {
        // состояние кнопки осталось прежним
        if ( _buttonCount != 0 ) _buttonCount--; // счетчик подтверждений - 1 с
ограничением на 0  }
    else {
        // состояние кнопки изменилось
        _buttonCount++; // +1 к счетчику подтверждений
        if ( _buttonCount >= _timeButton ) {
            // состояние сигнала достигло порога _timeButton
            flagPress= ! flagPress; // инверсия признака состояния
            _buttonCount= 0; // сброс счетчика подтверждений
            if ( flagPress == true ) flagClick= true; // признак клика кнопки
        }
    }
}
```

```
// при нажатой кнопке flagPress= true
// при отжатой кнопке flagPress= false
// при нажатии на кнопку flagClick= true
void Button::scanState() {
    if ( flagPress != digitalRead(_pin) ) {
        // признак flagPress = текущему состоянию кнопки
        // (инверсия т.к. активное состояние кнопки LOW)
        // т.е. состояние кнопки осталось прежним
        _buttonCount= 0; // сброс счетчика подтверждений состояния кнопки }
    } else {
        // признак flagPress не = текущему состоянию кнопки
        // состояние кнопки изменилось
        _buttonCount++; // +1 к счетчику состояния кнопки
        if ( _buttonCount >= _timeButton ) {
            // состояние кнопки не мянялось в течение заданного времени
            // состояние кнопки стало устойчивым
            flagPress= ! flagPress; // инверсия признака состояния
            _buttonCount= 0; // сброс счетчика подтверждений состояния кнопки
            if ( flagPress == true ) flagClick= true; // признак фронта кнопки на нажатие
        }
    }
}
```

// метод установки номера вывода и времени подтверждения
void Button::setPinTime(byte pin, byte timeButton) {

_pin= pin;
_timeButton= timeButton;
pinMode(_pin, INPUT_PULLUP); // определяем вывод как вход
}

// описание конструктора класса Button
Button::Button(byte pin, byte timeButton) {

_pin= pin;
_timeButton= timeButton;
pinMode(_pin, INPUT_PULLUP); // определяем вывод как вход
}

keywords.txt файлы

Button KEYWORD1

scanState KEYWORD2

filterAvarage KEYWORD2

setPinTime KEYWORD2

- Arduino IDE іске қосу.
- **Файл -> Настройки -> Размещение папки скетчей** мысалы, қою **D:\Arduino Projects**.
- Бұл қалтада қалта **libraries** жасаңыз (D:\Arduino Projects\libraries).
- **libraries** қалтасында жаңа **Button** кітапханасы үшін қалтаны жасаңыз (D:\Arduino Projects\libraries\Button).
- Осы қалтада **Button.h**, **Button.cpp** және **keywords.txt** файлдарын көшіріңіз.

/ К плате Ардуино подключены 2 кнопки и светодиод
* Каждое нажатие кнопки 1 инвертирует состояние светодиода на плате Ардуино
* Каждое нажатие кнопки 2 инвертирует состояние светодиода на макетной плате */*

#include <Button.h>

#define LED_1_PIN 13 // светодиод 1 подключен к выводу 13

#define BUTTON_1_PIN 12 // кнопка 1 подключена к выводу 12

#define BUTTON_2_PIN 11 // кнопка 2 подключена к выводу 11

#define LED_2_PIN 10 // светодиод 2 подключен к выводу 10

boolean ledState1; // переменная светодиода 1

boolean ledState2; // переменная светодиода 2

Button button1(BUTTON_1_PIN, 15); // создание объекта для кнопки 1

Button button2(BUTTON_2_PIN, 15); // создание объекта для кнопки 2

void setup() {

pinMode(LED_1_PIN, OUTPUT); // определяем выводы

светодиодов как выходы

pinMode(LED_2_PIN, OUTPUT);

}

```
// цикл с периодом 2 мс  
void loop() {
```

```
    button1.filterAvarage(); // вызов метода фильтрации по среднему для кнопки 1  
    button2.scanState(); // вызов метода ожидания стабильного состояния для  
кнопки 2
```

```
    // блок управления светодиодами 1
```

```
    if ( button1.flagClick == true ) {
```

```
        // был клик кнопки
```

```
        button1.flagClick= false; // сброс признака
```

```
        ledState1= ! ledState1; // инверсия состояние светодиода
```

```
        digitalWrite(LED_1_PIN, ledState1); // вывод состояния светодиода  
    }
```

```
    // блок управления светодиодами 2
```

```
    if ( button2.flagClick == true ) {
```

```
        // был клик кнопки
```

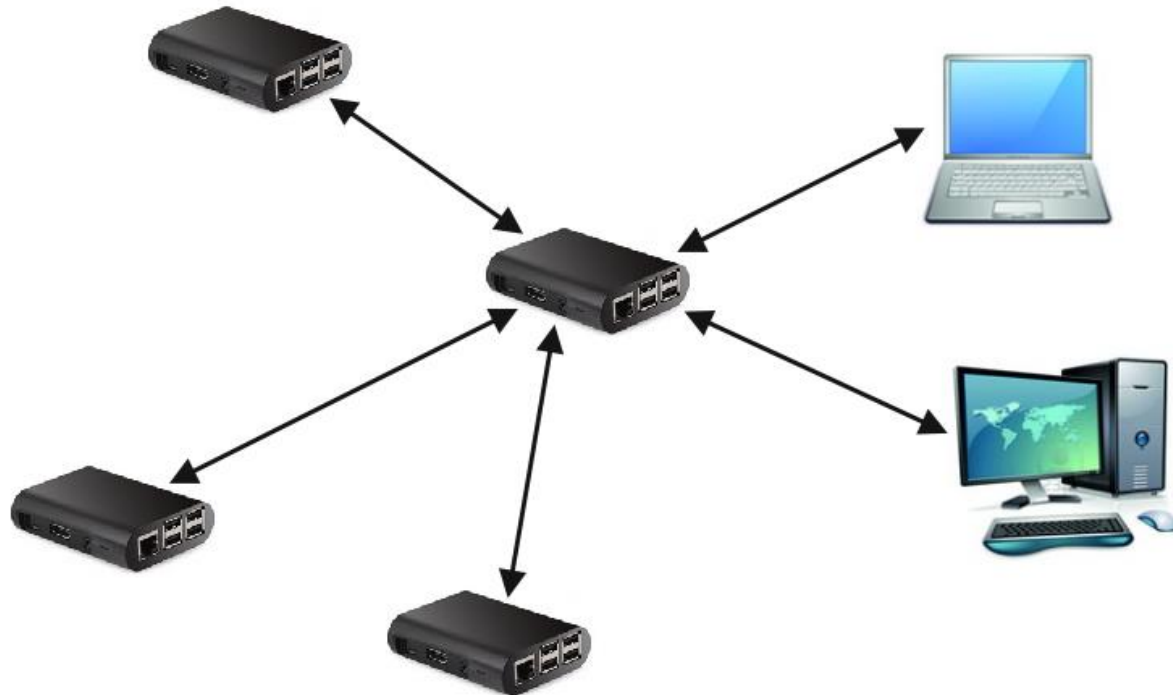
```
        button2.flagClick= false; // сброс признака
```

```
        ledState2= ! ledState2; // инверсия состояние светодиода
```

```
        digitalWrite(LED_2_PIN, ledState2); // вывод состояния светодиода  
    }
```

```
    delay(2); // задержка 2 мс  
}
```

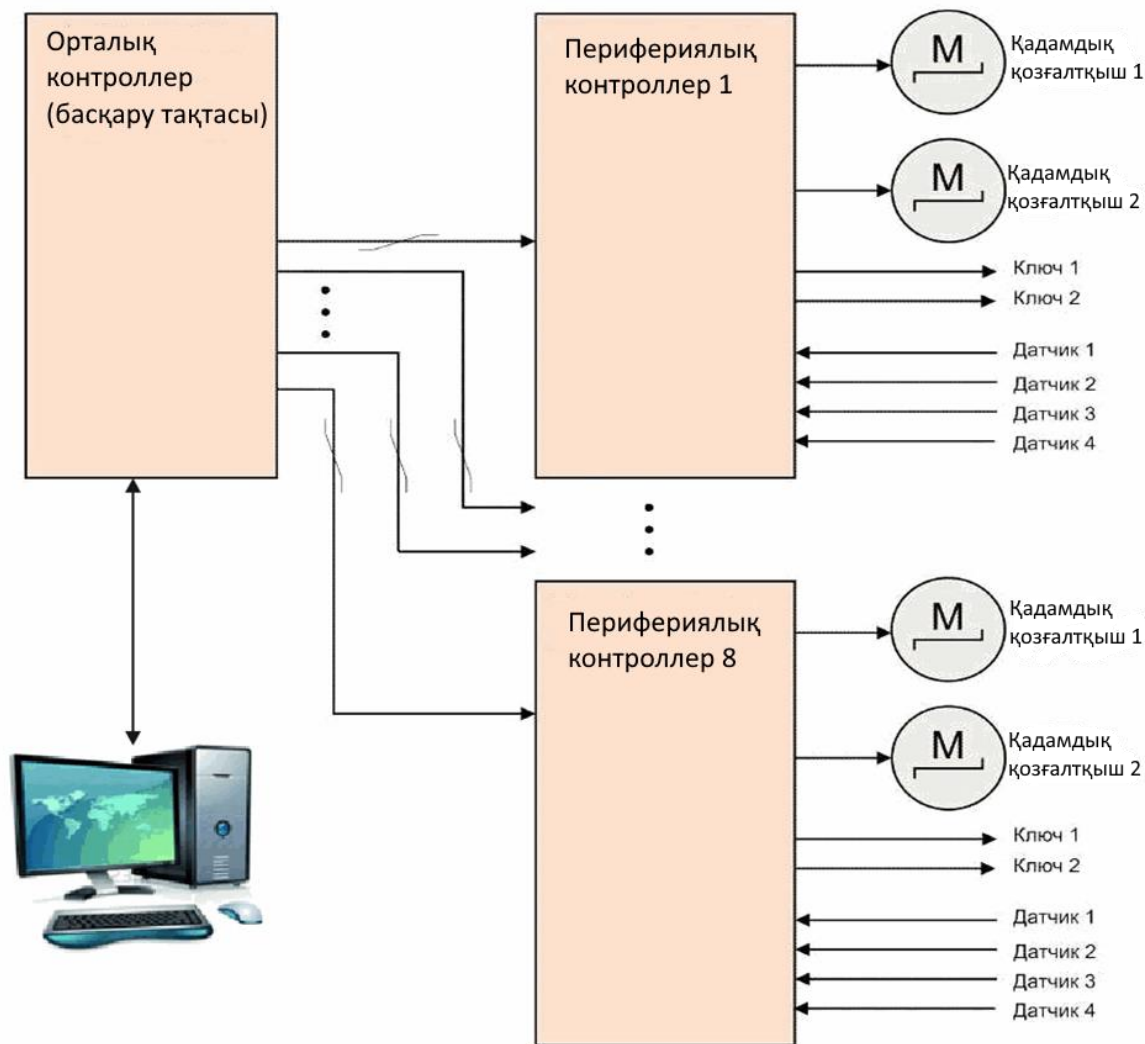
Ардуино төлемдері арасындағы деректермен алмасу



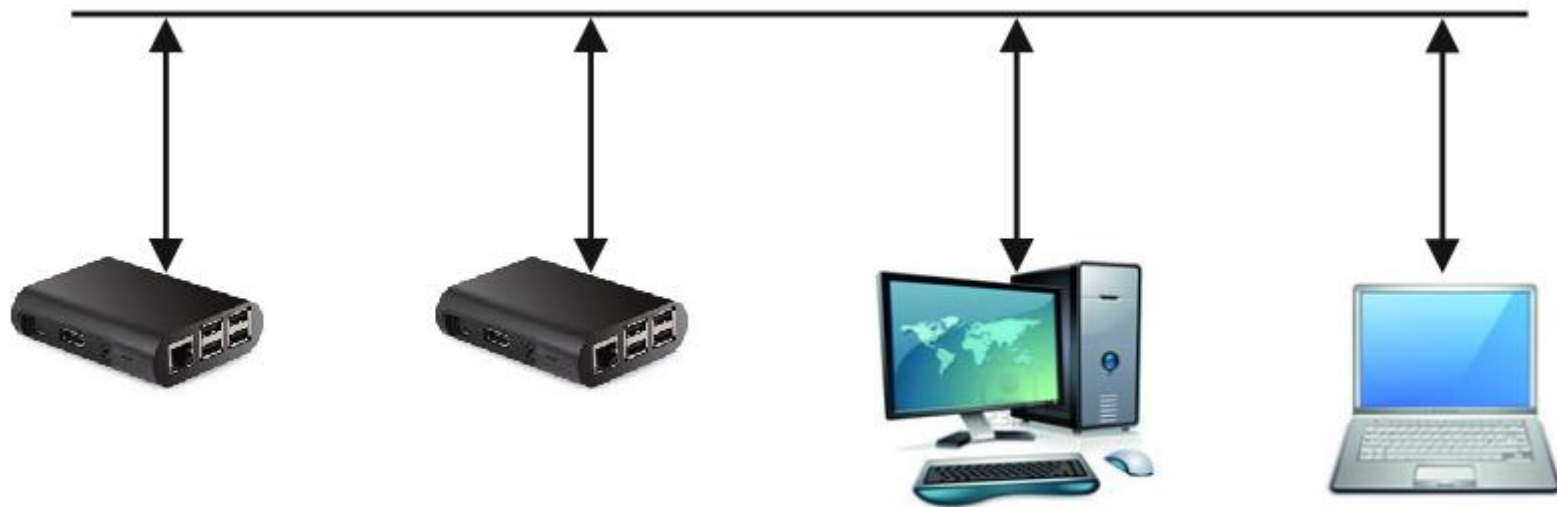
Шағын басқару және бақылау жүйелерінің құрылымдық схемасы



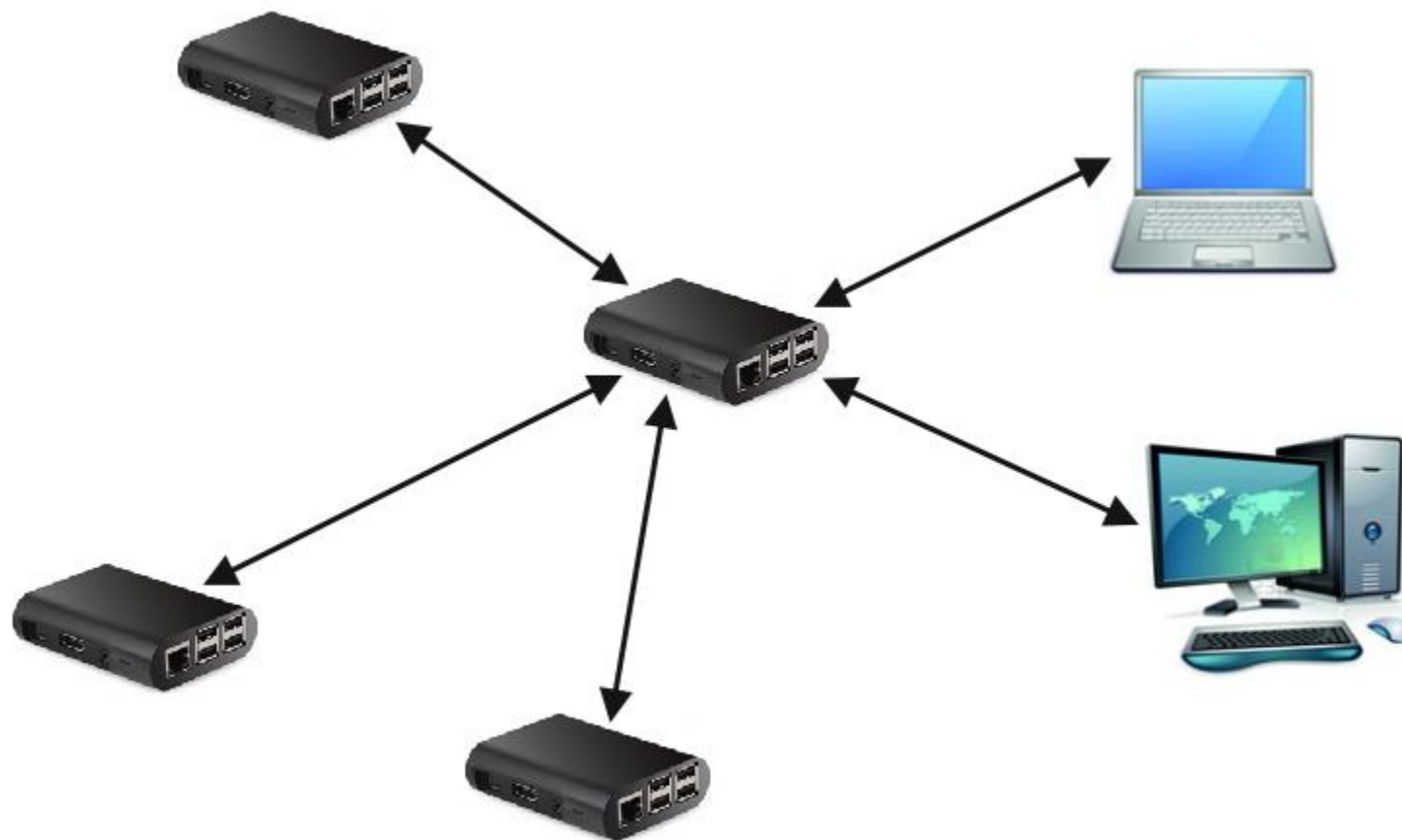
Ампулаларды толтыру және тығыздау машинасын басқару жүйесі.



топология Жалпы шина



Радиалды интерфейс (жұлдыз топологиясы)



UART интерфейсі

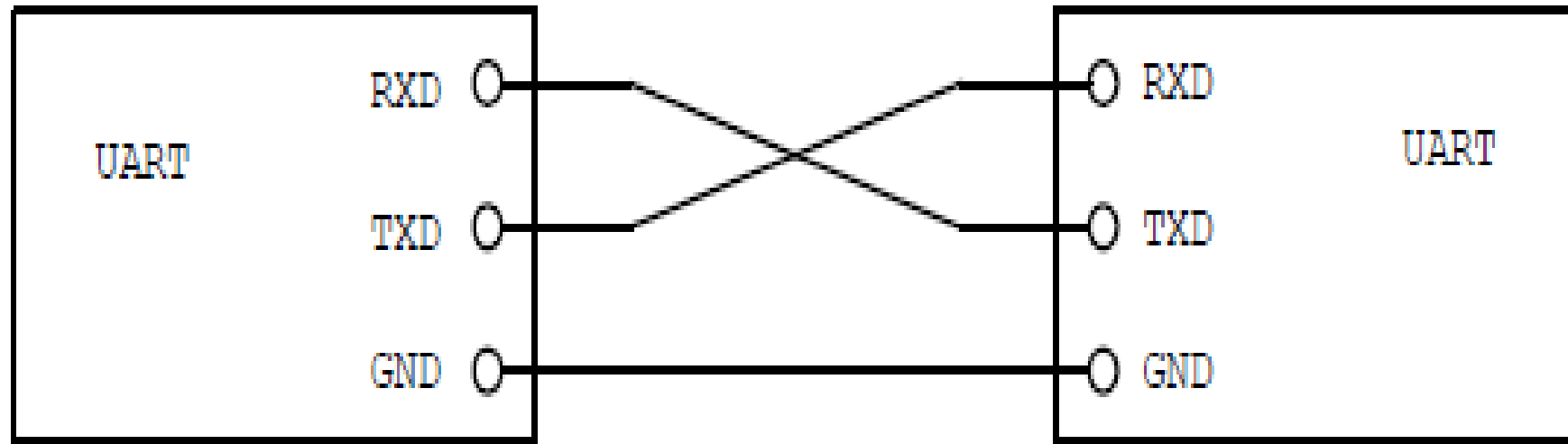
UART-те деректер алмасу үшін 2 сигнал бар:

- RX – деректерді қабылдау өтетін кіру,
- TX – бұл деректер жіберілетін шығыс.

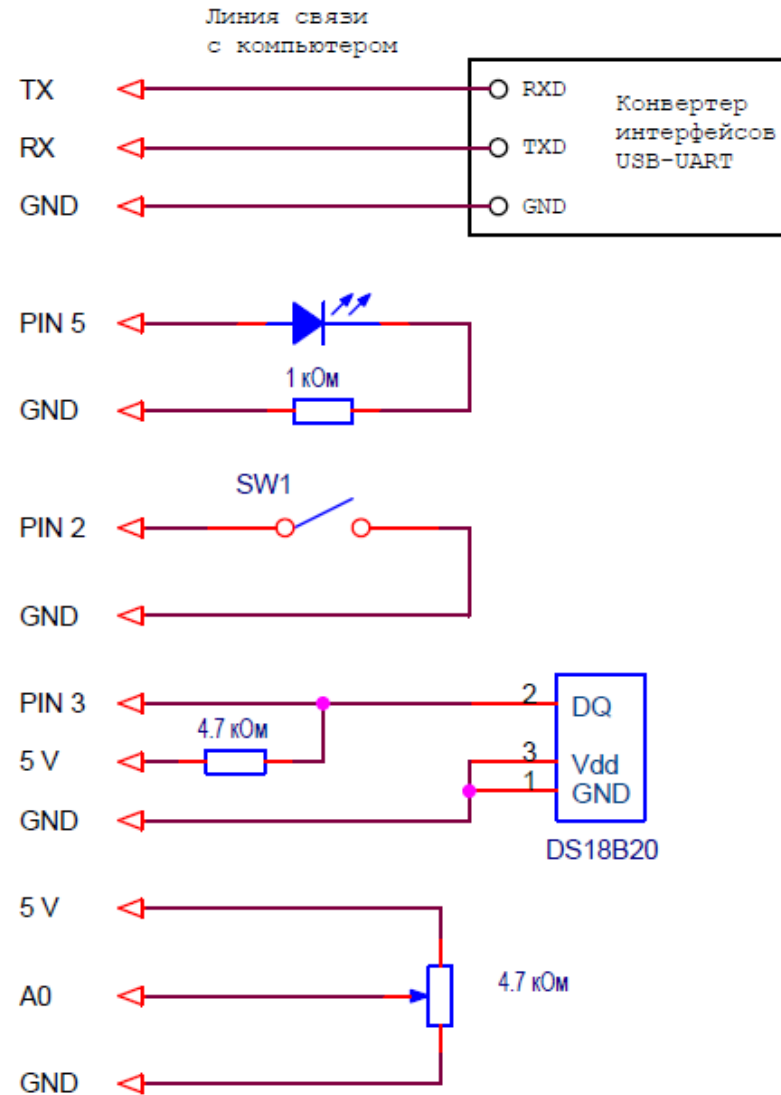
Екі сигнал да дискретті, CMOS логикалық деңгейлері бар, яғни.:

- логикалық 0 деңгейі шамамен 0 В;
- логикалық деңгейі 1 шамамен 5 В.

Қосылу



Жергілікті контроллер схемасы



алмасу протоколы

**Команда: Адрес -> Операция коды -> Деректер -> Бақылау коды
->**

Жауап: <- Деректер <- Бақылау коды

Пәрмен пішімі

Байт нөмірі	Сан пішімі	Мақсат
0	byte	Жұмыс коды + контроллер адресі (0x10)
1	byte	Жарық диодының жай-күйі (кіші бит)
2	byte	Бақылау коды (байт 0 ^ байт 1 ^ 0xe5)

Жауабы хаттамада

Байт нөмірі	Сан пішімі	Мақсат
0 ... 3	float	Температура
4 ... 7	float	Кернеу
8	byte	Түйме күйі (кіші бит)
9	byte	Резерв
10, 11	int	Бақылау сомасы (байттар сомасы $0 \dots 9 \wedge 0\text{ха}1\text{е}3$)

*// локальный контроллер
// радиальный интерфейс UART*

#include <MsTimer2.h>

#include <OneWire.h>

#include <Button.h>

OneWire sensTmp (3); // датчик подключен к выводу 3

Button button1(2, 30); // кнопка подключена к выводу 2

float temperature; // температура

float voltage; // напряжение

byte bufSensTmp[9]; // буфер данных датчика температуры

int timeCount; // счетчик времени

void setup() {

MsTimer2::set(1, timerInterrupt); // прерывания по таймеру 1 мс

MsTimer2::start(); // разрешаем прерывание по таймеру

Serial.begin(9600); // скорость 9600

}

```
void loop() {  
  //----- измерение температуры  
  if (timeCount < 50) {  
    timeCount= 50;  
    sensTmp.reset(); // сброс шины  
    sensTmp.write(0xCC, 1); // пропуск ROM  
    sensTmp.write(0x44, 1); // инициализация измерения  
  }  
  // задержка 0,9 сек  
  if(timeCount > 950) {  
    timeCount= 0;  
    sensTmp.reset(); // сброс шины  
    sensTmp.write(0xCC, 1); // пропуск ROM  
    sensTmp.write(0xBE, 1); // команда чтения памяти датчика  
    sensTmp.read_bytes(bufSensTmp, 9); // чтение памяти датчика, 9 байтов  
    if ( OneWire::crc8(bufSensTmp, 8) == bufSensTmp[8] ) { // проверка CRC  
      // данные правильные  
      temperature= (float)((int)bufSensTmp[0] | (((int)bufSensTmp[1]) << 8)) * 0.0625 +  
        0.03125;  
    }  
    else temperature= -200.; // ошибка измерения температуры  
  }  
}
```



```
//----- измерение напряжения  
voltage= (float)(analogRead(A0)) * 5. / 1024.;
```

```
//----- проверка измерений
```

```
Serial.print("T= ");
```

```
Serial.print(temperature);
```

```
Serial.print(" U= ");
```

```
Serial.print(voltage);
```

```
if (button1.flagPress == true) Serial.println(" PRESS");
```

```
else Serial.println(" FREE");
```

```
}
```

```
//----- обработчик прерывания 1 мс
```

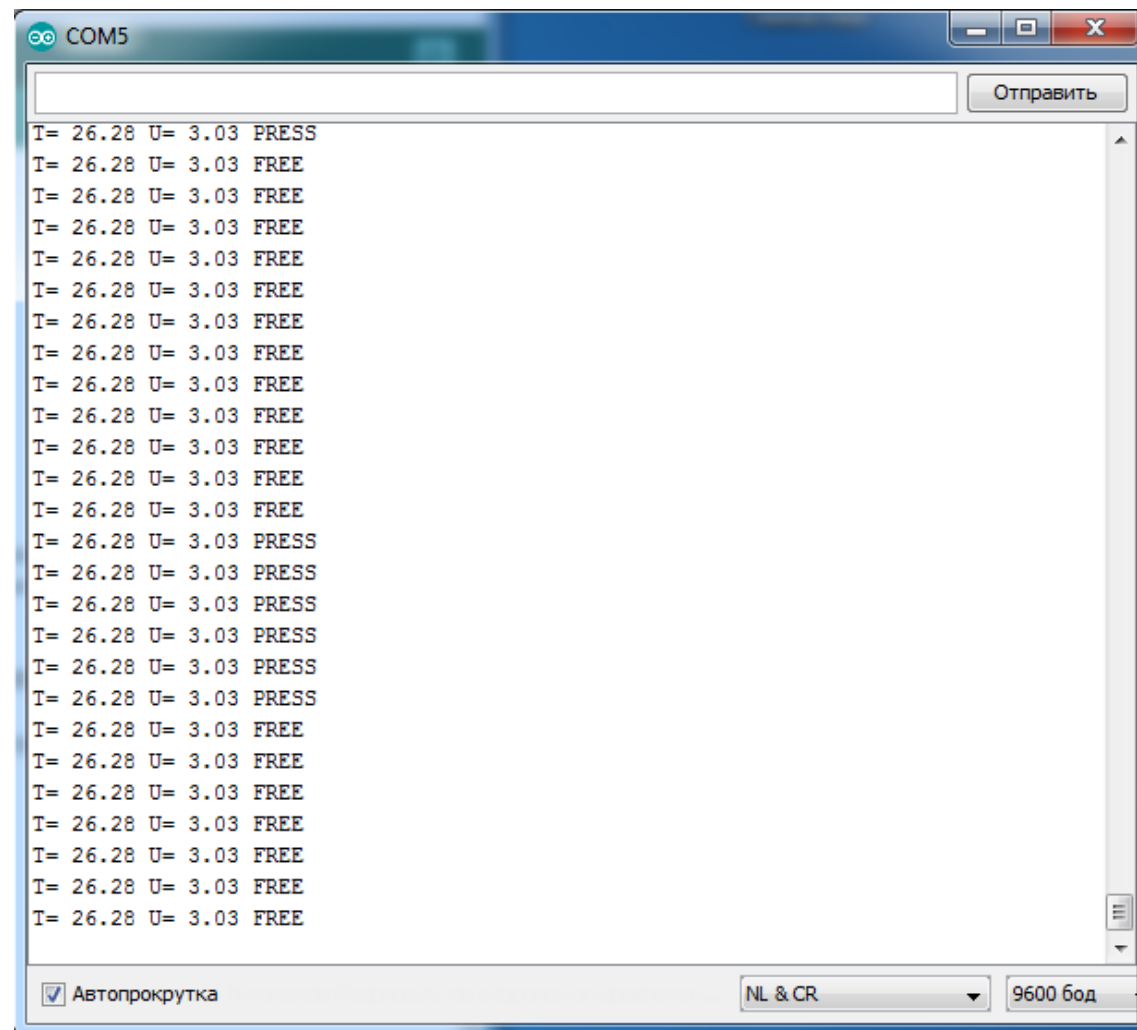
```
void timerInterrupt() {
```

```
timeCount++;
```

```
button1.scanState(); // вызов метода обработки состояния кнопки
```

```
}
```

Тексеру



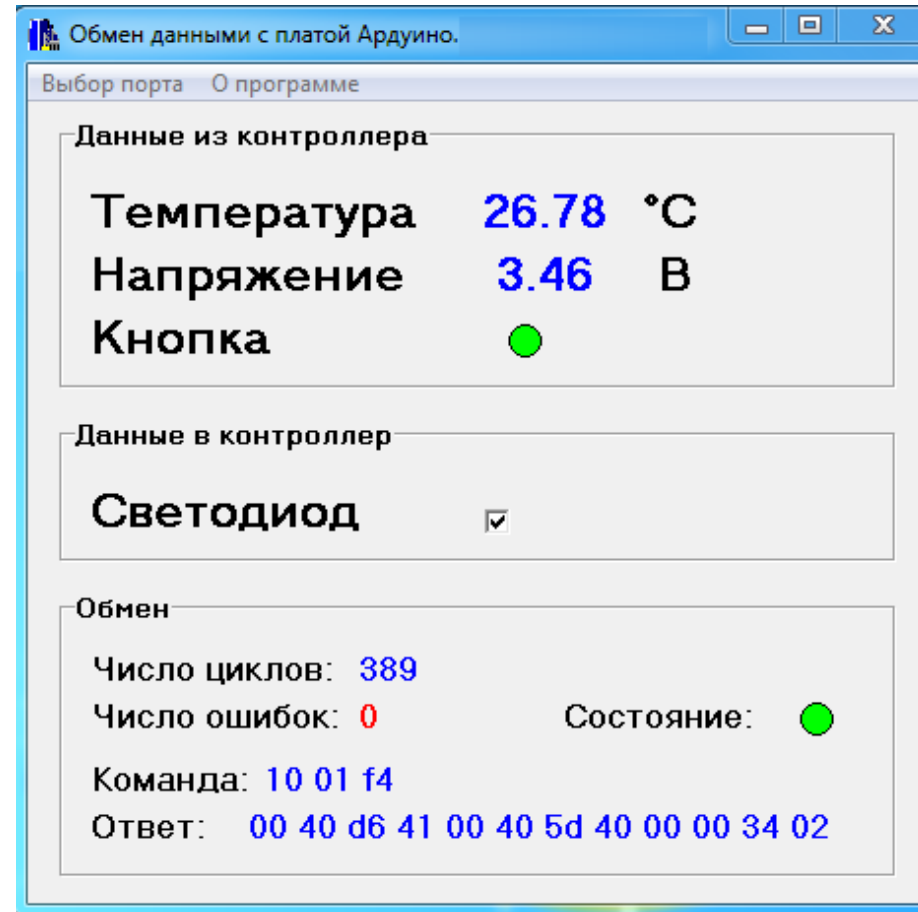
перезгрузка результатов измерений в буфер
noInterrupts;
** (float *)dataSerialBuf = temperature;*
** (float *)(dataSerialBuf+4) = voltage;*
if (button1.flagPress == true) dataSerialBuf[8]= 1;
else dataSerialBuf[8]= 0;
dataSerialBuf[9]= 0;
interrupts;
}

```
//----- обмен данными
timeOutCount++;
n= Serial.available(); // число принятых байтов
if (n == 0) timeOutCount= 0; // данных нет
else if (n == 3) {
// принята команда, 3 байта
// чтение команды в буфер
byte buf[3];
buf[0]= Serial.read();
buf[1]= Serial.read();
buf[2]= Serial.read();
// проверка
if ( (buf[0] == 0x10) && ((buf[0] ^ buf[1] ^ 0xe5) == buf[2]) ) {
// правильно
if ( (buf[1] & 1) == 0) digitalWrite(5, LOW); // управление светодиодом
else digitalWrite(5, HIGH);
```

```
// ответ
unsigned int sum= 0; // контрольная сумма
for (int i=0; i<10; i++) {
    Serial.write(dataSerialBuf[i]);
    sum += dataSerialBuf[i];
}
// контрольная сумма ответа
sum ^= 0xa1e3;
Serial.write( * ((byte *)& sum));
Serial.write( * (((byte *)& sum) + 1));
}
else {
    // неправильно, сброс порта
    timeOutCount= 0;
    while (true) { if (Serial.read() == 0xffff) break;}
}
}
```

```
else if (n > 3) {  
    // принято больше данных, неправильно, сброс порта  
    timeOutCount= 0;  
    while (true) { if (Serial.read() == 0xffff) break;}  
}  
  
else {  
    // не все байты приняты, проверка тайм-аута  
    if (timeOutCount > TIME_OUT) {  
        // сброс порта  
        timeOutCount= 0;  
        while (true) { if (Serial.read() == 0xffff) break;}  
    }  
}
```

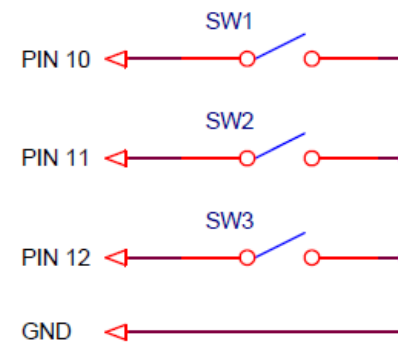
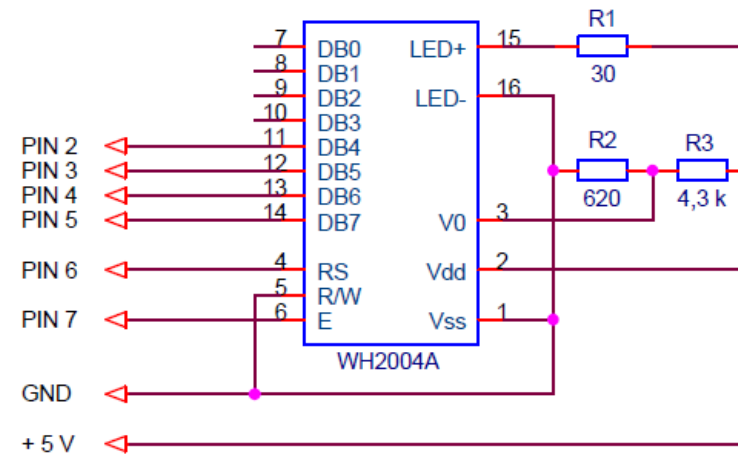
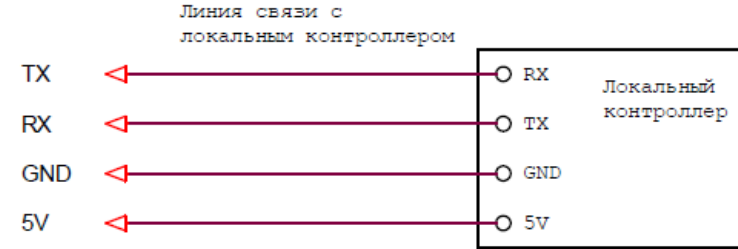
Жоғарғы деңгейдегі бағдарлама



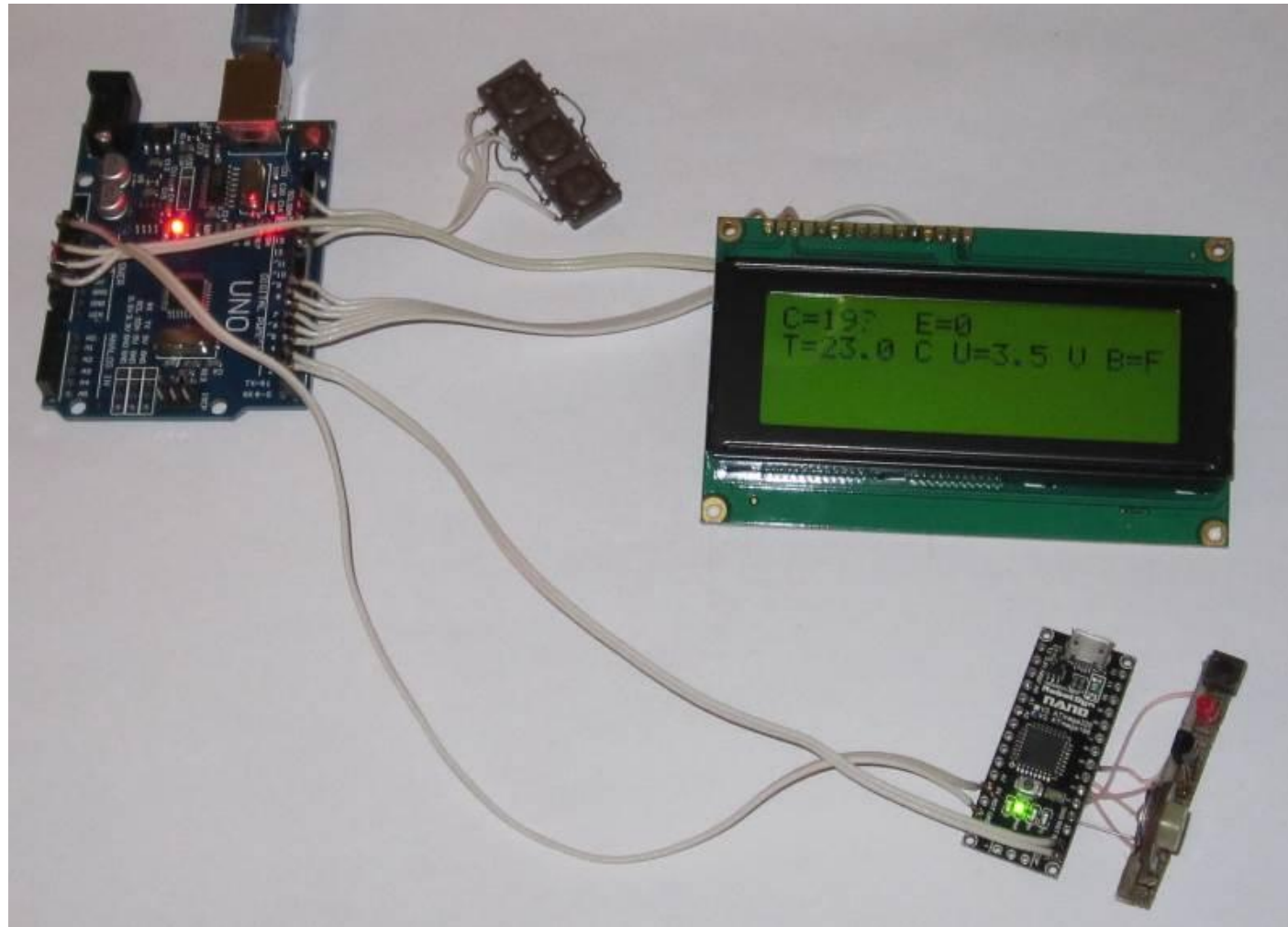
Обмен данными между платами Ардуино через интерфейс UART



Схема центрального контроллера



Собранная система из двух контроллеров



интерфейс между основной программой и
модулем обмена данными по сети

*byte dataFromLC[10]; // буфер данных для приема из локального
контроллера*

*byte dataToLC; // данные для передачи в локальный контроллер
(светодиод)*

byte stateCommin; // состояние обмена

// (0 - успешно закончено, 1 - пуск, 2 - продолжается, 3 - ошибка)

скетч программы управления обменом через интерфейс UART

```
#include <MsTimer2.h>

#define TIME_OUT 25 // время тайм-аута ответа (мс)

byte dataFromLC[10]; // буфер данных для приема из локального контроллера
byte dataToLC; // данные для передачи в локальный контроллер (светодиод)
byte stateComm; // состояние обмена
// (0 - успешно закончено, 1 - пуск, 2 - продолжается, 3 - ошибка)
byte timeOutCount; // счетчик времени тайм-аута обмена

void setup() {
  MsTimer2::set(1, timerInterrupt); // прерывание по таймеру 1 мс
  MsTimer2::start(); // разрешаем прерывание
  Serial.begin(9600); // 9600 бод
}

void loop() {
}
```

- *//----- обработчик прерывания 1 мс*
void timerInterrupt() {
- *//----- обмен данными*
timeOutCount++; // счетчик тайм-аута ответа
- *if ((stateCommun == 0) || (stateCommun == 3)) {*
// обмен закончен без ошибкой или с ошибкой
// ожидание следующего пуска
timeOutCount= 0;
}
- *else if (stateCommun == 1) {*
// пуск обмена, посылка команды
Serial.write(0x10); // код операции
Serial.write(dataToLC); // данные (светодиод)
Serial.write(0x10 ^ dataToLC ^ 0xe5); // контрольный код
stateCommun= 2; // переход на ожидание ответа
}

```
else if (stateCommun == 2) {  
    // ожидание ответа от локального контроллера  
    if (Serial.available() == 12) {  
        // принято 12 байтов  
        // чтение данных в буфер, проверка контрольной суммы  
        byte buf[10];  
        unsigned int sum= 0; // контрольная сумма  
        for (int i=0; i<10; i++) {  
            buf[i]= Serial.read();  
            sum += buf[i];  
        }  
        sum ^= 0xa1e3;  
        if ( ((* ((byte *)(& sum))) == Serial.read()) && ((* (((byte *)(& sum)) + 1)) ==  
Serial.read())) {  
            // контрольная сумма правильная  
            // перегрузка принятых данных в dataFromLC  
            for (int i=0; i<10; i++) {  
                dataFromLC[i] = buf[i];  
            }  
            stateCommun= 0; // операция успешно закончена  
        }  
    }  
}
```

```
else {  
    // контрольная сумма неправильная  
    // очистка буфера порта  
    while (true) { if (Serial.read() == 0xffff) break;}  
    stateCommun= 3; // окончание операции с ошибкой  
}  
}  
else {  
    // проверка тайм-аута  
    if (timeOutCount > TIME_OUT) {  
        // ошибка тайм-аута (ответ не пришел)  
        // очистка буфера порта  
        while (true) { if (Serial.read() == 0xffff) break;}  
        stateCommun= 3; // окончание операции с ошибкой  
    }  
}  
}  
else {  
    // сбой программы  
    stateCommun= 3;  
}  
}
```



Основная программа в цикле loop()

```
void loop() {  
  // инициализация обмена с локальным контроллером  
  // состояние кнопки  
  if (button1.flagPress == true) dataToLC= 1;  
  else dataToLC= 0;  
  stateComm= 1; // инициализация обмена  
  cyclCount++; // счетчик циклов  
  delay(50); // время на завершения обмена
```

```
if (stateCommun == 0) {  
    // данные успешно получены  
    // вывод данных на дисплей  
    disp.clear(); // очистка экрана  
    disp.print("C=");  
    disp.print(cyclCount);  
    disp.print(" E=");  
    disp.print(errorCount);  
    disp.setCursor(0, 1);  
    disp.print("T=");  
    disp.print(* ((float *)dataFromLC),1);  
    disp.print(" C U=");  
    disp.print(* ( ((float *)dataFromLC) +1 ),1);  
    if ( (dataFromLC[8] & 1) == 0) disp.print(" V B=F");  
    else disp.print(" V B=P");  
}  
else if (stateCommun == 3) {  
    // обмен завершен с ошибкой  
    errorCount++; // счетчик ошибок
```

```
disp.clear(); // очистка экрана
disp.print("C=");
disp.print(cyclCount);
disp.print(" E=");
disp.print(errorCount);
disp.setCursor(0, 1);
disp.print("ERROR");
}

delay(200);
}
```

Проверка





C=585 E=98
T=24.2 C U=3.5 U B=F

Стандартты UART интерфейсінің шуға төзімділігі және физикалық ортасы

Жергілікті желінің физикалық ортасы

Жергілікті желі туралы айтқанда, біз ең болмағанда тақталарды, көбінесе модульдерді немесе бөлек, құрылымдық аяқталған құрылғыларды желіге қосуды айтамыз. Бұл желінің құрамдас бөліктері (абоненттер) физикалық түрде бір-бірінен әртүрлі, жиі маңызды қашықтықта орналасуы мүмкін, әртүрлі байланыс желілерімен байланысты.

Жергілікті желінің физикалық ортасы

Абоненттер арасындағы деректер жергілікті желінің физикалық ортасы арқылы беріледі. Бұл желінің пайдаланушы қасиеттерін негізінен физикалық орта анықтайды. Егер біз осы немесе басқа желілік интерфейс туралы бірінші сұрақ қойсақ, ол қандай жылдамдықты қамтамасыз етеді. Содан кейін сұрақтардың екінші қатары әдетте:

- абоненттер арасындағы максималды қашықтық қандай;
- құрылғылар қандай кабельмен қосылған;
- жүйенің кедергілерге қаншалықты сезімталдығы;
- Гальваникалық оқшаулау және т.б. бар ма?

Жергілікті желінің физикалық ортасы

Бұл сұрақтардың барлығы деректерді берудің физикалық ортасына қатысты. Интерфейстің әрбір түрінің физикалық ортаға қойылатын өз талаптары бар. Бұл талаптар деректер жылдамдығына байланысты бірдей интерфейс үшін әртүрлі болуы мүмкін.

Әрі қарай интерфейстің әрбір түрі үшін параметрлері бар кестені ұсынамыз. Және сіз барлық дерлік параметрлер физикалық ортаны тікелей сипаттайтынын немесе оған қойылатын талаптарды анықтайтынын көресіз.

Деректерді беру сигналдарының кедергіден қорғалуының жалпы мәселелері

Радиоэлектрондық құрылғылардың электромагниттік үйлесімділігі өте күрделі тақырып. Оның өзіне арналған жеке сайты болуы керек. Бұл пән бойынша кітаптар ТОЕ заңдарына (электротехниканың теориялық негіздері), формулаларға, диаграммаларға, диаграммаларға толы. Бірақ іс жүзінде мұның бәрінің маңызы шамалы, өйткені өлшеуге, бағалауға болмайтын құбылыспен күресуге тура келеді. Әрине, электромагниттік кедергі деңгейі өлшенеді, бірақ бұл өте күрделі қымбат жабдықты қажет етеді. Өлшемдер белгілі бір жағдайларда орындалады. Содан кейін құрылғы мүлдем басқа жағдайларда, басқа кедергі деңгейімен және басқа соңғы нәтижемен жұмыс істейді.

Деректерді беру сигналдарының кедергіден қорғалуының жалпы мәселелері

Байланыс желісі бойынша деректерді дұрыс жіберуге кедергі келтіретін келесі факторларды бөлуге болады:

- паразиттік желі параметрлерімен байланысты деректерді беру желісіндегі сигналдың бұрмалануы: белсенді кедергі, индуктивтілік, сыйымдылық;
- өзара индуктивтілік пен сыйымдылықпен байланысты әртүрлі сигналдардың сызықтарының өзара әсері;
- бөгеуілдер, сыртқы немесе ішкі электромагниттік өрістердің кедергілері;
- гальваникалық оқшауланбаған құрылғылардың жалпы сымында жердегі токтардың немесе қуат көзінің ағып кетуіне байланысты пайда болатын жерге токтар.

Деректерді беру сигналдарының кедергіден қорғалуының жалпы мәселелері

Деректерді жіберу қателеріне біздің оларды байланыс желілері арқылы беру және қабылдау жолымыз әсер етеді. Анау. бұл жіберу кезінде сигналдардың бұрмалануының қаншалықты рұқсат етілгенін анықтайтын интерфейс параметрлері және физикалық орта бұл талаптарды қамтамасыз етуі керек.

жүйенің шуға төзімділігін анықтайтын желілік интерфейстердің параметрлері

- Кіріс сигналдарының деңгейі. Шуға төзімділікті анықтайтын маңызды параметрлердің бірі. Байланыс желісіндегі сигнал амплитудасы неғұрлым үлкен болса, оның фонында кедергі азырақ байқалады, соғұрлым оларды бөлу оңайырақ болады.
- Қабылдағыш өнімділігі. Жоғары жылдамдықты кірістер қысқа кедергілерге жауап береді, бұл шу иммунитетін айтарлықтай төмендетеді. Бұл сипат деректер жылдамдығы талаптарына қайшы келеді. Жоғары жылдамдықта жоғары жылдамдықты қабылдағыштар өте қажет. Бірақ тіпті 9600 жылдамдықта болса да, барлық кедергілерді ұстайтын ультра жылдам оптопарларды орнатуға болмайды.

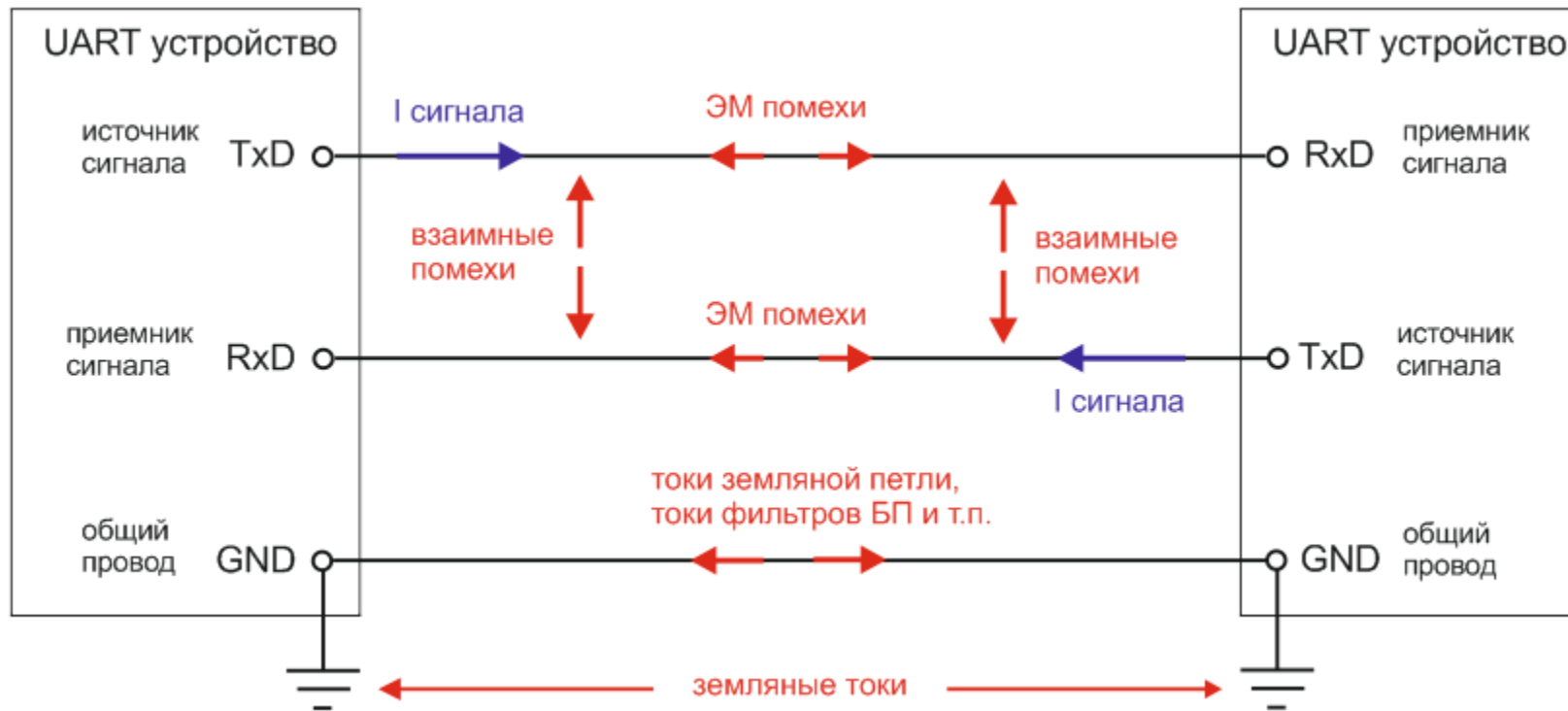
жүйенің шуға төзімділігін анықтайтын желілік интерфейстердің параметрлері

- Сигнал қабылдағыштардың кіріс кедергісі. Ол неғұрлым аз болса, жұмыс істеу үшін кедергіге соғұрлым көп ток қолданылуы керек, кірістің шуға төзімділігі соғұрлым жоғары болады. Бірақ қабылдағыштардың төмен кіріс кедергісі таратқыштардан жоғары шығыс токтарын талап етеді. Сондай-ақ, кіріс кедергісі ұзын жолдардағы сигналдардың бұрмалануына әсер етеді. Бірақ бұл туралы кейінірек.
- Таратқыштардың шығыс кедергісі. Алдыңғы абзацтағы ескертуден басқа, бұл параметр жолдағы сигналдың бұрмалануына да әсер етеді. Кейде әдейі көтеріледі.

жүйенің шуға төзімділігін анықтайтын желілік интерфейстердің параметрлері

- Дифференциалды сигналдарды қолдану желінің шуға қарсы иммунитетін айтарлықтай арттырады. Бұл туралы кейінірек, деректерді берудің осы әдісімен интерфейстерді пайдаланған кезде.
- Құрылғылар мен сигналдарды бір-бірінен бөлек гальваникалық оқшаулау шуға төзімділікті жақсартудың ең жақсы және жан-жақты әдістерінің бірі болып табылады.

бөлек сымдар мен кедергілерді қолданатын 2 UART құрылғысының қосылу схемасы



UART интерфейсінің жалпы қасиеттері мен параметрлері

Параметр	Мағынасы
Топология	Радиалды интерфейс
Байланыс желісі	Ортақ сыммен 2 сигнал
Гальваникалық оқшаулау	Жоқ
Деректер алмасу режимы	Асинхронды, толық дуплексті
Сигналдар деңгейі	CMOS, 0 – 5 В
Стандартты қолдану	Жалпы қуаты бар гальваникалық байланысты құрылғылар арасында деректер алмасу.

RS-232 және RS-422 радиалды интерфейстері

RS-232 және RS-422 радиалды интерфейстері

Алдыңғы сабақта шуға төзімділікке көбірек әсер ететін интерфейс параметрлері келтірілген. Бірінші орында – байланыс желісіндегі сигналдар деңгейі. Сигнал амплитудасы неғұрлым үлкен болса, кедергі үшін сигналды қабылданбайтын күйге бұрмалау қиынырақ болады. Мысалы, стандартты UART интерфейсінде:

- логикалық «0» деңгейі шамамен 0 В;
- логикалық деңгей «1» шамамен 5 В;
- кіріс шегі шамамен 2,5 В.

RS-232 интерфейсінің жалпы сипаттамасы

Бұл соңғы кездегі ең көп таралған интерфейстердің бірі. Бұл кез келген дербес компьютердегі кәдімгі құрылғы болды.

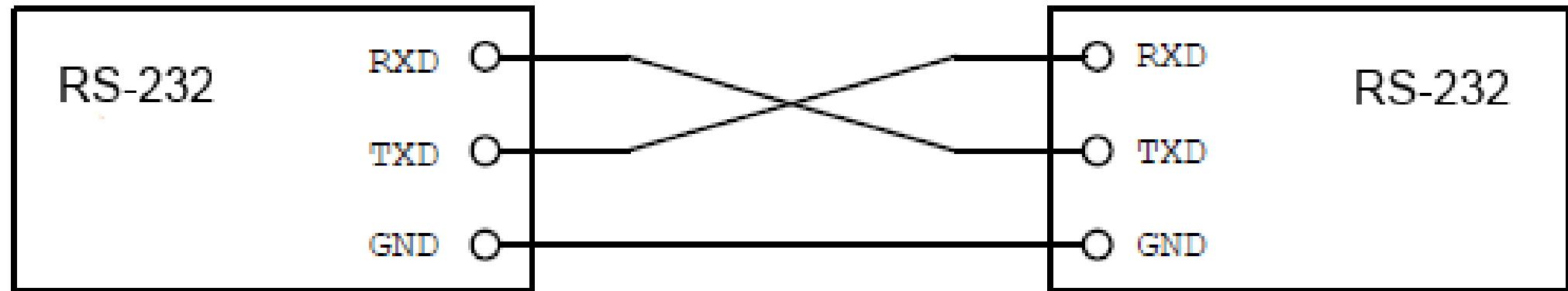
Компьютерлерде RS-232 COM порты, аудармада – байланыс интерфейсі деп аталады. Барлық дерлік жабдық COM порты арқылы компьютерге қосылды.

Әдетте, RS-232 заманауи аналық платаларда да бар. Көбінесе ол жүйелік блоктың артқы қабырғасында көрсетілмейді. Егер компьютерде COM порты болмаса, оны әрқашан кабельге салынған қарапайым USB-COM түрлендіргішінің көмегімен іске асыруға болады.

USB - COM түрлендіргіші



RS-232 арқылы құрылғыларды қосу схемасы



RS-232 параметрлері

Параметр	Мағынасы
Логикалық «0» таратқыш кернеуі	+5 ... + 15 В
Логикалық «1» таратқыш кернеуі	- 5 ... - 15 В
Логикалық «0» қабылдағыш кернеуі	более + 3 В
Логикалық «1» қабылдағыш кернеуі	менее – 3 В

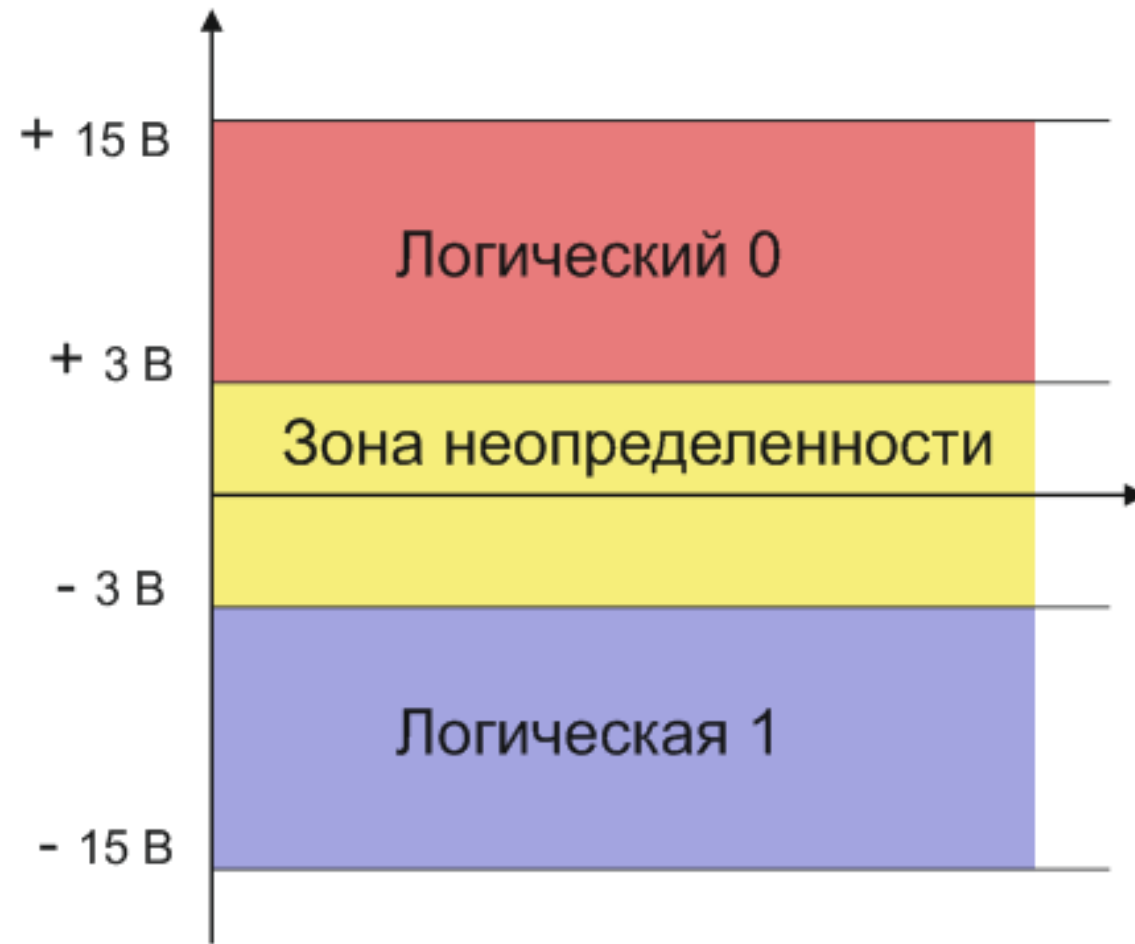
RS-232 параметрлері

Әдетте, 0/1 сигналының логикалық деңгейлері +12 / - 12 В кернеулеріне сәйкес келеді. Қабылдағыштың шектері анық нормаланған: 0/1 +3 / -3 В кернеулеріне сәйкес келеді. -3 диапазонында. ... +3 В, сигнал күйі анықталмаған деп саналады. Сигнал деңгейі қарама-қарсы шекке жеткенше өзгеріссіз қалады.

Мұндай сигнал параметрлерімен жалған дабылды тудыру үшін кедергі беру тізбегінде кернеуді тудыруы керек:

- Логика 1 күйі үшін + 15 В («-12 В» «+ 3 В» дейін арттыру);
- - логикалық 0 күйі үшін 15 В («+12 В» «- 3 В» дейін азайтады).

RS-232 параметрлері



RS-232 параметрлері

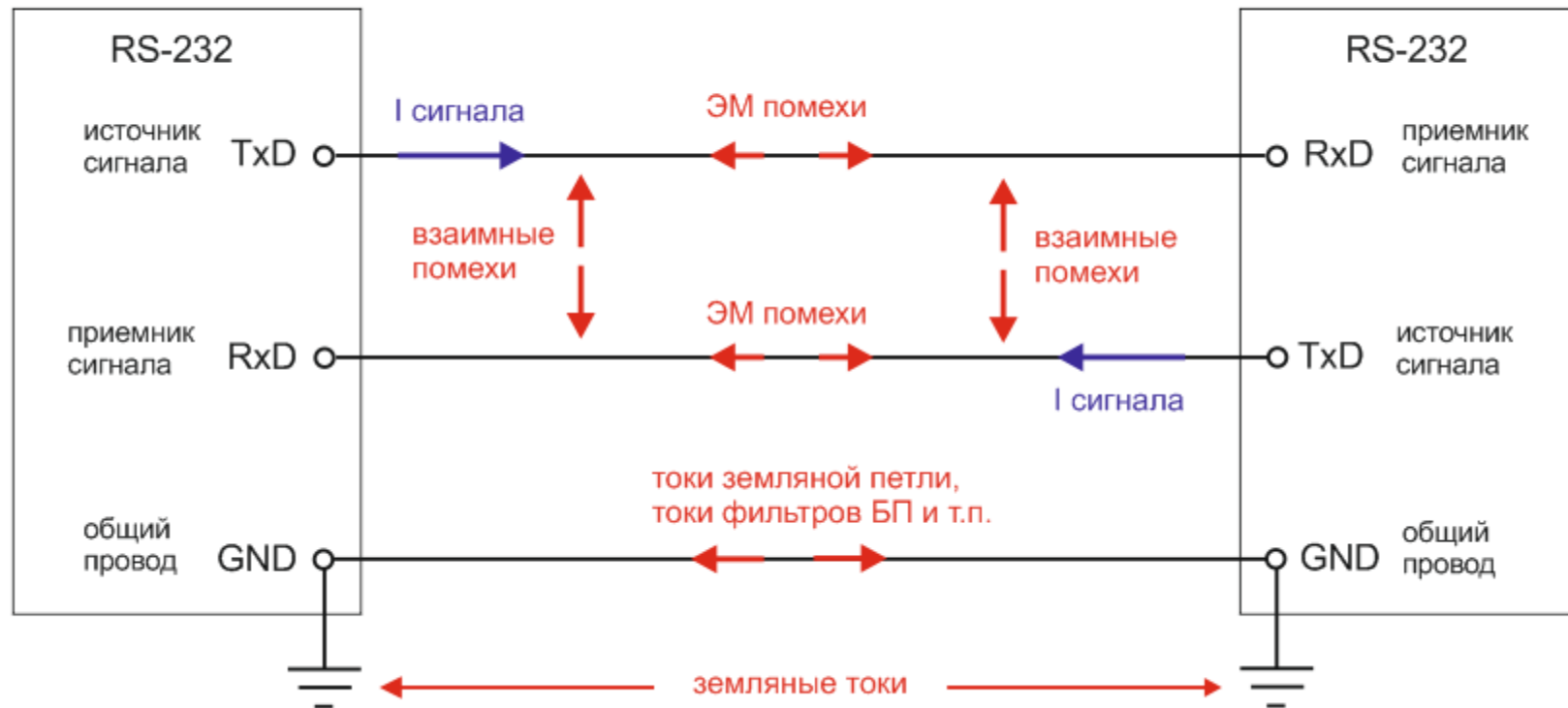
2,5 В тең UART үшін ұқсас шу кернеуінің мәнімен салыстырыңыз. Сигналдардың амплитудасы мен жауап беру шегінің жоғарылауы кедергілердің барлық түрлеріне бірдей қолайлы әсер етеді:

- сыртқы электромагниттік өрістердің бөгеуілдері мен кедергілері;
- өзара әсер ету байланыс желісі;
- жердегі шу және жалпы сымдағы ағып кету токтары.

Барлық басқа UART мәселелері RS-232-де қалады:

- гальваникалық оқшаулаудың болмауы;
- бұралған жұптарды тиімді пайдалануға мүмкіндік бермейтін жалпы СЫМ;
- жердегі контурдағы шу.

РС-232 байланыс желілеріндегі сигналдарға кедергілердің әсер ету схемасы



RS-232 параметрлері

Дегенмен, сигналдар деңгейінің бір көтерілуі байланыс желісінің максималды ұзындығын айтарлықтай арттыруға мүмкіндік берді. RS-232 стандарты 15 м абоненттер арасындағы максималды рұқсат етілген қашықтықты қалыпқа келтіреді. Ал бұл қарапайым экрандалмаған сымдармен қосылуға арналған.

Белгілі бір жағдайларға байланысты (қорғалған сымдар, беріліс жылдамдығын азайту, жалпы жерге қосу және т.б.) құрылғылар арасындағы қашықтық бірнеше ондаған метрге жетуі мүмкін.

RS-232 интерфейсінің стандарттары

RS-232 интерфейсінің халықаралық стандарттары

- ITU-Tv.24;
- ITU-Tv.28.

Кеңестік ГОСТ

- ГОСТ 23675-79;
- ГОСТ 18145-81.

Негізгі параметрлер

Параметр	Мағынасы
Топология	радиалды интерфейс
Байланыс желісі	Жалпы сыммен сигналдар (2-8).
Гальваникалық оқшаулау	Жоқ
Тасымалдау жылдамдығы	460 кбит/с дейін
Байланыс желісінің максималды ұзындығы	15 м

Негізгі параметрлер

Қабылдаушы	
Логикалық «0» кернеуі	+3 В жоғары
Логикалық «1» кернеуі	- 3 В төмен
Кіріс кедергісі	3000 ... 7000 Ом
Кіріс кернеуі	$\pm 3 \dots \pm 15$ В
Кіріс сыйымдылығы	2500 пФ артық емес

Негізгі параметрлер

Таратқыш	
Қысқа тұйықталу және үзіліс	Уақытты шектеусіз рұқсат етіледі
Ашық тізбектің шығыс кернеуі	± 15 В артық емес
Қысқа тұйықталу тогы	0,5 А артық емес
сыйымдылық жүктемесі	2500 пФ дейін

RS-232 интерфейсінің ажыратқыштары

Бізге белгілі TxD және RxD сигналдарына қосымша интерфейс стандарты деректер ағынын басқаруға арналған тағы бірнеше қосымша сигналдарды сипаттайды. Бұл сигналдар компьютердің COM портында жүзеге асырылады. Оларды бағдарламадан ерікті түрде басқаруға болады.

Әдетте, әмбебап кіріс және шығыс ретінде қосымша сигналдар қолданылады. Мысалы, Arduino IDE-ден бағдарламаны жүктеген кезде DTR сигналы Arduino тақталарының микроконтроллерін бастапқы қалпына келтіреді. Біз олардың стандартты мақсатын егжей-тегжейлі сипаттамаймыз.

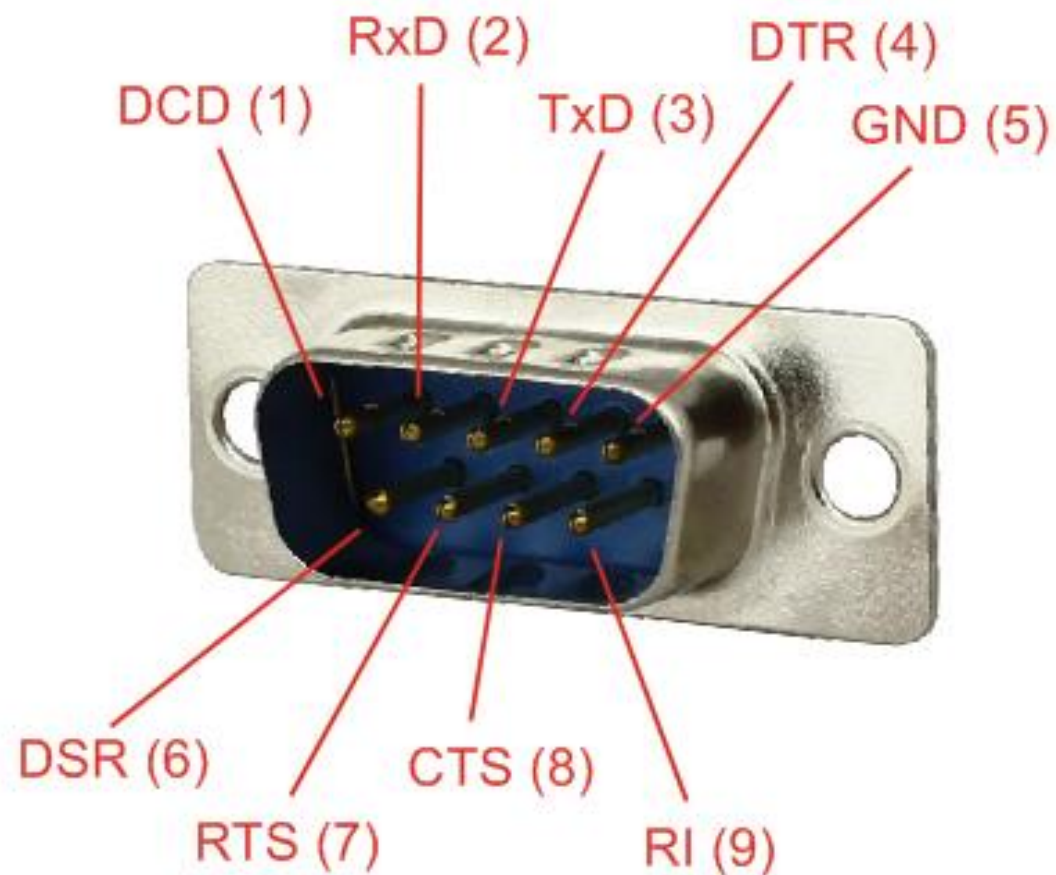
RS-232 интерфейсінің ажыратқыштары

Бастапқыда RS-232 интерфейсінде 25 істікшелі DB-25 қосқыштары қолданылды. Содан кейін 9 істікшелі DB-9 стандартты қосқыш болды.

Қазіргі уақытта стандартты RS-232 интерфейс қосқышы DB-9 болып табылады.

Екі жағдайда да блоктық бөліктің жағында тығындар, ал кабель бөлігінде розеткалар қолданылады.

RS-232 интерфейсінің ажыратқыштары



RS-232 пин тағайындау

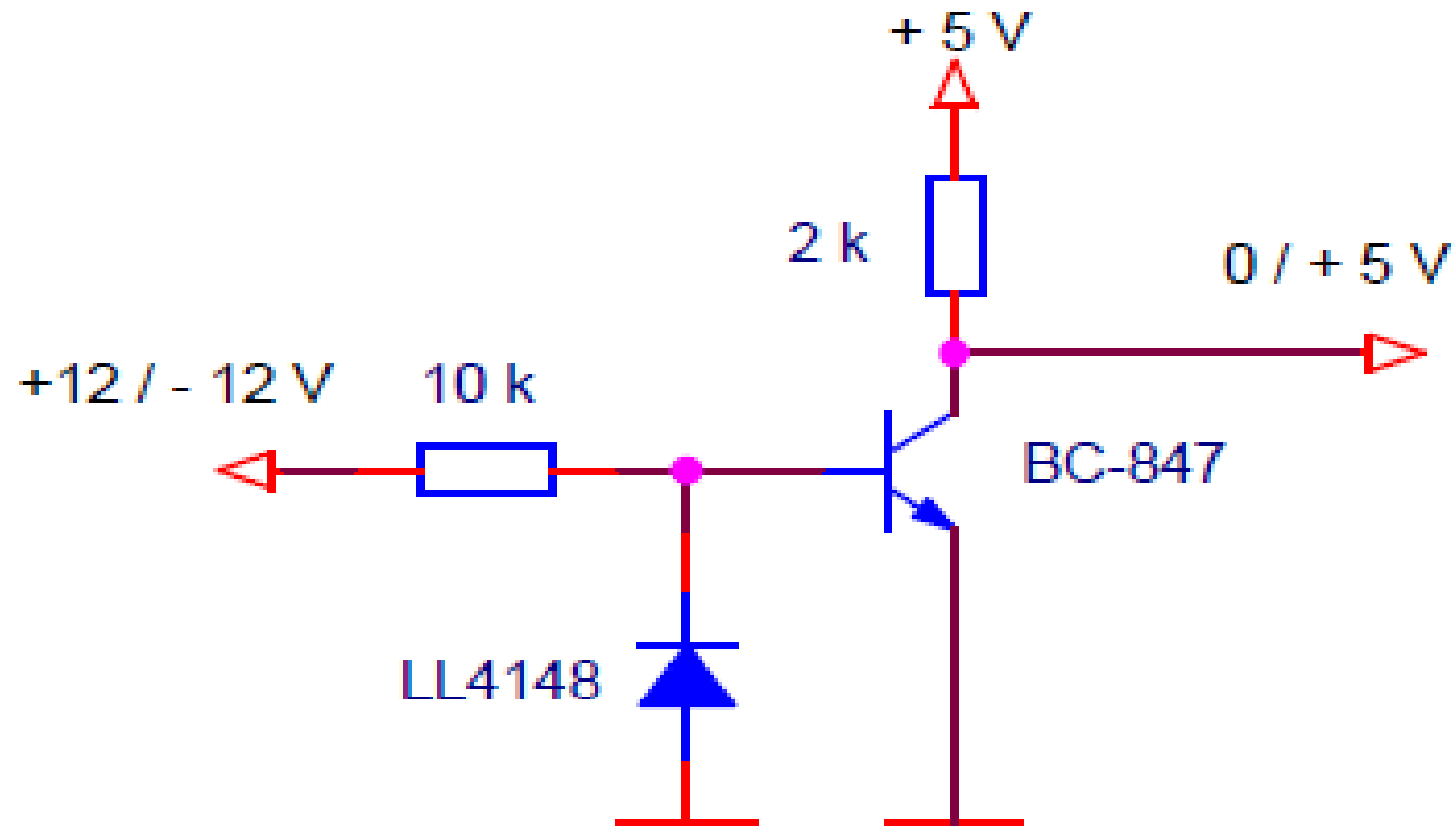
DB-25 контактісі	DB-9 контактісі	Сигнал атауы	Бағыт	Сипаттама
8	1	DCD	Кіру	Тасымалдаушының болуы. Алынған сигнал деңгейі қалыпты, модем қосылған.
3	2	RxD	Кіру	Деректер қабылдау. Басқа құрылғыдан алынған деректер.
2	3	TxD	Шығару	Деректерді тасымалдау. Деректер басқа құрылғыға жіберілді.
20	4	DTR	Шығару	Қабылдаушының дайындығы. Құрылғының деректерді қабылдауға дайын екенін көрсетеді.
7	5	GND		Жалпы сым
6	6	DSR	Кіру	Таратқыш дайын. Құрылғы деректерді тасымалдауға дайын.
4	7	RTS	Шығару	Деректерді тасымалдау сұрауы. Басқа құрылғыны деректерді тасымалдау режиміне қояды.
5	8	CTS	Кіру	Тасымалдауға дайын. Басқа құрылғы жіберуге дайын.
22	9	RI	Кіру	Қоңырау сигналы. Қоңырау индикаторы (телефон қоңырауы).

RS-232 схемасын жүзеге асыру

UART интерфейсінен RS-232 жасау үшін сигнал деңгейінің түрлендіргіштерін қосу жеткілікті. Түрлендіргіштер ешқандай логикалық әрекеттерді орындамайды. Олар жай ғана 0/5В логикалық деңгейлерді +12/-12V деңгейлеріне және керісінше түрлендіреді.

Түрлендіргіштерді дискретті элементтерде іске асыруға болады. Мұнда инвертивті транзисторлық қосқышқа негізделген қабылдағыш схемасы берілген.

Инвертивті транзисторлы қосқышқа негізделген қабылдағыш схемасы



RS-232 схемасын жүзеге асыру

Дискретті элементтерде таратқыштарды жүзеге асыру әлдеқайда қиын. Екі полярлы қосқышты және оған екі қуат көзін қажет + 12 В және - 12 В. Кейде транзисторлық қосқыштар 0 / 5 В шығыс сигналын қалыптастыру үшін қолданылады. Кейбір RS-232 қабылдағыштары бұл сигналмен жұмыс істейді, кейбіреулері жұмыс істемейді. Кез келген жағдайда мұндай сигналдармен интерфейстің қалыпты жұмысына кепілдік берілмейді.

Толыққанды екі жақты алмасуды жүзеге асыру үшін біріктірілген RS-232 түрлендіргіштерін қолданған дұрыс. Олардың көпшілігі бар.

RS-232 түрлендіргіштері

Микросхема	Өндіруші
MAX232	Maxim Integrated Products
SP232	Sipex
ADM232	Analog Devices

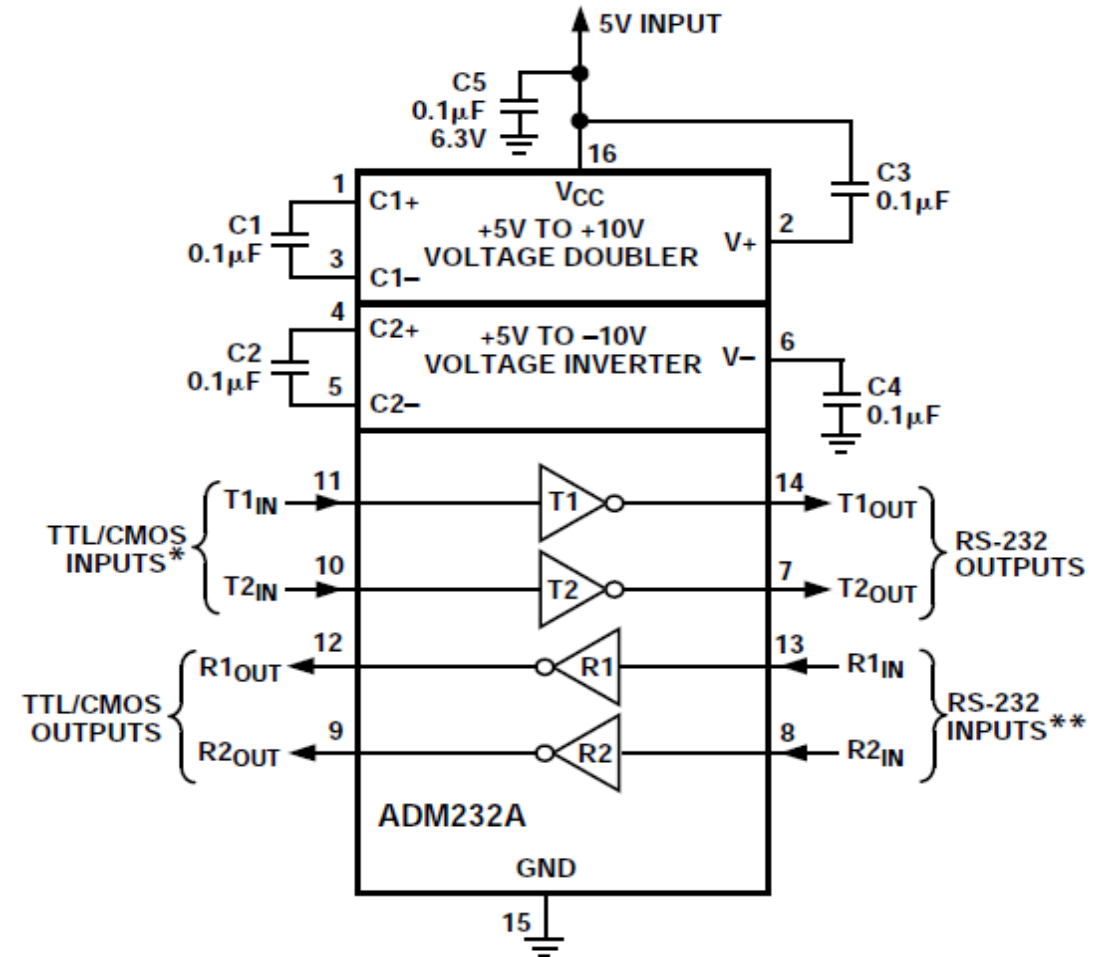
RS-232 схемасын жүзеге асыру

Бұл әртүрлі өндірушілердің микросұлбалары, бірақ функциялары, параметрлері және түйреуіш тағайындаулары бірдей.

2 кіріс және 2 шығыс RS-232 сигналдары үшін деңгей түрлендіргіштері 16 істікшелі пакетте жүзеге асырылады.

Түрлендіргіштер 5 В бір кернеумен қоректенеді. Таратқыштар үшін қажетті + 12 В және - 12 В кернеулер ішкі конденсатор инверторлары арқылы жасалады. Микросұлба 5 сыртқы компоненттерді, барлық конденсаторларды қосуды қажет етеді.

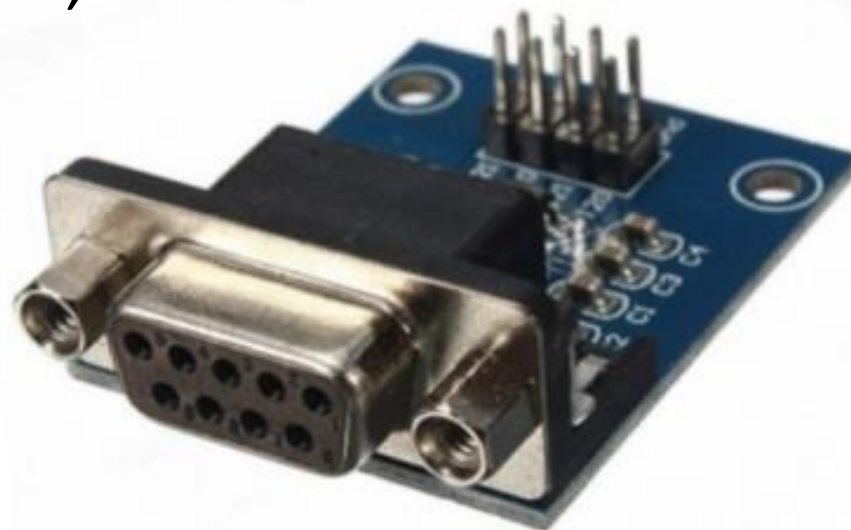
ADM232



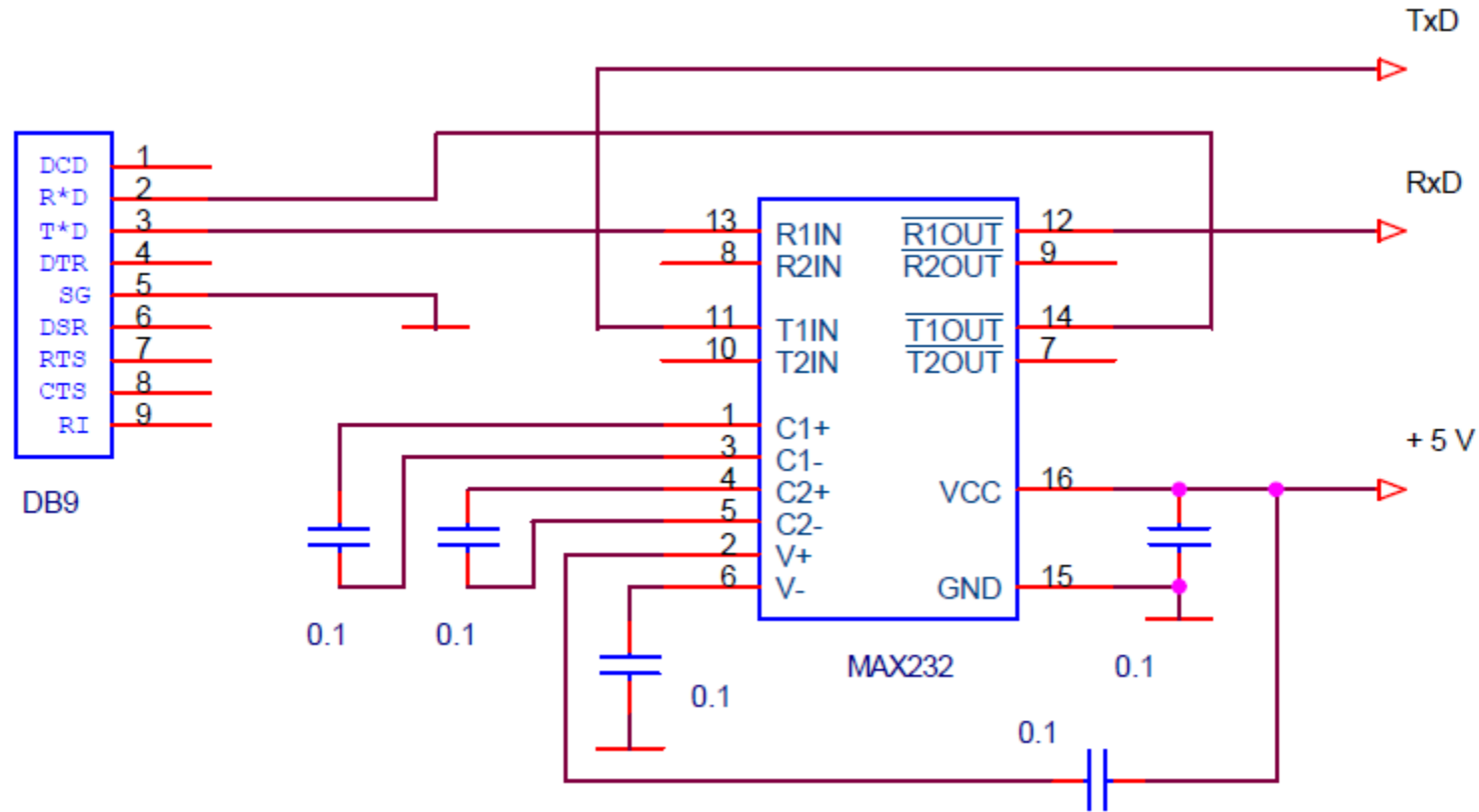
Arduino тақтасын RS-232 интерфейсі арқылы қосу

Мүмкін, жоғарыда жазылғанның бәрінен кейін, Arduino тақтасын компьютерге қосу немесе Arduino тақталарын RS-232 арқылы бір-біріне қосу ешқандай қиындық тудырмайды.

Тақтаға RS-232 деңгей түрлендіргішін қосу қажет. Сіз дайын модульді пайдалана аласыз, мысалы, осы.



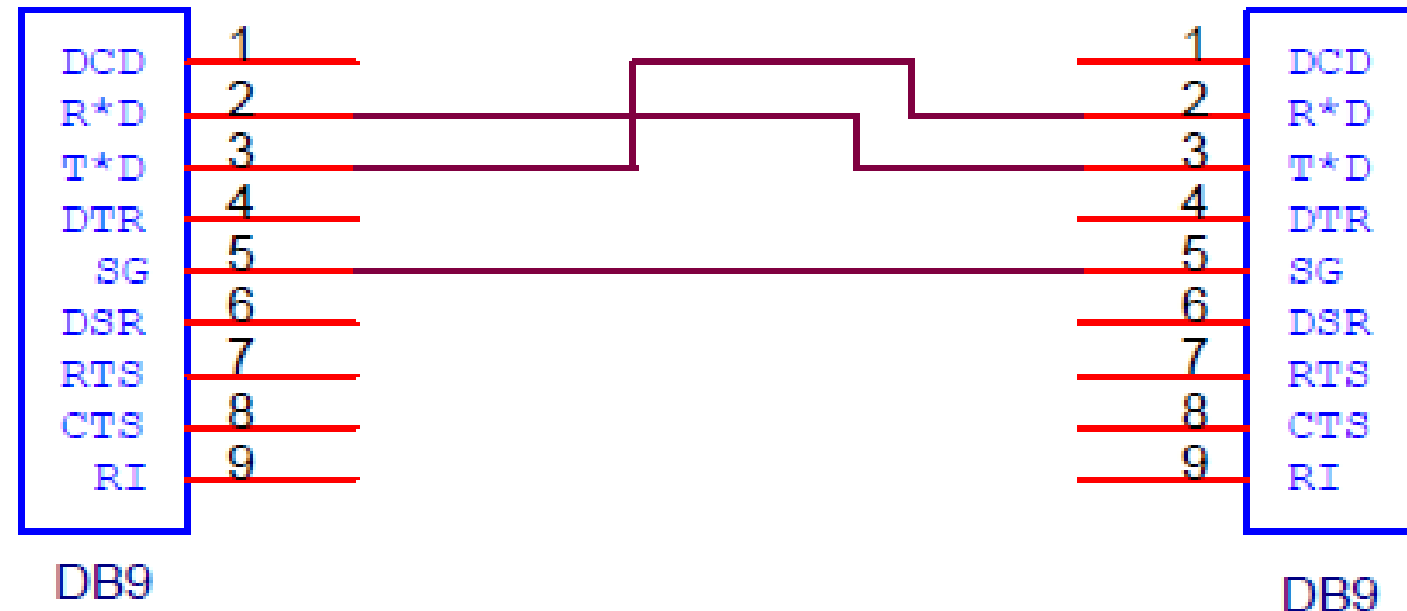
MAX32 чипіндегі Arduino тақтасына арналған RS-232 түрлендіргішінің схемасы



MAX32 чипіндегі Arduino тақтасына арналған RS-232 түрлендіргішінің схемасы

Бұл кабель бар түрлендіргіш тізбегі. RS-232 қосқышы құрылғы корпусында орнатылған болса, TxD және RxD сигналдары DB-9 қосқышында ауыстырылуы керек.

Содан кейін құрылғыларды қосу үшін «кроссовер» кабелі қолданылады.



MAX32 чипіндегі Arduino тақтасына арналған RS-232 түрлендіргішінің схемасы

Құрылғыны компьютерге стандартты COM порты арқылы қосуға болады, егер бар болса немесе USB-RS232 түрлендіргішін пайдалануға болады. Олардың көптеген түрлері бар.



RS-422 интерфейсі

Бұл сигнал беру дифференциалды түрде болатын радиалды интерфейс. Әрбір сигналды қосу үшін екі сымның (сызықтың) бұралған жұбы қолданылады. Таратқыштар желілерде антифазалық логикалық деңгейлерді құрайды, ал қабылдағыштар желілер арасындағы кернеу айырмашылығын қабылдайды. Нәтижесінде жүйенің шуға төзімділігі айтарлықтай артады.

Сигналдарды беру әдісі, RS-422 электрлік параметрлері RS-485 интерфейсінің талаптарына толығымен сәйкес келеді. Жалғыз айырмашылығы - RS-422 - радиалды интерфейс, ал RS-485 - шиналық интерфейс. Біріншісі арқылы бір-біріне тек 2 құрылғыны қосуға болады, ал екінші интерфейс арқылы бір байланыс желісімен бірнеше құрылғыны қосуға болады.

RS-422 негізгі параметрлері

Параметр	Мағынасы
Топология	Радиалды интерфейс
Байланыс желісі	2 бұралған жұп және жалпы сым
Гальваникалық оқшаулау	Жоқ
Деректер алмасу режимі	Дуплекс
Сигнал беру әдісі	Дифференциалды
Байланыс желісінің максималды ұзындығы	1200 м
Максималды жіберу жылдамдығы	10 Мбит/с

RS-485 интерфейсі

RS-485 интерфейсі шағын өнеркәсіптік автоматтандыру желілерін ұйымдастыруға арналған ең кең таралған интерфейс болуы мүмкін.

Бұған іске асырудың қарапайымдылығымен оның жоғары техникалық сипаттамалары ықпал етеді. RS-485 интерфейсі қарапайым аппараттық құралдармен желілерді құруға мүмкіндік береді:

- шина топологиясы;
- деректерді беру ортасы ретінде бұралған жұппен;
- байланыс желісінің ұзындығы 1200 м жетуі мүмкін;
- деректерді беру жылдамдығы 10 Мбит/с дейін.

RS-485 интерфейсі

RS-485 негізіндегі таратылған жүйелерді басқару үшін ModBus қоса алғанда көптеген стандартты хаттамаларды пайдалануға болады. Интерфейс арнайы хаттамалары бар желілерді құруға мүмкіндік береді. RS-485 аппараттық қамтамасыз ету үшін микроконтроллерге интеграцияның төмен дәрежесіндегі бір ғана микросұлбаны қосу жеткілікті.

RS-485 интерфейсі

RS-485 ANSI TIA/EIA-485-A:1998 сипатталған. Стандарт тек электрлік және уақыттық параметрлерді анықтайды. Ол шарт жасамайды:

- алмасу хаттамасы;
- кабельдер мен қосқыштардың түрлері;
- желі абоненттерінің гальваникалық оқшаулануы.

RS-485 стандартының негізгі параметрлері

Параметр	Мағынасы
Топология	Шина
Байланыс желісі	бұралған жұп
Гальваникалық оқшаулау	Стандартпен белгіленбеген
Деректер алмасу режимі	Полудуплекс
Сигнал беру әдісі	Дифференциалдық сигналдар
Желі абоненттерінің саны	32-ге дейін, қайталағыштармен ұзартылуы мүмкін
Байланыс желісінің максималды ұзындығы	1200 м
Максималды жіберу жылдамдығы	10 мБит/сек

Деректерді беру тәсілі RS-485

RS-485 интерфейс стандарты келесі сигналдарды анықтайды:

- A – инвертивті емес;
- B – төңкеру;
- C – жалпы сызық (қосымша сигнал).

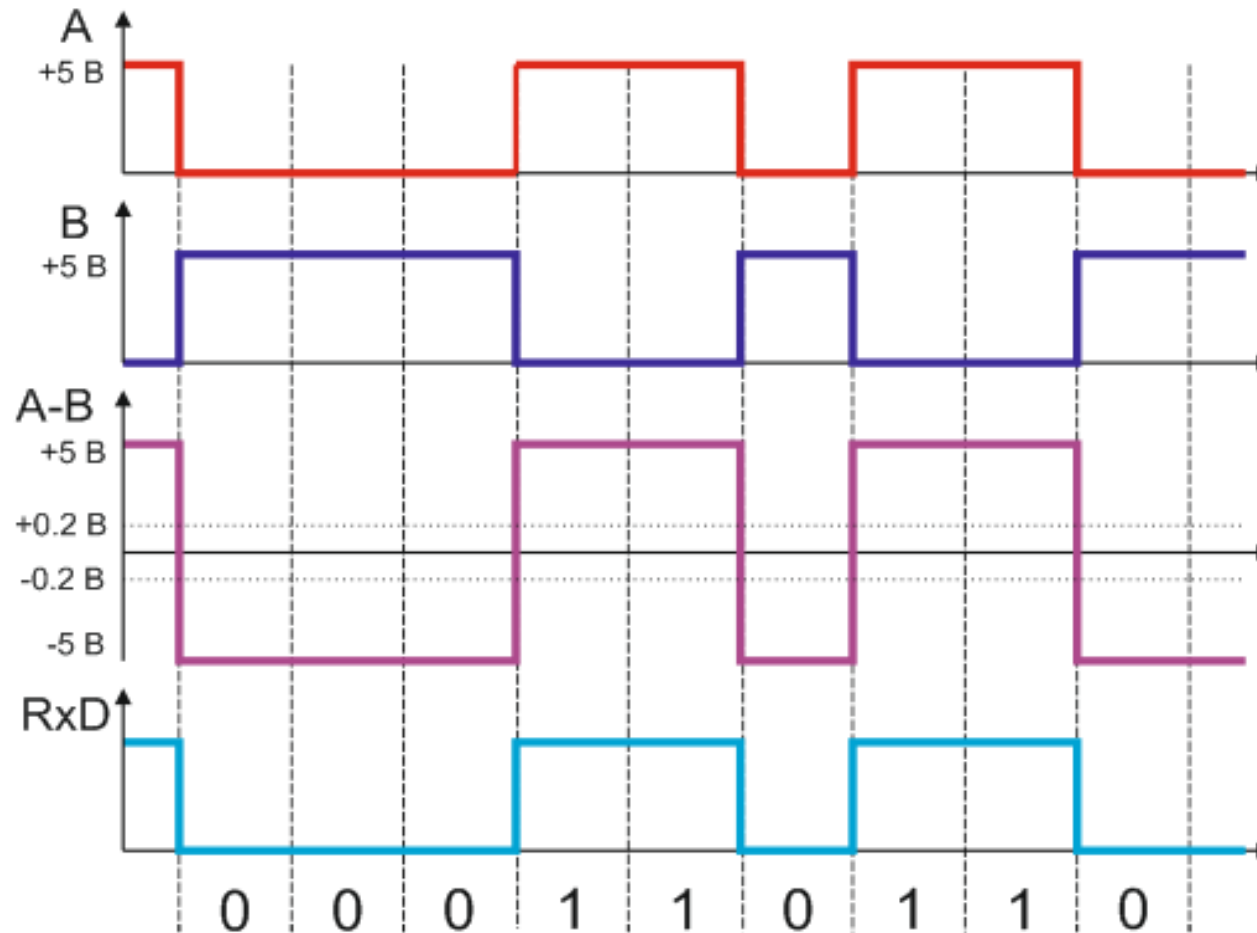
Кейде альтернативті сигнал белгілеулері қолданылады:

- Деректер+ / Деректер-;
- D+ / D-;
- + / -.

Деректерді беру тәсілі RS-485

Интерфейс деректерді берудің дифференциалды әдісін пайдаланады. Ақпарат екі антифазалық А және В сигналдары арқылы беріледі және RS-485 шинасының күйі жалпы С желісіне қатысты А және В желілері арасындағы потенциалдар айырмасымен анықталады. Жерге қатысты әрбір желінің кернеуі кез келген нәрсе болуы мүмкін, бірақ $-7 \dots +12$ В диапазонында.

Способ передачи данных RS-485



Деректерді беру тәсілі RS-485

RS-485 дифференциалды қабылдағыштар мен таратқыштарды пайдалануды талап етеді.

Таратқыштар кернеу айырмашылығы кемінде 1,5 В (стандарт бойынша) 2 антифазалық сигналды құрайды.

Мәліметтерді қабылдау үшін А және В желілері арасындағы кернеу айырмашылығын бөлетін дифференциалды қабылдағыштар қолданылады. Егер айырмашылық 200 мВ-тан жоғары, бірақ +12 В-қа дейін болса, желі күйі логикалық бірлікке тең деп саналады. Кернеу айырмашылығы -200 мВ-тан аз, бірақ -7 В төмен емес желі логикалық нөлдік күйде болады.

Деректерді беру тәсілі RS-485

Желінің белсенді кедергісіндегі кедергі деңгейі мен кернеудің төмендеуі 1,3 В жетуі мүмкін екенін есептеу оңай (таратқыштың шығыс кернеуі 1,5 В минус қабылдағыштардың жауап шегі 0,2 В). Мұндай маржа айтарлықтай белсенді кедергісі бар ұзын байланыс желілерінде интерфейстің жұмысын қамтамасыз етеді. Байланыс желісінің максималды ұзындығы (1200 м) осы параметрмен анықталады. Таратқыштардың шығысындағы нақты кернеу айырмашылығы 5 В жетуі мүмкін.

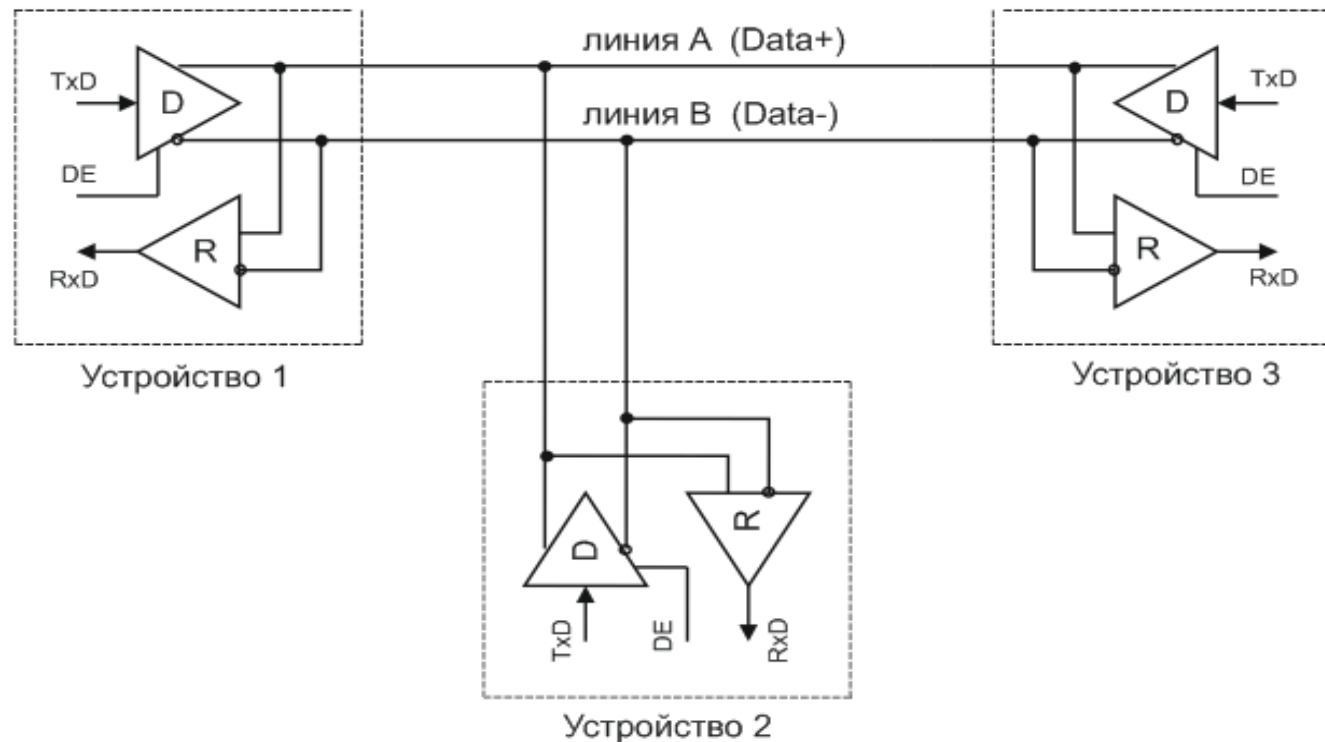
Деректерді беру тәсілі RS-485

А және В сызықтары С жерге қатысты симметриялы. Олардағы интерференция мен интерференция пішіні мен шамасы бойынша ұқсас индукцияланады. Дифференциалды қабылдағыштарда желілердегі кернеулер алынып тасталады, сигнал оқшауланады және кедергі кернеуі нөлге тең болады. Әрине, нақты жағдайларда әрқашан сызықтар мен жүктемелердің шамалы асимметриясы бар, бұл шығыс сигналында шудың пайда болуына әкеледі, бірақ ол айтарлықтай әлсірейді.

Интерфейстің таратқыштары мен қабылдағыштарының симметриясының арқасында электромагниттік кедергімен күресуде маңызды әсер байланыс желісі ретінде бұралған жұпты пайдалану арқылы қамтамасыз етіледі. Көрші бұрылыстардағы қабылдау токтары бір-біріне қарама-қарсы бағытталған және бірін-бірі жоққа шығарады.

RS-485 желі топологиясы

Әдетте, RS-485 интерфейстері бар құрылғылар «Жалпы шина» топологиясымен желіге қосылған. Абоненттер қосымша жалпы сыммен бір екі сымды байланыс желісі арқылы параллель қосылады.



RS-485 желі топологиясы

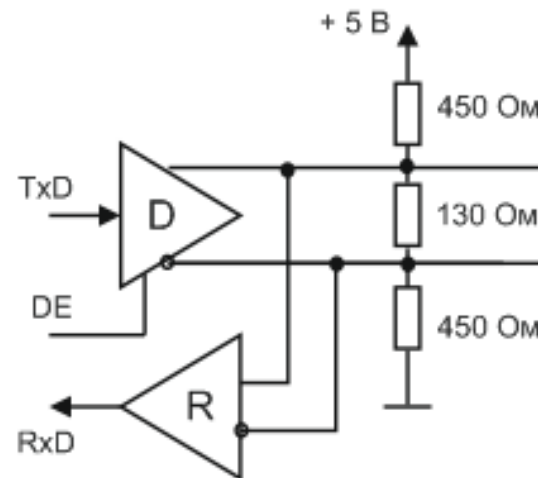
Әрбір абонент желіге дифференциалды таратқыш (D) және қабылдағыш (R) арқылы қосылады. Бір уақытта тек бір желі таратқышы белсенді (қосұлы) болуы мүмкін. Барлық басқа таратқыштар үшінші (жоғары кедергі) күйде болуы керек. Таратқыштың күйі бөлек сигналмен (DE) басқарылады.

Деректер алмасудың жалпы реттілігі келесідей көрінеді. Негізгі құрылғы таратқышты қосады, деректерді жібереді, содан кейін өшеді және жауап алады. Қазіргі уақытта барлық басқа құрылғылар таратқыштары ажыратылған күйде. Бағынышты құрылғы деректерді қабылдайды, содан кейін таратқышты қосып, жауапты негізгі құрылғыға жібереді.

RS-485 желі топологиясы

Әрине, барлық таратқыштар өшіріліп, «желі ауада ілініп тұрған» сәттер болады. Арнайы шаралар қабылдасаныз, желінің күйі белгісіз болады. Қабылдағыштардың шығыстары кез келген деңгейде болуы мүмкін.

Егер сіз қабылдағыштың инвертивті емес кірісін резисторлар арқылы қуат шинасына, ал инвертивті кірісті жерге қоссаңыз, бұл белгісіздікті жоюға болады.



RS-485 желі топологиясы

Резисторлардың кедергісі қабылдағыштардың кірістері арасында жауап беру шегінен (200 мВ) кем емес ауытқуды тудыратындай болуы керек. Терминатор резисторларын есептеу кезінде бұл резисторларды ескеру қажет.

Барлық таратқыштар өшірілген кезде желі күйінің белгісіздігін жоюдың басқа нұсқалары бар. Бірақ олардың барлығы хаттамалар деңгейінде.

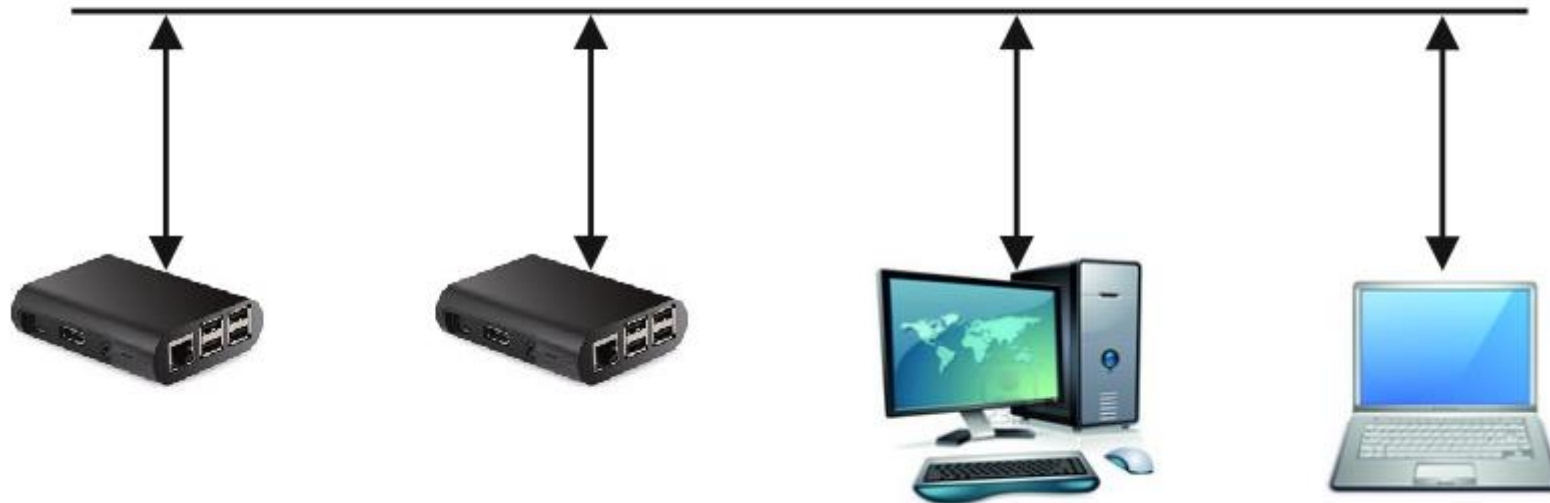
Айырбастау басында кодтардың қызмет көрсету тізбегін жіберуге болады. Бірақ бұл алмасуды айтарлықтай қиындатады, қосымша деректерді беруді талап етеді.

RS-485 желі топологиясы

Егер желіде әрқашан белсенді құрылғы болса, онда опция бар - екінші таратқыш қосулы болған кезде таратқышты өшіру, бірақ екеуі де логикалық «1» күйінде. Мастер деректерді жіберіп жатыр делік. Содан кейін ол таратқыштың шығысын логикалық «1» күйіне орнатады. Бағынышты құрылғы өзінің таратқышын қосады, сонымен қатар логикалық «1» күйінде. Әрі қарай, Мастер таратқышты сөндіреді, ал құл деректерді жібере бастайды. Желі ешқашан үзілмейді. Мұндай алгоритм таратқыштарды ауыстыру үшін уақыт интервалдарын өңдей отырып, дәл синхрондауды қажет етеді.

Тағы бір жағымсыз жаңғырық. Құрылғының таратқышы жіберетін барлық нәрсе өзінің қабылдағышын қабылдайды. Біз мұны ескеруіміз керек. Кейбір жүйелерде жаңғырық деректері хаттаманың бөлігі ретінде өңделеді. Басқаларында жіберу кезінде қабылдағыштың жұмысына тыйым салынады.

ModBus RTU сериялық байланыс протоколы



ModBus алмасу протоколы

Қарапайым әмбебап ModBus алмасу хаттамасы бар, онда деректер мен функциялардың артықтығы барынша азайтылады. Бұл шағын үлестірілген жүйелерді ұйымдастырудың ең кең таралған протоколы болуы мүмкін.

Хаттаманың жалпы сипаттамасы

ModBus шағын жергілікті желілерде деректер алмасуға арналған ашық хаттама болып табылады. Әдетте, ол RS-232, RS-485, RS-422 интерфейстері арқылы, TCP/IP, UDP желілерінде деректерді беру үшін қолданылады. Қарапайымдылығы мен әмбебаптығының арқасында ModBus кең тарады және шағын үлестірілген есептеу жүйелерінде де-факто стандартына айналды. Қазіргі заманғы контроллерлердің барлығы дерлік ModBus желілерін қолдайды.

Хаттаманың жалпы сипаттамасы

ModBus желісінде контроллерлер әдетте «жалпы шина» топологиясында қосылады. Контроллерлердің өзара әрекеттесуі басты-басты модельге (мастер-құл) сәйкес жүреді.

Желіде негізгі құрылғы - шебер бар. Сондай-ақ бірнеше құлдық құрылғылар - құлдар. Айырбастауды тек шебер ғана жасай алады.

Транзакция (деректер алмасудағы операциялар тізбегі) сұраныс пен жауаптан тұрады.

Негізгі құрылғы сұрауды кез келген бағынышты контроллерге жібере алады немесе барлық бағынышты құрылғыларға бір уақытта хабар тарата алады.

Сұраныстағы мекенжайды анықтаған бағынышты құрылғы жауап береді.

Шеберден сұрау міндетті түрде функция кодын қамтиды, яғни. не істеу. Сондай-ақ, функцияға байланысты сұрауда деректер болуы мүмкін.

Хаттаманың жалпы сипаттамасы

ModBus протоколының 3 нұсқасы бар.

- ModBus ASCII — мәтіндік протокол. Ол тек ASCII таңбаларын пайдаланады. Әрбір байт екі он алтылық таңба ретінде беріледі.
- ModBus RTU — сандық протокол. Мәліметтер екілік түрде беріледі. Желі арқылы берілетін байт протокол нөмірі болып табылады.
- ModBus TCP — TCP/IP желілерінде мәліметтерді тасымалдау протоколы.

Modbus RTU протоколы

Құрылғылар (контроллерлер) желіге «Жалпы шина» топологиясы бойынша қосылады. ModBus стандарты 247 контроллерге дейін бірге жұмыс істеуге мүмкіндік береді.

Әрбір контроллерде деректер бар, шын мәнінде қандай құрылғылар алмасады.

ModBus стандарты деректердің 4 түрін анықтайды.

Modbus RTU протоколы

Деректер түрі	Мөлшері	Рұқсат етілген операциялар
Жалау тізілімі (Coils)	1 бит	Жазу және оқу
Сандық кірістер (Discrete Inputs)	1 бит	Оқу
Регистрлерді ұстау (Holding Registers)	16 бит	Жазу және оқу
Енгізу регистрлері (Input Registers)	16 бит	Оқу

Modbus RTU протоколы

Деректердің бұл функционалды бөлінуі жойылды және іс жүзінде қолданылмайды. Барлық деректер ақырында контроллердің жадынан оқылады және олардың кірістерден немесе кіріс регистрлерінен қайдан түскені соншалықты маңызды емес.

Іс жүзінде тек осы Холдинг тізілімдері қолданылады. Деректерге қол жеткізу 16 биттік мекенжай арқылы жүзеге асырылады. Ақырында:

- әрбір контроллердегі деректер 16 разрядты регистрлердің кестесі болып табылады;
- деректер кестесінде 65536 жазбаға дейін болуы мүмкін;
- элементті нөмірлеу 0-ден басталады.

Modbus RTU протоколы

Негізгі құрылғы транзакцияны бастайды - деректер алмасу. Мәміле жеке (сұраныс-жауап) немесе таратылуы мүмкін (барлық бағыныштылар бір уақытта адрестеледі). Транзакция бір сұрау шеңберінен және бір жауап жақтауынан тұрады. Таратылым транзакциясында тек сұрау жақтауы пайдаланылады.

Жақтау деректері үздіксіз ағын ретінде беріледі. Мәліметтерді жіберу арасындағы үзіліс 1,5 таңбаны жіберу уақытынан аспауы керек. Жаңа кадрдың белгісі 3,5 таңбаны беру үшін қажетті уақыт ішінде желіде алмасудың болмауы (үнсіздік) болып табылады. Егер осы уақыт ішінде желі желісі белсенді емес күйде болса, онда тәуелді құрылғылар бірінші қабылданған деректерді кадрдың басы ретінде қабылдайды.

Modbus RTU протоколы

Жалпы, сұрау жақтауының келесі пішімі болады.

Пауза 3,5 Т	Адрес 8 бит	Функция 8 бит	Деректер N * 8 бит	Тексеру сомасы 16 бит	Пауза 3,5 Т
----------------	----------------	------------------	-----------------------	-----------------------------	----------------

Адрес (8 бит).

Кадр адрестік өрістен басталады, ол 8 биттен тұрады. Шеберден хабарлама жіберілетін бағынышты құрылғының мекенжайын қамтиды. Әрбір бағыныңқы 1-ден 247-ге дейінгі бірегей мекенжайға ие болуы керек. Және тек адрестік бағыныңқы иесінің сұрауына жауап беруі керек. Жауап шеберге қай құлдың қосылғанын айтады.

0 мекенжайы хабар тарату режимінде пайдаланылады. Барлық бағыныңқылар сұрауда көрсетілген функцияны орындайды, бірақ растау жіберілмейді.

Функция (8 бит).

Функция өрісі адрестелген құлға қандай операцияны орындау керектігін айтады.

Операцияның қате орындалғаны туралы шеберге хабарлау үшін бағыныңқы жауапта функция байтының жоғары биті 1-ге орнатылады. Егер операция сәтті болса, ең маңызды бит 0 болады.

Функция кодтарының 3 категориясы бар.

- стандартты командалар. ModBus протокол стандартымен анықталған кодтар.
- Пайдаланушы командалары. 65...72 және 100...110 кодтары үшін пайдаланушы ерікті функцияны өзі тағайындай алады.
- сақталған командалар. Бұл бастапқыда стандартпен анықталмаған, бірақ әртүрлі өндірушілердің құрылғыларында бұрыннан қолданылған кодтар.

Функция (8 бит).

ModBus контроллерлерінің басым көпшілігі тек 3 функцияны пайдаланады.

Функция коды	Аты	Сипаттама
03	READ HOLDING REGISTERS	Бір немесе бірнеше ұстау регистрлерінің мәндерін оқу
06	FORCE SINGLE REGISTER	Бір ұстау тізіліміне жазыңыз
16	FORCE MULTIPLE REGISTERS	Бірнеше ұстау регистрлерінің ретімен жазылуы

Жауап пішімі функцияға байланысты. Көптеген жағдайларда қалыпты жауап сұрауды толығымен немесе ішінара қайталайды.

Деректер ($N * 8$ бит).

Деректер өрісі көрсетілген функцияны орындау үшін бағынышты контроллерге қажетті ақпаратты қамтиды немесе өрісте шебердің сұрауы бойынша бағынышты құрылғы арқылы жіберілген деректер бар. Деректер өрісінің ұзындығы мен пішімі функция кодына байланысты. Кейбір хабарларда деректер болмауы мүмкін.

Деректер ($N * 8$ бит).

Әрбір деректерде 16 бит (2 байт) бар. Деректер алдымен жоғары байт тасымалданады. Мысалы, 0 және 1 адрестері бар регистрлерді сериялық тасымалдау келесідей болуы керек:

жоғары байт тіркеу 0 ->

Төмен байтты тіркеу 0 ->

регистрдің жоғары байты 1 ->

регистрдің төмен байты 1 ->.

4 байттық нөмірді тасымалдау кезінде, мысалы, өзгермелі нүктемен, реттілік бірдей болады. Сан екі 16 разрядты регистрге бөлінеді және олардың әрқайсысында ең маңызды байт бірінші беріледі (1-> 0-> 3-> 2->).

Бақылау сомасы (16 бит).

Хабарлама деректерінің тұтастығын басқару өрісі. Фреймдердегі қателерді тексеруге мүмкіндік береді. Бұл деректерді тасымалдау кезінде орын алатын қателер.

Бақылау сомасы ретінде циклдік резервтік код (CRC) пайдаланылады. Трансмиссиялық кадрдың барлық биттері үлкен екілік санға жинақталған. Ол тудырушы көпмүшенің коды бойынша бөлінеді. Бөлімнің қалған бөлігі басқару коды болып табылады.

Бақылау сомасы (16 бит).

Көпмүшелік арифметикалық модуль 2 пайдаланылады. Бұл CRC есептеуіндегі барлық операциялар тасымалдаусыз арифметикалық амалдар екенін білдіреді. Алу және қосу тасымалдауды есепке алмастан, аздап орындалады, сондықтан бұл операциялар бірдей нәтиже береді және XOR операциясымен ауыстырылуы мүмкін. Бөлу кезінде дивидендтен бөлгішті алып тастаудың орнына XOR операциясы да қолданылады. Бұл CRC есептеудің жалпы принципі. Практикада есептеулер тиімдірек алгоритмдер арқылы орындалады.

Бақылау сомасы жіберу кадрын аяқтайды. Фреймді қабылдағаннан кейін бағынышты құрылғы сол алгоритмді пайдаланып алынған деректердің бақылау сомасын есептейді және оны кадрдың соңындағы жіберілген басқару кодымен салыстырады. Нәтижесінде кадрдың деректердің тұтастығы расталады.

Бақылау сомасы (16 бит).

ModBus RTU стандартында генератор полиномы $X^{16}+X^{15}+X^2+1$ бар стандартты CRC-16 циклдік коды қолданылады. Бұл 16 биттік код, екілік коэффициенттері 1 1000 0000 0000 0101 (он алтылықта 8005 сағ).

Хабарлама ең маңызды бит бірінші жіберілетін бір дәйекті екілік сан ретінде қарастырылады. Бұл сан X^{16} (солға жылжу 16 бит) көбейтіледі және $X^{16}+X^{15}+X^2+1$ (1 1000 0000 0000 0101) бөлінеді. 16-биттік қалдық (барлықтарымен алдын ала инициализацияланған) хабарламаны басқару коды болып табылады.

Логикалық қателерді өңдеу.

Тасымалдау кезінде деректердің бүлінуімен байланысты қателерден басқа, сұрау қатесіз қабылданған, бірақ орындалмайтын кезде логикалық қателер болуы мүмкін. Әдетте, мұндай қателер жарамсыз мекенжаймен, деректермен, кодтармен және т.б. ModBus контроллерінің көпшілігі келесі қате түрлерін қолдайды.

Логикалық қателерді өңдеу.

Қате коды	Аты	Сипаттама
01	ILLEGAL FUNCTION	Функция кодына контроллерде қолдау көрсетілмейді
02	ILLEGAL DATA ADDRESS	Деректер мекенжайы жарамсыз
03	ILLEGAL DATA VALUE	Жарамсыз деректер мәндері
04	SLAVE DEVICE FAILURE	Операцияны орындау кезінде қате орын алды

Жауапта қате орын алса, функция коды өрісінде ең маңызды бит орнатылады, содан кейін әдеттегі деректердің орнына қате коды жіберіледі.

03 функция - сақтау тіркелімдерін оқу

Бірнеше ұстау регистрлерінің мәндерін оқу үшін қолданылады. Сұраныс регистрлер кестесінің бірінші элементінің адресін жібереді, оның мәні оқылуға тиіс және оқылатын регистрлер саны. Мекенжай мен сан үшін 16 разрядты сандар қолданылады. Алдымен жоғары байт жіберіледі.

Сұралған деректер жауапта қамтылған. Деректер блогының алдында оқылған байттардың санын қамтитын байт беріледі.

Сақтау тізілімін оқу функциясы үшін сұрау пішімі.

Байт саны	Параметрдегі байт саны	Параметр	8 және 9 адрестері бар оқу регистрлерінің мысалы	
0	0	Контроллер мекенжайы	01	01
1	0	Функция	03	03
2	1	Регистрлердің бастапқы мекенжайы	0008	00
3	0			08
4	1	Регистрлер саны	0002	00
5	0			02
6	1	Тексеру сомасы	45C9	45
7	0			C9

Жауап (бағдарлы мекенжай, функция коды, оқылған байт саны, регистр мәндері).

Байт саны	Параметрдегі байт саны	Параметр	8 және 9 адрестері бар оқу регистрлерінің мысалы	
0	0	Адрес контроллера	01	01
1	0	Функция	03	03
2	0	Оқылған байттардың саны	04	04
3	1	Тіркеу мәні 8	12A5	12
4	0			A5
5	1	Тіркеу мәні 9	E020	E0
6	0			20
7	1	Тексеру сомасы	A770	A7
8	0			70

Функция 06 – бір ұстау регистріне жазу.

Бір регистрге жазу үшін қолданылады. Сұрау тізілімнің мекенжайын және оның мәнін береді. Сәтті болған жағдайда, бақылаушы жауап ретінде сұраудың көшірмесін жібереді.

Бір сақтау регистрінің жазу функциясының сұраныс форматы

Байт саны	Параметрдегі байт саны	Параметр	12A5 мәнін 9 тіркеуге жазу мысалы	
0	0	Контроллер адресі	01	01
1	0	Функция	06	06
2	1	Тіркеу адресі	0009	00
3	0			09
4	1	Регистр мәні	12A5	12
5	0			A5
6	1	Тексеру сомасы	9513	95
7	0			13

Жауап (сұранысты қайталайды).

Байт саны	Параметрдегі байт саны	Параметр	12A5 мәнін 9 тіркеуге жазу мысалы	
0	0	Контроллер адресі	01	01
1	0	Функция	06	06
2	1	Тіркеу адресі	0009	00
3	0			09
4	1	Регистр мәні	12A5	12
5	0			A5
6	1	Тексеру сомасы	9513	95
7	0			13

16-функция – ұстау регистрлеріне мәндерді жазу.

Кестеде ретімен орналасқан бірнеше регистрлерге жазу үшін қолданылады.

Сұраныс бірінші регистрдің мекенжайын, регистрлердің санын және олардың мәндерін қамтиды.

Жауап бастапқы мекенжайды және өзгертілген регистрлердің санын қайтарады.

Сақтау регистрін жазу функциясының сұрау пішімі

Байт саны	Параметрдегі байт саны	Параметр	8 және 9 адрестері бар регистрлерді жазу мысалы	
0	0	Контроллер адресі	01	01
1	0	Функция	10	10
2	1	Регистрлердің бастапқы адресі	0008	00
3	0			08
4	1	Регистрлер саны	0002	00
5	0			02
6	0	Байт есептегіші	04	04
7	1	Тіркеу мәні 8	12A5	12
8	0			A5
9	1	Тіркеу мәні 9	E020	E0
10	0			20
11	1	Тексеру сомасы	AF4A	AF
12	0			4A

Жауап (бағдарлама мекенжайы, функция коды, бастапқы мекенжай және регистрлер саны)

Байт саны	Параметрдегі байт саны	Параметр	8 және 9 адрестері бар регистрлерді жазу мысалы	
0	0	Контроллер адресі	01	01
1	0	Функция	10	10
2	1	Регистрлердің бастапқы адресі	0008	00
3	0			08
4	1	Регистрлер саны	0002	00
5	0			02
6	1	Тексеру сомасы	C00A	C0
7	0			0A

ModBus және арнайы протоколдар

Енді ModBus хаттамасын алдыңғы лекциядағы мамандандырылған хаттамамен салыстыруға болады.

Сұраныс.

Байт нөмірі	Сан пішімі	Мақсат
0	byte	Жұмыс коды + контроллер адресі (0x10)
1	byte	Жарық диодының жай-күйі (кіші бит)
2	byte	Бақылау коды (байт 0 ^ байт 1 ^ 0xe5)

ModBus және арнайы протоколдар

Жауап.

Байт нөмірі	Сан пішімі	Мақсат
0 ... 3	float	Температура
4 ... 7	float	Кернеу
8	byte	Түйме күйі (кіші бит)
9	byte	Резерв
10, 11	int	Бақылау сомасы (байттар сомасы $0 \dots 9 \wedge 0\text{ха}1\text{е}3$)

ModBus және арнайы протоколдар

Арнайы хаттамамен салыстырғанда:

- ModBus баяу, оның өнімділігі төмен. Ақпараттың бірдей көлемін алу үшін желі арқылы әлдеқайда көп деректер жіберіледі.
- Оны жүзеге асыру қиынырақ. Қосымша микроконтроллер мен желі ресурстарын қажет етеді.

ModBus және арнайы протоколдар

Бірақ көптеген жағдайларда артықшылықтар маңыздырақ.

- Күрделі бақылау сомасы мен ақпараттың артық болуына байланысты ондағы желі қателері сенімдірек анықталады, ал деректер сенімділігі жоғарырақ.
- Желі оңай кеңейеді. Жаңа құрылғыларды қосу өте оңай.
- ModBus стандартты протокол болып табылады. Әртүрлі өндірушілердің көптеген контроллерлері оны қолдайды.

ModBus және арнайы протоколдар

Стандартты емес хаттамаларды қолданудың негізгі себебі жүйенің есептеу ресурстарының жетіспеушілігі болуы мүмкін.

Рас, стандартты емес хаттаманы қолданудың тағы бір себебі бар. Әзірлеушілерден басқа ешкім арнайы протоколы бар құрылғыларға қосылып, оларды басқара алмайды. Анау. сапасы стандартты хаттамаларға мүлдем қарама-қайшы. Қарама-қайшылықтардың бірлігінің философиялық заңы әрекет етеді.

Көрсеткіштер және
сілтемелер

Көрсеткіштер

Көрсеткіштер - бағдарламалаудағы ең қиын тақырыптардың бірі, біз оны қарапайым және практикалық түрде түсіндіруге тырысамыз.

Микроконтроллер жадындағы мәліметтерді бейнелеуден бастайық: разрядтық операциялар туралы өткен сабақта біз ең аз адрестелетін жады блогы байт екенін талқыладық, яғни микроконтроллер жадындағы кез келген байтқа сілтеме жасай аламыз. Айнымалылармен жұмыс істегенде біз жадтағы деректердің мекенжайлары мен орналасуы туралы ойламаймыз, біз олардың атауларын оқу/жазу, функцияларға аргумент ретінде атауларды беру және деректермен басқа әрекеттерді орындау үшін ғана қолданамыз. Мәліметтер блоктарының адрестерін иелену жад тұрғысынан көп нәрсені жылдам және тиімді орындауға мүмкіндік береді.

Көрсеткіштер

Көрсеткіштер беретін мүмкіндіктердің бірнеше мысалдары:

- Көрсеткіштерді пайдалана отырып, кез келген деректерді (барлық түрдегі айнымалылар, құрылымдар) олармен кейіннен өңдеу үшін биттерге бөлуге болады (тасымалдау / жазу / оқу).
- Деректер блоктарының мекенжайларын функцияларға аргументтер ретінде беруге болады, сондықтан функция шақырылғанда айнымалылардың көшірмелері жасалмайды және код жылдамырақ жұмыс істейді. Басқаша айтқанда, функцияға берілген аргументті өзгертуге рұқсат беріңіз.
- Динамикалық жадпен «тікелей» жұмыс істеу, айнымалылар мен объектілер үшін жадты бөлу.

Көрсеткіштер

Көрсеткіш дегеніміз не? Бұл микроконтроллердің жадындағы деректер аймағының адресін (айнымалы/құрылым/ объект/ функция және т.б.) қамтитын айнымалы, дәлірек айтсақ, оның ең бірінші блогы, байт. Деректер блогындағы бірінші байттың мекенжайын біле отырып, сіз сол мекенжайдағы деректерді басқаруға қол жеткізе аласыз, бірақ бұл блоктың өлшемін білуіңіз керек. Сондықтан көрсеткішті құру кезінде ол қандай деректер типін көрсететінін көрсетеміз, ол кез келген деректер түрі болуы мүмкін.

«Адрес» операторлары

Бізде операторлар аз, бірақ олар C++ тілінің ең қуатты мүмкіндіктерінің бірі болып табылады.:

- & - жадтағы мекенжайды қайтарады
- * – көрсетілген мекенжайға қол жеткізу (оқу, жазу).
- -> – мүшелер мен әдістерге жанама қол жеткізу операторы (құрылымдар мен сыныптарға көрсеткіштер үшін). Көрсеткіш арқылы конструкцияның стенографиялық белгісі болып табылады: $a \rightarrow b$ (*a).b-ге тең.

«Адрес» операторлары

Біз осылай қалаған деректер түріне көрсеткіш жасай аламыз:

тип_данных* имя_указателя;

тип_данных * имя_указателя;

тип_данных *имя_указателя;

Иә, сіз көбейтумен шатастыра аласыз, бірақ компилятор шатастырмайды. Барлық үш жазу опциясы баламалы, сіз олардың кез келгенін басқа біреудің кодында немесе мақаласында кездестіре аласыз.

«Кәдімгі» айнымалыларға көрсеткіштер

Көрсеткішпен жұмыс айнымалы мәнін оның адресі арқылы оқуға/өзгертуге мүмкіндік береді. Түсініктемелері бар мысалды қараңыз:

```
byte b;    // просто переменная типа byte
```

```
b = 10;    // b теперь 10
```

```
byte* ptr; // ptr – переменная "указатель на объект типа byte"
```

```
ptr = &b;  // указатель ptr хранит адрес переменной b
```

```
*ptr = 24; // b теперь равна 24 (записываем по адресу &b)
```

```
byte s;    // переменная s
```

```
s = *ptr;  // s теперь тоже равна 24 (читаем по адресу &b)
```

«Кәдімгі» айнымалыларға көрсеткіштер

Күрделі ештеңе жоқ сияқты: біз байт (байт * ptr) үшін ptr көрсеткішін жасап, оған b айнымалысының адресін жаздық: ptr = &b. Енді біз b айнымалысына ptr көрсеткіші арқылы билік жүргіземіз, оның мәнін *ptr = мән ретінде өзгертуге болады; немесе *ptr; ретінде оқыңыз.

Функцияға адресті беруге және функция ішіндегі оның адресіндегі айнымалының мәнін өзгертуге тырысайық. Int айнымалысының адресін алатын функция және сол айнымалының квадраттары болсын.

```
void square(int* val) {  
    *val = *val * *val;  
}
```

«Кәдімгі» айнымалыларға көрсеткіштер

Біз оны осылай қолданамыз:

```
int value = 7; // создали переменную  
square(&value); // передали её адрес в функцию  
// тут value уже == 49
```

Неліктен бұл тәсіл жақсы? Біз функционалдық сабақтағыдай айнымалының көшірмесін жасамадық, жай мекенжайды жіберіп, мәндерді тікелей өзгерттік.

Массивтерге көрсеткіштер

Бізде массивтер туралы бөлек сабақ болды, мен ол жерде мен сізді жүктеп, «массивтер қайдан келетінін» айтқан жоқпын, өйткені массивтер шын мәнінде көрсеткіш және оның достары. Массив аты осы массивтің бірінші элементінің көрсеткіші болып табылады және массивті жариялағанда элемент түрін көрсетеміз. Яғни, `myArray[0] == *myArray` немесе `myArray == &myArray[0]`. Кодты жазуды және оқуды жеңілдету үшін төртбұрышты жақшалар енгізіледі, бірақ іс жүзінде ол келесідей жұмыс істейді: `a[b] == *(a + b)!`

Массив – көрсетілген типтегі «айнымалылармен» толтырылған жад аймағы және біз оларға адрес бойынша қол жеткізе аламыз.

Мысал

```
void setup() {  
  Serial.begin(9600);  
  // работаем без [] скобок  
  byte myArray[] = {1, 2, 3, 4, 5};  
  // выведет 1 2 3 4 5  
  for (byte i = 0; i < 5; i++) {  
    Serial.print(*(myArray + i));  
    Serial.print(' ');  
  }  
  Serial.println();  
  // работаем с отдельным указателем
```

```
  int myArray2[] = {10, 20, 30, 40, 50};  
  int* ptr2 = myArray2; // указатель на  
  массив  
  // выведет 10 20 30 40 50  
  for (byte i = 0; i < 5; i++) {  
    Serial.print(*ptr2);  
    ptr2++;  
    Serial.print(' ');  
  }  
}
```

Мысал

```
void setup() {  
    Serial.begin(9600);  
    int myArray[] = {1, 2, 3, 4, 5, 6};  
    Serial.println(sumArray(myArray));  
}  
  
int sumArray(int* arrayPtr) {  
    int sum = 0;  
    // суммируем массив  
    for (byte i = 0; i < 6; i++) sum +=  
        arrayPtr[i];  
    return sum; // возвращаем
```

Мысал

```
void setup() {  
    Serial.begin(9600);  
    int myArray[] = {1, 2, 3, 4, 5, 6};  
    // выводим сумму массива  
    Serial.println( sumArray(myArray,  
sizeof(myArray)/sizeof(int)) );  
    // передали массив и его размер  
    в количестве элементов  
}
```

```
long sumArray(int* arrayPtr, int  
arrSize) {  
    long sum = 0;  
    for (int i = 0; i < arrSize; i++) sum  
+= arrayPtr[i];  
    return sum; // возвращаем
```

Функция көрсеткіші

Функцияның жадта өз адресі де бар, оған қол жеткізуге болады. Сіз жай ғана көрсеткіш арқылы функцияны шақыра аласыз немесе оны басқа функцияға аргумент ретінде бере аласыз. Функция көрсеткіші осылай жарияланады:

`возвращаемый_тип_данных (*имя)(аргументы)`

Содан кейін көрсеткішке кез келген функцияның мекенжайын жай ғана атау ретінде беруге болады (массивтердегі сияқты мекенжай операторы қажет емес). Көрсеткіш арқылы функцияны шақыру үшін * операторы да қажет емес.

Мысал

```
void setup() {  
    Serial.begin(9600);  
    void (*ptrF)(byte a); // указатель на  
    функцию (она объявлена ниже)  
    ptrF = printByte;    // даём адрес функции }  
    printByte  
    ptrF(125); // вызов printByte через  
    указатель (выведет 125)  
    int (*ptrFunc)(byte a, byte b); // сделаем  
    ещё указатель  
    ptrFunc = sumFunc; // на функцию  
    sumFunc  
    // вызовем printByte, которой передадим  
    результат sumFunc  
    // через указатель ptrFunc  
    ptrF(ptrFunc(10, 30)); // выведет 40  
  
void printByte(byte b) {  
    Serial.println(b);  
}  
  
int sumFunc(byte a, byte b) {  
    return (a + b);  
}  
  
void loop() {}
```

Құрылымдарға/сыныптарға көрсеткіш

Құрылымдар мен сыныптар құрама деректер типтері болып табылады, олармен көрсеткіштер арқылы әрекеттесу механизмі сәл өзгеше. Құрылымды, оған көрсеткішті жасап, сол арқылы құрылымға сілтеме жасайық:

Мысал

```
struct myStruct {  
    byte myByte;  
    int myInt;  
};
```

```
// создадим структуру someStruct  
myStruct someStruct;  
  
// указатель типа myStruct* на  
структуру someStruct
```

```
myStruct* p = &someStruct;
```

```
// пишем по адресу в  
someStruct.myInt
```

```
p -> myInt = -666;
```

```
/*(*p).myInt = -666; // или так, см.  
начало урока
```

void меңзер

Барлық алдыңғы мысалдарда біз бұрын белгілі деректер түрі үшін көрсеткіш жасадық. Бірақ белгісіз деректер түріне мекенжай тағайындағыңыз келсе ше? `void * ptr` - кез келген түрдегі `void` көрсеткішін жасауға болады. Әрі қарай жұмыста көрсеткішті қажетті деректер түріне өзгерту керек:

```
float Fval = 0.254; // переменная float
```

```
void* ptrV = &Fval; // указатель на любой тип (дали ему float, он не против)
```

```
// создали Fptr - указатель на float
```

```
// и преобразовали неизвестный ptrV во float
```

```
float* Fptr = (float*)ptrV;
```

```
// теперь *Fptr равен 0.254
```

Указатель на void

Здесь мы преобразовали ptrV, который был void* (указатель на void) в указатель на float при помощи (float*). Иногда это может быть удобно, например при передаче данных разного формата при помощи одной “универсальной функции”. Также можно встретить преобразование типа указателя через cast:

```
float* Fptr = static_cast<float*>(ptrV);
```

Разбивка на байты

Иногда бывает нужно передать какой-нибудь кусок данных, а потом принять его с другой стороны. Либо записать эти данные на какой-нибудь внешний носитель (EEPROM, карта памяти и подобное), а потом считать его обратно. Нужен универсальный инструмент, который запишет любую дату, а потом корректно её прочитает. Для решения этой задачи можно использовать указатели, делается это следующим образом: создаём указатель на тип `byte`, присваиваем ему адрес блока данных любого типа, выполнив преобразование (`byte*`). Получим просто указатель на первый байт данных. Зная длину (размер в байтах) нашего куска данных, можем считать его побайтно, просто прибавляя к адресу единицу!

Пример

```
// большое число
uint32_t bigVal = 123456789;
// указатель ptrB на адрес &bigVal
// приведённый к (byte*)
byte* ptrB = (byte*)&bigVal;
// разбиваем на байты
byte bigVal_1 = *ptrB;
byte bigVal_2 = *(ptrB + 1);
byte bigVal_3 = *(ptrB + 2);
byte bigVal_4 = *(ptrB + 3);
// попробуем собрать обратно
```

```
// понадобится новая переменная
// такого же типа, что и первая
(uint32_t)
uint32_t newBig;
byte* ptrN = (byte*)&newBig;
// и собираем 4 байта обратно!
*ptrN = bigVal_1;
*(ptrN + 1) = bigVal_2;
*(ptrN + 2) = bigVal_3;
*(ptrN + 3) = bigVal_4;
// в этом месте newBig равна
123456789
```

Разбивка на байты

Таким образом можно “разобрать” и “собрать” любой набор данных (массив любого типа, структуру), зная его размер. Задачу можно решить более красиво, используя массив байтов для чтения и записи. Рассмотрим примеры с приведением типа указателя через (byte*), через void* и через template:

Пример через (byte*)

```
// буфер
byte buffer[20];
// структура для теста
struct myStruct {
    byte val1;
    int val2;
    long val3;
    float val4;
};
void setup() {
    // === тест с переменными ===
    long a = 123456789;
    long b = 0;
    // разбиваем блок данных a на байты
    // и сохраняем в buffer
    writeData((byte*)&a, sizeof(a));
    // собираем блок данных b
    // из буфера buffer
    readData((byte*)&b, sizeof(b));
```

```
// здесь b == 123456789
// === тест со структурами ===
// создаём структуру
myStruct transmit;
// присваиваем значение члену val4
transmit.val4 = 3.1415;
// "приёмная" структура
myStruct recieve;
// разбиваем блок данных transmit на байты
// и сохраняем в buffer
writeData((byte*)&transmit, sizeof(transmit));
// собираем блок данных recieve
// из буфера buffer
readData((byte*)&recieve, sizeof(recieve));

// здесь recieve.val4 == 3.1415
}
void writeData(byte* data, int length) {
    int i = 0;
```

```
while (length--) {
    buffer[i] = *(data + i);
    i++;
}
void readData(byte* data, int length) {
    int i = 0;
    while (length--) {
        *(data + i) = buffer[i];
        i++;
    }
}
void loop() {}
```

Пример через void*

```
// буфер
byte buffer[20];
// структура для теста
struct myStruct {
    byte val1;
    int val2;
    long val3;
    float val4;
};
void setup() {
    // === тест с переменными ===
    long a = 123456789;
    long b = 0;
    // разбиваем блок данных a на байты
    // и сохраняем в buffer
    writeData(&a, sizeof(a));
    // собираем блок данных b
    // из буфера buffer
    readData(&b, sizeof(b));
```

```
// здесь b == 123456789
// === тест со структурами ===
// создаём структуру
myStruct transmit;
// присваиваем значение члену val4
transmit.val4 = 3.1415;
// "приёмная" структура
myStruct recieve;
// разбиваем блок данных transmit на
байты
// и сохраняем в buffer
writeData(&transmit, sizeof(transmit));
// собираем блок данных recieve
// из буфера buffer
readData(&recieve, sizeof(recieve));

// здесь recieve.val4 == 3.1415
}
void writeData(void* data, int length) {
    uint8_t* dataByte = (uint8_t*)data;
```

```
int i = 0;
while (length--) {
    buffer[i] = *(dataByte + i);
    i++;
}
void readData(void* data, int length) {
    uint8_t* dataByte = (uint8_t*)data;
    int i = 0;
    while (length--) {
        *(dataByte + i) = buffer[i];
        i++;
    }
}
void loop() {}
```

Пример через template

```
// буфер
```

```
byte buffer[20];
```

```
// структура для теста
```

```
struct myStruct {
```

```
    byte val1;
```

```
    int val2;
```

```
    long val3;
```

```
    float val4;
```

```
};
```

```
void setup() {
```

```
    // === тест с переменными ===
```

```
    long a = 123456789;
```

```
    long b = 0;
```

```
    // разбиваем блок данных a на байты
```

```
    // и сохраняем в buffer
```

```
    writeData(a);
```

```
    // собираем блок данных b
```

```
    // из буфера buffer
```

```
    readData(b);
```

```
// здесь b == 123456789
```

```
// === тест со структурами ===
```

```
// создаём структуру
```

```
myStruct transmit;
```

```
// присваиваем значение члену val4
```

```
transmit.val4 = 3.1415;
```

```
// "приёмная" структура
```

```
myStruct recieve;
```

```
// разбиваем блок данных transmit на  
байты
```

```
// и сохраняем в buffer
```

```
writeData(transmit);
```

```
// собираем блок данных recieve
```

```
// из буфера buffer
```

```
readData(recieve);
```

```
// здесь recieve.val4 == 3.1415
```

```
}
```

```
template <typename T>
```

```
void writeData(T &data) {
```

```
    const uint8_t *ptr = (const uint8_t*)  
&data;
```

```
    for (uint16_t i = 0; i < sizeof(T); i++) {  
        buffer[i] = *ptr++;
```

```
    }
```

```
}
```

```
template <typename T>
```

```
void readData(T &data) {
```

```
    uint8_t *ptr = (uint8_t*) &data;
```

```
    for (uint16_t i = 0; i < sizeof(T); i++) {  
        *ptr++ = buffer[i];
```

```
    }
```

```
}
```

```
void loop() {}
```

Разбивка на байты

Данные примеры отличаются только вариантом передачи аргумента-адреса и его обработки:

- В первом случае мы приводим указатель к (`byte*`) при передаче аргумента в функцию. Также передаём размер блока данных при помощи `sizeof()`
- Во втором случае у нас `void*` и ему всё равно, какой тип данных ему передадут. Далее мы переводим указатель в `uint8_t` через `reinterpret_cast`. Также передаём размер блока данных при помощи `sizeof()`
- Третий вариант – через шаблонную функцию, которой вообще без разницы, какой у данных тип и размер: она принимает данные по ссылке. Далее делаем указатель на первый байт и прямо внутри функции вычисляем размер блока данных через `sizeof()`. Это самый мощный и универсальный вариант.

Примечание: все три примера занимают одинаковый объём в памяти.

Ссылки

Ссылки – по сути те же самые указатели, но с другим синтаксисом.
В отличие от указателя, ссылка

Должна быть инициализирована при создании

Не может быть изменена в процессе работы программы

тип_данных &имя_ссылки = данные;

Ссылки

Ссылка может работать в тех же случаях, что и указатель:

- “Перехватывать управление” данными, т.н. ссылка в качестве “псевдонима”, через неё можно читать и писать
- Быть аргументом в функциях
- Осуществлять более удобный доступ к данным

В отличие от указателя, ссылка не может быть изменена, т.е. всегда указывает на те данные, которые ей показали при инициализации!

Ссылки на типы данных

Рассмотрим первый пример с указателями в этой главе, но с использованием ссылок. Пример полностью аналогичный с точки зрения происходящего, но вместо указателей используются ссылки:

```
byte b;    // просто переменная типа byte
```

```
b = 10;    // b теперь 10
```

```
byte &link = b; // link – переменная "ссылка на объект типа byte"
```

```
link = 24; // b теперь равна 24 (записываем через ссылку)
```

```
byte s;    // переменная s
```

```
s = link; // s теперь тоже равна 24 (читаем по ссылке)
```

Ссылки на типы данных

Вспомним пример с функцией, возводящей переменную в квадрат, и перепишем её через ссылки: передадим ссылку на переменную в качестве аргумента функции.

```
void setup() {  
    int value = 7; // создали переменную  
    square(value); // передали её в функцию  
    // тут value уже == 49  
}  
  
void square(int &val) { // val - ссылка на аргумент!  
    val = val * val;  
}
```

Работа программы полностью идентична примеру с указателем, но в таком применении ссылки гораздо удобнее, чем указатели: нужно писать меньше символов, код выглядит более читаемым.

Ссылка на String-строку

Строка тоже является объёмом данных, на который можно сослаться. Эту тему вынесем в отдельный раздел, потому что он очень важный: строки являются тяжёлым инструментом, который при неправильном использовании может привести к внезапному переполнению оперативной памяти и микроконтроллер зависнет. Основной момент, где нам понадобятся адреса – передача строки в качестве аргумента. Если передавать строку по значению – в оперативной памяти будет создан новый экземпляр строки, то есть на протяжении работы функции у нас в памяти будут две одинаковые строки! Если память достаточно сильно занята, а строка большая – может случиться беда.

Пример

```
void setup() {  
    Serial.begin(9600); // для отладки  
    String myString = "hello world! Hello again!";  
    printString(myString);  
}  
void printString(String string) {  
    Serial.println(string);  
}  
void loop() {}
```

Ссылка на String-строку

Внутри функции `printString()` в памяти существуют две одинаковые строки, а при её вызове тратится время на выделение памяти под копию строки и копирование её туда.

Оптимизировать такой сценарий очень просто: передавать строку не по значению, создавая её копию, а передавая по адресу адрес через ссылку:

Пример

```
void setup() {  
  Serial.begin(9600); // для отладки  
  String myString = "hello world! Hello again!";  
  printString(myString);  
}  
void printString(String &string) {  
  Serial.println(string);  
}  
void loop() {}
```

Такой вариант работы со строкой быстрее и безопаснее для тяжёлых скетчей и больших строк.

Ссылка на структуру

Использование ссылок позволяет упростить доступ ко вложенным структурам:

```
struct Values {  
    int value1;  
    float value2;  
};  
struct BigStruct {  
    Values values; // элемент типа  
    Values
```

```
    int otherValue;  
};  
BigStruct myStruct;  
// делаем ссылку на элемент float  
float &link = myStruct.values.value2;  
// присваиваем через ссылку  
link = 3.14;  
// здесь myStruct.values.value2 ==  
3.14
```

